Bernoulli Types

An Algebra of Approximate Computation

Alexander Towell

May 24, 2026

© 2026 Alexander Towell.

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (`https://creativecommons.org/licenses/by-nc-sa/4.0/`).

The Python companion library `bernoulli` is released separately under the MIT License.

ORCID: `0000-0001-6443-9897`
Web: `https://metafunctor.com`
Source: `https://github.com/queelius/bernoulli-book`

Contents

Preface **vii**

I The Atom **1**

1 The Noisy Bit **3**
1.1 Three Faces of a Noisy Bit . . . 3
1.2 Quantifying the Noise . . . 6
1.3 A First Look at the Boolean Universe . . . 11
1.4 Sampling and Estimating Rates . . . 14
1.5 Why "Atom"? . . . 16
Bibliographic Notes . . . 20
Exercises . . . 21

2 Binary Channels and Model Order **27**
2.1 The Binary Symmetric Channel . . . 27
2.2 The Asymmetric Binary Channel . . . 33
2.3 Composition of Channels . . . 38
2.4 Model Order . . . 44
Bibliographic Notes . . . 51
Exercises . . . 53

3 Boolean Algebra Under Noise **59**
3.1 AND Under Noise . . . 59
3.2 OR and NOT . . . 66
3.3 The Composition Theorem . . . 72
3.4 Miller-Rabin: A Worked Walkthrough . . . 81
3.5 Composition is the Bridge to Sets . . . 88
Bibliographic Notes . . . 92
Exercises . . . 94

4 Two Axioms and Kronecker Factorization 99
4.1 Why Axioms? . . . 99
4.2 Axiom 1: Element-Wise Independence . . . 102
4.3 Axiom 2: Conditional Independence of Block Error Rates . . . 107
4.4 The Kronecker Factorization Theorem . . . 114
4.5 The Bernoulli[bool] Type, Formally . . . 123
4.6 What Comes Next . . . 129
Bibliographic Notes . . . 131
Exercises . . . 133

II From Bits to Sets 137

5 Approximate Sets and Predicates 139
5.1 From Atom to Set . . . 139
5.2 The Bloom Filter From Scratch . . . 143
5.3 The Predicate Algebra . . . 150
5.4 One-Sided vs Two-Sided Approximate Sets . . . 155
5.5 What Comes Next . . . 158
Bibliographic Notes . . . 161
Exercises . . . 162

6 Composition of Sets 169
6.1 Union of Approximate Sets . . . 169
6.2 Intersection, Complement, Difference . . . 174
6.3 Cancellation and the Loss of One-Sidedness . . . 179
6.4 The Perfect Filter Line . . . 184
6.5 What Comes Next . . . 187
Bibliographic Notes . . . 189
Exercises . . . 190

7 Classification Measures 197
7.1 The Setup Revisited . . . 197
7.2 The Binomial Estimator: Properties . . . 199
7.3 Confidence Intervals Properly . . . 204
7.4 Derived Measures . . . 210
7.5 Worked Application: Bloom Filter Diagnostics . . . 216
Bibliographic Notes . . . 220
Exercises . . . 221

8 Information-Theoretic Bounds **227**
8.1 Shannon Entropy Refresher 227
8.2 Channel Capacity in the BSC 230
8.3 The Entropy Floor for Approximate Sets 233
8.4 The Bloom Filter's Gap 238
8.5 What Comes Next 242
Bibliographic Notes 243
Exercises 245

III From Sets to Maps **249**

9 Approximate Functions and n-ary Channels **251**
9.1 From Approximate Sets to Approximate Functions 251
9.2 N-ary Channels 255
9.3 The n-ary Kronecker Theorem 260
9.4 Capacity for n-ary Channels 266
9.5 Bridge to Sketches 272
Bibliographic Notes 273
Exercises 275

10 Sketches and Estimators **279**
10.1 Sketches as Bernoulli of T 279
10.2 CountMin 282
10.3 HyperLogLog 287
10.4 MinHash and LSH 293
10.5 Bridge 299
Bibliographic Notes 300
Exercises 302

11 Randomized Algorithms **307**
11.1 Miller-Rabin Formal 307
11.2 Las Vegas vs Monte Carlo 311
11.3 RP, BPP, and Bernoulli Type 316
11.4 Sample Complexity 321
11.5 Bridge 326

12 Approximate Relations **329**

13 The Algebra of Approximate Types **331**

IV Optimality and Construction **333**

14 Optimal Bernoulli Sets **335**

15 Perfect Hash Filters **337**

V The Boundary and Beyond **339**

16 Approximate but Not Bernoulli **341**

17 Cipher Maps and Trapdoor Computing **343**

18 Encrypted Search as Relaxed Trapdoor **345**

A Probability Refresher **347**

B The Five Core Principles **349**

C Library Walkthrough **351**

D Selected Exercise Solutions **353**

Preface

This is a placeholder preface. The final preface will:

- State the book's thesis and audience.
- Declare prerequisites: undergraduate probability, basic discrete math, comfort with proof, working Python knowledge for the notebook companion.
- Explain the lockstep notebook discipline (some chapters carry notebooks; theory-heavy chapters do not, by design).
- Provide a reading guide for the smart-undergrad-upward reader (chapters that admit a soft landing) and the adjacent-field grad student (the modal reader).

Part I

The Atom

Chapter 1

The Noisy Bit

After this chapter you will be able to:

- Recognize a noisy bit when you see one, whether it arrives as a biased coin, a binary symmetric channel, or a binary classifier's verdict.
- Quantify a noisy bit with its false-positive and false-negative rates, and sample to estimate them.
- Name the noisy bit as the atomic object of the framework, and point at the trajectory that develops its algebra across the remaining chapters of Part I.

1.1 Three Faces of a Noisy Bit

Three systems, drawn from three different corners of applied probability, each produce a single Boolean output that may be wrong. The first is a biased coin that reports an external state. The second is a one-bit communication channel that occasionally corrupts what it transmits. The third is a binary classifier that emits a verdict about each input it sees. We look at each in turn, with no formalism beyond informal arithmetic. The question to hold throughout: what is the abstract structure that all three share?

Face 1: The biased coin

Imagine a coin that is not quite fair, and whose bias depends on how it has been held. When you palm it "in" (cupped, warm side down) and flip, it lands heads with probability τ. When you palm it "out" (cool side

down) and flip, it lands heads with probability ϵ. The numbers τ and ϵ are properties of the coin and how it is being held. They need not be equal, and neither needs to be close to 1/2.

This is a contrived setup, on purpose. Read it as a thought-experiment device that lets us write a single object that has two possible inputs (in versus out), one Boolean output (heads versus tails), and two independent parameters for how reliably the output tracks the input. The same flip mechanism is at work in both cases; what changes is how the coin is being held. In the language we are building toward, "in" is a positive condition and "out" is a negative condition, and the coin reports each with its own characteristic reliability. The biased coin returns as a running example throughout Part I, first in Section 2.2 as an asymmetric channel and again as the worked example of the formal definition in Chapter 4.

Face 2: The binary symmetric channel

A single bit is transmitted across a noisy line: a wire, a wireless link, a chip-to-chip trace, or any one-bit communication path subject to disturbance. The line sends $\top$ or $\bot$ ($\top$ for true, $\bot$ for false, used throughout the book) from sender to receiver, and for each bit independently the line behaves as if a small probabilistic adversary acts on it. With probability ε the line flips the bit; with probability $1 - \varepsilon$ it passes the bit through unchanged. The crossover probability ε is the same regardless of which symbol is being transmitted: a $\top$ flips to $\bot$ with probability ε, and a $\bot$ flips to $\top$ with the same probability ε.

This object is the binary symmetric channel, and it is the simplest model of a noisy one-bit communication line. The word *channel* is borrowed from Shannon's information theory and is the standard name for any single-input, single-output stochastic transmission. The symmetry assumption (one parameter for both flip directions) is a strong one. Many real communication systems are asymmetric: a transistor pulled to ground may be more reliable than one held high; a photon detector may have a small dark-count rate but never an extinguished-photon false negative. Such systems need two parameters, one per direction, and we will treat them in Section 2.2. For now, the BSC is the cleanest starting point because it isolates the phenomenon of bit-flip noise from the question of whether one direction is noisier than the other. The BSC is the most studied noisy channel in information theory, and the chapter 2 development of it (Section 2.1) will give it both a formal definition and a matrix representation.

Face 3: The classifier verdict

A spam filter examines each incoming message and emits a verdict: spam ($\top$) or ham ($\bot$). Some messages are genuinely spam; some are genuinely ham; the filter does its best on each. Like any imperfect classifier, it sometimes flags ham as spam (a false positive) and sometimes misses spam, marking it as ham (a false negative). The rate at which it flags ham as spam is the false-positive rate ε; the rate at which it misses spam is the false-negative rate ω. In general these two numbers are not equal, and the asymmetry matters: a spam filter that drops legitimate mail at 1% is a different product from one that lets 1% of spam through.

Notice that the spam filter, unlike the BSC, needs two parameters by design rather than by happenstance. The two error directions are conceptually distinct: a false positive misroutes legitimate mail, and a false negative leaks spam into the inbox. These are different harms with different costs. A useful classifier therefore comes with two rates, not one, and the user of the classifier wants to know both. The classifier verdict is our most realistic noisy-bit setting, because it forces us to take two-parameter error structure seriously from the start. The vocabulary of false positives and false negatives is the bridge to the contingency-table arithmetic we develop in Section 1.2, and the spam filter itself returns as a worked example throughout the chapter.

The unifying observation

We have three systems: a coin held in or out, a wire transmitting one bit, and a filter emitting one verdict. They live in different worlds. The first is physical, the second is informational, the third is algorithmic. Yet at the level of input-output behavior they look the same. Each takes a Boolean input ($\top$ or $\bot$, by whatever name) and produces a Boolean output. Each output is, with some probability, the correct one, and with the remaining probability the wrong one. The error behavior in every case is characterized by at most two numbers: the rate of reporting $\top$ when the truth is $\bot$, and the rate of reporting $\bot$ when the truth is $\top$. The BSC happens to have a single parameter because its two rates are constrained to be equal; the coin and the spam filter need two. In all three cases, two numbers suffice to describe everything probabilistic at the single-query level.

We will call the abstract structure that all three share *a noisy bit*.

The three sections used three vocabularies, by design: each face is more natural in its own letters. The reconciliation is direct. The biased coin's τ is the true-positive rate $\tau = 1 - \omega$, and its ϵ is the false-positive rate ε. The

classifier's ε, ω are the same quantities under different names. The binary symmetric channel is the special case where these two rates are forced to be equal: a single $\varepsilon = \omega$ suffices.

The naming is deliberate. It captures three commitments we will hold to throughout Part I. First, the object is atomic: a noisy bit has one Boolean input slot and one Boolean output slot, and nothing larger. (We will see in chapters 2 and 4 that everything bigger, multi-bit channels, repeated-trial algorithms like Miller-Rabin, classifier ensembles, is built up from noisy bits.) Second, the structure is purely behavioral: we say nothing about how the noise arises, only about its rates. A biased coin, a thermal photon detector, and a logistic-regression classifier may have totally different mechanisms and identical noisy-bit profiles, and our framework treats them identically. Third, the description is parsimonious: at most two numbers per noisy bit, no more. The rest of Part I unfolds the algebra of how these atoms compose, decompose, and combine.

Where the chapter goes from here

Section 1.2 fixes the vocabulary by introducing the 2×2 contingency table that gives ε and ω a precise meaning, and connects this notation to the classifier vocabulary of precision and recall. Section 1.3 walks the four-cell Boolean universe (two true inputs, two observed outputs) qualitatively, deferring the joint-probability formalism to chapter 4. Section 1.4 asks how we estimate ε and ω from data, hooking the chapter's accompanying notebook. Section 1.5 closes the chapter by saying why this object earns the title "atom" and previews the algebraic development of chapters 2, 3, and 4. The three faces return at named points in each of those chapters; by the end of Part I, each will have been treated at four progressively tighter levels of formality, culminating in a single definition that retroactively justifies everything we say about it now.

1.2 Quantifying the Noise

The closing observation of Section 1.1 was that every noisy bit, biased coin, binary symmetric channel, classifier verdict, is characterized by at most two error rates: the rate of reporting $\top$ when the truth is $\bot$, and the rate of reporting $\bot$ when the truth is $\top$. That phrasing is suggestive but informal. What does "rate" mean precisely, what is the denominator, and how do we measure these numbers from data? Those are the questions of this section.

The path is short. We will introduce the 2×2 contingency table that records the outcome of n observations of a noisy bit, name the four rates one can compute from that table, and connect those rates to the precision and recall that practitioners of binary classification already know. By the end of this section the reader can read off the error rates of any noisy bit from a confusion matrix, and can move freely between (ε, ω) on the one hand and $(\text{precision}, \text{recall})$ on the other.

The contingency table

Suppose we run a noisy bit n times. On each trial we know the true input, either $\top$ or $\bot$, and we observe an output, also either $\top$ or $\bot$. The result of n trials is a list of n ordered pairs $(\text{truth}, \text{output})$, and each pair falls into exactly one of four joint states. A pair where the truth is $\top$ and the output is $\top$ is a *true positive*. A pair where the truth is $\top$ but the output is $\bot$ is a *false negative*, the noisy bit reported a $\bot$ when it should have reported $\top$. A pair where the truth is $\bot$ but the output is $\top$ is a *false positive*. A pair where the truth is $\bot$ and the output is $\bot$ is a *true negative*. A standing source of student confusion is that "false negative" names the *system output*, not the truth: a false negative is a $\bot$ output produced when the truth was $\top$.

Let TP, FN, FP, and TN denote the counts of each joint state across the n trials. By construction $\text{TP} + \text{FN} + \text{FP} + \text{TN} = n$. We arrange these four counts in a 2×2 table indexed by truth (rows) and observed output (columns):

	output $\top$	**output** $\bot$	total
truth $\top$	TP	FN	TP + FN
truth $\bot$	FP	TN	FP + TN
total	TP + FP	FN + TN	n

Table 1.1: The 2×2 contingency table for n observations of a noisy bit. Each of the n trials contributes one count to exactly one cell. The row totals record how many trials had each truth value; the column totals record how many trials had each observed output.

Table 1.1 is the empirical record of one experiment. Two conventions matter for reading it. First, rows are conditioned on truth: the top row holds only those trials whose true input was $\top$, and the bottom row holds only those trials whose true input was $\bot$. Second, the diagonal cells (TP and

TN) record agreement between truth and output, and the off-diagonal cells (FN and FP) record disagreement. A perfectly accurate noisy bit produces all of its mass on the diagonal; a maximally noisy one produces it all off the diagonal.

From counts to rates

The two error rates of a noisy bit are conditional: each is normalized by the row total of the truth on which it is conditioned. The *false-positive rate* is the fraction of $\bot$-inputs that produce a $\top$-output,

$$\varepsilon \;=\; \frac{\mathrm{FP}}{\mathrm{TN}+\mathrm{FP}},$$

and the *false-negative rate* is the fraction of $\top$-inputs that produce a $\bot$-output,

$$\omega \;=\; \frac{\mathrm{FN}}{\mathrm{TP}+\mathrm{FN}}.$$

Each rate divides one off-diagonal cell by its row total, not by n. The denominator FP + TN counts only the trials whose true input was $\bot$, and FN + TP counts only the trials whose true input was $\top$. The conditioning matters: it makes ε and ω properties of the noisy bit itself, not of the particular mixture of inputs we happen to have observed.

The two complementary rates are the *true-negative rate* $\eta = 1-\varepsilon$ and the *true-positive rate* $\tau = 1-\omega$. The true-negative rate is the fraction of $\bot$-inputs reported correctly; the true-positive rate is the fraction of $\top$-inputs reported correctly. The true-positive rate is also called *sensitivity* in clinical and classification settings, and the true-negative rate is called *specificity*.

Notation sidebar

Rate symbols and macros. The four rates used throughout this book have fixed Greek letters and fixed `alex.sty` macros.

- ε (`\fprate`) false-positive rate. The rate at which a $\bot$-input is reported as $\top$.
- ω (`\fnrate`) false-negative rate. The rate at which a $\top$-input is reported as $\bot$.
- τ (`\tprate`) true-positive rate, $= 1-\omega$. Also called *sensitivity* or *recall*.
- η (`\tnrate`) true-negative rate, $= 1-\varepsilon$. Also called *specificity*.

In the biased-coin vocabulary of Section 1.1, τ was written τ and ε was written ϵ; here we adopt names that emphasize what each rate measures.

The two free parameters of a generic noisy bit are ε and ω; the other two are derived. The binary symmetric channel of Section 2.1 is the constrained case in which $\varepsilon = \omega$, and one parameter suffices. A consolidated cheat sheet, including how these symbols connect to the five core principles of the framework, appears in Appendix B.

Cross-walk to precision and recall

A practitioner of binary classification looks at Table 1.1 with a different vocabulary in mind. The standard metrics for a classifier are *precision* and *recall*:

$$\text{precision} \;=\; \frac{\text{TP}}{\text{TP}+\text{FP}}, \qquad \text{recall} \;=\; \frac{\text{TP}}{\text{TP}+\text{FN}}.$$

Read off the table: precision divides TP by the *column* total of trials the noisy bit *called* $\top$, and recall divides TP by the *row* total of trials whose true input was $\top$. Recall therefore coincides with the true-positive rate: $\text{recall} = \tau = 1 - \omega$.

Precision is different. Its denominator is the column total TP+FP, which depends not only on the noisy bit's error rates but also on how many $\top$- and $\bot$-inputs were in the sample. Concretely, if the sample contains $n_\top$ positives and $n_\bot$ negatives with $n_\top + n_\bot = n$, then $\text{TP} \approx n_\top \tau$ and $\text{FP} \approx n_\bot \varepsilon$ (treating τ and ε as the population rates, this is the expected-count decomposition; for the finite-sample version see Section 1.4), so

$$\text{precision} \;\approx\; \frac{n_\top \tau}{n_\top \tau + n_\bot \varepsilon} \;=\; \frac{n_\top (1-\omega)}{n_\top (1-\omega) + n_\bot \varepsilon}.$$

Two facts follow. First, precision is a function not only of (ε, ω) but also of the input mix $n_\top / n_\bot$, the *prevalence* of positives in the sample. Recall depends only on the noisy bit. Second, precision and recall together determine ε and ω only when the prevalence is also known. The pair (ε, ω) is intrinsic to the noisy bit; the pair (precision, recall) is intrinsic to the noisy bit together with a workload.

This is why the framework of this book uses (ε, ω) as the primary description of a noisy bit, and treats precision and the related accuracy as derived measures. The full development of these derived measures lives in

the companion paper on classification measures; here we note only that they are expressible in terms of the four cells of Table 1.1 and the prevalence.

Worked example: a spam-filter run

Suppose a spam filter is run on a labelled mailbox of $n = 100$ messages, of which 40 are spam ($\top$) and 60 are ham ($\bot$). The filter's outputs produce the contingency table

	flagged spam	**flagged ham**	total
true spam	35	5	40
true ham	3	57	60
total	38	62	100

Row totals: $35 + 5 = 40$ spam, and $3 + 57 = 60$ ham. Column totals: $35 + 3 = 38$ flagged, and $5 + 57 = 62$ not flagged. Grand total 100. The table is internally consistent.

The four rates are

$$\varepsilon = \frac{3}{60} = 0.05, \quad \omega = \frac{5}{40} = 0.125, \quad \eta = \frac{57}{60} = 0.95, \quad \tau = \frac{35}{40} = 0.875.$$

By construction $\eta = 1 - \varepsilon$ and $\tau = 1 - \omega$, and both identities check.

The two classifier metrics are

$$\text{precision} = \frac{35}{35 + 3} = \frac{35}{38} \approx 0.921, \qquad \text{recall} = \frac{35}{35 + 5} = \frac{35}{40} = 0.875.$$

Recall matches τ, as the cross-walk predicts. Precision lies between τ and 1, reflecting that the column total 38 is smaller than the row total 40 because the filter misses 5 spam messages. The prevalence of spam in this sample is $40/100 = 0.4$; a sample with a different prevalence would yield a different precision from the same (ε, ω), illustrating the point of the previous subsection.

What we have, what comes next

A noisy bit is characterized by two numbers, ε and ω, each a conditional rate read off the contingency table. The complementary rates τ and η are derived, as are the classifier metrics precision and recall. The symbols $\varepsilon, \omega, \tau, \eta$ recur throughout the book and refer always to the macros listed in the notation sidebar.

Section 1.3 now walks the four-cell Boolean universe of joint truth-output pairs whose conditional rates this section has just defined, treating the cells qualitatively before Chapter 4 assigns each cell a joint probability. Section 1.4 returns to the present section's contingency-table arithmetic from the data-analytic side, asking how reliably ε and ω can be estimated from finite n.

1.3 A First Look at the Boolean Universe

A noisy bit has two possible inputs and two possible outputs. The truth on any given trial is either $\top$ or $\bot$, and the reported value is also either $\top$ or $\bot$. Cross those two binary choices and four input-output combinations exhaust the joint behavior of the device. You have already seen this 2×2 grid in Section 1.2, but as a table of counts; here we look at it as a table of *outcome types*, the four ways a noisy bit's trial can turn out. We arrange the combinations in a 2×2 outcome matrix indexed by the true input (rows) and the observed report (columns):

	report $\top$	**report** $\bot$
truth $\top$	correct positive	false negative
truth $\bot$	false positive	correct negative

The four cells are the universe of joint behavior. Anything the noisy bit can do on a single trial is one of these four outcomes; there are no others. The chapter from here on is about the structure of these four cells, what populates each one, how the populations move when the device is composed with another, and how the rates of Section 1.2 fit on top of them.

The four cells, narratively

Take the cells one at a time.

Truth $\top$*, report* $\top$*.* The input was $\top$ and the device reported $\top$. Call this a *correct positive*: the noisy bit got it right on a $\top$-input. In the spam-filter vocabulary of Section 1.1 this is a spam message correctly flagged. In the biased-coin face this is the in-handheld trial that came up heads as it usually does. In the binary-channel face this is the bit that traveled the wire without flipping.

Truth $\top$*, report* $\bot$*.* The input was $\top$ and the device reported $\bot$. This is the *false negative*: the noisy bit missed. The spam filter dropped a spam message into the inbox. The coin came up tails on a trial that should usually

have produced heads. The wire flipped a transmitted $\top$ into a $\bot$. Whatever the mechanism, the system replied with the wrong answer in the $\top$-to-$\bot$ direction.

Truth $\bot$, report $\top$. The input was $\bot$ and the device reported $\top$. This is the *false positive*: the noisy bit fired when it should have stayed silent. The spam filter flagged a legitimate message. The coin came up heads on a trial when heads was unlikely. The wire flipped a $\bot$ into a $\top$.

Truth $\bot$, report $\bot$. The input was $\bot$ and the device reported $\bot$. The *correct negative*: the noisy bit stayed silent when it should have. The ham message passed through, the unlikely-heads trial produced tails as expected, the $\bot$-bit traveled the wire unchanged.

Those are the four cells. Their names are exactly the four labels of the contingency table in Section 1.2: TP, FN, FP, TN. This is not a coincidence; it is the same four-way split seen from a different angle. The contingency table of Section 1.2 records what happens when we run the noisy bit n times and *count* how many trials land in each cell. The outcome matrix above records the *cells themselves*, abstracted from any particular run. The four cells of the joint distribution and the four cells of the empirical contingency table are the same four cells. The empirical table is a sample drawn from the joint distribution over those cells.

Why this is a "universe"

The input domain $\{\top, \bot\}$ has exactly two elements. We will call this set the *Boolean universe* and write it $U = \{\top, \bot\}$. The word *universe* is borrowed from set-theoretic usage: it is the ambient set out of which subsets are formed and over which predicates are evaluated. For our purposes the Boolean universe is the smallest non-trivial universe of interest. With one element, no nontrivial predicate exists. With two, we can already ask whether an input is one specific value or the other, which is the simplest discrimination a classifier can perform.

The smallness is the point. Part I of this book develops the algebra of this universe and nothing larger. Every result, every composition rule, every error-rate formula in the chapters that follow is a statement about predicates whose input is a single Boolean. The reason for that restriction is pedagogical and structural: the Boolean universe is the *atom* of the framework, and every richer universe (multi-bit inputs, finite enumerations, arbitrary algebraic types) is built up from Booleans by Kronecker products in Part II and beyond. If we understand the algebra of the four cells, we understand a great deal about systems that look more complicated on the

surface but reduce, on inspection, to compositions of noisy bits.

The deferral

A fully formal description of the four cells would assign each one a probability mass. Given the rates of Section 1.2 and the prevalence of $\top$ in the input stream, the four cells receive four numbers that sum to one. The resulting object is a joint distribution over the four pairs (truth, report). Multiplying it out involves the rates ε, ω, τ, η, and the prior probability of a $\top$-input. We will not write that joint distribution here.

The reason for the deferral is structural, not coy. The formal definition of the object we are circling, the type whose instances are exactly the noisy bits with two free rate parameters, lives in Section 4.5. That section earns the formal step by first introducing the two axioms that make the type well-posed (in chapter 4 §§4.2 and 4.3) and then proving the Kronecker factorization that justifies the parameter economy (chapter 4 §4.4). Writing the joint PMF in §1.3 would either repeat that machinery or assert it without support. Both are bad pedagogy. The honest path is to acknowledge that the four cells admit a joint distribution, name where that distribution gets written down, and continue. Section 4.5 returns to this picture with the formal definition; the table of joint probabilities, the channel matrix, and the worked biased-coin instance all appear there.

A qualitative example: the biased coin

The biased coin from Section 1.1 runs through the book, and it is worth seeing what the four cells look like for a specific instance. Take $\varepsilon = 0.1$ and $\omega = 0.2$: the coin reports a $\top$ when the truth is $\bot$ one trial in ten on average, and reports a $\bot$ when the truth is $\top$ one trial in five on average.

In the outcome matrix this means the following. Of every five $\top$-trials, roughly four land in the *correct positive* cell and roughly one lands in the *false negative* cell: the true-positive rate is $\tau = 1 - \omega = 0.8$ and the false-negative rate is $\omega = 0.2$. Of every ten $\bot$-trials, nine land in the *correct negative* cell and one lands in the *false positive* cell: $\eta = 1 - \varepsilon = 0.9$ and $\varepsilon = 0.1$.

These four conditional shares (four-fifths, one-fifth, one-tenth, nine-tenths) say what fraction of each *row* the noisy bit places in each *column*. They do not say what fraction of all trials lands in each cell, because that fraction also depends on how many trials had truth $\top$ versus truth $\bot$ in the first place. The bridge from row-conditional rates to outright cell probabilities is

exactly the joint distribution we are deferring. Section 4.5 takes the bridge.

Where the chapter goes from here

This section has named four cells of the joint distribution but refused to write down the joint distribution itself. Section 1.4 takes the next concrete step: you can *measure* what is in those cells, by sampling, even before Section 4.5 writes the joint formula. Estimation and formalization meet later; in Section 1.4 we start estimating. That returns the chapter to concrete arithmetic and sets up the question, picked up by chapters 2 through 4, of what an algebra of these noisy bits looks like.

1.4 Sampling and Estimating Rates

Section 1.2 defined what ε and ω mean as conditional rates read off a contingency table. Section 1.3 walked the joint structure of the four cells of that table. Both treated the rates as known. In practice they are not. A spam filter arrives without a label saying "my false-positive rate is 0.03"; a randomized algorithm comes with a worst-case error bound but not a tight one; a noisy channel must be characterized from observed traffic. The honest question is: given a finite sample of trials, how do we estimate the rates, and how confident should we be in the estimate?

This section answers in three short pieces. Binomial counts give the estimator. The central limit theorem gives an approximate confidence interval. A notebook simulation lets the reader watch both work and fail. The full theoretical treatment of the estimators (bias, variance, efficiency, alternative interval forms) lives in chapter 7 (Part II), which earns the formal step after the algebra of approximate sets is in place.

Binomial counts

Run a noisy bit n times. Of those n trials, suppose n_0 have truth $\bot$ and $n_1 = n - n_0$ have truth $\top$. The two row totals of the contingency table of Table 1.1 are n_0 and n_1. For each of the n_0 trials with truth $\bot$, the noisy bit independently reports $\top$ with probability ε; the count of such reports is the false-positive count FP. Independence across the n_0 trials makes the count a binomial random variable,

$$\mathrm{FP} \;\sim\; \mathrm{Binomial}(n_0, \varepsilon).$$

The same construction on the n_1 trials with truth $\top$ gives $\mathrm{FN} \sim \mathrm{Binomial}(n_1, \omega)$.

The natural empirical estimators are the row-conditional proportions

$$\hat{\varepsilon} \;=\; \frac{\mathrm{FP}}{n_0}, \qquad \hat{\omega} \;=\; \frac{\mathrm{FN}}{n_1}.$$

These are exactly the formulas of Section 1.2 with empirical counts in the numerator. The estimators have two properties worth stating at this stage and deferring the proof for later. First, each estimator is unbiased: $\mathbb{E}[\hat{\varepsilon}] = \varepsilon$ and $\mathbb{E}[\hat{\omega}] = \omega$. Second, the variances are

$$\mathrm{Var}(\hat{\varepsilon}) \;=\; \frac{\varepsilon(1-\varepsilon)}{n_0}, \qquad \mathrm{Var}(\hat{\omega}) \;=\; \frac{\omega(1-\omega)}{n_1}.$$

Both shrink as $1/n_0$ or $1/n_1$, so the estimator improves with more trials of the corresponding truth class. The variance numerator $\varepsilon(1-\varepsilon)$ peaks at $\varepsilon = 1/2$, where the noisy bit is hardest to characterize, and vanishes at $\varepsilon \in \{0, 1\}$, where a single trial settles the matter. Chapter 7 derives both facts.

A confidence interval

For large n_0, the central limit theorem applies to the standardized estimator. The result is that $\hat{\varepsilon}$ is approximately Normal with mean ε and variance $\varepsilon(1-\varepsilon)/n_0$. Replacing the unknown ε in the variance with its empirical estimate gives the standard Wald confidence interval at level $1-\alpha$,

$$\hat{\varepsilon} \;\pm\; z_{\alpha/2}\sqrt{\frac{\hat{\varepsilon}(1-\hat{\varepsilon})}{n_0}},$$

where $z_{\alpha/2}$ is the upper-$\alpha/2$ quantile of the standard Normal. The analogous interval for ω replaces $\hat{\varepsilon}$ with $\hat{\omega}$ and n_0 with n_1.

The Wald form is convenient and asymptotically correct, but it is also known to under-cover at small n_0 when ε sits near 0 or 1. The interval can even extend below 0 or above 1, which is a sign that the Normal approximation has broken down. Alternative intervals (Wilson, Clopper-Pearson, Jeffreys) handle the boundary case better. Chapter 7 develops the full theory; for now the Wald interval is the existence statement that an interval is available.

The chapter 1 notebook

The accompanying notebook `python/notebooks/ch01-noisy-bit.ipynb` makes the abstract statements concrete. It simulates a binary symmet-

ric channel with a known crossover probability and lets the reader vary the sample size across several orders of magnitude.

The notebook does four things. First, it runs many independent replications at a fixed true ε for each sample size and plots $\hat{\varepsilon}$ as a function of n_0 for n_0 sweeping from tens of samples to tens of thousands. The plot shows the empirical estimator settling toward the true rate, with a visible $1/\sqrt{n_0}$ band of fluctuation traced by the spread of the replications. The unbiasedness statement of the previous subsection is visible in the centering of the band; the variance formula is visible in its width.

Second, at a single fixed n_0 it runs many independent replications, constructs the Wald interval at the 95% level for each, and counts the fraction of intervals that actually contain the true ε. The CLT promises this fraction will approach 95%; the reader sees it land close to nominal at a moderate sample size.

Third, it sweeps n_0 across several orders of magnitude and plots the empirical coverage at each size. For small n_0 the coverage falls below 95%, exactly the under-coverage promised above, and the gap closes as n_0 grows. The reader watches the asymptotic guarantee fail in the regime where it is supposed to fail.

Fourth, it sweeps the true ε from 0.5 down to 0.01 at a fixed n_0 and shows the coverage gap widening as the rate approaches the boundary. The pattern is a small but real warning: an estimator that is asymptotically correct can still mislead at the sample sizes practitioners actually use. The negative-space chapters of Part III return to that point with examples in which the Wald approximation fails by enough to matter.

What comes next

We can now define a noisy bit (Section 1.2), walk its joint structure (Section 1.3), and estimate its rates from finite data with calibrated uncertainty. The chapter has done its concrete work. Section 1.5 zooms back out and explains why this single, narrow object, the noisy bit on two values with two free rates, deserves to be called the framework's atom and not merely its first example.

1.5 Why "Atom"?

The noisy bit has been built up carefully over four sections. We have seen its three faces (Section 1.1), named its two rates (Section 1.2), walked its joint structure (Section 1.3), and shown how to estimate the rates from

finite samples (Section 1.4). The opening overview of the chapter promised something more: the noisy bit would be the framework's *atom*, not merely its first example. It is worth pausing on what that word is doing here, because the rest of the book takes the claim as a working hypothesis.

A noisy bit is atomic in a precise sense. It has no internal Bernoulli structure to peel back. The input is a single Boolean value, the output is a single Boolean value, and the entire behavior is captured by two numbers, the false-positive rate ε and the false-negative rate ω. You cannot decompose a noisy bit into smaller noisy parts, because a smaller part would have a sub-Boolean input or output, and there is nothing below a Boolean. The atom is the place where the internal accounting stops and the external accounting begins. Every larger object the book builds, from chained channels to noisy AND gates to randomized algorithms to approximate sets to general algebraic types, will turn out to be assembled, by some specific composition rule, from noisy bits. The atom is the smallest object that the framework's machinery applies to without itself invoking the machinery.

A second sense of atomicity is worth flagging now. Section 2.4 will define the *model order* of a noisy system: the number of equivalence classes the input domain splits into. A noisy bit has model order two ($B_1 = \{\top\}$, $B_2 = \{\bot\}$), the smallest non-trivial value; everything richer is built by composition from objects whose model order is exactly two. The noisy bit is atomic not only because Booleans are indivisible but because its channel matrix has the smallest non-trivial structure any noisy system can have.

That commitment is empty until we discharge it. The next three chapters do the discharging in three different directions.

Chapters 2 through 4: the algebra of the atom

Chapter 2 (Chapter 2) gives the noisy bit a formal shape. The two-rate description of Section 1.2 becomes a 2×2 row-stochastic matrix Q. The binary symmetric channel, where $\varepsilon = \omega$, is one specialization; the asymmetric binary channel, where the two rates may differ, is the general case; the Z-channel, where one rate is forced to zero, is a useful boundary. The chapter introduces *model order* as a way to talk about how many distinct rates a channel actually has, and shows that the noisy bit sits at model order two, the minimum non-trivial value. Chapter 2 also takes the first step in composition: channels can be chained, and the matrix product of two row-stochastic matrices is another row-stochastic matrix, so a chain of channels is itself a channel. The atom is closed under serial composition.

Chapter 3 (Chapter 3) raises the question that classical logic forgot to

ask. AND, OR, and NOT are familiar operations on Boolean values. What do they do to noisy bits? If the two operands of an AND each carry their own ε and ω, what are the effective rates of the result? The answer is the composition theorem (Section 3.3): the output is itself a noisy bit, with rates computable in closed form from the rates of the operands. The Boolean operations stay closed when their operands stop being exact. The chapter develops the closed-form rules and applies them to several worked examples. The atom is closed under the operations of Boolean algebra.

Chapter 4 (Chapter 4) axiomatizes. Chapters 1 through 3 silently assume that error events at distinct inputs are independent, and that the rates themselves do not drift across the universe of inputs. Chapter 4 names these as Axiom 1 (Section 4.2, element-wise independence) and Axiom 2 (Section 4.3, conditional independence of per-block rates). The Kronecker factorization theorem (Section 4.4) shows what those two axioms purchase: the joint distribution over a block of inputs factors as a tensor product of the per-element channel matrices, which is what makes the per-element description sufficient. Chapter 4 then writes the formal definition Bernoulli[bool] $= (X, Y, Q)$ (Section 4.5), where X is the input domain, Y is the output report, and Q is the channel matrix that chapter 2 will introduce. The triple ties the chapter 1, chapter 2, and chapter 4 threads together in a single object, and chapter 4 shows that every concrete object chapters 1 through 3 worked with is an instance of it. The atom is characterized axiomatically, not just by example.

Miller-Rabin: a familiar algorithm in unfamiliar dress

One example deserves its own preview because it is the first place the framework will pay an unfamiliar dividend. The Miller-Rabin primality test is among the most widely deployed randomized algorithms in computer science; every cryptographic library uses it. The standard treatment presents Miller-Rabin as a clever piece of number theory attached to a clever piece of probability. Section 3.4 takes a different view. One round of Miller-Rabin is exactly a noisy bit, with $\omega = 0$ (a prime never fails a witness test) and $\varepsilon \leq 1/4$ (the strong-liar count for any composite is at most a quarter of the witness pool). The k-round test is the AND of k such noisy bits, and the famous bound $\varepsilon_k \leq (1/4)^k$ is not a separate piece of analysis but a direct application of the composition theorem of Section 3.3. The number theory contributes the per-round $1/4$ bound; everything else is the algebra of the atom. We will see the calculation in full in chapter 3.

Part II: sets out of atoms

Chapter 1 has shown what a single noisy bit looks like. A natural next question is what happens when many of them are arranged into a structure. Part II asks the version of this question that membership-test data structures invite. A set S over a universe U induces, for each element $u \in U$, a Boolean query, namely "is u in S?". An approximate set is one in which each such query is a noisy bit. Bloom filters, the canonical example, are a recipe for producing a noisy-bit instance per universe element with one-sided error ($\omega = 0$) and a tunable ε. The atom becomes a building block, and the project of Part II is the algebra of the resulting structures: how set operations propagate error rates, how classification measures quantify quality, and how much space an approximate set must consume to hit a given accuracy target. Each of these threads is grounded in the atom but is not reducible to it; the atom is the unit, not the whole story.

Part III: lifting the atom past Boolean

The Boolean domain is convenient but not fundamental. Once the machinery of Part II is in place, the natural question is whether the same construction works for richer codomains. Part III replaces $\{\top, \bot\}$ with an arbitrary algebraic type T. Approximate maps, approximate relations, and approximate values of structured types all become available. The two-element model order of chapter 2 ceases to be a ceiling. The atom of chapter 1 is the simplest non-trivial special case of a much larger family of atoms, one for each algebraic type, and the closures and composition rules of Part I generalize accordingly. Where Part II takes the atom and builds a collection out of it, Part III takes the atom and changes the alphabet it works in.

Closing

This has been a forward glance. The chapter ahead, namely Chapter 2, picks up where Section 1.4 left off and gives the noisy bit its first formal vocabulary. Before that, the next and final section of this chapter, the bibliographic notes, names the antecedents of what we have just done: Shannon's channel framing, the contingency-table tradition from medical statistics, the precision-and-recall vocabulary from information retrieval, and the original Miller and Rabin papers on primality testing. The historical record will tell us that almost every component of chapter 1 was already in use somewhere; what is new is the decision to treat them all as one object and to call that object an atom. Chapter 1 has done what chapter 1 should do, namely

shown the framework's atom, named it, and pointed at the algebra the rest of Part I unfolds; everything that follows is composition, and the atom is what gets composed.

Bibliographic Notes

Shannon and the channel framing. The noisy-channel reading of Face 2 in Section 1.1—and the binary symmetric channel that opens Chapter 2 (Section 2.1)—descend directly from Shannon [Sha48]. Shannon's 1948 paper introduced the discrete memoryless channel as the mathematical object for noisy communication: a conditional distribution $P(\text{output} \mid \text{input})$ together with a capacity defined as the supremum of the mutual information between input and output. The binary symmetric channel is among the simplest of Shannon's channels, and it remains the standard pedagogical entry point to the channel framework, both in his original paper and in essentially every information-theory textbook that has followed.

The contingency-table tradition. The 2×2 contingency table that anchors Section 1.2 predates the modern probability framing of this book by decades. It grew up in epidemiology, biostatistics, and medical diagnostic testing, where the natural objects of study are tests with a true-positive (sensitivity) and a true-negative (specificity) rate, and where the population prevalence of the underlying condition is part of every working calculation. The vocabulary of sensitivity, specificity, and the cells of the 2×2 table travelled from clinical screening into machine learning largely unchanged; what differs across fields is which marginal the practitioner conditions on, and which derived rate gets the spotlight. The cross-walk at the end of Section 1.2—from (ε, ω) to precision and recall—is the modern statement of a much older bilingual conversation between epidemiology and signal detection.

Precision, recall, and information retrieval. The vocabulary of *precision* and *recall* arrived in this book by a different route, from information retrieval. In the IR setting one issues a query against a corpus and asks: of the documents returned, what fraction are relevant (precision); of the relevant documents in the corpus, what fraction were returned (recall). The textbook reference is Manning, Raghavan, and Schütze [MRS08], whose Chapter 8 develops the precision-recall vocabulary, the precision-recall curve, and the F-measure as the standard combined score. Chapter 1's emphasis on

the difference between (ε, ω) and (precision, recall)—that the first pair is intrinsic to a noisy bit and the second depends also on the workload—is the contingency-table analogue of the IR distinction between a system's intrinsic discriminating ability and its observed behaviour on a given query mix.

Miller, Rabin, and the primality tease. The Miller-Rabin algorithm previewed in Section 1.5 and developed in Section 3.4 has two distinct intellectual parents. Miller [Mil76] introduced the witness-based structure under the Extended Riemann Hypothesis; Rabin [Rab80] removed the hypothesis by randomizing the choice of witness and proved the unconditional $1/4$ bound on the per-round false-positive rate. Chapter 3's reading of Miller-Rabin—one round of the test as a Bernoulli[bool] with $\omega = 0$ and $\varepsilon \leq 1/4$, the k-round protocol as a composition—restates that result in the language of this text. The full historical record, the proof of the $1/4$ bound, and the companion treatment of randomized algorithms more broadly are taken up in Section 3.5.

The Bernoulli papers. The atom of this chapter, the noisy bit, is the order-2 Boolean instance of the formal Bernoulli framework developed in our companion papers. The two-axiom model and the rate-propagation theorems originate in Towell [Tow26a]; the extension from approximate sets to approximate values over arbitrary algebraic types—and, in particular, the treatment of Bernoulli[bool] as a value-with-error type—is the subject of Towell [Tow26f]. The present book [Tow26b] reorganizes that material as a textbook, starting from the noisy bit, building outward to sets, maps, relations, and the algebraic types that the companion paper studies in their full generality. Chapter 1's contribution is the decision to treat the biased coin, the binary symmetric channel, and the classifier verdict as three faces of one object and to call that object an atom; almost every other ingredient of the chapter has a longer history in one of the traditions noted above.

Exercises

1. ⋆ **Confusion matrix to rate conversion.** A binary classifier is evaluated on a labelled test set and produces the following counts: TP = 90, FP = 10, FN = 10, TN = 90.

   (a) Compute the empirical false-positive rate $\hat{\varepsilon}$ and false-negative rate $\hat{\omega}$ using the row-conditional proportions from Section 1.2.

Be explicit about which counts go in the numerator and which in the denominator.

(b) Compute the precision $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$ and the recall $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$.

(c) In one sentence, explain why $\hat{\varepsilon}$ and precision are *different* numbers even though both involve the false-positive count. (*Hint:* which marginal does each one condition on?)

2. ⋆ **Three real-world noisy bits and one non-example.** The three faces of Section 1.1 (BSC, biased coin, classifier verdict) are stand-ins for many real systems. Name *three* systems of your own that you think of as noisy bits in the sense of Section 1.2 (a Boolean-output system with stable per-input rates).

(a) For each of the three, identify what plays the role of the Boolean input and Boolean output, then write a rough estimate of ε and ω.

(b) Construct one explicit *non-example* whose per-input error rates are not stable: they change over time, or depend on which queries came before. The error behaviour is real, but the rates are not the stable numbers the noisy-bit model of Section 1.2 assumes.

(c) In one sentence, identify the stability property of the noisy-bit model that your non-example violates. We return to non-examples like this in chapter 16, which makes the failure modes of the model its explicit subject.

3. ⋆ **Joint table to rates.** A noisy bit is described by the joint table below, where the truth has prior $\Pr\{X=\top\}=0.3$ and $\Pr\{X=\bot\}=0.7$:

	$Y=\top$	$Y=\bot$
$X=\top$	0.27	0.03
$X=\bot$	0.07	0.63

The four cell entries are joint probabilities $\Pr\{X,Y\}$, and they sum to 1.

(a) Verify that the row sums match the priors.

(b) Compute the row-conditional false-positive rate $\varepsilon = \Pr\{Y = \top | X = \bot\}$ and the false-negative rate $\omega = \Pr\{Y = \bot | X = \top\}$ by dividing the appropriate cell by its row sum (Section 1.2).

(c) Confirm that ε and ω are properties of the noisy bit itself, not of the prior $\Pr\{X = \top\}$: recompute them for a different prior (say $\Pr\{X = \top\} = 0.5$) with the same conditional behaviour, and check that they are unchanged. In one sentence, explain why this independence from the prior is what lets the noisy-bit model get by with two rates.

4. ⋆⋆ **Empirical estimation under varying n (notebook-tied).** Open the chapter 1 companion notebook `python/notebooks/ch01-noisy-bit.ipynb`. Use the binary symmetric channel simulator with true crossover $\varepsilon = 0.10$ and run the estimator at $n \in \{10, 100, 1000, 10000\}$.

   (a) Tabulate $\hat{\varepsilon}$ for each value of n from a single run, and the absolute deviation $|\hat{\varepsilon} - \varepsilon|$ from the true rate.

   (b) Plot $\hat{\varepsilon}$ as a function of n on a log-n axis, with the true rate drawn as a horizontal reference line. Describe in one sentence how the spread of the estimator around the true rate shrinks with n.

   (c) Compare the rate at which the empirical deviation shrinks to the predicted standard error $\sqrt{\varepsilon(1-\varepsilon)/n}$ from Section 1.4. Is the observed convergence consistent with a $1/\sqrt{n}$ shrinking band?

5. ⋆⋆ **The trivial biased coin.** A trivial biased coin is a noisy bit that ignores the input entirely: it returns $\top$ with probability p and $\bot$ with probability $1 - p$, regardless of whether the truth is $\top$ or $\bot$.

   (a) Using the definitions in Section 1.2, write ε and ω as functions of p.

   (b) At what value of p is the trivial biased coin *useless* for the classification task, in the sense that ε and ω are both equal? Compute that pair of rates.

   (c) At what values of p does the trivial biased coin achieve its *best* behaviour, in the sense that one of the two rates is exactly zero? Which rate, at which value of p?

   (d) Comment briefly on the asymmetry: a constant function is a perfect classifier for one truth class and a maximally bad one for the other. The biased-coin parameterization makes this trade-off visible.

6. ⋆⋆ **NOT swaps the two rates.** Let b be a noisy bit with rates (ε, ω). Define $b' = \neg b$ to be the noisy bit obtained by swapping the

output: whenever b reports $\top$, b' reports $\bot$, and vice versa. (No additional randomness is introduced; b' is a deterministic relabelling of the output of b.)

(a) Write out the four conditional probabilities $\Pr\{Y = \top | X = \top\}$, $\Pr\{Y = \bot | X = \top\}$, $\Pr\{Y = \top | X = \bot\}$, $\Pr\{Y = \bot | X = \bot\}$ for b, following the row-conditional convention of Section 1.2.

(b) Apply the output swap to b and write out the same four conditional probabilities for b'.

(c) Read off the rates of b'. Show that $\varepsilon' = \omega$ and $\omega' = \varepsilon$.

(d) Conclude that NOT acts on the parameter pair as the swap $(\varepsilon, \omega) \mapsto (\omega, \varepsilon)$. In one sentence, interpret this geometrically as a reflection in the parameter plane $[0, 1]^2$.

7. $\star\star\star$ **Threshold sweep and the error-rate trade-off curve.** A score-based classifier produces a score $s(x) \in [0, 1]$ on each input x and reports $\top$ if and only if $s(x) > \theta$ for a threshold θ. For each value of θ, the system is a noisy bit with rates $\varepsilon(\theta)$ and $\omega(\theta)$ that depend on θ.

(a) As θ increases from 0 to 1, describe what happens to $\varepsilon(\theta)$ and $\omega(\theta)$: which one increases, which one decreases, and why. (You do not need a formula; reason from the threshold rule of Section 1.2.)

(b) At $\theta = 0$ the classifier reports $\top$ for almost every input; at $\theta = 1$ it reports $\bot$ for almost every input. Give the (ε, ω) pair at each of these two extremes.

(c) The curve traced by $(\varepsilon(\theta), \omega(\theta))$ as θ sweeps from 0 to 1 is the trade-off curve between the two error rates. The standard *ROC curve* plots the true-positive rate against the false-positive rate, which is the same information re-axed. Sketch (in words) the shape of the (ε, ω) trade-off curve, and explain why a classifier cannot in general drive both rates to zero at once.

8. $\star\star$ **Sample-size formula via the CLT bound.** Section 1.4 gives the variance of the empirical false-positive estimator as $\mathrm{Var}\,(()\,\hat{\varepsilon}) = \varepsilon(1 - \varepsilon)/n_0$. Suppose we want the standard error to satisfy $\sqrt{\mathrm{Var}\,(()\,\hat{\varepsilon})} \leq \sigma$ for some target $\sigma > 0$.

(a) Solve the inequality $\sqrt{\varepsilon(1 - \varepsilon)/n_0} \leq \sigma$ for n_0. Write the smallest integer n_0 that satisfies the bound as a function of ε and σ.

(b) Evaluate n_0 for the practical target $\sigma = 0.01$ at $\varepsilon \in \{0.01, 0.05, 0.10, 0.50\}$. Tabulate the four sample-size requirements.

(c) Identify the value of ε that requires the largest sample size. In one sentence, explain why this maximum sits at $\varepsilon = 1/2$ rather than at the boundary (cf. Section 1.4, which observes that the variance numerator peaks at 1/2).

9. ⋆⋆ **Informal LLN argument for $\hat{\varepsilon} \to \varepsilon$.** Argue informally that under iid sampling, the empirical estimator $\hat{\varepsilon}$ from Section 1.4 converges to the true rate ε in probability as $n_0 \to \infty$.

(a) State precisely the iid assumption you are using: each of the n_0 negative-truth queries produces a Bernoulli(ε) output independent of all the others. Write the empirical count FP as a sum of n_0 such Bernoulli variables.

(b) Recall (or look up) Chebyshev's inequality: for any random variable Z with finite variance and any $\delta > 0$, $\Pr\{|Z - \mathbb{E}[Z]| \geq \delta\} \leq \mathrm{Var}\,(()\,Z)/\delta^2$. Apply this to $Z = \hat{\varepsilon}$, using the unbiasedness statement $\mathbb{E}[\hat{\varepsilon}] = \varepsilon$ and the variance formula $\mathrm{Var}\,(()\,\hat{\varepsilon}) = \varepsilon(1-\varepsilon)/n_0$ from Section 1.4.

(c) Show that the right-hand side of the Chebyshev bound goes to zero as $n_0 \to \infty$ for any fixed $\delta > 0$. Conclude that $\hat{\varepsilon} \to \varepsilon$ in probability.

(d) In one sentence, identify which assumption from part (a) is most likely to fail in a real data system, and what kind of correlation across queries would prevent the Chebyshev argument from going through.

10. ⋆⋆⋆ **Stable rates, correlated errors.** The noisy-bit model rests on two assumptions: the per-input rates are stable across queries, and the noise events on different queries are independent. Construct an explicit Boolean-input, Boolean-output system with *stable* per-input rates whose errors are nonetheless *correlated* across queries, so that a naive iid estimator misjudges its behaviour.

(a) Construct the system. Specify the input domain (Boolean), the output domain (Boolean), and the mechanism that determines the output given the input. The mechanism must produce a well-defined Boolean output on every query and must have well-defined, stable per-input rates ε and ω, but the error events on successive queries must be statistically dependent in a stated way.

(b) Identify precisely how the independence assumption fails: describe the dependence between the error event on one query and the error event on the next, and confirm in one sentence that the per-input rates themselves are still stable (so it is independence, not stability, that breaks).

(c) Argue that a naive observer who computes $\hat{\varepsilon}$ and $\hat{\omega}$ from a sample of queries, and then treats those estimates as if the queries were iid, would misjudge the system. Be specific: the point estimates may be fine, but what quantity (an error bar, a coverage claim, a sample-size requirement) would they get wrong, and in which direction?

(d) Chapter 16 is the negative-space chapter of the book: its explicit subject is what fails to fit the model. Your construction is a candidate worked example for that chapter; in one sentence, state the conceptual lesson your example would carry there.

11. ⋆⋆⋆ **Stress test of the notebook coverage estimator.** The chapter 1 notebook `python/notebooks/ch01-noisy-bit.ipynb` runs a Wald-interval coverage check that empirically counts the fraction of 95% intervals containing the true ε. Section 1.4 flags the under-coverage caveat: the Wald form is asymptotically correct but is known to under-cover at small n_0 when ε sits near 0 or 1.

(a) Identify parameter regimes (combinations of n_0 and ε) under which you predict the empirical coverage will fall visibly below 0.95. Justify your prediction by reference to the under-coverage caveat in Section 1.4.

(b) Run the notebook's coverage estimator in two regimes of your choosing: one where you predict good coverage (close to 0.95) and one where you predict bad coverage. Report the two empirical coverage numbers.

(c) Explain in one paragraph *why* the Wald form under-covers in the bad regime. The argument should reference the Normal approximation underlying the Wald form and identify what fails when ε sits near a boundary or n_0 is small. (No rigorous derivation is needed; the explanation should be sound.)

(d) State which of the alternative intervals named in Section 1.4 (Wilson, Clopper-Pearson, Jeffreys) you would expect to perform better in your bad-coverage regime, and briefly explain the intuition. Full derivations are deferred to chapter 7.

Chapter 2

Binary Channels and Model Order

After this chapter you will be able to:

- Define a binary channel formally and write its 2×2 transition matrix.
- Distinguish symmetric (BSC) from asymmetric (Z-channel and general) binary channels and connect each to model order.
- Compose two binary channels via matrix product and read off the resulting channel.
- State the Boolean ceiling at order 2: any predicate over the input domain $\{\top, \bot\}$ has model order at most 2.

2.1 The Binary Symmetric Channel

One bit, one channel

Section 1.1 introduced three framings of the noisy bit: a biased coin, a noisy communication channel, and a classifier's verdict. All three share a common structure that deserves a name and a precise description. The *binary symmetric channel* is that common structure, stripped to its essentials.

The input is a single Boolean value: $\top$ ("in the set", "heads", "positive") or $\bot$ ("not in the set", "tails", "negative"). The output is also Boolean. The channel transmits the input faithfully most of the time, but occasionally flips it. The defining property of the *symmetric* channel is that the flip

probability is the same regardless of which input is being transmitted: a positive is misreported as negative with the same probability that a negative is misreported as positive.

This symmetry is a genuine constraint. Most real noisy Boolean systems do not satisfy it: a spam filter is far more likely to mis-classify legitimate mail as spam (false positive) than to miss actual spam (false negative). The BSC is the ideal symmetric case, and understanding it precisely is the right starting point before the asymmetric generalization of Section 2.2.

Definition 2.1.1 (Binary Symmetric Channel)**.** *A* binary symmetric channel *with crossover probability* $\varepsilon \in (0,1)$ *is a random map* $\phi : \{\top, \bot\} \to \{\top, \bot\}$ *satisfying*

$$\Pr\{\phi(x) \neq x\} \;=\; \varepsilon \quad \textit{for every } x \in \{\top, \bot\}.$$

The parameter ε *is the* crossover probability*: the probability that the output differs from the input.*

Two things are worth noting immediately. First, the crossover probability is the same for both inputs: $\Pr\{\phi(\top) \neq \top\} = \Pr\{\phi(\bot) \neq \bot\} = \varepsilon$. This is the symmetry. Second, ε plays the role of both the false-positive rate and the false-negative rate simultaneously: in the Bernoulli framework's notation from Section 1.1, the BSC has $\varepsilon = \omega$ (using the convention that ε is the rate of misreporting a negative as positive and ω is the rate of misreporting a positive as negative). The BSC is the special case where these two rates collapse to a single parameter.

The channel matrix

A single parameter and a prose definition are useful, but a matrix representation is more useful still. It displays all four input-output probabilities at once, supports direct computation, and generalizes cleanly to the asymmetric case in Section 2.2 and to multi-query systems in Chapter 4.

We organize the channel as a 2×2 matrix Q, called the *channel transition matrix* or simply the *channel matrix*. The convention throughout this book is:

Notation. *The channel matrix* Q *for a binary channel is a* 2×2 *stochastic matrix.* ***Rows*** *are indexed by the input block: row 1 corresponds to block* $B_1 = \{\top\}$ *(positives), row 2 to block* $B_2 = \{\bot\}$ *(negatives).* ***Columns*** *are indexed by the output label: column 1 corresponds to output* $\top$ *(reported positive), column 2 to output* $\bot$ *(reported negative). The* (i, j)*-entry of* Q *is*

the probability of observing output j given that the input belongs to block B_i. Each row sums to 1: Q is row-stochastic.

Under this convention, the four entries of Q have specific names in the confusion-table vocabulary. For the order-2 Boolean model with $B_1 = \{\top\}$ and $B_2 = \{\bot\}$:

	output $\top$	output $\bot$
input $\top$ (B_1)	τ (true-positive rate)	ω (false-negative rate)
input $\bot$ (B_2)	ε (false-positive rate)	η (true-negative rate)

So the general order-2 channel matrix is

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

where we use $\tau = 1 - \omega$ and $\eta = 1 - \varepsilon$ from the row-stochastic constraint.

For the BSC, the symmetry constraint $\varepsilon = \omega$ collapses this to a single parameter:

$$Q_{\mathrm{BSC}} \;=\; \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

Both rows sum to 1. Both diagonal entries equal $1-\varepsilon$ (correct output). Both off-diagonal entries equal ε (flip). The two rows are simply permutations of each other, which is the matrix expression of symmetry. The BSC has a single free parameter; the general order-2 channel has two. We return to this reduction in Section 2.4.

Reading the matrix

Before computing with Q, it is worth reading it carefully for a small ε. Take $\varepsilon = 0.05$ as a concrete case.

$$Q \;=\; \begin{pmatrix} 0.95 & 0.05 \\ 0.05 & 0.95 \end{pmatrix}.$$

Reading row 1: a positive input ($\top$) is correctly reported as positive ($\top$) with probability 0.95 and incorrectly reported as negative ($\bot$) with probability 0.05. Reading row 2: a negative input ($\bot$) is incorrectly reported as positive ($\top$) with probability 0.05 (a false positive) and correctly reported as negative ($\bot$) with probability 0.95. The channel is "mostly right" with a small error floor.

The channel matrix carries more information than the crossover probability alone: it tells you the probability of every input-output pair simultaneously, without requiring you to remember which rate is which or convert between TPR and FNR in your head. Once we introduce multiple queries in Section 2.3 and Section 4.4, the matrix form pays for itself many times over.

Worked example: BSC with $\varepsilon = 0.05$

We verify all four probabilities and three derived quantities for $\varepsilon = 0.05$.

The matrix.

$$Q = \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \varepsilon & 1-\varepsilon \end{pmatrix} = \begin{pmatrix} 0.95 & 0.05 \\ 0.05 & 0.95 \end{pmatrix}.$$

Row sums. Row 1: $0.95 + 0.05 = 1.00$. Row 2: $0.05 + 0.95 = 1.00$. Both rows sum to 1, confirming that Q is row-stochastic.

Single-query error probability. If the input is drawn uniformly from $\{\top, \bot\}$, each with probability $1/2$, the probability of an incorrect output is

$$\Pr\{\text{error}\} = \tfrac{1}{2}\cdot\Pr\{\phi(\top) \neq \top\} + \tfrac{1}{2}\cdot\Pr\{\phi(\bot) \neq \bot\} = \tfrac{1}{2}\cdot 0.05 + \tfrac{1}{2}\cdot 0.05 = 0.05.$$

Under a uniform prior on the input, the error probability equals the crossover probability ε. This is not a coincidence: symmetry means the marginal error rate does not depend on the input distribution, as long as $\varepsilon = \omega$.

Two specific joint probabilities. Suppose the input is positive ($\top$). The probability of a correct positive output is $Q_{11} = 0.95$. The probability of a false-negative output is $Q_{12} = 0.05$.

Now suppose the input is negative ($\bot$). The probability of a false positive is $Q_{21} = 0.05$. The probability of a correct negative is $Q_{22} = 0.95$.

Accuracy. For a uniform prior, accuracy $= \frac{1}{2}\cdot 0.95 + \frac{1}{2}\cdot 0.95 = 0.95 = 1 - \varepsilon$. As expected, the BSC with crossover 0.05 has 95% accuracy under a uniform input distribution.

The BSC as the symmetric special case of order 2

The source papers in the Bernoulli framework use two conventions that are worth aligning before proceeding. Some sections call the BSC a "first-order" model (one free parameter), while Chapter 4 calls it the *symmetric special*

case of the order-2 model. These are both correct, depending on what is being counted.

Model order, formalized in Section 2.4, counts the number of distinct partition blocks in the input domain, not the number of free parameters. The Boolean input domain $\{\top, \bot\}$ has two elements, and in the order-2 model these form two blocks: $B_1 = \{\top\}$ (positives) and $B_2 = \{\bot\}$ (negatives). The model has two blocks and therefore model order 2, regardless of whether the BSC's symmetry constraint $\varepsilon = \omega$ further reduces the free parameter count from 2 to 1.

Put differently: model order is about the partition of the input domain (how many distinct error regimes exist), while parameter count is about how many numbers you need to write down (which may be less than the model order, if the channel is symmetric or otherwise constrained). The BSC is order 2 with one free parameter; the general asymmetric channel of Section 2.2 is order 2 with two free parameters.

Section 4.3 makes the connection to partition blocks explicit in the context of Axiom 2. The convention $B_1 = \{\top\}$, $B_2 = \{\bot\}$ used there matches the row labeling of Q throughout this chapter and in Chapter 4.

Connection to chapter 1: the three faces revisited

Section 1.1 presented three framings of the same abstract object. We can now describe each one precisely using the BSC channel matrix.

The biased coin. A coin with heads probability $1-\varepsilon$ and tails probability ε, where "heads" is the correct output, is a BSC. The two sides of the coin are the two elements of $\{\top, \bot\}$; the flip probability is ε for both sides, because the coin is symmetric. The channel matrix is exactly Q_{BSC} above. We generalize to an asymmetric coin, with different head-bias and tail-bias, in Section 2.2.

The noisy communication channel. This is Shannon's BSC in its original form [Sha48]: a bit sent over a channel where each transmitted bit is flipped independently with probability ε. The input alphabet is $\{0, 1\}$ and the channel matrix is Q_{BSC} with the mapping $1 \leftrightarrow \top$ and $0 \leftrightarrow \bot$.

The classifier verdict. A binary classifier with equal false-positive and false-negative rates is a BSC. In practice, classifiers rarely have $\varepsilon = \omega$; the symmetric case is a simplifying assumption. When the two rates are unequal, the classifier is better modeled as an asymmetric binary channel, the subject of Section 2.2.

The unifying point: all three framings share the same channel matrix Q_{BSC} with the same single crossover parameter ε. The names differ; the probability model is identical.

What makes symmetry a strong constraint

The BSC's symmetry constraint, $\varepsilon = \omega$, is restrictive but often useful as a first approximation. Three properties follow from it.

First, the channel matrix Q_{BSC} is a *circulant* matrix: its rows are cyclic permutations of each other. This makes the algebra of composing BSCs, discussed in Section 2.3, particularly clean.

Second, the *capacity* of the BSC, defined as the maximum mutual information between input and output, is $1 - H_2(\varepsilon)$ bits per use, where $H_2(\varepsilon) = -\varepsilon \log_2 \varepsilon - (1-\varepsilon)\log_2(1-\varepsilon)$ is the binary entropy function. Capacity is maximized at $\varepsilon = 0$ (a perfect channel, transmitting one bit per use without error) and at $\varepsilon = 1$ (a perfectly anti-correlated channel, which also transmits one bit per use after relabeling outputs); it is minimized at $\varepsilon = 1/2$, where the output is statistically independent of the input and no information is transmitted. Exercise 4 works through the derivation.

Third, the BSC is its own *inverse channel* in the following sense: composing a BSC with crossover ε with another BSC with the same crossover ε yields a BSC with crossover $2\varepsilon(1-\varepsilon)$, which is strictly greater than ε for $0 < \varepsilon < 1/2$. Two imperfect copies of the same symmetric channel make things worse, not better. The formula for composed BSC crossovers is derived in full in Section 2.3.

Notation in this chapter

Throughout chapters 2 and beyond, the following convention applies:

- Q always denotes a channel transition matrix, with rows indexed by input block and columns indexed by output label.
- ε (macro: `\fprate`, rendered as ε) is the false-positive rate: the probability that a negative input is reported as positive.
- ω (macro: `\fnrate`, rendered as ω) is the false-negative rate: the probability that a positive input is reported as negative.
- $\tau = 1 - \omega$ is the true-positive rate, and $\eta = 1 - \varepsilon$ is the true-negative rate.

- For the BSC, $\varepsilon = \omega$, and we use ε for the single crossover parameter throughout.

The source papers in this research program sometimes use (FPR, TPR) rather than (FPR, FNR) as the parameter pair. The two choices are equivalent: $\text{TPR} = 1 - \text{FNR}$ is a deterministic function, so any statement about one pair translates directly to the other.[1]

Looking ahead

The BSC is the symmetric special case of a more general object. In Section 2.2 we lift the constraint $\varepsilon = \omega$ and allow the two error rates to be independent: the resulting *asymmetric binary channel* covers Bloom filters (where $\omega = 0$ by construction) and medical tests (where FPR and FNR can be tuned independently by adjusting a threshold). In Section 2.3 we ask what happens when two channels are applied in sequence, and derive the composed channel's matrix from the two input matrices via matrix multiplication. In Section 2.4 we formalize what we have been calling "order 2" and prove the Boolean ceiling theorem: because the Boolean domain has exactly two elements, every binary channel has model order at most 2.

2.2 The Asymmetric Binary Channel

Lifting the symmetry constraint

The Binary Symmetric Channel of Section 2.1 is a useful starting point, but its defining property—equal error rates for both inputs—is rarely satisfied in practice. A spam filter flags legitimate mail as spam far more often than it lets genuine spam through. A Bloom filter returns false positives but never false negatives. A medical screening test is deliberately tuned to minimize one kind of error at the cost of the other.

These systems are not defective BSCs; they are different channels altogether. When we allow the false-positive rate ε and the false-negative rate ω to be independent parameters, we obtain the *asymmetric binary channel*: the same two-element input and output domains, the same row-stochastic channel matrix Q, but no constraint tying ε to ω.

[1] Specifically, the companion paper [Tow26a] uses (FPR, TPR) in places. In converting source material to this textbook, we have replaced that convention with (FPR, FNR) throughout, using the alex.sty macros ε and ω.

Definition 2.2.1 (General asymmetric binary channel)**.** *An* asymmetric binary channel *with false-positive rate* $\varepsilon \in [0,1)$ *and false-negative rate* $\omega \in [0,1)$ *is a random map* $\phi : \{\top, \bot\} \to \{\top, \bot\}$ *satisfying*

$$\Pr\{\phi(\top) \neq \top\} = \omega,$$
$$\Pr\{\phi(\bot) \neq \bot\} = \varepsilon.$$

The channel transition matrix is

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

where row 1 corresponds to input block $B_1 = \{\top\}$ *(positives), row 2 to block* $B_2 = \{\bot\}$ *(negatives), and columns to output labels* $\top$ *(left) and* $\bot$ *(right), following the convention established in Section 2.1.*

Two special cases are immediately visible in the matrix. Setting $\omega = \varepsilon$ recovers the BSC of Definition 2.1.1: the two rows become identical up to a reflection, and the single crossover parameter ε is enough to describe the channel. Setting $\omega = 0$ makes row 1 become $(1, 0)$: a positive input is always reported correctly. This is the *Z-channel*, the subject of the next subsection. The name of each special case and its connection to a familiar system is spelled out in full in Section 2.4.

The Z-channel: one-sided error

The most widely encountered asymmetric channel in computer science is the *Z-channel*. In the variant used here, the transmission $\top \to \top$ is perfect, and the only error is $\bot \to \top$: a negative input incorrectly reported as positive.[2]

Definition 2.2.2 (Z-channel)**.** *A* Z-channel *with false-positive rate* $\varepsilon \in (0,1)$ *is an asymmetric binary channel with* $\omega = 0$*. Its channel matrix is*

$$Q_Z \;=\; \begin{pmatrix} 1 & 0 \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

The Z-channel has a sharp asymmetry: false negatives are impossible ($\omega = 0$), while false positives occur with probability ε. This makes an output of $\bot$ fully reliable—it cannot be a mistake—while an output of $\top$ is uncertain.

[2]Information-theory texts often define the Z-channel with the opposite asymmetry—false negatives only, no false positives ($\omega > 0$, $\varepsilon = 0$), so that $\bot \to \bot$ is perfect and the only error is $\top \to \bot$. Both variants are equivalent up to relabeling of inputs and outputs. The convention here aligns with the Bloom filter and other positive approximate sets, where the false-positive direction is the natural error mode.

Bloom filters as Z-channels. The Bloom filter is the canonical Z-channel instance from data structures. A Bloom filter answers membership queries for a set S: given an element x, it returns either *present* ($\top$) or *absent* ($\bot$). The filter is constructed so that

$$x \in S \implies \text{Bloom returns } \top \quad \text{(no false negatives)},$$

while a non-member $x \notin S$ may still trigger $\top$ with some probability ε (a false positive). In the channel-matrix language, the Bloom filter is exactly a Z-channel: $\omega = 0$ by construction, ε controlled by the filter's bit-width and hash count. When the Bloom filter returns absent ($\bot$), the answer is certain; when it returns present ($\top$), the answer is probabilistic. This semantic asymmetry—reliable "no", uncertain "yes"—is the hallmark of all Z-channel systems.

The biased coin, revisited

Section 1.1 introduced the biased coin as one framing of the noisy bit. In Section 2.1 we identified it as a BSC: a coin with the same flip probability for heads and tails. But a coin can be *doubly biased*: the probability of landing heads on a flip from heads may differ from the probability of landing tails on a flip from tails. Such a coin is not a BSC; it is an asymmetric binary channel.

Concretely: let $\top$ represent heads and $\bot$ represent tails. Suppose a flipped-heads coin shows tails with probability ω, and a flipped-tails coin shows heads with probability ε, where $\omega \neq \varepsilon$. The channel matrix for this coin is exactly Q from Definition 2.2.1. The BSC of Section 2.1 was the special case $\omega = \varepsilon$, where the double bias collapses to a single bias. The asymmetric channel is the general case that was always there, waiting for ω and ε to differ.

Physical examples are easy to construct. A coin that spins on a surface comes to rest heads or tails with probabilities that depend on its initial face. A weighted die, approximated as a two-sided object, has different effective flip rates for each face. A market that can only move a small amount in one direction (a price floor or ceiling) is an asymmetric channel on the direction of movement. In every case, the natural description is an asymmetric binary channel with two independent parameters ε and ω.

Medical tests as asymmetric channels

Medical screening tests illustrate the asymmetric channel in a domain where the two error types have very different consequences. A test for a serious

disease might be deliberately calibrated to err on the side of sensitivity: it flags nearly every true case (ω small) at the cost of a higher false-alarm rate (ε larger). A confirmatory test later in the diagnostic pipeline does the opposite: it trades sensitivity for specificity (ε small, ω possibly larger).

In both cases, the test is fully described by the matrix Q from Definition 2.2.1. The *screening test* has small ω and larger ε; the *confirmatory test* has small ε and larger ω. Neither is a BSC. The channel-matrix representation makes the trade-off visible at a glance: the top row (positives) and bottom row (negatives) can be tuned independently by adjusting the decision threshold.

This is precisely the receiver operating characteristic (ROC) curve familiar from binary classification: as the threshold moves, (ε, ω) traces a curve in the unit square $[0,1]^2$, and each point on the curve is a distinct asymmetric binary channel.

Worked example: $\varepsilon = 0.05$, $\omega = 0.10$

To make the asymmetric channel concrete, we work through the channel matrix and several derived quantities for $\varepsilon = 0.05$ and $\omega = 0.10$.

The matrix.

$$Q = \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix} = \begin{pmatrix} 0.90 & 0.10 \\ 0.05 & 0.95 \end{pmatrix}.$$

Row sums. Row 1: $0.90 + 0.10 = 1.00$. Row 2: $0.05 + 0.95 = 1.00$. Both rows sum to 1, confirming that Q is row-stochastic.

Reading the matrix. A positive input ($\top$) is correctly reported as positive with probability 0.90 (true-positive rate $\tau = 1 - \omega$) and incorrectly reported as negative with probability 0.10 (false-negative rate ω). A negative input ($\bot$) is incorrectly reported as positive with probability 0.05 (false-positive rate ε) and correctly reported as negative with probability 0.95 (true-negative rate $\eta = 1 - \varepsilon$). The two error rates are distinct: $\omega = 0.10 \neq 0.05 = \varepsilon$. This channel is not a BSC.

Error-rate asymmetry. The ratio of the two error rates is $\omega/\varepsilon = 0.10/0.05 = 2$: false negatives are twice as likely as false positives. For the BSC with $\varepsilon = \omega = 0.075$ (the mean of the two rates), the matrix would be

$$Q_{\mathrm{BSC}} = \begin{pmatrix} 0.925 & 0.075 \\ 0.075 & 0.925 \end{pmatrix},$$

which has identical off-diagonal entries. Comparing this with the asymmetric Q above makes the structural difference plain: the off-diagonal entries 0.10 and 0.05 are a factor of 2 apart.

Joint probabilities under a uniform prior. If the input is drawn uniformly from $\{\top, \bot\}$ (prior probability 1/2 for each), the joint probabilities of each input-output pair are:

	output $\top$	output $\bot$	total
input $\top$	$\frac{1}{2} \cdot 0.90 = 0.450$	$\frac{1}{2} \cdot 0.10 = 0.050$	0.500
input $\bot$	$\frac{1}{2} \cdot 0.05 = 0.025$	$\frac{1}{2} \cdot 0.95 = 0.475$	0.500
total	0.475	0.525	1.000

Accuracy. Under the uniform prior, accuracy is the sum of the diagonal joint probabilities: $0.450+0.475 = 0.925$. This is the average of $\tau = 0.90$ and $\eta = 0.95$: accuracy $= \frac{1}{2}(0.90 + 0.95) = 0.925$. Compare this with the BSC above: the BSC would achieve accuracy 0.925 as well, but by having both error rates at 0.075 rather than one at 0.05 and one at 0.10. The asymmetric channel and this particular BSC have the same accuracy under a uniform prior, but their error structures are different—a point that matters when the prior is not uniform or when the two error types carry different costs.

The asymmetric channel as the general order-2 case

The BSC has one free parameter (ε, with the constraint $\varepsilon = \omega$ reducing the two-parameter family to one). The asymmetric binary channel has two free parameters (ε and ω, with no constraint between them). Section 2.4 formalizes what "order 2" means: the Boolean input domain $\{\top, \bot\}$ partitions into two blocks, $B_1 = \{\top\}$ and $B_2 = \{\bot\}$, each with its own per-block error rate. The asymmetric channel assigns ω to B_1 and ε to B_2 independently; the BSC additionally forces $\omega = \varepsilon$.

In this sense, the BSC is not a different type of channel from the asymmetric channel: it is the *symmetric special case* of the same order-2 structure. The constraint $\omega = \varepsilon$ reduces the number of free parameters from 2 to 1 without changing the partition of the input domain.

The distinction between model order (the number of partition blocks) and parameter count (how many numbers are needed to describe the channel) is a recurring theme. Section 2.4 makes it precise and proves the Boolean ceiling theorem: because the Boolean domain has exactly two elements, any binary channel has model order at most 2. There is no room for a more fine-grained partition, no matter how many free parameters the channel has.

Looking ahead

The Z-channel and general asymmetric channel are the natural habitat of most real-world approximate Boolean systems. In Section 2.3 we compose two binary channels—which may each be symmetric, asymmetric, or Z-channel—via matrix multiplication and read off the resulting channel's error rates directly from the product. The asymmetric channel's two-parameter freedom makes the composition richer than the BSC case, and the matrix formalism handles both cases uniformly.

2.3 Composition of Channels

Channels in series

The channels we have studied so far—the BSC of Section 2.1 and the asymmetric binary channel of Section 2.2—model a single noisy step. Many real systems chain two or more such steps together. A signal passes through a first amplifier that introduces some noise, then through a second that adds more. A classification pipeline applies one model to filter candidates and a second to rank them. A Bloom filter checks membership; a downstream system then applies its own fallible test to the reported positives.

In each case we have a *serial pipeline*: an element enters the first channel Q_1, its output becomes the input to the second channel Q_2, and the combined system produces a final output. We want to know: what is the channel matrix of the composed system?

Definition 2.3.1 (Serial composition of binary channels)**.** *Let Q_1 and Q_2 be two binary channel matrices. The* serial composition $Q_2 \circ Q_1$ *is the channel formed by applying Q_1 first and Q_2 second. Its channel matrix is the ordinary matrix product*

$$Q_{\text{total}} \;=\; Q_2\, Q_1.$$

The (i, j)-entry of Q_{total} is the probability of observing final output j given that the original input belongs to block B_i.

The right-to-left convention $Q_2 Q_1$ is standard: in matrix notation, the *rightmost* factor is applied first. The input vector (or row) is transformed by Q_1, and the intermediate output is then transformed by Q_2. The law of total probability is what makes the product work:

$$(Q_{\text{total}})_{ij} \;=\; \sum_{k} (Q_1)_{ik}\,(Q_2)_{kj}.$$

Reading this sum: $(Q_1)_{ik}$ is the probability that the first channel maps input block B_i to intermediate output k; $(Q_2)_{kj}$ is the probability that the second channel maps intermediate input k to final output j. Summing over all intermediate values k gives the total probability that B_i produces final output j, which is exactly the (i, j)-entry of the composed channel.

Worked example: composing two BSCs

The Binary Symmetric Channel of Section 2.1 is the cleanest test case for composition. We show that composing two BSCs always yields a third BSC, and we derive the composed crossover probability explicitly.

Let Q_1 be a BSC with crossover ε_1 and Q_2 be a BSC with crossover ε_2:

$$Q_1 \;=\; \begin{pmatrix} 1-\varepsilon_1 & \varepsilon_1 \\ \varepsilon_1 & 1-\varepsilon_1 \end{pmatrix}, \qquad Q_2 \;=\; \begin{pmatrix} 1-\varepsilon_2 & \varepsilon_2 \\ \varepsilon_2 & 1-\varepsilon_2 \end{pmatrix}.$$

We compute $Q_{\text{total}} = Q_2 Q_1$ directly:

$$\begin{aligned} Q_2 Q_1 &= \begin{pmatrix} 1-\varepsilon_2 & \varepsilon_2 \\ \varepsilon_2 & 1-\varepsilon_2 \end{pmatrix} \begin{pmatrix} 1-\varepsilon_1 & \varepsilon_1 \\ \varepsilon_1 & 1-\varepsilon_1 \end{pmatrix} \\ &= \begin{pmatrix} (1-\varepsilon_2)(1-\varepsilon_1) + \varepsilon_2\varepsilon_1 & (1-\varepsilon_2)\varepsilon_1 + \varepsilon_2(1-\varepsilon_1) \\ \varepsilon_2(1-\varepsilon_1) + (1-\varepsilon_2)\varepsilon_1 & \varepsilon_2\varepsilon_1 + (1-\varepsilon_2)(1-\varepsilon_1) \end{pmatrix}. \end{aligned}$$

Observe that the top-left and bottom-right entries are equal, and the top-right and bottom-left entries are equal. That is precisely the structure of a BSC matrix. Denote the off-diagonal entry as $\varepsilon_{\text{total}}$:

$$\begin{aligned} \varepsilon_{\text{total}} &= (1-\varepsilon_2)\varepsilon_1 + \varepsilon_2(1-\varepsilon_1) \\ &= \varepsilon_1 - \varepsilon_1\varepsilon_2 + \varepsilon_2 - \varepsilon_1\varepsilon_2 \\ &= \varepsilon_1 + \varepsilon_2 - 2\varepsilon_1\varepsilon_2. \end{aligned} \tag{2.1}$$

So the composed channel is

$$Q_{\text{total}} \;=\; \begin{pmatrix} 1-\varepsilon_{\text{total}} & \varepsilon_{\text{total}} \\ \varepsilon_{\text{total}} & 1-\varepsilon_{\text{total}} \end{pmatrix},$$

a BSC with crossover

$$\varepsilon_{\text{total}} \;=\; \varepsilon_1(1-\varepsilon_2) + (1-\varepsilon_1)\varepsilon_2.$$

This is the formula stated in Definition 2.3.1: two BSCs in series produce a third BSC.

Reading the composition formula

The formula $\varepsilon_{\text{total}} = \varepsilon_1(1-\varepsilon_2) + (1-\varepsilon_1)\varepsilon_2$ has a direct probabilistic interpretation. An error in the composed channel occurs if and only if *exactly one* of the two channels flips its input. There are two ways this can happen:

- The first channel flips (probability ε_1) and the second does not (probability $1-\varepsilon_2$), contributing $\varepsilon_1(1-\varepsilon_2)$.
- The first channel does not flip (probability $1-\varepsilon_1$) and the second does (probability ε_2), contributing $(1-\varepsilon_1)\varepsilon_2$.

If both channels flip, the two errors cancel: the input is back to its original value. If neither flips, the input passes through unchanged. The composed error is therefore the XOR of the two individual errors, and the probability of the XOR of two independent Bernoulli trials with parameters ε_1 and ε_2 is exactly $\varepsilon_1(1-\varepsilon_2) + (1-\varepsilon_1)\varepsilon_2$.

Two channels make things worse. For $0 < \varepsilon_1, \varepsilon_2 < 1/2$, we have

$$\varepsilon_{\text{total}} = \varepsilon_1 + \varepsilon_2 - 2\varepsilon_1\varepsilon_2 > \varepsilon_1.$$

Adding a second channel in series strictly increases the crossover probability (unless one of the channels is perfect, $\varepsilon = 0$, or one is a maximal-error channel, $\varepsilon = 1$). Two imperfect BSCs in series are noisier than either alone. The only way to reduce the crossover in a serial pipeline is to make the intermediate channel a *corrector*, which is outside the binary-channel model we study here.

Numerical example. Take $\varepsilon_1 = 0.05$ and $\varepsilon_2 = 0.10$. Then

$$\varepsilon_{\text{total}} = 0.05 \cdot 0.90 + 0.95 \cdot 0.10 = 0.045 + 0.095 = 0.140.$$

The composed channel has crossover 0.14, strictly larger than either 0.05 or 0.10. As a matrix:

$$Q_{\text{total}} = \begin{pmatrix} 0.86 & 0.14 \\ 0.14 & 0.86 \end{pmatrix}.$$

Both rows sum to 1 and the off-diagonal entries are equal—the composed channel is indeed a BSC.

General asymmetric channels compose by the same rule

The BSC is the symmetric special case. The general asymmetric binary channel of Definition 2.2.1 follows the same rule: the composed channel matrix is the matrix product Q_2Q_1, and the composed error rates follow from the law of total probability exactly as above.

Let Q_1 have false-positive rate ε_1 and false-negative rate ω_1, and let Q_2 have ε_2 and ω_2:

$$Q_1 = \begin{pmatrix} 1-\omega_1 & \omega_1 \\ \varepsilon_1 & 1-\varepsilon_1 \end{pmatrix}, \qquad Q_2 = \begin{pmatrix} 1-\omega_2 & \omega_2 \\ \varepsilon_2 & 1-\varepsilon_2 \end{pmatrix}.$$

The composed matrix Q_2Q_1 yields combined rates (reading off the $(1,2)$-entry for the false-negative rate and the $(2,1)$-entry for the false-positive rate):

$$\begin{aligned} \omega_{\text{total}} &= (1-\omega_2)\omega_1 + \omega_2\varepsilon_1, \\ \varepsilon_{\text{total}} &= (1-\varepsilon_2)\varepsilon_1 + \varepsilon_2(1-\omega_1). \end{aligned}$$

This is Theorem 3.7 from the companion paper [Tow26a], stated there in terms of the approximate-identity function composition $\widetilde{\mathrm{id}}_{\varepsilon,\tau} \circ \widetilde{\mathrm{id}}_{\varepsilon',\tau'}$. The matrix-product derivation above is the channel-matrix reformulation of the same result.

Note that the composition of two asymmetric channels is *not* generally a BSC even when the individual channels happen to have equal overall crossover rates: the asymmetry in Q_1 and Q_2 generally produces different false-positive and false-negative rates in Q_2Q_1. The BSC is the special case in which the symmetry $\varepsilon = \omega$ is preserved under composition because both inputs to the formula are symmetric, which forces $\varepsilon_{\text{total}} = \omega_{\text{total}}$.

Serial composition vs. parallel queries

It is essential to distinguish the *serial* composition of this section from the *parallel* query structure studied in Section 4.4.

- **Serial composition** (this section): an element passes through channel Q_1, and the *output* of Q_1 becomes the *input* to channel Q_2. The combined system acts on a single element, producing a single final output. The joint channel matrix is the ordinary matrix product Q_2Q_1, which is a 2×2 matrix (the same size as each factor).

- **Parallel queries** (Section 4.4): m independent queries are applied to m *distinct* elements, each through its own copy of the channel. The combined output is an m-tuple. The joint channel matrix is the Kronecker product $Q_1 \otimes Q_2 \otimes \cdots \otimes Q_m$, which is a $2^n \times 2^m$ matrix, growing *exponentially* in m.

The two compositions are structurally different and answer different questions. Serial composition (matrix product) asks: what is the end-to-end error rate when one channel's output feeds another? Parallel composition (Kronecker product) asks: what is the joint output distribution when we query m elements simultaneously?

Theorem 4.4.1 shows that the Kronecker product structure arises because Axioms 1 and 2 force independence across parallel queries. The matrix product of this section arises from the law of total probability applied to the intermediate state in a serial pipeline. These are complementary, not competing, structures. When both matter—for instance, when k parallel pipelines each consist of multiple stages in series—the combined channel is a Kronecker product of composed matrices:

$$\left(Q_2^{(1)} Q_1^{(1)}\right) \otimes \left(Q_2^{(2)} Q_1^{(2)}\right) \otimes \cdots ,$$

where each parenthesized factor is a serial composition computed by ordinary matrix multiplication. The Kronecker product is applied *across* parallel pipelines; the ordinary product is applied *within* each pipeline.

Composition is associative but not commutative

An immediate algebraic consequence of the matrix-product interpretation is that serial channel composition is *associative*: composing three channels as $(Q_3 Q_2) Q_1$ or $Q_3 (Q_2 Q_1)$ gives the same result, since matrix multiplication is associative. This means that chaining any number of binary channels in series is unambiguous: the order of application matters, but the grouping does not.

Commutativity, however, fails in general. The product $Q_2 Q_1$ need not equal $Q_1 Q_2$: applying the first channel before the second is not the same as applying the second before the first, unless the two channels have some special symmetry.

For the BSC, commutativity *does* hold: two BSC matrices commute because each is a polynomial in the same circulant permutation matrix. Concretely, if Q_1 and Q_2 are both BSCs, then $Q_2 Q_1 = Q_1 Q_2$ by the symmetry of the composition formula $\varepsilon_{\text{total}} = \varepsilon_1(1 - \varepsilon_2) + (1 - \varepsilon_1)\varepsilon_2$, which is

symmetric in ε_1 and ε_2. This symmetry is a property of the BSC's circulant structure and is *lost* in the asymmetric channel.

Exercise 2.7 constructs an explicit pair of asymmetric channels that do not commute, illustrating that the order of channels in a serial pipeline matters outside the BSC special case.

The stationary point

For a BSC with crossover $\varepsilon < 1/2$, the composition formula shows that $\varepsilon_{\text{total}} = \varepsilon_1 + \varepsilon_2 - 2\varepsilon_1\varepsilon_2 > \max(\varepsilon_1, \varepsilon_2)$ whenever $0 < \varepsilon_1, \varepsilon_2 < 1/2$. As we chain more and more copies of the same BSC in series, the crossover approaches the stationary value $1/2$ from below. At $\varepsilon = 1/2$, the channel output is statistically independent of the input: the channel transmits no information. This convergence to the stationary point mirrors the figure in the companion paper [Tow26a] (Figure 3.1), which shows both TPR and FPR converging to the stationary value $\varepsilon/(\varepsilon+\omega)$ under repeated composition of the general asymmetric channel.

Looking ahead: two perspectives on the same idea

Channel composition via matrix product is the channel-theoretic perspective on a result that chapter 3 will reframe algebraically. Section 3.3 will show that composing k approximate identity channels in series produces a composed false-positive rate (and, symmetrically, a composed hit rate η_{total}) satisfying

$$\eta_{\text{total}} \;=\; 1 - \prod_{i=1}^{k} (1 - \eta_i),$$

where η_i is the hit rate of the i-th stage. This is the same calculation as the matrix product, viewed through the Boolean-algebra lens: the composition theorem in chapter 3 is the algebraic restatement of what Definition 2.3.1 computes.

At the other end of the formalism, Section 4.4 and Theorem 4.4.1 give the *parallel* analog: m independent channels queried on m distinct elements produce a joint confusion matrix that is the Kronecker product $Q_1 \otimes \cdots \otimes Q_m$. The Kronecker product is the structure forced by Axioms 1 and 2; the matrix product of this section needs neither axiom, only the law of total probability.

Understanding which composition is relevant—serial (matrix product) or parallel (Kronecker product)—is one of the key skills this book develops. The corollary Corollary 4.4.1.1 gives the parallel-query factorization; the

formula of this section gives the serial-pipeline composition. Both structures appear throughout Parts II and III.

2.4 Model Order

Two channels, one question

We have now seen two versions of the binary channel. The Binary Symmetric Channel of Section 2.1 has a single crossover parameter ε, constrained so that both inputs flip with the same probability. The asymmetric binary channel of Section 2.2 lifts that constraint and allows two independent parameters ε and ω, one for each input block. When we compose two such channels in series (Section 2.3), the resulting channel inherits the two-block structure, regardless of how many parameters either constituent channel carries.

Why exactly two blocks? And could a binary channel have more? These questions are not about specific channels; they are about the structure of the input domain. The answer is given by the concept of *model order*, which we define in this section, connect to the channels developed in §§2.1–2.3, and cap with a theorem.

Partitions and blocks

A *partition* of a set S is a collection of non-empty, pairwise disjoint subsets $B_1, \ldots, B_n \subseteq S$ whose union is S:

$$B_i \cap B_j = \emptyset \text{ for } i \neq j, \qquad B_1 \cup B_2 \cup \cdots \cup B_n = S.$$

Each subset B_i is called a *block*. A partition tells us which elements are "the same" for the purpose of the model: elements in the same block are treated identically.

In the Bernoulli framework, a partition of the input domain is central to the channel model. An element x in block B_i uses the per-block error rate assigned to B_i; elements in different blocks may use different rates. This is precisely the structure of Axiom 2 in Section 4.3: the per-block rates $\{\alpha_1, \ldots, \alpha_n\}$ are conditionally independent given the objective set, one rate per block B_i.

Definition 2.4.1 (Model order)**.** *Let $\phi : D \to D$ be a binary channel with input domain D. A* partition of D compatible with ϕ *is a partition $\{B_1, \ldots, B_n\}$ of D such that every element within the same block B_i has the*

same conditional error probability:

$$\Pr\{\phi(x) \neq x\} = \alpha_i \quad \textit{for all } x \in B_i.$$

The model order *of ϕ is the number of blocks in the finest compatible partition of D: the partition that assigns distinct blocks to elements with distinct error probabilities and groups together only those elements that genuinely share a common rate. We denote the model order by n.*

Two remarks on this definition. First, the finest compatible partition is well-defined: we can always take the coarsest partition (all of D as one block) if the error probability is the same for every input, or refine to singletons if every input has its own distinct rate. The model order is the count of blocks in the finest such partition. Second, model order is a property of the channel as a whole, not of any particular parameterization: it reflects the true number of distinct error regimes in the input domain, regardless of how the channel is described.

Model order 2 for the Boolean domain

The Boolean input domain is $D = \{\top, \bot\}$. There are exactly two elements. The possible partitions of D are:

1. The *trivial partition*: one block $\{B_1\} = \{\top, \bot\}$, both inputs in a single block. This means $\Pr\{\phi(\top) \neq \top\} = \Pr\{\phi(\bot) \neq \bot\}$: the same error rate for both inputs.

2. The *discrete partition*: two blocks $B_1 = \{\top\}$ and $B_2 = \{\bot\}$, each input in its own block. This means each input has its own error rate, which need not be equal.

No other partition exists. The input domain has two elements, and a partition of a 2-element set has at most two blocks. The model order of any binary channel is therefore at most 2.

The two channels from §§2.1–2.2 correspond exactly to these two partitions.

Model order 1: the trivial partition. When a binary channel assigns the same error rate ε to both inputs, only one block is needed: $\{B_1\} = \{\top, \bot\}$. The channel matrix is

$$Q = \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

which is exactly the BSC of Definition 2.1.1. A single shared rate ε describes the channel; the model order is 1.

Model order 2: the discrete partition. When the two inputs have distinct error rates $\varepsilon \neq \omega$, no single block can cover both; the finest compatible partition has two blocks, $B_1 = \{\top\}$ and $B_2 = \{\bot\}$, and the model order is 2. The channel matrix is

$$Q = \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

which is the general asymmetric channel of Definition 2.2.1. Two parameters, ε and ω, are required.

The BSC as the symmetric special case of order 2. There is a subtlety that is easy to overlook. The BSC has model order 1: its finest compatible partition has a single block because both inputs have the same error rate ε. We can, however, always embed a model-order-1 channel into the order-2 framework by using the discrete partition $B_1 = \{\top\}$, $B_2 = \{\bot\}$ and then imposing the additional constraint $\varepsilon = \omega$. This is exactly how Section 2.1 described the BSC: as an order-2 channel with the symmetry constraint. Saying "the BSC is the symmetric special case of the order-2 model" means that we work in the order-2 channel family (two blocks, two parameters) and then restrict to the symmetric subcase where the two parameters are equal.

Both descriptions are correct: the BSC's finest compatible partition has one block (model order 1 by the definition above), but it is also a degenerate member of the two-parameter, two-block family (the symmetric special case of order 2). The distinction matters primarily when comparing channels across families: the BSC genuinely lives in the order-1 class by partition count, and the asymmetric channel genuinely requires order 2. Practically, we will work in the order-2 framework throughout chapters 2–4, treating the BSC as a constrained member of that family, because the two-block partition is the natural one for the Boolean domain.

The Boolean ceiling theorem

The preceding analysis leads immediately to the central theorem of this section.

Theorem 2.4.1 (Boolean ceiling)**.** *Any binary channel with input domain* $\{\top, \bot\}$ *has model order at most* 2*.*

Proof. Let $\phi : \{\top, \bot\} \to \{\top, \bot\}$ be any binary channel. We must show that every compatible partition of the input domain $D = \{\top, \bot\}$ has at most 2 blocks.

A partition of D is a collection of non-empty, pairwise disjoint subsets whose union is $D = \{\top, \bot\}$. Since $|D| = 2$, any partition can have at most $|D| = 2$ blocks: a partition with more than 2 blocks would require more than 2 elements in the union, but $|D| = 2$. Concretely, the partitions of a 2-element set are:

1. $\big\{\{\top, \bot\}\big\}$: the trivial partition, one block.
2. $\big\{\{\top\},\ \{\bot\}\big\}$: the discrete partition, two blocks.

No other partition exists. The model order, defined as the number of blocks in the finest compatible partition, is therefore either 1 or 2, and in particular is at most 2. □

The proof is elementary because the claim is a direct consequence of the input domain's cardinality. A partition of any k-element set has at most k blocks; for the Boolean domain, $k = 2$, giving the ceiling of 2. The real content of the theorem is that this ceiling is achieved: the asymmetric channel of Section 2.2 is a binary channel with model order exactly 2, so the bound is tight. And the BSC shows that the ceiling need not be reached: a binary channel can have model order 1 when the two inputs happen to share a common error rate.

Model order vs. parameter count

Model order and parameter count are related but distinct, and conflating them is one of the most common sources of confusion in this framework. It is worth being explicit.

- **Model order** is the number of partition blocks in the finest compatible partition of the input domain. It is a structural property of the channel, determined by how many distinct error regimes actually exist in D. For the Boolean domain, model order is at most 2 by the Boolean ceiling theorem.
- **Parameter count** is the number of free real numbers required to specify the channel. For a single Boolean channel, an n-th order model has n free parameters (one per block), but the total parameter count

for m joint queries grows with m: under Axioms 1 and 2, the per-source-block convention gives $n(2^m - 1)$ parameters in general, reduced by Kronecker factorization to nm (Corollary 4.4.1.2).

The key distinction: model order is about partition granularity (how many distinct error regimes), while parameter count is about how many numbers are needed to specify the model (which depends on the axioms and the query structure). Confusion matrices and model order are orthogonal concepts: model order constrains which rows of Q are identical (or more precisely, which inputs can be put in the same block), while the independence axioms constrain the internal structure of each row and how rows interact across multiple queries.

Three examples make the distinction concrete.

Example 1: BSC, single query. Model order 1 (one block, $\varepsilon = \omega$). Parameter count: 1.

Example 2: Asymmetric channel, single query. Model order 2 (two blocks, $\varepsilon \neq \omega$). Parameter count: 2.

Example 3: Asymmetric channel, m queries under Axioms 1 and 2. Model order 2. Parameter count under axioms: $2m$ (two parameters per query, by Corollary 4.4.1.2). Parameter count without axioms, per-source-block convention: $2(2^m - 1)$.

In Example 3, model order does not change with m: the channel is still order 2, with two blocks, regardless of the query count. What changes with m is the parameter count. The axioms drive the dramatic reduction from $2(2^m - 1)$ to $2m$: without Axiom 1 (element-wise independence) and Axiom 2 (cross-block rate independence, Axiom 2), the Kronecker factorization of Corollary 4.4.1.2 would not hold. The axioms are doing all the work; model order is not.

This clarification matters for reading the companion papers correctly. Some sections of the source papers use "order" loosely to refer to the number of free parameters in the single-query setting, where both quantities agree (an n-th order model on one Boolean query has n parameters). In the multi-query setting, or whenever the independence axioms are in view, it is essential to keep the two concepts separate. Throughout this book, *model order* always means the number of partition blocks, as in Definition 2.4.1.

How the channels of §§2.1–2.3 fit

We can now locate every channel of this chapter within the model-order framework.

The BSC (Section 2.1). Model order 1 by finest-partition count, but studied in the order-2 framework (as the symmetric special case). Channel matrix: Q_{BSC} with $\varepsilon = \omega$. Free parameters in Q: 1.

The asymmetric binary channel (Section 2.2). Model order 2. Channel matrix: general Q with independent ε and ω. Free parameters in Q: 2.

The Z-channel (Definition 2.2.2). Model order 2 (the two blocks have genuinely different rates: $\omega = 0$ for $B_1 = \{\top\}$ and $\varepsilon > 0$ for $B_2 = \{\bot\}$). Free parameters in Q: 1 (only ε, since ω is known to be 0). This illustrates that model order and free parameter count are independent: the Z-channel has model order 2 but only one free parameter.

Composed channels (Section 2.3). Composing two binary channels produces a binary channel. The model order of the composed channel is at most 2 (by the Boolean ceiling theorem, since the output is still a binary channel over the same domain $\{\top, \bot\}$). The parameter count of the composed channel depends on the input channels' rates: the BSC-BSC composition gives one parameter ($\varepsilon_{\text{total}}$), while the asymmetric-asymmetric composition generally gives two.

Worked illustration: ordering by blocks

The following table organizes the four channels of this chapter by their block structure under the discrete partition $\{B_1, B_2\} = \{\{\top\}, \{\bot\}\}$:

Channel	α_1 (rate for $B_1 = \{\top\}$)	α_2 (rate for $B_2 = \{\bot\}$)	Model order
BSC	ε	ε $(= \alpha_1)$	1
Z-channel	0	ε $(\neq 0)$	2
Asymmetric	ω	ε (free)	2
Perfect channel	0	0	1

Reading the table: the BSC's two blocks share the same rate ε, so the finest compatible partition merges them into one block, giving model order 1. The perfect channel (zero error everywhere) likewise merges both blocks, giving model order 1. The Z-channel and asymmetric channel have $\alpha_1 \neq \alpha_2$, so the finest compatible partition has two blocks, giving model order 2.

The table also makes visible why the Z-channel has model order 2 despite having only one free parameter (ε): the two blocks have different rates (0 and $\varepsilon > 0$), so the finest partition must keep them separate. Parameter

count reflects how many numbers we write down; model order reflects how many blocks are needed.

Axiom 2 and the per-block structure

The partition-block structure of model order connects directly to the formal treatment in Section 4.3. Axiom 2 (Axiom 2) asserts that the per-block error rates $\alpha_1, \ldots, \alpha_n$ are conditionally independent given the objective set. In the order-2 Boolean case, this reduces to:

$$f(\varepsilon, \omega|A) = f(\varepsilon|A) \cdot f(\omega|A),$$

the independence of the false-positive rate (rate for block $B_2 = \{\bot\}$) and the false-negative rate (rate for block $B_1 = \{\top\}$). The model order is what determines the dimension of this independence claim: order-2 gives a two-factor joint density; order-n gives an n-factor joint density.

The Boolean ceiling theorem limits this: because the Boolean domain has model order at most 2, the Axiom 2 independence claim for Boolean channels involves at most two independent rates. This makes the framework particularly tractable: every Boolean channel can be described by at most two parameters, and the Kronecker factorization of Corollary 4.4.1.2 reduces the multi-query parameter count to $2m$ rather than $2(2^m - 1)$.

Why the ceiling matters

The Boolean ceiling theorem is not a limitation of the framework; it is a feature. Any predicate over $\{\top, \bot\}$ lives in a world with exactly two possible error regimes: one for each element of the domain. No matter how complex the underlying computation, no matter how many intermediate steps a channel goes through, the final Boolean predicate has model order at most 2. The two-block partition $\{B_1, B_2\} = \{\{\top\}, \{\bot\}\}$ is the universal structure for Boolean channels.

This has a practical consequence. When we apply the Bernoulli framework to a new Boolean predicate—a membership test, a primality check, a classifier's verdict—we know in advance that the predicate is either order 1 (if the error probability is the same for all inputs) or order 2 (if the error probabilities differ by input sign). There are no other cases. The Boolean ceiling closes off the space of possible structures and reduces the question of model identification to a single binary choice.

Chapter 9 will revisit the model-order concept for input domains with more than two elements: when $|D| = k$ for $k > 2$, the same partition-block

machinery applies and the ceiling rises to k, but the analysis of channels, parameter counts, and independence axioms remains structurally the same.

Looking ahead

Section 4.3 placed the partition-block structure at the center of Axiom 2. Corollary 4.4.1.2 showed how the Kronecker factorization drives the parameter count from $2(2^m - 1)$ down to $2m$ for order-2 Boolean predicates under the axioms. In Part II we will see how the order-2 Boolean case interacts with the Bernoulli set operations: union, intersection, and complement all produce channels whose model order is at most 2 (because they remain binary channels over the Boolean domain). The Boolean ceiling theorem is why the entire Boolean algebra of Bernoulli sets remains tractable.

The broader setting, where the input domain has cardinality greater than 2, belongs to Part III. A single sentence is enough for now: the same partition-block machinery that gives the Boolean ceiling at order 2 generalizes to n-ary channels with model order at most $|D|$, covered in Chapter 9.

Bibliographic Notes

Shannon and the channel framework. The channel matrix formalism used throughout this chapter descends directly from Shannon [Sha48]. Shannon's paper introduced the discrete memoryless channel as the mathematical object for studying noisy communication: a channel is specified by a conditional distribution $P(\text{output} \mid \text{input})$, and the capacity of the channel is the maximum mutual information between input and output over all input distributions. The binary symmetric channel of Section 2.1 is one of Shannon's central running examples; the crossover parameter ε and the capacity formula $1 - H(\varepsilon)$ are both in that 1948 paper. The treatment of the asymmetric binary channel in Section 2.2 is a natural extension of Shannon's framework to the case where the two input symbols face independent error probabilities. For a modern textbook development of binary channels, channel capacity, and the channel coding theorem, see Cover and Thomas [CT06], especially Chapters 7 and 8.

The Bernoulli papers. The connection between binary channels and the Bernoulli framework is drawn in Towell [Tow26a], which introduces the two-axiom model and proves the composition theorem that Section 2.3 reformulates in matrix-product language. The matrix-product form—that serial

composition of two binary channels corresponds to matrix multiplication of their channel matrices—is a reorganization of the rate-propagation formula in that paper; see the companion notes in Section 4.6 for a precise account of what is new in this text and what descends from the companion papers.

The model-order treatment of Section 2.4, and in particular the definition of model order as the number of partition blocks in the finest compatible partition of the input domain (Definition 2.4.1), follows Towell [Tow26f]. That paper extends the Bernoulli framework from Boolean sets to approximate values over arbitrary algebraic types; the order-2 Boolean case developed in this chapter is the simplest instance of its general theory. The Boolean Ceiling Theorem (Theorem 2.4.1)—that any binary channel over $\{\top, \bot\}$ has model order at most 2—is explicit in the algebraic-types paper as the base case of the more general statement that a channel over a k-element domain has model order at most k.

Model order versus degrees of freedom. A clarification that deserves explicit attention is the distinction between model order and parameter count, developed in Section 2.4 under the heading "Model order vs. parameter count." This distinction is easy to miss in the source papers, because for a single Boolean query the two quantities agree: an n-th order channel on one Boolean input has n partition blocks and also n free parameters, one per block. The difference surfaces in the multi-query setting and is worth recording here, as it is flagged as a source of confusion in the companion repository's planning notes (`papers/CLAUDE.md`, Terminology section). When m queries are issued jointly, a general order-n channel on m inputs has $n(2^m - 1)$ parameters in the per-source-block convention, but the independence axioms (Axioms 1 and 2) force a Kronecker factorization that reduces this to nm. The model order does not change: it remains n, determined by the partition of the single-query input domain. What changes is the parameter count, and the axioms are entirely responsible for the reduction. Model order constrains which rows of the channel matrix Q can be identified; the axioms constrain how the channel matrix factorizes across queries. These are orthogonal constraints.

Further reading. Chapter 4's Bibliographic Notes (Section 4.6) provide additional context for the Kronecker product and its role in the multi-query factorization, and for the relationship between the Bernoulli axioms and the memoryless channel. Readers who want a deeper grounding in information theory beyond what Part I requires will find Cover and Thomas [CT06] the

standard reference.

Exercises

1. ⋆**Writing the channel matrix.** Let $\varepsilon = 0.07$ and $\omega = 0.12$. Write the 2×2 channel matrix Q for the asymmetric binary channel (Definition 2.2.1) with these parameters. Verify that both rows sum to 1 (row-stochastic condition). Then write the matrix for the special case $\varepsilon = \omega = 0.07$ and identify which named channel (Definition 2.1.1) it gives.

2. ⋆**Composing two BSCs by hand.** Let Q_1 be a BSC with crossover $\varepsilon_1 = 0.05$ and Q_2 be a BSC with crossover $\varepsilon_2 = 0.10$ (Definition 2.1.1).

   (a) Write out Q_1 and Q_2 explicitly as 2×2 matrices.

   (b) Compute the matrix product $Q_{\text{total}} = Q_2\, Q_1$ entry by entry (Definition 2.3.1).

   (c) Using the formula $\varepsilon_{\text{total}} = \varepsilon_1(1 - \varepsilon_2) + (1 - \varepsilon_1)\varepsilon_2$ from Equation (2.1), verify that $\varepsilon_{\text{total}} = 0.14$ and that Q_{total} is a BSC with this crossover.

3. ⋆**Identifying model order.** For each of the following systems, state the model order (Definition 2.4.1) and justify your answer in one sentence.

   (a) A BSC with crossover $\varepsilon = 0.03$ (Definition 2.1.1).

   (b) A Bloom filter (Definition 2.2.2).

   (c) An asymmetric binary channel with $\varepsilon = 0.05$ and $\omega = 0.08$ (Definition 2.2.1).

   (d) The composition of a BSC followed by a Z-channel (Definition 2.3.1).

4. ⋆⋆ **Deriving BSC capacity.** The capacity of a channel is the maximum mutual information $I(X;Y)$ between input X and output Y over all input distributions. Let $p = \Pr\{X = \top\}$ be the prior probability of a positive input, so $\Pr\{X = \bot\} = 1 - p$.

   (a) For a BSC with crossover ε (Definition 2.1.1), write out $\Pr\{Y = \top\}$ and $\Pr\{Y = \bot\}$ in terms of p and ε.

   (b) Recall that mutual information satisfies $I(X;Y) = H(Y) - H(Y|X)$, where H denotes binary entropy. Show that $H(Y|X) = H_2(\varepsilon)$ regardless of p, where $H_2(\varepsilon) = -\varepsilon \log_2 \varepsilon - (1 - \varepsilon)\log_2(1 - \varepsilon)$.

(c) Show that $H(Y)$ is maximized at $p = 1/2$, and conclude that the BSC capacity is $1 - H_2(\varepsilon)$.

(d) Evaluate the capacity at $\varepsilon = 0.05$ (the running example from Section 2.1) and at $\varepsilon = 0.5$. Interpret both values.

5. ⋆⋆**Bloom filter as a Z-channel.** The Bloom filter satisfies $\omega = 0$: it never reports a false negative (Definition 2.2.2).

(a) Write the channel matrix Q for a Bloom filter with false-positive rate $\varepsilon = 0.05$ and verify that the matrix satisfies $\omega = 0$.

(b) A query returns "not in set." Show that, under the Z-channel model (Section 2.2), this answer carries no uncertainty: the true membership status is determined exactly.

(c) Argue that no amount of serial composition of Z-channels (Definition 2.3.1) with $\omega = 0$ can produce a non-zero false-negative rate. Use the matrix-product form to support your argument.

(*Hint for (c):* consider what the composition formula from Equation (2.1) becomes when $\omega_1 = \omega_2 = 0$.)

6. ⋆⋆**The composition of two BSCs is a BSC.** Prove the general case of the composition result implicit in Equation (2.1): for any two BSC matrices Q_1 and Q_2 with crossovers ε_1 and ε_2, the product $Q_{\text{total}} = Q_2 Q_1$ is again a BSC.

(a) A 2×2 stochastic matrix is a BSC (Definition 2.1.1) if and only if both of its off-diagonal entries are equal. Show that Q_{total} satisfies this condition by multiplying the two symmetric matrices and comparing off-diagonal entries.

(b) Identify the crossover of Q_{total} in terms of ε_1 and ε_2.

(c) The composition of BSCs is therefore *closed* in the class of BSCs. Give a one-sentence interpretation: why is closure a useful property for building serial pipelines out of symmetric components?

7. ⋆⋆**When does an asymmetric channel reduce to a BSC?** An asymmetric binary channel (Definition 2.2.1) has parameters ε and ω.

(a) State the algebraic condition on (ε, ω) that makes the asymmetric channel's matrix equal to a BSC matrix (Definition 2.1.1).

(b) In the (ε, ω) parameter space $[0, 1)^2$, describe the set of parameter pairs that correspond to BSCs. What geometric object is this set?

(c) At the boundary of this set, perturb ω by a small amount $\delta > 0$ while fixing ε. Compute the new Q and identify the model order (Definition 2.4.1) of the resulting channel.

(d) Explain in one paragraph why the BSC is a measure-zero special case within the space of asymmetric channels, and what that implies for real-world data systems.

8. ⋆⋆**Constructing a model-order-2 channel from two model-order-1 channels.** Two deterministic channels each have model order 1: the identity channel (crossover $\varepsilon = 0$) and the complementing channel (crossover $\varepsilon = 1$). Both are trivially order-1 BSCs.

   (a) Compute the serial composition of the identity channel followed by the complementing channel (Definition 2.3.1). What channel does the composition produce?

   (b) Now consider a non-degenerate construction: compose a BSC with crossover $\varepsilon_1 = 0.1$ with an asymmetric channel having $\varepsilon_2 = 0.2$, $\omega_2 = 0.3$. Compute $Q_{\text{total}} = Q_2 Q_1$ and verify the model order (Definition 2.4.1) of the result.

   (c) Describe in general how serial composition (Section 2.3) affects the partition structure. Does composing two order-n channels always yield an order-n channel? Does it always yield an order-m channel for some fixed m related to the input domain?

   (*Hint:* focus on the input domain of the composed channel, not the intermediate domain.)

9. ⋆⋆⋆**Rigorous proof of the Boolean ceiling.** Theorem 2.4.1 states that any binary channel over the input domain $\{\top, \bot\}$ has model order at most 2. Give a self-contained proof from the definitions.

   (a) State the definition of model order (Definition 2.4.1) precisely: a partition of $\{\top, \bot\}$ is a collection of non-empty pairwise-disjoint subsets whose union is $\{\top, \bot\}$. List all distinct partitions of the two-element set $\{\top, \bot\}$.

   (b) Show that no partition of $\{\top, \bot\}$ can have more than 2 blocks. (The argument is purely combinatorial: a partition into k blocks of a 2-element set requires $k \leq 2$.)

   (c) Conclude that the model order of any channel over $\{\top, \bot\}$ is at most 2, and give the two distinct channel classes that correspond

to model order 1 (the BSC) and model order 2 (the general asymmetric channel).

(d) Contrast with the n-ary case: state, without proving, how the ceiling changes for an input domain of cardinality k (foreshadows Chapter 9 without entering Part III content).

10. $\star\star\star$**Channel composition: associative but not commutative.** Let Q_1, Q_2, Q_3 be binary channel matrices and define serial composition as matrix product (Definition 2.3.1).

(a) *Associativity.* Prove that $(Q_3Q_2)Q_1 = Q_3(Q_2Q_1)$. (This follows directly from associativity of matrix multiplication; state the argument clearly and identify what it implies for building pipelines of arbitrary length.)

(b) *Non-commutativity by counterexample.* Let Q_1 be a BSC with crossover $\varepsilon_1 = 0.1$ and Q_2 be an asymmetric channel with $\varepsilon_2 = 0.4$ and $\omega_2 = 0.1$. Compute Q_2Q_1 and Q_1Q_2 explicitly. Show that they are not equal.

(c) Interpret the non-commutativity result: the order in which two noisy stages appear in a serial pipeline matters. Give a one-sentence practical implication for designing a two-stage classification pipeline where one stage has higher false-positive rate and the other has higher false-negative rate.

11. $\star\star\star$**A 3-input partition and model order 3.** The binary domain $\{\top, \bot\}$ supports at most model order 2 (Theorem 2.4.1). This exercise explores what happens when the input domain grows.

(a) Let the input domain be $\{1, 2, 3\}$ (a 3-element set). List all distinct partitions of this domain. How many distinct values of model order are possible?

(b) Write a 3×2 channel matrix Q for a channel from the 3-element input domain to the binary output domain $\{\top, \bot\}$, choosing a partition into 2 blocks $B_1 = \{1, 2\}$ and $B_2 = \{3\}$ with per-block error rates $\varepsilon_1 = 0.05$, $\varepsilon_2 = 0.20$. Verify that both rows in each block share the same entries and that the matrix is row-stochastic.

(c) Write a second channel matrix using the 3-block partition $B_1 = \{1\}$, $B_2 = \{2\}$, $B_3 = \{3\}$ with rates $\varepsilon_1 = 0.05$, $\varepsilon_2 = 0.15$, $\varepsilon_3 = 0.30$. State the model order of this channel.

(d) Explain in one paragraph why the Kronecker factorization (Theorem 4.4.1) discussed in the context of parallel queries (chapter 4 §4.4) extends naturally to n-ary input domains, and identify the one change in the parameter count. (This is a preview of Chapter 9; no detailed proof is expected.)

Chapter 3

Boolean Algebra Under Noise

After this chapter you will be able to:

- Derive AND, OR, and NOT under noise from first principles, using the per-block rate parameters of the input Bernoulli[bool] instances.
- State and apply the composition theorem $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$ for chained identities-with-error.
- Analyze Miller-Rabin's k-round error from first principles by treating one witness as a Bernoulli[bool] instance and composing.
- Locate the hidden independence assumption that Chapter 4 will make explicit.

3.1 AND Under Noise

We have, by now, a clear picture of a single noisy bit: a Bernoulli[bool] instance with a false-positive rate ε and a false-negative rate ω, as defined formally in Definition 4.5.1. The binary symmetric channel of Section 2.1 was the symmetric special case $\varepsilon = \omega$. The biased-coin running example, introduced in Section 1.1, is the general asymmetric case.

The first question chapter 3 must answer is: what happens when we combine two noisy bits using a Boolean operation? AND is the natural place to start, because it is the most commonly composed gate in practice, the one behind the Bloom filter's k-hash membership query, and the operation

that underlies Miller-Rabin's k-round test. Once we understand AND under noise, OR and NOT follow without new machinery, as Section 3.2 shows.

Setup: two independent noisy bits

Let $\tilde{b}_1$ and $\tilde{b}_2$ be two Bernoulli[bool] instances with rates $(\varepsilon_1, \omega_1)$ and $(\varepsilon_2, \omega_2)$ respectively. Each instance has its own channel matrix:

$$Q_i \;=\; \begin{pmatrix} 1-\omega_i & \omega_i \\ \varepsilon_i & 1-\varepsilon_i \end{pmatrix}, \quad i = 1, 2.$$

Row 1 of Q_i governs positive inputs (true value $\top$); row 2 governs negative inputs (true value $\bot$). Columns are the output labels $\top$ and $\bot$, following the convention from Section 2.1.

We apply the AND function to the two observed outputs. The *true* value of the AND is the AND of the two *true* inputs. We want to know: what is the probability that the AND of the noisy observations agrees with the AND of the true values?

Axioms invoked: a flag before we compute

Before proceeding to the case analysis, we must flag the independence assumptions the derivation below will silently use. We state them explicitly here because Part I's aim is to make these assumptions visible, not merely convenient.

Axiom 1 (Section 4.2, Axiom 1) asserts that error events at distinct inputs to a single predicate are mutually independent. In the AND setting, $\tilde{b}_1$ and $\tilde{b}_2$ evaluate two distinct input elements. Axiom 1 therefore licenses the factorization: the event that $\tilde{b}_1$ makes an error is independent of the event that $\tilde{b}_2$ makes an error. Every product of probabilities in the case analysis below rests on this factorization.

Axiom 2 (Section 4.3, Axiom 2) asserts that the per-block error rates are conditionally independent of each other given the objective set. For $\tilde{b}_1$, this means ε_1 and ω_1 are independent parameters; likewise for $\tilde{b}_2$. As a consequence (Proposition 4.3.1), the channel matrix Q_i is stable across queries: the rate seen by one query of $\tilde{b}_1$ is the same rate that applies to any other query of the same instance. This is what allows us to read the rates ε_i and ω_i as fixed constants throughout the case analysis, rather than tracking a time-varying stochastic process.

Both axioms are in play. Axiom 1 handles the across-instance independence; Axiom 2 handles the within-instance rate stability. Chapter 4 proves

both axioms in full (Section 4.2 and Section 4.3) and shows, in Section 4.4, that the two together are exactly the content of the Kronecker factorization theorem. What we do now is use these axioms deliberately, not accidentally.

Truth table for AND: four cases

The AND of two Boolean values is $\top$ if and only if both inputs are $\top$. There are four combinations of true inputs; we analyze each.

True b_1	True b_2	Correct AND output	Pr[AND output $= \top$]
$\top$	$\top$	$\top$	$(1-\omega_1)(1-\omega_2)$
$\top$	$\bot$	$\bot$	$(1-\omega_1)\varepsilon_2$
$\bot$	$\top$	$\bot$	$\varepsilon_1(1-\omega_2)$
$\bot$	$\bot$	$\bot$	$\varepsilon_1\varepsilon_2$

The rightmost column gives Pr[observed AND $= \top$] for each true-input combination. It does not give the probability that the AND is *correct*; that requires also knowing which output is correct for that row. Let us work through each case in turn.

Case 1: both true inputs are $\top$. The correct output is $\top$. For the AND of observed outputs to equal $\top$, both $\tilde{b}_1$ and $\tilde{b}_2$ must report $\top$. Each does so correctly with probability $1-\omega_i$ (the true-positive rate). By Axiom 1, the two events are independent:

$$\Pr[\tilde{b}_1 = \top,\ \tilde{b}_2 = \top \mid b_1 = \top, b_2 = \top] \ = \ (1-\omega_1)(1-\omega_2).$$

This is the AND's true-positive probability.

Case 2: $b_1 = \top$, $b_2 = \bot$. The correct output is $\bot$ (since $b_2 = \bot$). For the AND of observed outputs to incorrectly equal $\top$, $\tilde{b}_1$ must report $\top$ (correctly, with probability $1-\omega_1$) *and* $\tilde{b}_2$ must report $\top$ (incorrectly, a false positive, with probability ε_2). By Axiom 1:

$$\Pr[\tilde{b}_1 = \top,\ \tilde{b}_2 = \top \mid b_1 = \top, b_2 = \bot] \ = \ (1-\omega_1)\varepsilon_2.$$

This is a false positive conditional on the input pattern $(\top, \bot)$.

Case 3: $b_1 = \bot$, $b_2 = \top$. Symmetric to Case 2. The AND output is $\top$ incorrectly when $\tilde{b}_1$ makes a false positive and $\tilde{b}_2$ reports correctly:

$$\Pr[\tilde{b}_1 = \top,\ \tilde{b}_2 = \top \mid b_1 = \bot, b_2 = \top] \ = \ \varepsilon_1(1-\omega_2).$$

Case 4: both true inputs are $\bot$. The correct output is $\bot$. The AND output is $\top$ incorrectly only when both $\tilde{b}_1$ and $\tilde{b}_2$ make false positives simultaneously:

$$\Pr[\tilde{b}_1 = \top,\ \tilde{b}_2 = \top \mid b_1 = \bot, b_2 = \bot] \ = \ \varepsilon_1\varepsilon_2.$$

Closed-form output rates

The AND gate's output is a new Boolean value. To characterize it as a Bernoulli[bool] instance, we need to identify its false-positive rate and false-negative rate with respect to the *true AND output*.

The true AND output is $\top$ if and only if $b_1 = b_2 = \top$. The true AND output is $\bot$ in all three remaining cases.

False-negative rate of AND. A false negative occurs when the true output is $\top$ (meaning $b_1 = b_2 = \top$) but the observed AND is $\bot$. From Case 1, the probability that the observed AND is $\top$ when the true output is $\top$ is $(1 - \omega_1)(1 - \omega_2)$. Therefore, the false-negative rate is:

$$\omega_\wedge \;=\; 1 - (1 - \omega_1)(1 - \omega_2) \;=\; \omega_1 + \omega_2 - \omega_1\omega_2. \tag{3.1}$$

This is the probability that at least one of the two instances reports $\bot$ when the true value is $\top$. The union-of-events formula applies because both must succeed for the AND to succeed.

False-positive rate of AND. A false positive occurs when the true output is $\bot$ (meaning at least one of b_1, b_2 is $\bot$) but the observed AND is $\top$. The true output is $\bot$ for three input patterns: $(\top, \bot)$, $(\bot, \top)$, and $(\bot, \bot)$.

The observed AND is $\top$ in each of these three cases with the probabilities from Cases 2, 3, and 4 above. The false-positive rate of the AND gate is therefore a mixture over these three patterns, weighted by their prior probabilities. Let π denote the prior probability that $b_1 = \top$, and let $\pi_1 = \pi$, $\pi_2 = \pi$ (assuming the two bits are drawn from the same prior for simplicity; the general formula follows with π_1 and π_2 distinct).

The conditional-on-pattern false-positive events are:

$$\begin{aligned}
&\Pr[\wedge = \top \mid b_1 = \top, b_2 = \bot] = (1 - \omega_1)\varepsilon_2,\\
&\Pr[\wedge = \top \mid b_1 = \bot, b_2 = \top] = \varepsilon_1(1 - \omega_2),\\
&\Pr[\wedge = \top \mid b_1 = \bot, b_2 = \bot] = \varepsilon_1\varepsilon_2.
\end{aligned}$$

These are three distinct conditional false-positive probabilities, one per input pattern. In the Bernoulli framework, the false-positive rate of the AND gate is the probability that the observed AND is $\top$ given that the true AND is $\bot$. Because the three patterns that yield true AND $= \bot$ have different likelihoods of a false $\top$ output, the effective false-positive rate of the AND gate depends on the input distribution.

For the special case where both bits are independently queried about the same underlying question (both $\pi_1 = \pi_2 = \pi$, and the two bits are drawn

independently), the marginal false-positive rate is:

$$\varepsilon_\wedge(\pi) \;=\; \frac{\pi(1-\pi)(1-\omega_1)\varepsilon_2 + (1-\pi)\pi\,\varepsilon_1(1-\omega_2) + (1-\pi)^2\varepsilon_1\varepsilon_2}{\pi(1-\pi) + (1-\pi)\pi + (1-\pi)^2}. \quad (3.2)$$

The denominator is the probability that the true AND is $\bot$, which equals $1-\pi^2$.

The input-distribution-free bounds. In most practical analyses, we do not know the exact prior π. What we can say regardless of π is the following. Since we are taking the AND of two outputs, the AND gate can only output $\top$ if both inputs report $\top$. This means:

$$\varepsilon_\wedge \;\leq\; \min(\varepsilon_1, \varepsilon_2),$$

since the AND gate can falsely report $\top$ only when both individual instances report $\top$, which is at least as unlikely as each one doing so alone. The upper bound follows from the cases directly: no single case has a false-positive probability exceeding $\max(\varepsilon_1, \varepsilon_2)$, and the worst case (Case 2 with $\omega_1 = 0$) approaches ε_2.

The most important special case is when both instances are independent, identically noisy copies: $\varepsilon_1 = \varepsilon_2 = \varepsilon$, $\omega_1 = \omega_2 = \omega$. The worst-case false-positive input pattern is the $(\bot, \bot)$ case, giving ε^2. For the $(\top, \bot)$ and $(\bot, \top)$ patterns, the false-positive probability is $(1-\omega)\varepsilon$, which for small ω is approximately ε. The effective false-positive rate is therefore bounded above by $(1-\omega)\varepsilon$, and the dominant pattern depends on the input prior. When both bits query the same two-valued population symmetrically, the AND gate reduces the false-positive rate dramatically compared to a single query: the $(\bot, \bot)$ contribution goes as ε^2, which for small ε is negligible. This is the mechanism behind Miller-Rabin's exponential error reduction, which we examine in Section 3.4.

AND as a Bernoulli[bool] **operation.** The AND gate's output is a new Boolean value. Its false-negative rate $\omega_\wedge$ in (3.1) depends only on the input rates, not on the input prior. Its false-positive rate $\varepsilon_\wedge$ depends on the input prior through (3.2), unless we are in a special case that eliminates the dependence (for example, the $(1-\omega)\varepsilon_2$ bound applies when $b_1 = \top$ is guaranteed).

The conclusion is that AND is a binary operation on Bernoulli[bool] instances that produces a new Bernoulli[bool] instance. The type is closed under AND: each output is characterized by a pair of rates, derived from the input rates via the formulas above. The derivation invoked both axioms

explicitly: Axiom 1 to factor probabilities across the two instances, and Axiom 2 to treat each instance's rates as stable constants throughout the computation.

Worked example: AND of two noisy coin observers

The biased-coin thread from Section 1.1 returns here with its most concrete appearance so far. We have two observers of the same coin flip.

Setup. A coin lands heads ($\top$) with probability $\pi = 0.5$ (fair coin). Two independent observers $\tilde{b}_1$ and $\tilde{b}_2$ each report the outcome, but with different noise levels:

- Observer 1: $\varepsilon_1 = 0.10$, $\omega_1 = 0.05$ (more likely to miss heads than to hallucinate heads).
- Observer 2: $\varepsilon_2 = 0.08$, $\omega_2 = 0.12$ (more likely to miss heads than to hallucinate, but with different magnitudes).

We declare "heads" (output $\top$) only if both observers agree: output $\tilde{b}_1 \wedge \tilde{b}_2$.

False-negative rate of AND. From (3.1):

$$\omega_\wedge = \omega_1 + \omega_2 - \omega_1\omega_2 = 0.05 + 0.12 - (0.05)(0.12) = 0.17 - 0.006 = 0.164.$$

Requiring both observers to report "heads" when the coin actually landed heads raises the false-negative rate from the individual rates of 5% and 12% to 16.4%. This is the cost of the AND gate: it is harder to get two correct $\top$ reports simultaneously than just one.

Conditional false-positive rates, by pattern.

$$\begin{aligned}
&\Pr[\wedge = \top \mid b_1 = \top, b_2 = \bot] = (1 - 0.05)(0.08) = 0.95 \times 0.08 = 0.076,\\
&\Pr[\wedge = \top \mid b_1 = \bot, b_2 = \top] = (0.10)(1 - 0.12) = 0.10 \times 0.88 = 0.088,\\
&\Pr[\wedge = \top \mid b_1 = \bot, b_2 = \bot] = (0.10)(0.08) = 0.008.
\end{aligned}$$

The AND gate is hardest to fool when both true values are $\bot$: the probability of a false $\top$ output drops to 0.8%, compared to 7.6% or 8.8% when one true value is $\top$.

Effective false-positive rate (fair coin, $\pi = 0.5$). With a fair coin, the

four true-input patterns have probabilities:

$$\begin{aligned}
\Pr[b_1 = \top, b_2 = \top] &= 0.5 \times 0.5 = 0.25,\\
\Pr[b_1 = \top, b_2 = \bot] &= 0.5 \times 0.5 = 0.25,\\
\Pr[b_1 = \bot, b_2 = \top] &= 0.5 \times 0.5 = 0.25,\\
\Pr[b_1 = \bot, b_2 = \bot] &= 0.5 \times 0.5 = 0.25.
\end{aligned}$$

The true AND is $\bot$ when any of the last three patterns hold, giving $\Pr[\text{true AND} = \bot] = 0.75$. The marginal probability that the observed AND is $\top$ when the true AND is $\bot$ is the weighted average:

$$\begin{aligned}
\text{numerator} &= 0.25 \times 0.076 + 0.25 \times 0.088 + 0.25 \times 0.008\\
&= 0.019 + 0.022 + 0.002 = 0.043.
\end{aligned}$$

The effective false-positive rate is:

$$\varepsilon_\wedge \;=\; \frac{0.043}{0.75} \;\approx\; 0.057.$$

Comparison to individual rates. The two individual observers have $\varepsilon_1 = 0.10$ and $\varepsilon_2 = 0.08$. Their AND gate reduces the effective false-positive rate to approximately 0.057 under a fair-coin prior. The reduction is meaningful but not dramatic, because the dominant false-positive patterns are those where one true value is $\top$ (so only one observer needs to make an error); only the all-negative pattern benefits from requiring both observers to err simultaneously.

The false-negative rate rises from 5% and 12% individually to 16.4%. The AND gate trades false-negative accuracy for false-positive suppression. In applications where false positives are costly and false negatives are tolerable, this is a favorable trade. In applications where false negatives are costly (a safety system that must not miss a $\top$ event), requiring both observers to agree is counterproductive.

Summary of the example. Two Bernoulli[bool] observers with rates $(0.10, 0.05)$ and $(0.08, 0.12)$ combine under AND to produce a new Bernoulli[bool] instance with $\omega_\wedge = 0.164$ and $\varepsilon_\wedge \approx 0.057$ (at $\pi = 0.5$). The rates follow closed-form formulas derived from the four truth-table cases. Both Axiom 1 and Axiom 2 were invoked: Axiom 1 to factor the joint probability of both observers reporting $\top$ into a product of individual probabilities; Axiom 2 to treat each observer's rates as stable constants rather than time-varying quantities.

Looking ahead

The AND derivation gives us one gate. Section 3.2 will derive OR and NOT under the same methodology: NOT is elementary (it swaps ε and ω), and OR follows via De Morgan's law applied to noisy gates. Together, AND, OR, and NOT generate all of Boolean algebra.

Once we have all three gates, Section 3.3 asks what happens when we chain them: if AND has false-positive rate $\varepsilon_\wedge$, and we then AND the result with a third noisy bit, what is the total false-positive rate? The answer is the composition theorem, $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$, which describes how error rates accumulate in chains of identity-with-error operations.

That composition theorem is precisely what makes Miller-Rabin's k-round bound derivable from first principles, as Section 3.4 will show. The k-round Miller-Rabin test is, in the framework's language, a k-fold AND of Bernoulli[bool] witnesses. The derivation we just completed for two witnesses extends directly to k; the formulas are the same; only the indices change.

3.2 OR and NOT

Section 3.1 derived what happens when we AND two Bernoulli[bool] instances. We found that AND is a binary operation on the type: two instances with rates $(\varepsilon_1, \omega_1)$ and $(\varepsilon_2, \omega_2)$ combine to produce a new instance with closed-form rates $\varepsilon_\wedge$ and $\omega_\wedge$. The computation invoked both axioms: Axiom 1 to factor joint probabilities, and Axiom 2 to treat the per-instance rates as stable constants.

OR and NOT call for the same treatment. As we will see, NOT is immediate from the channel-matrix perspective, and OR follows from De Morgan's law applied to noisy gates. We then verify the De Morgan derivation by re-deriving OR from the truth table directly, confirming that both paths agree. The section closes by identifying what the three gates together buy us: closure of Bernoulli[bool] under all of Boolean algebra.

NOT under noise

Let $\tilde{b}$ be a Bernoulli[bool] instance with channel matrix

$$Q = \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

Its NOT, $\neg\tilde{b}$, reports the complement of the observed output. What are the rates of the complemented instance?

We answer this by tracing the meaning of the four outcomes. If the true value of $\tilde{b}$ is b and the observed output is $\hat{b}$, then $\neg\tilde{b}$ has true value $\neg b$ and observed output $\neg\hat{b}$.

False-positive rate of $\neg\tilde{b}$. A false positive of $\neg\tilde{b}$ is the event that the true value of $\neg b$ is $\bot$—meaning the true value of b is $\top$—while the observed output of $\neg\tilde{b}$ is $\top$—meaning the observed output of $\tilde{b}$ is $\bot$. In other words, a false positive of $\neg\tilde{b}$ is a *false negative* of $\tilde{b}$. The rate is:

$$\varepsilon_\neg \;=\; \omega.$$

False-negative rate of $\neg\tilde{b}$. A false negative of $\neg\tilde{b}$ is the event that the true value of $\neg b$ is $\top$—meaning $b = \bot$—while the observed output of $\neg\tilde{b}$ is $\bot$—meaning $\hat{b} = \top$. In other words, a false negative of $\neg\tilde{b}$ is a *false positive* of $\tilde{b}$. The rate is:

$$\omega_\neg \;=\; \varepsilon.$$

Summary. The NOT operation acts on the channel matrix by swapping the roles of false positives and false negatives:

$$Q_{\neg\tilde{b}} \;=\; \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \omega & 1-\omega \end{pmatrix}. \tag{3.3}$$

In rate notation: $(\varepsilon, \omega) \overset{\neg}{\mapsto} (\omega, \varepsilon)$.

This is the simplest operation in the Boolean algebra of noisy bits. The channel matrix is not recomputed from a truth table; it is obtained by reading the rows of Q in the opposite order. Complementing a predicate swaps the positive and negative blocks; the row that formerly governed positives now governs negatives, and vice versa.

NOT is its own inverse. Applying NOT twice returns the original:

$$(\varepsilon_\neg, \omega_\neg) \;=\; (\omega, \varepsilon) \;\overset{\neg}{\mapsto}\; (\varepsilon, \omega).$$

In channel-matrix terms, the row-swap (3.3) applied twice restores Q exactly. More precisely, NOT is an *involution* on Bernoulli[bool]: $(\neg)^2 = \mathrm{id}$.

This is the noisy-gate analog of the classical fact $\neg\neg b = b$. Under noise, the equality is at the level of the channel matrix, not the individual observation: applying NOT twice to a Bernoulli[bool] instance returns an instance with the same rates as the original, not the same realized output on any given query.

OR via De Morgan's law

De Morgan's law says $b_1 \vee b_2 = \neg(\neg b_1 \wedge \neg b_2)$. We apply this identity at the gate level, using NOT (derived above) and AND (from Section 3.1).

Let $\tilde{b}_1$ and $\tilde{b}_2$ be two independent Bernoulli[bool] instances with rates $(\varepsilon_1, \omega_1)$ and $(\varepsilon_2, \omega_2)$ respectively.

Step 1: Negate each input. By the NOT formula above:

$$\neg\tilde{b}_1 \text{ has rates } (\omega_1, \varepsilon_1),$$
$$\neg\tilde{b}_2 \text{ has rates } (\omega_2, \varepsilon_2).$$

Step 2: AND the negated inputs. We apply the AND formula from Section 3.1 to $\neg\tilde{b}_1$ and $\neg\tilde{b}_2$ with their swapped rates. For two Bernoulli[bool] instances with rates (a_1, c_1) and (a_2, c_2), the AND gate's false-negative rate is $c_1 + c_2 - c_1 c_2$ (from Equation (3.1)).

For the AND of $\neg\tilde{b}_1$ and $\neg\tilde{b}_2$, the roles of ε and ω are swapped relative to the original instances, so $a_i = \omega_i$ and $c_i = \varepsilon_i$:

$$\omega_{\neg\wedge\neg} \;=\; \varepsilon_1 + \varepsilon_2 - \varepsilon_1\varepsilon_2. \tag{3.4}$$

The FPR of the AND of the negated inputs is, again from the AND formula, now with the input FPRs equal to ω_1, ω_2 (the swapped values). It depends on the input prior over the negated values; but we will see in Step 3 that the outer NOT maps it to the OR's FNR, which we handle directly from the truth table.

Step 3: Negate the result. The outer NOT swaps the rates of $\neg\tilde{b}_1 \wedge \neg\tilde{b}_2$:

$$\varepsilon_\vee \;=\; \omega_{\neg\wedge\neg} \;=\; \varepsilon_1 + \varepsilon_2 - \varepsilon_1\varepsilon_2, \tag{3.5}$$

and $\omega_\vee = \varepsilon_{\neg\wedge\neg}$.

The FPR of the OR gate, $\varepsilon_\vee = \varepsilon_1 + \varepsilon_2 - \varepsilon_1\varepsilon_2$, is the inclusion-exclusion formula for the probability that at least one of two independent events occurs. It is input-distribution-free: it depends only on the individual false-positive rates, not on the prior over input patterns. This is the noisy-gate analog of the classical fact that $\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$ for independent events.

OR via the direct truth table

We now re-derive the OR false-positive rate directly from the truth table, to confirm that both paths yield the same result.

The OR of the observed outputs is $\top$ if *at least one* of $\hat{b}_1$, $\hat{b}_2$ is $\top$. The true OR output, $b_1 \vee b_2$, is $\top$ if at least one of b_1, b_2 is $\top$.

True b_1	True b_2	Correct OR output	Pr[OR output $= \bot$]
$\top$	$\top$	$\top$	$\omega_1\omega_2$
$\top$	$\bot$	$\top$	$\omega_1(1-\varepsilon_2)$
$\bot$	$\top$	$\top$	$(1-\varepsilon_1)\omega_2$
$\bot$	$\bot$	$\bot$	$(1-\varepsilon_1)(1-\varepsilon_2)$

The rightmost column gives Pr[observed OR $= \bot$] for each true-input combination (the OR is $\bot$ only when both observed outputs are $\bot$, which by Axiom 1 has probability equal to the product of the individual $\bot$-reporting probabilities).

False-positive rate of OR. A false positive of the OR gate occurs when the true OR is $\bot$—meaning both true inputs are $\bot$—but the observed OR is $\top$. This is the complement of both observed outputs being $\bot$:

$$\varepsilon_\vee \;=\; \Pr[\hat{b}_1 = \top \text{ or } \hat{b}_2 = \top \mid b_1 = \bot, b_2 = \bot] \;=\; 1-(1-\varepsilon_1)(1-\varepsilon_2) \;=\; \varepsilon_1+\varepsilon_2-\varepsilon_1\varepsilon_2. \tag{3.6}$$

This matches (3.5) from the De Morgan derivation exactly.

False-negative rate of OR. A false negative of the OR gate occurs when the true OR is $\top$—meaning at least one true input is $\top$—but the observed OR is $\bot$. The observed OR is $\bot$ when *both* observed outputs are $\bot$.

The three cases where the true OR is $\top$ are $(\top, \top)$, $(\top, \bot)$, and $(\bot, \top)$:

$$\begin{aligned}
\Pr[\hat{b}_1 = \bot, \hat{b}_2 = \bot \mid b_1 = \top, b_2 = \top] \;&=\; \omega_1\omega_2, \\
\Pr[\hat{b}_1 = \bot, \hat{b}_2 = \bot \mid b_1 = \top, b_2 = \bot] \;&=\; \omega_1(1-\varepsilon_2), \\
\Pr[\hat{b}_1 = \bot, \hat{b}_2 = \bot \mid b_1 = \bot, b_2 = \top] \;&=\; (1-\varepsilon_1)\omega_2.
\end{aligned}$$

As with the AND's false-positive rate, the OR's false-negative rate depends on the input distribution: different mixtures of the three patterns yield different marginal FNR values. For the special case $\varepsilon_1 = \varepsilon_2 = 0$ (no false positives), the three conditional FNRs simplify to $\omega_1\omega_2$, ω_1, and ω_2 respectively. The dominant pattern is then whichever positive-input combination is most common: if most queries have exactly one positive input, the FNR of OR is close to ω_1 or ω_2 individually (OR requires only one correct positive report, but it still misses when the positive input's reporter makes a false negative).

In contrast, the FPR of OR (3.6) is fully determined by the individual FPRs alone: the only way the true OR is $\bot$ is if both true inputs are $\bot$, and

the only way the observed OR is then $\top$ is if at least one of the reporters makes a false positive. Both the input pattern and the conditional false-positive probability are distribution-free. This asymmetry between the OR's FPR and FNR is the dual of the AND's asymmetry: AND has a distribution-free FNR and a distribution-dependent FPR; OR has a distribution-free FPR and a distribution-dependent FNR.

Summary of the three gates

Gate	FPR	FNR
$\tilde{b}_1 \wedge \tilde{b}_2$ (AND)	depends on input prior (see §3.1)	$\omega_1 + \omega_2 - \omega_1\omega_2$
$\neg\tilde{b}$ (NOT)	ω	ε
$\tilde{b}_1 \vee \tilde{b}_2$ (OR)	$\varepsilon_1 + \varepsilon_2 - \varepsilon_1\varepsilon_2$	depends on input prior

The table crystallizes the fundamental duality between AND and OR. Under noise, AND suppresses false positives at the cost of inflating false negatives (the FNR of AND grows monotonically with the number of ANDed instances), while OR suppresses false negatives at the cost of inflating false positives (the FPR of OR grows monotonically with the number of ORed instances). NOT is the exchange operator that converts one regime into the other.

Worked check: consistency at $\varepsilon = \omega$. For the symmetric case $\varepsilon_1 = \varepsilon_2 = \varepsilon$ and $\omega_1 = \omega_2 = \omega$:

$$\omega_\wedge = 2\omega - \omega^2,$$
$$\varepsilon_\vee = 2\varepsilon - \varepsilon^2.$$

For a BSC with crossover probability $\varepsilon = \omega$, these are symmetric: the AND gate inflates errors for positives by the same formula as the OR gate inflates errors for negatives. De Morgan's duality is exact, as expected.

Boolean algebra closure under noise

AND, OR, and NOT together generate all of two-valued Boolean algebra. Every Boolean function of n variables can be expressed as a formula built from AND, OR, and NOT (this is the completeness of the connective basis $\{\wedge, \vee, \neg\}$). We have now derived the channel matrices for each of the three generators. The key structural observation is the following.

Proposition 3.2.1 (Closure of Bernoulli[bool] under Boolean operations)**.** *If* $\tilde{b}_1, \ldots, \tilde{b}_n$ *are* Bernoulli[bool] *instances, then every Boolean formula* $\varphi(\tilde{b}_1, \ldots, \tilde{b}_n)$ *built from AND, OR, and NOT is also a* Bernoulli[bool] *instance. Its channel matrix has entries expressible as polynomials in the input rates* $\varepsilon_1, \omega_1, \ldots, \varepsilon_n, \omega_n$*.*

Proof sketch. By induction on the structure of φ. The base case is a single instance $\tilde{b}_i$, which is Bernoulli[bool] by assumption. For the inductive step:

NOT: if $\tilde{b}$ is Bernoulli[bool] with rates (ε, ω), then $\neg\tilde{b}$ is Bernoulli[bool] with rates (ω, ε) by the derivation above.

AND: if $\tilde{b}_1$ and $\tilde{b}_2$ are Bernoulli[bool] instances, then $\tilde{b}_1 \wedge \tilde{b}_2$ is Bernoulli[bool] by the derivation in Section 3.1, with FNR given by (3.1) and FPR by (3.2). Both are polynomial functions of the input rates.

OR: by De Morgan's law, $\tilde{b}_1 \vee \tilde{b}_2 = \neg(\neg\tilde{b}_1 \wedge \neg\tilde{b}_2)$. The outer and inner NOTs each preserve Bernoulli[bool] membership (by the NOT case); the AND of the negated instances is Bernoulli[bool] (by the AND case). The composition is therefore Bernoulli[bool].

Since every Boolean formula is built by finitely many applications of AND, OR, and NOT, the induction terminates and the result holds. □

Closure is not merely a formal nicety. It is what makes the Bernoulli framework composable: we can build complex Boolean circuits from noisy gates and know that every intermediate and final result is still a Bernoulli[bool] instance. Error propagation through the circuit is computable in closed form; it does not require re-deriving the framework from scratch at each stage.

Looking ahead to composition

The OR gate illustrates a structural point about the composition theorem, which Section 3.3 will state precisely.

The FPR of OR is $\varepsilon_1 + \varepsilon_2 - \varepsilon_1\varepsilon_2$. For small ε_1 and ε_2, this is approximately $\varepsilon_1 + \varepsilon_2$: error rates *add* under OR. Equivalently, the probability of no error is $(1 - \varepsilon_1)(1 - \varepsilon_2)$: no-error probabilities *multiply*.

The AND gate, dually, has FNR equal to $\omega_1 + \omega_2 - \omega_1\omega_2$: a symmetric formula showing that false-negative error rates add under AND. The no-false-negative probability $(1 - \omega_1)(1 - \omega_2)$ again factors as a product.

Both formulas are instances of the same pattern. When we chain k independent identity-with-error stages—each of which fails independently with rate η_i—the total no-error probability is $\prod_i(1 - \eta_i)$, and the total error rate is $\eta_{\text{total}} = 1 - \prod_i(1 - \eta_i)$. Section 3.3 proves this formula in its

general form and shows that the AND and OR formulas above are special cases. Once we have the composition theorem in hand, Miller-Rabin's k-round error bound becomes immediate: k rounds of witness-ANDing drive the FPR to $(1/4)^k$ because each round is an independent stage of error suppression, and the no-error probability multiplies.

The gates derived in this section—NOT, OR, AND, and their interaction via De Morgan—are the moving parts of that composition argument. They are also the gates that give the Bernoulli framework its claim to being a complete algebra, not merely a collection of special-purpose error-rate formulas.

3.3 The Composition Theorem

Section 3.1 showed that the AND of k independent Bernoulli[bool] witnesses drives the false-positive rate down: requiring all k to agree on $\top$ makes a false $\top$ output unlikely. Section 3.2 showed the dual: the OR of k witnesses drives the false-negative rate down. Both sections closed with a preview of the same structural pattern: the probability that the chain produces *no* error factors as a product.

This section makes that pattern precise. We state and prove the *composition theorem*: for k independent identity-with-error stages chained in series, the end-to-end error rate is

$$\eta_{\text{total}} = 1 - \prod_{i=1}^{k} (1 - \eta_i),$$

where η_i is the per-stage error rate. We then examine what this formula does and does not capture, paying particular attention to the BSC running thread—because the BSC has a subtlety, a cancellation structure, that the composition theorem does not capture. Understanding that gap is essential to using the theorem correctly.

Setup: chaining identity-with-error stages

The model is simple. We have an original Boolean value $b \in \{\top, \bot\}$. A single *identity-with-error* stage passes b through a channel that reports the correct value with probability $1 - \eta_i$ and reports the wrong value with probability η_i. Call the stage's output $\hat{b}_i$. The stage is not a BSC in general: we will say more shortly about what the error rate η_i means. For now, think of it as the probability that the stage makes *any* error, regardless of direction.

We chain k such stages: $b \to \hat{b}_1 \to \hat{b}_2 \to \cdots \to \hat{b}_k$. The final output $\hat{b}_k$ is what we observe. The question is: what is the probability that $\hat{b}_k \neq b$?

For this question to have a clean answer, we need a clean independence structure. We assume:

1. **Chain-stage independence.** The error event at stage i is independent of the error event at every other stage $j \neq i$. This is a chain-level assumption: it says that the k stages do not share error-inducing correlations along the pipeline.

2. **Each error is one-directional.** Once stage i makes an error, the subsequent stages in the chain receive the wrong value and propagate it. Errors do not cancel. This is the identity-with-error regime: any single error in the chain corrupts the end-to-end output.

The second assumption is worth flagging before the theorem, not after. It is the source of the BSC subtlety we will return to.

The composition theorem

Theorem 3.3.1 (Composition theorem, identity-with-error chains)**.** *Let b be a Boolean value. Let $\hat{b}_1, \hat{b}_2, \ldots, \hat{b}_k$ be the successive outputs of k chained identity-with-error stages, where stage i has error rate $\eta_i \in [0, 1]$ and the error events across stages are mutually independent.*

Under the identity-with-error assumption (any single error in the chain corrupts the end-to-end output), the end-to-end error rate satisfies

$$\eta_{\text{total}} \;=\; 1 - \prod_{i=1}^{k} (1 - \eta_i). \tag{3.7}$$

Equivalently, letting $\eta_i = 1 - \eta_i$ denote the per-stage correctness probability*, the end-to-end correctness probability is*

$$\eta_{\text{total}} \;=\; \prod_{i=1}^{k} \eta_i. \tag{3.8}$$

Proof. We prove the correctness-probability form (3.8) by induction on k, then recover (3.7) by complementation.

Base case, $k = 1$. A single stage with error rate η_1 has correctness probability $\eta_1 = 1 - \eta_1$. The formula gives $\prod_{i=1}^{1} \eta_i = \eta_1$. Correct.

Inductive step. Assume the result holds for $k-1$ stages: the first $k-1$ stages in the chain produce a correct output with probability $\eta_1 \cdots \eta_{k-1}$. Now append stage k with error rate η_k.

Under the identity-with-error assumption, the final output $\hat{b}_k$ equals b if and only if *both* of the following hold:

- The output of the first $k-1$ stages is correct, with probability $\eta_1 \cdots \eta_{k-1}$ (by the inductive hypothesis).
- Stage k does not make an error, with probability η_k.

By chain-stage independence, these two events are independent. Their joint probability is:

$$\eta_{\text{total}} \;=\; (\eta_1 \cdots \eta_{k-1}) \cdot \eta_k \;=\; \prod_{i=1}^{k} \eta_i.$$

Taking the complement, the end-to-end error rate is

$$\eta_{\text{total}} = 1 - \eta_{\text{total}} = 1 - \prod_{i=1}^{k} (1 - \eta_i). \qquad \square$$

Remark on the identity-with-error assumption. The inductive step rests on the phrase "if and only if both." This is only valid when any error at any stage is fatal to correctness. If stage k receives an already-wrong value and makes an error, it *could* accidentally correct the previous error—returning to the right answer. The composition theorem excludes this possibility by assumption. For channels where cancellation is possible (notably the BSC), the formula (3.7) does not give the exact end-to-end error rate, and the matrix-product computation of Section 2.3 must be used instead. We return to this point with numerical evidence in the BSC worked example below.

Connection to the AND and OR formulas

Before the BSC example, let us confirm that the composition theorem unifies the formulas already derived in Sections 3.1 and 3.2.

AND as composition. In Section 3.1, the false-negative rate of AND was:

$$\omega_{\wedge} \;=\; 1 - (1 - \omega_1)(1 - \omega_2).$$

This is exactly (3.7) with $k = 2$ and $\eta_i = \omega_i$. Interpreting AND as a two-stage identity-with-error composition in the positive block: stage 1 fails to

deliver a $\top$ output with probability ω_1; stage 2 independently fails with probability ω_2; the AND of the two outputs is $\top$ only if both succeed. The identity-with-error assumption is satisfied in the positive block because if either stage makes a false negative (reports $\bot$ when the true value is $\top$), the AND output is $\bot$ regardless of the other stage. There is no cancellation in the $\top$ direction.

Extending to k independent Bernoulli[bool] instances all having true value $\top$, with false-negative rates $\omega_1, \ldots, \omega_k$: the k-fold AND has false-negative rate

$$\omega_{\wedge,k} \;=\; 1 - \prod_{i=1}^{k} (1 - \omega_i).$$

When all stages have the same rate ω, this simplifies to $1 - (1-\omega)^k$.

OR as composition. In Section 3.2, the false-positive rate of OR was:

$$\varepsilon_{\vee} \;=\; 1 - (1-\varepsilon_1)(1-\varepsilon_2).$$

This is (3.7) with $k = 2$ and $\eta_i = \varepsilon_i$. Interpreting OR as a two-stage identity-with-error composition in the negative block: stage 1 makes a false positive with probability ε_1; stage 2 independently with probability ε_2; the OR output is $\bot$ only if both avoid false positives. The identity-with-error assumption holds in the $\bot$ direction because once either stage makes a false positive (reports $\top$ when the true value is $\bot$), the OR output is $\top$ regardless.

Extending to k stages with false-positive rates $\varepsilon_1, \ldots, \varepsilon_k$: the k-fold OR has false-positive rate

$$\varepsilon_{\vee,k} \;=\; 1 - \prod_{i=1}^{k} (1 - \varepsilon_i).$$

The BSC running thread: where the formula applies and where it does not

The binary symmetric channel (BSC) from Section 2.3 is the principal running example for composition in this book. We use it here to illustrate both what the composition theorem captures and, critically, where its identity-with-error assumption breaks down.

What chapter 2 §2.3 derived. Section 2.3 computed the crossover probability of two BSCs in series using the matrix product $Q_{\text{total}} = Q_2 Q_1$. The result (Equation (2.1)) was:

$$\varepsilon_{\text{total}} \;=\; \varepsilon_1 + \varepsilon_2 - 2\varepsilon_1\varepsilon_2.$$

For equal crossovers $\varepsilon_1 = \varepsilon_2 = \varepsilon$, this becomes:

$$\varepsilon_{\text{total}} \;=\; 2\varepsilon - 2\varepsilon^2. \tag{3.9}$$

What the composition theorem gives. If we naively apply (3.7) with $\eta_i = \varepsilon$ for both stages and $k = 2$, we obtain:

$$\eta_{\text{total}} \;=\; 1 - (1-\varepsilon)^2 \;=\; 2\varepsilon - \varepsilon^2. \tag{3.10}$$

These two formulas are different.

$$\underbrace{2\varepsilon - 2\varepsilon^2}_{\text{correct BSC composition, §2.3}} \quad \neq \quad \underbrace{2\varepsilon - \varepsilon^2}_{\text{naive §3.3 formula}} \quad .$$

For $\varepsilon = 0.1$:

$$\begin{aligned} \text{correct (§2.3):}\ & 2(0.1) - 2(0.01) = 0.18, \\ \text{naive (§3.3):}\ & 2(0.1) - (0.01) = 0.19. \end{aligned}$$

The naive formula overestimates the BSC's end-to-end error rate.

The discrepancy is not a bug in either formula. It reveals the identity-with-error assumption in action.

The cancellation structure of the BSC. In a BSC, a "flip" swaps $\top \leftrightarrow \bot$. Two flips cancel: $\top \xrightarrow{\text{flip}} \bot \xrightarrow{\text{flip}} \top$. The end-to-end output is wrong if and only if an *odd* number of flips occurred across the chain. The end-to-end output is right if an *even* number of flips occurred (including zero).

For $k = 2$ stages with equal crossover ε, the parity decomposition is:

$$\begin{aligned} \Pr[\text{exactly 0 flips}] &= (1-\varepsilon)^2, \\ \Pr[\text{exactly 1 flip}] &= 2\varepsilon(1-\varepsilon), \\ \Pr[\text{exactly 2 flips}] &= \varepsilon^2. \end{aligned}$$

The end-to-end error probability is the odd-flip probability:

$$\Pr[\text{odd number of flips}] = 2\varepsilon(1-\varepsilon) = 2\varepsilon - 2\varepsilon^2.$$

This matches the §2.3 formula (3.9) exactly. The "at-least-one-flip" probability that the naive §3.3 formula computes is

$$\Pr[\text{at least one flip}] = 1 - (1-\varepsilon)^2 = 2\varepsilon - \varepsilon^2,$$

which includes the event that both stages flip—an event that *cancels* to correctness. The composition theorem counts this event as an error; the BSC does not. Hence the overestimate.

Summary of the distinction. The composition theorem formula $\eta_{\text{total}} = 1 - \prod(1 - \eta_i)$ answers the question: *what is the probability that at least one stage in the chain makes an error?* For the identity-with-error model—where a single error corrupts the end-to-end output irrecoverably—this is also the end-to-end error rate. But for the BSC, two errors cancel, so the relevant question is: *what is the probability that an odd number of stages flip?* The BSC composition formula from Section 2.3 answers this second question.

The two questions agree only in special cases:

- **One-sided error channels: FNR $= 0$.** If $\omega = 0$ (a positive approximate set), then stage i can only make errors in one direction: a false positive cannot be "uncorrected" by a later false positive, because the false positive takes a $\perp$ input to $\top$, and a subsequent channel with FNR $= 0$ will report $\top$ correctly. There is no cancellation in the positive direction. The composition theorem formula gives the exact accumulated false-positive rate.

- **One-sided error channels: FPR $= 0$.** Symmetrically, if $\varepsilon = 0$, the only errors are false negatives, and two false negatives do not cancel (a missed positive remains missed). The composition theorem applies.

- **AND / OR gates.** As shown above, AND with FNR rates and OR with FPR rates are both in the identity-with-error regime in their respective dominant error direction. No cancellation is possible.

- **Miller-Rabin's k-round test.** The Miller-Rabin witness test has FNR $= 0$ per round (it never falsely declares a prime composite). The only error is a false positive, and a false positive cannot be "uncanceled" by subsequent rounds—each round independently either detects the compositeness or doesn't. The composition theorem applies directly: $\varepsilon_k \leq (1/4)^k$. We work through this in Section 3.4.

For the BSC, neither condition holds: the BSC has $\varepsilon = \omega$, and two flips cancel. The correct composition formula is the one in Section 2.3: matrix product, not the composition theorem of this section. This is not a contradiction; it is two different theorems answering two different questions about two different models.

The channel-matrix product view

The matrix-product computation in Section 2.3 gives the BSC composition formula (with cancellation): two flips that cancel return the chain to the correct answer, and the resulting end-to-end crossover is $\varepsilon_1 + \varepsilon_2 - 2\varepsilon_1\varepsilon_2$, not $1-(1-\varepsilon_1)(1-\varepsilon_2)$. The composition theorem of this section applies to a different underlying model: each stage either preserves correctness or destroys it, with no possibility of recovery ("once lost, stays lost"). The BSC's bidirectional flip structure is incompatible with this monotone correctness assumption. The two formulas describe different physical processes; neither is wrong, and they answer different questions about different models.

Sister structure: Kronecker for parallel queries

The composition theorem governs *serial* chains: one query passes through k stages in sequence. The Kronecker factorization theorem (Theorem 4.4.1) governs *parallel* queries: m queries on m distinct elements, each through its own copy of the channel.

In the serial case (this section), correctness probabilities multiply along the chain:

$$\eta_{\text{total}} \;=\; \eta_1 \cdot \eta_2 \cdots \eta_k.$$

In the parallel case (Section 4.4), the joint confusion matrix factors as a Kronecker product:

$$Q^{(m)} \;=\; Q_1 \otimes Q_2 \otimes \cdots \otimes Q_m.$$

These two factorizations arise from different independence structures. The serial factorization uses *chain-stage independence*: the error events at distinct stages of the pipeline are independent. The Kronecker factorization uses *Axiom 1* (Axiom 1): the error events at distinct input elements are independent. The two independence assumptions can both hold simultaneously—and in well-designed systems, they do—but they are logically distinct.

An important practical consequence: when k parallel pipelines each consist of multiple stages in series, the combined channel matrix is

$$\left(Q_2^{(1)}Q_1^{(1)}\right) \otimes \left(Q_2^{(2)}Q_1^{(2)}\right) \otimes \cdots,$$

as Section 2.3 noted. The serial composition (matrix product) applies *within* each pipeline; the Kronecker product applies *across* parallel pipelines. The composition theorem of this section computes the correctness probability

of each pipeline's serial chain; the Kronecker theorem assembles the joint confusion matrix for the m parallel pipelines together.

A note on chain-stage independence vs. element-wise independence

The composition theorem assumes chain-stage independence. Axiom 1 (Axiom 1) guarantees element-wise independence: error events at distinct input elements are independent. These are closely related but operationally distinct.

In the Miller-Rabin example of Section 3.4, the k witness evaluations are drawn from a pool of witnesses. If we treat each witness as a distinct element of a predicate over the witness pool—which is the right recasting, as Section 3.4 details—then Axiom 1 licenses independence across witnesses. That independence is also chain-stage independence for the k-AND composition: the k AND stages each involve a distinct element, and their error events are independent by Axiom 1. In this setting, Axiom 1 implies chain-stage independence, and the composition theorem follows.

In contrast, consider a pipeline where stage i and stage j query the *same* element of the same Bernoulli predicate. If the implementation is deterministic (same input, same output), the two stages are perfectly correlated, not independent. Axiom 1 does not apply across repeated queries of the same element (Chapter 16 discusses this negative space). The composition theorem would not apply either: the chain-stage independence assumption fails when stages share elements.

The honest position is this: chain-stage independence is a structural property of the pipeline, not a consequence of Axiom 1 alone. In well-behaved implementations where each stage of the pipeline queries a distinct element, Axiom 1 implies chain-stage independence and the composition theorem holds. In pipelines where stages share elements or are correlated through shared randomness, the theorem requires additional justification or does not apply.

Worked numerical example: a 4-stage AND chain

To make the composition theorem concrete before we deploy it in Miller-Rabin, here is a worked example with four stages.

Let $\tilde{b}_1, \tilde{b}_2, \tilde{b}_3, \tilde{b}_4$ be independent Bernoulli[bool] instances, each a positive approximate set (FNR $= 0$), with false-positive rates:

$$\varepsilon_1 = 0.20, \quad \varepsilon_2 = 0.15, \quad \varepsilon_3 = 0.10, \quad \varepsilon_4 = 0.25.$$

We take their four-fold AND: declare $\top$ only if all four report $\top$.

Since $\omega_i = 0$ for all stages (positive approximate set), the four-fold AND has $\omega_\wedge = 0$ as well: the AND never falsely suppresses a true $\top$ (every stage correctly passes $\top$ inputs through).

The composition theorem gives the false-positive rate of the AND. With $\eta_i = \varepsilon_i$:

$$\eta_{\text{total}} = (1 - 0.20)(1 - 0.15)(1 - 0.10)(1 - 0.25) = 0.80 \times 0.85 \times 0.90 \times 0.75.$$

Computing step by step:

$$\begin{aligned} 0.80 \times 0.85 &= 0.680, \\ 0.680 \times 0.90 &= 0.612, \\ 0.612 \times 0.75 &= 0.459. \end{aligned}$$

So $\eta_{\text{total}} = 0.459$ and:

$$\varepsilon_{\wedge,4} = 1 - 0.459 = 0.541.$$

Is this better than each individual stage? Yes: the individual stages have $\varepsilon_i \in \{0.10, 0.15, 0.20, 0.25\}$. The AND gate has $\varepsilon_{\wedge,4} \approx 0.541$—actually worse than each individual stage on its own.

This counterintuitive result is not a paradox. The stages in this example are quite noisy (ε_i up to 0.25). The AND of four noisy stages accumulates the FNR in the positive direction (here zero, since FNR = 0) and pushes the FPR down *only* when $\varepsilon_i < 0.5$ and the product $\prod(1 - \varepsilon_i)$ is large. With four stages each above 0.10, the product of TNRs is only 0.459, so the AND gate's FPR is 0.541. To get an AND gate with FPR below any individual stage, we need the product of individual TNRs to exceed the best individual TNR, i.e., the other stages must compensate.

For comparison, if all four stages had $\varepsilon = 0.25$:

$$\varepsilon_{\wedge,4} = 1 - (0.75)^4 = 1 - 0.3164 = 0.684.$$

And if all four stages had $\varepsilon = 0.05$:

$$\varepsilon_{\wedge,4} = 1 - (0.95)^4 = 1 - 0.8145 = 0.185.$$

With low per-stage FPR, the AND gate's FPR is substantially lower than any individual stage. This is the regime Miller-Rabin exploits: with $\varepsilon \leq 1/4$ per round, a k-round AND achieves $\varepsilon_k \leq (1/4)^k$.

Looking ahead to Miller-Rabin

The composition theorem establishes the machinery. Section 3.4 deploys it against a canonical example: Miller-Rabin witnesses are Bernoulli[bool] instances with FNR $= 0$ and FPR $\leq 1/4$, so k independent rounds give the bound $\varepsilon_k \leq (1/4)^k$ from (3.7), a direct instance of the framework developed in this section.

3.4 Miller-Rabin: A Worked Walkthrough

Part I has built up a small but complete toolkit: Bernoulli[bool] instances with error rates (ε, ω), the Boolean algebra over noisy bits (Section 3.1), and the composition theorem that governs how error rates accumulate when instances are combined (Section 3.3). This section deploys that toolkit in full against a canonical example from number theory.

The Miller-Rabin primality test is one of the most widely used randomized algorithms in practice. It decides, with adjustable confidence, whether an integer n is prime. We will show that the algorithm IS a k-fold Bernoulli[bool] composition. Every formula in the Miller-Rabin analysis—the per-round false-positive bound $\varepsilon \leq 1/4$, the k-round bound $\varepsilon_k \leq (1/4)^k$, and the design rule "choose k large enough"—falls directly out of the framework developed in §§3.1–3.3. The algorithm contributes one non-trivial mathematical fact; everything else is the composition theorem applied.

Step 1: The witness predicate

Fix a positive odd integer $n > 3$ whose primality we wish to decide. A single *Miller-Rabin witness test* proceeds as follows.

Write $n - 1 = 2^r \cdot d$ with d odd and $r \geq 1$. Choose a witness $a \in \{2, 3, \ldots, n-2\}$ uniformly at random. Compute the sequence

$$a^d \bmod n, \quad a^{2d} \bmod n, \quad a^{4d} \bmod n, \quad \ldots, \quad a^{2^r d} \bmod n.$$

Declare n **composite** (the definitive outcome) if none of these values equals $\pm 1 \pmod n$, or more precisely: if $a^d \not\equiv \pm 1$ and $a^{2^j d} \not\equiv -1$ for all $j \in \{1, \ldots, r-1\}$. Declare n **probably prime** (the uncertain outcome) otherwise.

The test returns one of two verdicts:

- **"composite":** n is definitely not prime. No false negatives here—a prime number always passes the test (Fermat's little theorem guaran-

tees $a^{n-1} \equiv 1 \pmod{n}$ for any a coprime to n, and the sequence of square roots must eventually exhibit a trivial square root of 1).

- **"probably prime":** n might be prime, or might be a composite that "fooled" witness a. The latter is possible for composites with large *strong-liar sets*.

This is already the shape of a Bernoulli[bool] instance, as we make precise next.

Step 2: Characterizing the test as Bernoulli[bool] with FNR $= 0$, FPR $\leq 1/4$

The witness test is a predicate on an integer n:

$$W_a(n) \;=\; \begin{cases} \text{composite} & \text{if } a \text{ certifies compositeness,} \\ \text{probably prime} & \text{otherwise.} \end{cases}$$

Identify "composite" with $\bot$ (false, n is not prime) and "probably prime" with $\top$ (positive result, n might be prime). The true value of the predicate is whether n is actually prime.

In these terms, two error events are possible:

- **False negative (ω):** n is prime but $W_a(n) =$ composite. This cannot happen: if n is prime, then $a^d \equiv \pm 1 \pmod{n}$ or some $a^{2^j d} \equiv -1 \pmod{n}$, and the test correctly declares n probably prime. So $\omega = 0$.
- **False positive (ε):** n is composite but $W_a(n) =$ probably prime. This can happen when a is a *strong liar* for n: an element of $\{2, \ldots, n-2\}$ that passes the Miller-Rabin test even though n is composite.

The Miller-Rabin theorem states that for any composite n, the number of strong liars in $\{2, \ldots, n-2\}$ is at most $(n-3)/4$. A uniformly random witness therefore satisfies:

$$\varepsilon \;\leq\; \frac{1}{4}. \tag{3.11}$$

The $1/4$ bound is sharp: some composites (products of two primes of similar size) come close to having $(n-3)/4$ strong liars.

The single-round witness test is therefore a Bernoulli[bool] instance with $\omega = 0$ and $\varepsilon \leq 1/4$ (see Definition 4.5.1 for the formal type).

Predicate recasting: fitting Axiom 1. There is a subtle point here that deserves explicit attention, foreshadowed in Section 4.2 and Axiom 1.

The natural description of the Miller-Rabin test is: fix a single candidate n, and draw k independent random witnesses $a_1, a_2, \ldots, a_k \in \{2, \ldots, n-2\}$. In this framing, randomness enters via the witness draw, not via the input. The input n is *fixed* across all rounds. But Axiom 1 is a statement about distinct *input elements* to a single predicate: the error events at distinct inputs are independent.

To apply Axiom 1 cleanly, we recast the test. Define a new predicate

$$\tilde{P}\colon \{2, \ldots, n-2\} \;\to\; \{\text{composite, probably prime}\}$$

where $\tilde{P}(a)$ is the verdict of the witness test applied to n with witness a. Now the witness pool $\{2, \ldots, n-2\}$ plays the role of the *universe*, and distinct witnesses $a_i \neq a_j$ are distinct input elements. Axiom 1 is the claim that the error event at a_i (the event "a_i is a strong liar for n") is independent of the error event at a_j (the event "a_j is a strong liar for n"). Whether this independence actually holds for a given composite n is a fact about the structure of n's strong-liar set, not something that Axiom 1 grants by fiat. We return to this point in Step 5. What the recasting achieves is structural: distinct witnesses are now distinct input elements, and the Axiom 1 framework applies to the recast predicate $\tilde{P}$ in the same way it applies to any other Bernoulli[bool] predicate.

The k-round Miller-Rabin test is then the AND of k evaluations of $\tilde{P}$: declare "probably prime" only if all k witnesses return "probably prime". This is a k-fold AND of Bernoulli[bool] instances, exactly the operation analyzed in Section 3.1.

Step 3: Applying the composition theorem for k rounds

With the predicate recasting in place, the composition theorem (Section 3.3, Theorem 3.3.1) applies directly.

Each round is a Bernoulli[bool] instance with $\omega_i = 0$ and $\varepsilon_i \leq 1/4$. The k-fold AND has:

False-negative rate of the k-round test. Since $\omega_i = 0$ for all rounds, any prime will pass every round correctly. The AND of zero-false-negative stages has zero false-negative rate:

$$\omega_k \;=\; 1 - \prod_{i=1}^{k}(1-\omega_i) \;=\; 1 - 1^k \;=\; 0.$$

The k-round test never misidentifies a prime as composite, regardless of k.

False-positive rate of the k-round test. For the FNR $= 0$ (one-sided error) case, the composition theorem applies without the BSC cancellation caveat noted in Section 3.3: a false positive at any round cannot be "corrected" by a later round, because subsequent rounds with $\omega = 0$ correctly pass $\top$ inputs through. The k-round false-positive rate satisfies:

$$\varepsilon_k \;\leq\; \varepsilon^k \;\leq\; \left(\frac{1}{4}\right)^k. \tag{3.12}$$

Here the first inequality uses the fact that, when $\omega_i = 0$, the composition theorem's "at least one error" formula for the false-positive rate reduces to the product $\prod_{i=1}^{k} \varepsilon_i$, since each round making a false positive is the only way the chain can make a false positive.

To see this: the k-round AND declares "probably prime" if and only if every round declares "probably prime". For a composite n, this happens if and only if every witness a_i is a strong liar. By the Axiom 1 recasting, the events "a_i is a strong liar" are independent (under the worst-case assumption over n). The probability that all k witnesses are strong liars is at most $(1/4)^k$.

Equivalently: the correctness probability of the k-round test (the probability of correctly declaring a composite as composite) is at least $1 - (1/4)^k$, and the correctness probability of correctly identifying a prime as probably prime is exactly 1.

The three key error bounds for standard choices of k:

k	$\varepsilon_k \leq (1/4)^k$	Approximate
1	$1/4$	0.250
5	$(1/4)^5$	$\approx 9.8 \times 10^{-4}$
20	$(1/4)^{20}$	$\approx 9.1 \times 10^{-13}$

At $k = 20$ rounds, the probability of a false positive is below 10^{-12}: for practical purposes, the test is definitive.

Step 4: Pseudocode and numerical example

Worked example: $n = 341 = 11 \times 31$.

We choose $n = 341$ because it is a pedagogically useful case: 341 is a *Fermat pseudoprime to base 2*, meaning the naive Fermat test ($2^{n-1} \equiv 1 \pmod{n}$?) would incorrectly pass it. Indeed, $2^{340} \equiv 1 \pmod{341}$, so Fermat fails. Miller-Rabin does not.

Setup. We have $n - 1 = 340$. Factoring out powers of 2: $340 = 4 \times 85 = 2^2 \times 85$. So $r = 2$ and $d = 85$.

Algorithm 1: Miller-Rabin primality test (k rounds)

input : n: odd integer > 3; k: number of rounds ($k \geq 1$)
output : COMPOSITE (definitive) or PROBABLY-PRIME (with FPR $\leq (1/4)^k$)

```
1  function MillerRabin(n, k)
2      write n − 1 = 2^r · d with d odd;
3      for i ← 1 to k do
4          choose a uniformly at random from {2, ..., n − 2};
5          x ← a^d mod n;
6          if x = 1 or x = n − 1 then
7              continue ;            // witness a passes; try next
8          end
9          for j ← 1 to r − 1 do
10             x ← x^2 mod n;
11             if x = n − 1 then
12                 break ;            // found −1; witness passes
13             end
14         end
15         if x ≠ n − 1 then return COMPOSITE;
16     end
17     return PROBABLY-PRIME;
```

Witness $a = 2$. Compute the sequence:

$$a^d \bmod n = 2^{85} \bmod 341 = 32.$$
$$a^{2d} \bmod n = 32^2 \bmod 341 = 1024 \bmod 341 = 1.$$

The test checks: is $a^d \equiv \pm 1 \pmod{n}$? We have $32 \not\equiv 1$ and $32 \not\equiv 340 \equiv -1 \pmod{341}$. So we proceed to squaring.

Next, is $a^{2d} \equiv -1 \pmod{n}$? We have $1 \not\equiv 340 \pmod{341}$. We have exhausted all $r - 1 = 1$ squarings without finding -1.

Therefore, $2^{85} \equiv 32 \pmod{341}$ with $32 \neq \pm 1$, and $32^2 \equiv 1 \pmod{341}$. This is a *non-trivial square root of 1* modulo 341: a value $x \neq \pm 1$ with $x^2 \equiv 1$. The existence of a non-trivial square root of 1 is a certificate of compositeness (if n were prime, the only square roots of 1 in $\mathbb{Z}/n\mathbb{Z}$ would be ± 1). The algorithm returns COMPOSITE.

Why does 341 fool Fermat but not Miller-Rabin? The Fermat test checks $2^{340} \equiv 1 \pmod{341}$, which holds—341 is a Fermat pseudoprime. Miller-

Rabin is stronger: it checks not just $a^{n-1} \equiv 1$, but whether the sequence of square roots leading to 1 goes through -1 at some point. For a prime p, every square root of 1 in $\mathbb{Z}/p\mathbb{Z}$ is ± 1. For the composite 341, the Chinese Remainder Theorem produces elements that square to 1 without being ± 1 modulo 341; witness $a = 2$ exposes one of them.

Error bounds for $n = 341$. For a composite with $r = 2$, $d = 85$, and k rounds:

k	$\varepsilon_k \leq (1/4)^k$	Interpretation
1	0.250	1-in-4 chance of passing a random witness
5	$\approx 9.8 \times 10^{-4}$	Better than 1-in-1000
20	$\approx 9.1 \times 10^{-13}$	Effectively certain

In practice, $k = 20$ is the standard for cryptographic applications. At that level the false-positive probability is negligible compared to hardware error rates.

Step 5: The unification thesis

We are now in a position to state plainly what this section has shown.

Miller-Rabin primality testing is not a separate technique that happens to use randomness. It is an instance of the Bernoulli framework applied to a specific mathematical predicate.

- The single-round witness test is a Bernoulli[bool] instance with $\omega = 0$ and $\varepsilon \leq 1/4$. This follows from the Miller-Rabin theorem about strong-liar counts, not from the framework itself.
- The k-round test is the k-fold AND of Bernoulli[bool] witnesses. This is the composition of Section 3.1.
- The bound $\varepsilon_k \leq (1/4)^k$ is the composition theorem (Theorem 3.3.1) applied to the one-sided-error (FNR $= 0$) special case.
- The design rule "choose k large enough to drive ε_k below a target ϵ" is the composition theorem solved for k: $k \geq \log_{1/4}(\epsilon) = -2\log_2(\epsilon)$.
- The correctness of this design rule is guaranteed by the framework given one input from the number theory: $\varepsilon \leq 1/4$ per round.

Every structural step above was a consequence of the framework. The only fact that Miller-Rabin contributes is the 1/4 bound on the per-round

false-positive rate—that is the mathematical content of the Miller-Rabin theorem. The rest is Boolean algebra over noisy bits.

On the independence assumption. The analysis in Step 3 applied Axiom 1 to the recast predicate $\tilde{P}$, treating the error events at distinct witnesses as independent. It is worth being precise about what this means.

The $(1/4)^k$ bound is a *worst-case* bound over all composites n. It says: for any composite n, at most $1/4$ of witnesses are strong liars. Therefore, if we sample witnesses uniformly and independently, the probability that all k land in the strong-liar set is at most $(1/4)^k$.

The independence here is *over the uniform random draw of witnesses*, not a statement about the structure of the strong-liar set itself. For a given composite n with a fixed strong-liar set $L_n \subseteq \{2, \ldots, n-2\}$, the event "$a_i \in L_n$" and the event "$a_j \in L_n$" are independent when a_i and a_j are drawn independently and uniformly from $\{2, \ldots, n-2\}$, because membership of a uniform random element in L_n is independent across draws. This is sampling-with-replacement independence, which holds by construction, not by any special property of L_n.

So the application of Axiom 1 here is clean: we draw distinct witnesses, and the error events are independent by the uniform sampling protocol. The bound $(1/4)^k$ uses both the $|L_n| \leq (n-3)/4$ size bound (the mathematical content of Miller-Rabin) and the uniform independent sampling (the protocol that makes Axiom 1 applicable to the recast predicate $\tilde{P}$). Neither alone is sufficient; both are needed.

What this section was not. We have not given the most general treatment of randomized primality tests. The Las Vegas versus Monte Carlo distinction—and the question of what a *zero-error* primality test would require—belongs to chapter 11 (Chapter 11), where the Bernoulli framework connects to the full theory of randomized algorithms. The present section had one goal: to show that the core of Miller-Rabin is the composition theorem, nothing more.

Looking ahead

The Miller-Rabin example sits at one end of a spectrum. At the other end sits the Bloom filter: also a k-fold Bernoulli[bool] composition, also with a per-round false-positive rate that the composition theorem drives down, but drawn from a different application domain (approximate set membership rather than primality). Section 3.5 will make this connection explicit and show how the Boolean algebra of this chapter generalizes, in a single

structural step, to the algebra of approximate sets.

3.5 Composition is the Bridge to Sets

We have now arrived, via AND, OR, NOT, the composition theorem, and the Miller-Rabin walkthrough, at a complete picture of Boolean algebra under noise. A Bernoulli[bool] instance is a noisy channel on a two-element type. The Boolean operations close this type: AND, OR, NOT applied to Bernoulli[bool] instances produce new Bernoulli[bool] instances with derived error rates. The composition theorem (Section 3.3, Theorem 3.3.1) governs how those rates accumulate when instances are chained. Miller-Rabin (Section 3.4) showed the payoff: one input from number theory, everything else from the composition theorem.

But bool is only one type. Part II of this book builds approximate data structures over richer types: sets, maps, relations. The connection from Bernoulli[bool] to Bernoulli[S] is not a conceptual leap; it is a structural consequence of the same composition machinery developed in this chapter. Three named bridges make this concrete.

Bridge 1: A Bloom filter's m hash functions are m-fold AND composition

A Bloom filter tests set membership approximately. The classical construction uses m independent hash functions $h_1, h_2, \ldots, h_m$, each mapping a query element x to a bit position in a bit array. An element x is declared a member of the set if and only if all m bit positions are set in the array. The m-bit AND of hash outcomes is the filter's membership query.

In the Bernoulli[bool] language: each hash lookup is a Bernoulli[bool] instance. For a true member x, every hash position is set, so each individual lookup returns $\top$ with probability 1 (no false negatives: $\omega_i = 0$ for each hash). For a non-member x, each hash position is set independently with probability $p = 1 - (1 - 1/n)^k$ (where n is the bit-array size and k is the count of elements inserted), giving a per-hash false-positive rate $\varepsilon_i = p$. The Bloom filter's reported membership—"all m hash positions are set"—is the m-fold AND:

$$\bigwedge_{i=1}^{m} \tilde{h}_i(x)$$

where $\tilde{h}_i(x)$ is the i-th hash lookup result. The AND of Bernoulli[bool] instances with $\omega_i = 0$ is itself a Bernoulli[bool] instance (shown in Section 3.1), and its false-positive rate is the product of the individual false-positive rates by the composition theorem:

$$\varepsilon_{\text{filter}} \;\leq\; \prod_{i=1}^{m} \varepsilon_i \;=\; p^m.$$

This is exactly the classical Bloom filter false-positive rate formula, derived here as a direct consequence of the Boolean algebra of noisy bits.

This is not an analogy. A Bloom filter membership query is, definitionally, an m-fold AND of Bernoulli[bool] instances. The derivation above does not add anything beyond what this chapter has already established; it merely identifies the Bloom filter as an instance of a pattern already in hand.

The difference from Miller-Rabin. Both the Bloom filter and Miller-Rabin are m-fold AND compositions of Bernoulli[bool] instances with $\omega = 0$. The structural pattern is identical; the sources differ. Miller-Rabin's per-round $\varepsilon \leq 1/4$ bound is a theorem from number theory. The Bloom filter's per-hash ε is a design parameter controlled by the bit-array size and insertion count. In both cases, the composition theorem (Equation (3.12) for Miller-Rabin; the formula above for the Bloom filter) converts a per-stage guarantee into an end-to-end guarantee.

Chapter 5 develops the Bloom filter and its variants in full, including optimal parameter selection, the standard analysis of false-positive rate as a function of bit-array size, and the failure modes that chapter 16 (Chapter 16) examines.

Bridge 2: Set-operation error rates are AND, OR, NOT error rates

Bridge 1 takes a Bloom filter query—a Bernoulli[bool] instance—and deduces its error rates from the composition theorem. Bridge 2 lifts the same reasoning one level: from queries on individual elements to operations on *sets*.

Let $\tilde{A}$ and $\tilde{B}$ be two approximate sets, each a Bernoulli[S] instance for some universe S. For any element $x \in \mathsf{S}$, the membership indicator $x \in \tilde{A}$ is a Bernoulli[bool] instance with rates $(\varepsilon_A, \omega_A)$, and similarly $x \in \tilde{B}$ has rates $(\varepsilon_B, \omega_B)$.

Set operations on approximate sets are defined pointwise: for each x,

$$\begin{aligned} x \in \tilde{A} \cap \tilde{B} &\text{ is computed as } (x \in \tilde{A}) \wedge (x \in \tilde{B}), \\ x \in \tilde{A} \cup \tilde{B} &\text{ is computed as } (x \in \tilde{A}) \vee (x \in \tilde{B}), \\ x \in \tilde{A}^c &\text{ is computed as } \neg(x \in \tilde{A}). \end{aligned}$$

The error rates for intersection, union, and complement on approximate sets are therefore exactly the AND, OR, NOT error rates derived in Section 3.1 and the sections following it. For intersection:

$$\begin{aligned} \varepsilon_\cap &= \varepsilon_A \cdot \varepsilon_B, \\ \omega_\cap &= 1 - (1-\omega_A)(1-\omega_B) = \omega_A + \omega_B - \omega_A\omega_B. \end{aligned}$$

For union:

$$\begin{aligned} \varepsilon_\cup &= \varepsilon_A + \varepsilon_B - \varepsilon_A\varepsilon_B, \\ \omega_\cup &= \omega_A \cdot \omega_B. \end{aligned}$$

For complement, the rates simply swap:

$$\varepsilon_{A^c} = \omega_A, \qquad \omega_{A^c} = \varepsilon_A.$$

These formulas derive from nothing more than the AND, OR, NOT error rate formulas in chapter 3, applied one element at a time. The per-element membership query is a Bernoulli[bool] instance; the set operation is the Boolean operation on those instances.

This derivation is not the full story. To treat $\tilde{A} \cap \tilde{B}$ as a Bernoulli[S] instance requires that the per-element rates be uniform across S—that $\varepsilon_\cap(x)$ and $\omega_\cap(x)$ be the same for every x. This is exactly the rate-stability condition that chapter 4's axioms address. For well-behaved approximate set data structures (Bloom filters, quotient filters, cuckoo filters), this uniformity holds by construction. For malicious inputs or adversarial query sequences, it may fail; chapter 16 examines those failure modes.

Chapter 6 derives these formulas in full generality, works through the rate uniformity conditions, and shows how the Boolean algebra of Bernoulli[bool] lifts to a Boolean algebra of Bernoulli[S] instances.

Bridge 3: Approximate function composition lifts Boolean composition to maps

Bridges 1 and 2 both remain within the Boolean type: the output of the membership query is a bit. Bridge 3 generalizes to non-Boolean output types.

Let $f\colon X \to Y$ be a function from some input type X to some output type Y. An *approximate function* $\tilde{f}\colon X \to Y$ is a function that may return incorrect values. When $Y = \text{bool}$, this is precisely the Bernoulli[bool] setting: the error events are false positives and false negatives. When Y is an arbitrary type, errors are deviations from $f(x)$ in the output space; their structure depends on Y.

The composition machinery of this chapter generalizes from $Y = \text{bool}$ to arbitrary Y in a single structural step. Suppose we have two approximate functions:

$$\tilde{f}\colon X \to Y, \qquad \tilde{g}\colon Y \to Z.$$

The composed approximate function $\tilde{g} \circ \tilde{f}\colon X \to Z$ accumulates error from both stages. In the bool case, this accumulation is governed by the composition theorem of Section 3.3: the end-to-end error rate is derived from the per-stage rates via $\eta_{\text{total}} = 1 - \prod_i (1 - \eta_i)$. For general Y, the end-to-end error is a convolution of the two error distributions in Y and Z respectively; the Boolean formula is the special case where those distributions are Bernoulli distributions on a two-element type.

When chapter 9 introduces Bernoulli[M] for a map type $M = \text{Map}[X, Y]$, the composition machinery of this chapter is the foundation. Every derivation in the bool case carries over to the map case with Y-valued channels replacing scalar bits: the channel matrix Q grows from 2×2 to $|Y| \times |Y|$, but the composition law $Q_{\text{total}} = Q_k \cdots Q_1$ is unchanged. The Boolean algebra of approximate sets is the $Y = \text{bool}$ fiber of the full map algebra.

This is the sense in which Boolean composition is the bridge: not a metaphor, but a degenerate case. The bool formulas of this chapter are the $Y = \text{bool}$ specializations of formulas that hold for all output types. Chapter 9 will make the generalization explicit.

The chapter in one sentence

The three bridges are not independent observations. They are consequences of a single fact: Boolean operations on Bernoulli[bool] instances produce new Bernoulli[bool] instances, with error rates governed by the composition theorem. Bridge 1 identifies Bloom filters as m-fold AND. Bridge 2 identifies set operations as pointwise Boolean operations on membership indicators. Bridge 3 identifies the bool case as a special fiber of map composition.

Put differently: the composition theorem is a general fact about chained channels. In this chapter we have seen it in the Boolean case. Part II and Part III will see it in more general cases. The machinery does not change; only the output type does.

What comes next. The bibliographic notes (Section 3.5) place the Miller-Rabin analysis, the Bloom filter connection, and the chapter's derivations in the literature. Chapter 4 (Chapter 4), which this chapter has cited throughout, formalizes the independence axioms that our derivations assumed; reading chapters 3 and 4 in sequence closes the loop. Part II then builds on both chapters to develop the full algebra of approximate sets and maps.

Bibliographic Notes

The original Miller-Rabin papers. The primality test at the center of Section 3.4 has two distinct intellectual parents. Miller [Mil76] showed, assuming the Extended Riemann Hypothesis, that a deterministic variant of the witness test runs in polynomial time and is correct for all sufficiently large n. The witness-based structure—compute $a^d \bmod n$ and test for a non-trivial square root of unity—originates in that paper. Rabin [Rab80] removed the unproven hypothesis: his contribution was to show that, when the witness a is chosen uniformly at random from $\{2, \ldots, n-2\}$, the probability that a composite n passes the test is at most $1/4$, unconditionally. This is the bound that makes the composed k-round test practical; no number-theoretic conjecture is required. Chapter 3's Bernoulli reading of Miller-Rabin—treating a single witness as a Bernoulli[bool] instance with $\omega = 0$ and $\varepsilon \leq 1/4$—is a reformulation of Rabin's 1980 result in the language of this text.

Standard textbook treatments. Both the Miller-Rabin algorithm and the broader topic of randomized algorithms receive thorough coverage in two standard graduate texts. Cormen et al. [Cor+09] covers Miller-Rabin in Chapter 31 (Number-Theoretic Algorithms), including a proof of the $1/4$ FPR bound via the structure of strong liars modulo a composite; that proof provides the number-theoretic backing for the Bernoulli characterization in Section 3.4. Mitzenmacher and Upfal [MU17] covers randomized algorithms and probabilistic analysis more broadly, with particular attention to the concentration-of-error phenomena that arise when independent probabilistic tests are composed—the same setting as Section 3.3's chain model. Readers who want the full number-theoretic argument for the $1/4$ bound, or who want to see the Miller-Rabin test situated among other randomized primality tests, should consult either of these references before proceeding to Part III's treatment of randomized algorithms.

The Bernoulli papers and the composition theorem. The composition theorem of Section 3.3—that chaining k identities-with-error gives an end-to-end error rate $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$—is a Boolean-algebra reformulation of Theorem 3.7 in Towell [Tow26a]. That paper states the rate-propagation result in terms of the Bernoulli set model; Chapter 3 restates it at the Boolean-algebra layer, where the "set" is replaced by a single Bernoulli[bool] instance and the composition is explicitly a chain of identities-with-error rather than set-intersection operations. The AND, OR, and NOT rules of Section 3.1 are derived in Towell [Tow26f], which extends the Bernoulli framework from Boolean sets to approximate values over arbitrary algebraic types; the bool case is the base case of that general theory.

Three abstraction layers, one composition algebra. A thread that runs through Chapters 2, 3, and 4 is worth naming explicitly here. The same underlying composition algebra appears at three different abstraction layers:

- *Matrix product layer.* In Section 2.3 (Chapter 2), serial composition of two binary channels corresponds to matrix multiplication $Q_{\text{total}} = Q_2 Q_1$. The composition formula for the crossover probability follows from this product by reading off the $(1, 0)$ entry of the composed channel matrix.
- *Boolean-algebra layer.* In Section 3.3 (this chapter), the same composition appears as $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$ for chained identity-with-error gates. This is the Boolean-algebra rephrasing of the matrix-product formula; for a symmetric channel the two are not identical because BSC composition has a cancellation structure (two flips return to the original), but for one-way identities-with-error the product formula is exact.
- *Kronecker-product layer.* In Section 4.4 (Chapter 4), parallel composition of independent Bernoulli[bool] queries factors the joint confusion matrix as a Kronecker product (Theorem 4.4.1). The Kronecker product is the algebraic expression of the same independence structure that underlies the k-round product formula of Section 3.3.

The three layers are not three separate ideas; they are one idea viewed through three different algebraic lenses. Chapter 4's Bibliographic Notes (Section 4.6) situate the Kronecker layer in its own literature; Chapter 2's Bibliographic Notes (Section 2.4) situate the matrix-product layer. Seeing all three in the same framework is a contribution of this text.

Exercises

1. ⋆**Deriving OR rates from AND via De Morgan.** Let b_1 and b_2 be Bernoulli[bool] instances with false-positive rates ε_1, ε_2 and false-negative rates ω_1, ω_2. Using De Morgan's law $b_1 \vee b_2 = \neg(\neg b_1 \wedge \neg b_2)$ and the NOT rule from Section 3.2 (NOT swaps ε and ω), derive closed-form expressions for the false-positive and false-negative rates of $b_1 \vee b_2$. Verify that your answer matches the direct truth-table derivation given in Section 3.2.

2. ⋆**FPR after a 3-stage AND chain (worked).** Suppose three independent Bernoulli[bool] instances b_1, b_2, b_3 each have false-positive rate $\varepsilon = 0.20$ and false-negative rate $\omega = 0$.

   (a) Using the AND output-rate formula from Section 3.1, compute the false-positive rate ε_{12} of $b_1 \wedge b_2$. (*Hint:* with $\omega = 0$, several terms vanish; the only way to get a spurious $\top$ output is if both inputs report $\top$ when the true answer is $\bot$.)

   (b) Now compute the false-positive rate of $(b_1 \wedge b_2) \wedge b_3$, i.e., the full 3-stage AND.

   (c) Confirm that this agrees with the composition-theorem bound $\varepsilon_k \leq (\varepsilon)^k$ obtained by applying Theorem 3.3.1 in its special FNR-zero form, with $k = 3$ and the per-stage success probability $1-\varepsilon = 0.80$ on negative inputs.

3. ⋆**Computing Miller-Rabin's k for a target FPR.** Miller-Rabin guarantees a per-round false-positive rate of at most $\varepsilon_1 = 1/4$ (Equation (3.11)).

   (a) Using the k-round bound $\varepsilon_k \leq (1/4)^k$ (Equation (3.12)), find the smallest integer k such that $\varepsilon_k \leq 10^{-6}$. (*Hint:* take logarithms base 10 on both sides.)

   (b) Find the smallest k for $\varepsilon_k \leq 10^{-15}$ (a bound suitable for cryptographic applications).

   (c) Suppose a weaker test had $\varepsilon_1 = 1/2$ per round. How many rounds would be needed to achieve $\varepsilon_k \leq 10^{-6}$? Comment on the practical difference.

4. ⋆⋆**Verifying the composition theorem on a 4-stage example.** Consider a chain of four identity-with-error stages, each with per-stage error rate $\eta_i = 0.10$.

(a) Apply the composition theorem $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$ (Equation (3.7)) with $k = 4$ and $\eta_i = 0.10$ to find η_{total}.

(b) Verify the same answer by building up stage by stage: compute $\eta_{1\oplus 2} = 1 - (1 - \eta_1)(1 - \eta_2)$, then $\eta_{1\oplus 2\oplus 3}$, then the full 4-stage result. Confirm each step agrees with the cumulative formula.

(c) Let $p_{\text{correct}} = 1 - \eta_{\text{total}}$ be the probability that all four stages produce a correct output simultaneously. Express p_{correct} as $(1 - \eta)^k$ and evaluate. Interpret the result: what fraction of queries emerge error-free from a four-stage pipeline with 10% per-stage error?

5. ⋆⋆**NOT is its own inverse on** Bernoulli[bool]**.** Recall from Section 3.2 that the NOT operation on a Bernoulli[bool] instance with rates (ε, ω) swaps those rates, producing a new instance with rates (ω, ε).

(a) Apply NOT twice to a Bernoulli[bool] instance with rates (ε, ω). Show algebraically that the resulting rates are again (ε, ω).

(b) Interpret the involution $(\text{NOT})^2 = \text{id}$ in terms of the channel matrix Q from Definition 4.5.1: what matrix operation corresponds to NOT, and why does applying it twice return the original matrix?

(c) Does the same involution hold for AND and OR? That is, is there a notion of "applying AND to itself" that returns the identity? Explain briefly why the answer differs from the NOT case.

6. ⋆⋆**Implement Miller-Rabin in 20 lines and check empirical FPR.** Implement the Miller-Rabin primality test in the language of your choice. The pseudocode in Section 3.4 fits in roughly 15 lines; a full implementation with multi-round looping and result aggregation fits in 20.

(a) Test your implementation on the following inputs and report the outcome for each: $n = 2, 3, 4, 17, 100, 341, 561, 1009$. (Among these, 341 is a Fermat pseudoprime to base 2 but not a Miller-Rabin pseudoprime; 561 is a Carmichael number. Your implementation should correctly declare both composite with enough rounds.)

(b) To estimate the empirical false-positive rate, generate a random sample of 1,000 composite odd integers in the range $[10^4, 10^6]$

(e.g., products of two random primes). Run Miller-Rabin with $k = 1$ round on each and record how many are incorrectly declared "probably prime." Compare your empirical rate to the theoretical bound of $1/4$.

(c) Repeat part (b) with $k = 5$ and $k = 10$ rounds. Plot (or tabulate) empirical FPR vs. k. Does the empirical curve fall below the theoretical bound $(1/4)^k$ at each k?

7. ⋆⋆**Prove the composition theorem assuming Axiom 1.** The composition theorem (Theorem 3.3.1) states that k independent identity-with-error stages in series produce an overall error rate $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$.

   (a) State precisely what independence assumption is needed at each stage of the chain for the per-stage probabilities to multiply. In the language of Section 4.2, what does it mean for chain-stage error events to be mutually independent?

   (b) Prove the formula by induction on k: the base case $k = 1$ is immediate; for the inductive step, use the chain rule and the independence of the k-th stage from all previous stages to factor the joint error probability.

   (c) As noted in Section 3.3, this is *chain-stage independence*, which is distinct from the element-wise Axiom 1 (Axiom 1) even though both involve independence of error events. Write one paragraph explaining the operational difference: Axiom 1 governs *which elements* are queried; chain-stage independence governs *which stage* in the pipeline produces an error on a single query.

8. ⋆⋆**Construct a 3-bit ANDed-confidence example.** Let b_1, b_2, b_3 be three independent Bernoulli[bool] instances representing three noisy sensors, each observing whether a physical quantity x exceeds a threshold (true answer: $\top$). Suppose each sensor has $\varepsilon = 0.15$ and $\omega = 0.05$.

   (a) Compute the false-positive and false-negative rates of the AND of all three sensors, $b_1 \wedge b_2 \wedge b_3$, by successive application of the AND formula from Section 3.1. (*Hint:* compute $b_1 \wedge b_2$ first, then AND with b_3.)

   (b) Compute the false-positive and false-negative rates of the OR of all three sensors, $b_1 \vee b_2 \vee b_3$, using the De Morgan derivation from Section 3.2 or directly.

(c) The AND system emphasizes low FPR; the OR system emphasizes low FNR. In numerical terms, which objective does the ANDed configuration better achieve compared with a single sensor? Which objective is *worse* after ANDing? Justify your answer with the numbers from parts (a) and (b).

9. ⋆⋆⋆**Why the AND derivation in §3.1 tacitly assumes Axiom 2.** Re-read the derivation of the AND output rates in Section 3.1. The derivation treats ε_1 and ε_2 as fixed constants when writing the joint probability that both inputs produce a false $\top$.

   (a) Identify the exact expression in the derivation where the false-positive rates of the two inputs are multiplied together. Why does this step require that ε_1 and ε_2 be independent random quantities given the underlying set A? State which axiom (Axiom 2) licenses that factoring.

   (b) Suppose Axiom 2 (Axiom 2) fails for b_1 and b_2: specifically, suppose the false-positive rate of b_2 is a function of the false-positive rate of b_1, say $\varepsilon_2 = g(\varepsilon_1)$ for some nontrivial function g. Write a corrected expression for the false-positive rate of $b_1 \wedge b_2$ in this dependent case (in terms of $\mathbb{E}[\varepsilon_1 \cdot g(\varepsilon_1)]$ or a similar expectation).

   (c) Give a concrete scenario where this dependence arises naturally: for example, two sensors whose noise levels share a common environmental cause (correlated hardware failures, ambient temperature, etc.). Explain in two or three sentences why the simple product formula would overestimate the FPR reduction achieved by ANDing such sensors.

10. ⋆⋆⋆**De Morgan's laws hold for noisy Boolean gates.** The standard De Morgan laws are $\neg(A \wedge B) = \neg A \vee \neg B$ and $\neg(A \vee B) = \neg A \wedge \neg B$. Show that the same identities hold at the level of error rates for Bernoulli[bool] instances, using the rate-swapping NOT rule from Section 3.2 and the AND rate formulas from Section 3.1.

   (a) Start from the OR-via-De-Morgan derivation in Section 3.2. Write out the step-by-step substitution: apply NOT to each input, apply AND, then apply NOT to the result. Track the false-positive and false-negative rates through each step.

(b) Verify that the resulting rates for $\neg(b_1 \wedge b_2)$ equal those obtained by directly computing the rates of $(\neg b_1) \vee (\neg b_2)$, confirming the first De Morgan law.

(c) In classical Boolean algebra, De Morgan's laws are *definitional*: OR is *defined* to be NOT-AND-NOT. Is the same true in the Bernoulli framework, or is the agreement of rates a theorem that requires proof? Identify any assumption (such as the closure property in Proposition 3.2.1) that the argument needs.

11. $\star\star\star$**Constructing a system where the composition theorem fails.** The composition theorem (Theorem 3.3.1) requires that the per-stage error events be mutually independent. This exercise shows that the theorem can fail when that condition is violated – a preview of the negative-space discussion in Chapter 16.

(a) Construct a two-stage identity-with-error system in which the error events at the two stages are perfectly positively correlated: whenever stage 1 errs, stage 2 errs with probability 1, and whenever stage 1 is correct, stage 2 is also correct. Let the per-stage error rate be $\eta = 0.10$. Compute the true end-to-end error rate η_{total}.

(b) Compare to the composition-theorem prediction $\eta_{\text{total}}^{\text{thm}} = 1 - (1 - \eta)^2$. By how much does the actual error rate exceed the prediction, in absolute terms and as a ratio?

(c) Generalise to k perfectly correlated stages. Show that the actual error rate is η (the per-stage rate) regardless of k, while the composition-theorem prediction approaches 0 as $k \to \infty$. In one paragraph, explain what this failure mode means for designing multi-stage pipelines in practice: when is the independence assumption most likely to be violated, and what structural property of the underlying system causes the correlation?

Chapter 4

Two Axioms and Kronecker Factorization

After this chapter you will be able to:

- State Axioms 1 and 2 of the Bernoulli model formally.
- Prove the Kronecker factorization theorem and derive the parameter count drop from $n(2^m - 1)$ to $2m$.
- Write the formal definition of Bernoulli[bool] and recognize every example from chapters 1–3 as an instance.

4.1 Why Axioms?

The three chapters behind us built something real. We derived the error rates for AND and OR gates operating on noisy bits (Section 3.1), showed that composition of channels produces predictable compound rates, and in Section 3.3 proved the composition theorem: when k independent noisy channels are chained in series, their true-positive rates compose multiplicatively. We then applied that theorem immediately to Miller–Rabin primality testing, watching a per-round false-positive rate of at most $1/4$ collapse to $(1/4)^k$ after k rounds. The derivations were concrete and the results were correct.

But correct derivations can hide assumptions. Chapters 1–3 made *two* independence assumptions so naturally that we did not pause to name them. This chapter names them, states them precisely, and then shows exactly what mathematical structure they license. That structure is a precise struc-

tural theorem showing what those assumptions cost and what they buy; it is the formal payoff for the care we are about to take.

The first hidden assumption: element-wise independence

Recall the composition theorem from Section 3.3. When we wrote the true-negative rate for k channels chained in series as $\eta_{\text{total}} = 1 - \prod_{i=1}^{k}(1 - \eta_i)$, we were treating each channel's contribution to the end-to-end error as independent of every other channel's contribution. One query enters the chain; it passes through channel 1, whose output feeds channel 2, and so on; the overall error rate is a product of independent per-stage terms. No correlation across stages was allowed, and the product formula relied on that fact throughout.

A different scenario where Axiom 1 matters: two classifiers trained on overlapping data from the same source. Their errors on a new input x will tend to correlate: when one misclassifies x, the other is more likely to as well. The product formula breaks. The effective error rate is worse than the product formula predicts, because the errors reinforce rather than cancel.

We will call the assumption that justifies the product formula *Axiom 1.* Roughly, Axiom 1 asserts independence of error events across distinct input elements; the precise statement and its scope appear in Section 4.2. The practical effect is that joint error distributions factor across distinct queries, which is what made the composition theorem's product form possible. When Axiom 1 holds, every formula in chapters 1–3 that relies on independence is automatically justified. When Axiom 1 fails, those formulas become bounds at best and misleading at worst. Chapter 3's Miller–Rabin analysis, for instance, requires Axiom 1: the k witness tests must be independent for the per-round bound to compound as $(1/4)^k$. If the witnesses were correlated, the product would underestimate the true composite error rate.

The second hidden assumption: stable block error rates

The derivation of AND under noise in Section 3.1 used specific values ε and ω, the false-positive and false-negative rates, treating them as fixed properties of each input. We implicitly assumed that querying an input repeatedly would yield the same error behavior each time: that ε does not drift from query to query, that the channel is stationary across uses.

This is not the only conceivable behavior. A Markov-correlated channel, for instance, lets the error rate on query $t+1$ depend on what happened at query t. If the channel just made a false positive, it might be more likely

to make another. The rates are not stable; they are history-dependent. We did not model this, and for good reason: the stationary case is the tractable and practically important one. But we should say we are assuming it.

We have been assuming Axiom 2 since Section 3.1; let us make it explicit. Axiom 2 says: the error rates within each partition block of the input domain are the same for every query, unconditionally. There is one channel matrix Q per input-block-pair, and it does not depend on which query in a sequence we are currently processing. Under Axiom 2, the channel is genuinely memoryless within each block: its statistical behavior is the same on the hundredth query as on the first.

This is a meaningful restriction, not a triviality. It rules out fatigue, warm-up effects, adversarial adaptation, and history dependence. It also makes the math work cleanly. We formalize Axiom 2 in Section 4.3.

What the axioms buy

Once both axioms are in place, something striking follows. The joint confusion matrix describing m independent queries on the channel is not a free matrix with exponentially many parameters. Under Axioms 1 and 2, it *must* factor into a product of m small per-query matrices, one factor per query. The exponential-to-linear collapse in parameter count, made precise in Section 4.4, is the formal payoff of this chapter.

This is not merely an algebraic convenience. It is the theorem that makes the Bernoulli framework useful in practice. Without it, a fifty-query analysis would require specifying an astronomically large matrix. With it, you need fifty pairs of numbers. The saving is exponential.

We prove this Kronecker factorization theorem, derive the parameter count corollary, and work through explicit examples in Section 4.4. The theorem is the chapter's climax and the formal vindication of every computation in chapters 1–3.

The formal definition

Once the axioms and the Kronecker theorem are in hand, we can at last write down the formal definition of the type Bernoulli[bool] without fear of circularity. Chapter 1 named the noisy bit informally. Chapter 2 gave it a channel matrix. Chapter 3 showed how to compose these objects. Chapter 4 closes by showing that all of those things are instances of a single precise definition: a triple (X, Y, Q) where X and Y are the input and output domains $\{\top, \bot\}$, Q is a 2×2 stochastic matrix, and the whole structure

satisfies Axioms 1 and 2. We work this out in Section 4.5, including the biased-coin example, introduced in Section 1.1 and deliberately left without a full PMF in Section 1.3 until we had the formal machinery to handle it.

The trajectory of this chapter, then, is: axioms first (Sections 4.2 and 4.3), then the theorem they make possible (Section 4.4), then the definition the theorem earns us (Section 4.5).

By the end, the reader who worked through chapters 1–3 will recognize every example, every formula, and every derivation as an instance of one coherent definition. That is the point.

4.2 Axiom 1: Element-Wise Independence

The previous section sketched two kinds of independence that chapters 1–3 quietly assumed. This section makes the first of them precise.

A brief note on framing before we begin. In Section 4.1 we used a chain-of-stages example as motivation: k noisy channels in series, with end-to-end error rate $1-\prod_{i=1}^{k}(1-\eta_i)$. That example invokes a kind of independence, but it is not the same statement as Axiom 1. Axiom 1 governs independence across distinct input elements of a *single* approximate predicate. Stage-wise independence in a composition chain is a separate assumption: closely related, often co-occurring, but operationally distinct. Both are needed for the chain formula; both fail in similar circumstances; and in well-behaved implementations both hold. We state Axiom 1 cleanly here, on a single predicate. The reader who wants to recover the chain formula from these axioms should track the two independences separately and combine them as the composition theorem of **??** prescribes.

The formal statement

Let U be the universal set, the domain of all possible inputs to the approximate membership predicate. For a fixed objective set $A \subseteq U$, the predicate either correctly reports the true answer or makes an error. For each $x \in U$, define the error indicator

$$E_x \;=\; \mathbf{1}\big[\tilde{A}(x) \neq \mathbf{1}_A(x)\big],$$

where $\tilde{A}(x)$ is the approximate predicate's output on input x and $\mathbf{1}_A(x)$ is the exact answer. $E_x = 1$ means an error occurred at x; $E_x = 0$ means the answer was correct.

Axiom 1 (Element-wise independence)**.** *The family $\{E_x : x \in U\}$ is mutually independent given the objective set A and the error rates. In particular, for any distinct $x, y \in U$, the pairwise factorization*

$$\mathrm{P}\left(E_x = 1,\, E_y = 1\right) \;=\; \mathrm{P}\left(E_x = 1\right) \cdot \mathrm{P}\left(E_y = 1\right)$$

holds. Furthermore, within each partition block B_i of U (defined formally in Section 4.3), the error indicators $\{E_x : x \in B_i\}$ are identically distributed.

The notation $\tilde{A}(x)$ is deliberately abstract: it stands for any approximate membership test, whether implemented as a Bloom filter, a probabilistic data structure, a noisy sensor, or a randomized algorithm. Axiom 1 is a property of the *process* that generates the answers, not of any particular implementation.

Consequence: factored joint distributions

Axiom 1 is not merely a philosophical assertion. It has an immediate and quantitative consequence for joint query distributions.

Proposition 4.2.1 (Factored joint distribution)**.** *Under Axiom 1, the joint distribution of the error indicators over any finite set of distinct inputs $\{x_1, \ldots, x_n\} \subseteq U$ factors as a product of n marginal distributions:*

$$\mathrm{P}\left(E_{x_1} = e_1,\, \ldots,\, E_{x_n} = e_n\right) \;=\; \prod_{i=1}^{n} \mathrm{P}\left(E_{x_i} = e_i\right)$$

for any choice of $e_1, \ldots, e_n \in \{0, 1\}$. Equivalently, the joint distribution of the observed responses $\tilde{A}(x_1), \ldots, \tilde{A}(x_n)$ factors as

$$\mathrm{P}\left(\tilde{A}(x_1) = s_1, \ldots, \tilde{A}(x_n) = s_n | A\right) \;=\; \prod_{i=1}^{n} \mathrm{P}\left(\tilde{A}(x_i) = s_i | A\right),$$

where each factor equals ε, η, τ, or ω depending on whether $x_i \in A$ and whether s_i is a correct response.

Proof. The first identity is the definition of mutual independence for the family $\{E_x : x \in U\}$, which Axiom 1 asserts directly. Mutual independence of the family $\{E_x : x \in U\}$ (the primary assertion of Axiom 1) implies, by definition, mutual independence of any finite subfamily; hence the joint distribution over $\{x_1, \ldots, x_n\}$ factors into the stated product of n marginals. The second identity follows because $\tilde{A}(x_i) = s_i$ is a deterministic function of

E_{x_i} and the exact answer $\mathbf{1}_A(x_i)$: knowing A determines $\mathbf{1}_A(x_i)$, and given that, $\tilde{A}(x_i) = s_i$ iff E_{x_i} takes the appropriate value. The product therefore inherits its factored form from the independence of $\{E_{x_i}\}$. □

This proposition is the engine behind every multi-query formula in the book. Whenever we write a joint probability over several distinct inputs as a product of marginals, we are using this proposition, which rests on Axiom 1.

The factored joint distribution also licenses a key observation about the joint confusion matrix of m simultaneous queries. Without any independence assumption, that matrix lives in a space of dimension $n(2^m - 1)$ for an n-th order model. Axiom 1 forces the rate-conditional joint matrix to factor as a Kronecker product of per-query 2×2 matrices, collapsing the parameter count to at most $2m$. That is the content of Section 4.4; we state it here as a forward reference so the reader knows why Axiom 1 matters at the level of the whole framework.

Example 1: The Bloom filter's independence assumption

A Bloom filter [Blo70] answers membership queries on a set A stored in a bit array of m bits, using k independent hash functions. When k hashes are applied to a new element x, each hash selects a bit position; x is declared a member if and only if all k selected positions are 1.

For elements $x \notin A$, the probability that all k positions happen to be set (a false positive) is approximately $(1 - e^{-kn/m})^k$, where $n = |A|$. This familiar formula is derived under an explicit assumption: for two distinct non-members x and y, the event "x is a false positive" is independent of the event "y is a false positive." Without this assumption, the formula gives neither an exact probability nor a useful bound; instead, one would need to account for correlations between all pairs of non-members, producing a quantity with no closed form.

Axiom 1 is exactly the assumption the Bloom filter's analysis needs. The filter's k hash functions are chosen to be independent, and the hash values for distinct elements are treated as unrelated. Whether or not this is precisely true in a given implementation, it is the assumption under which the false-positive rate formula is derived, and it is the assumption that the Bernoulli framework formalizes.

We will analyze the Bloom filter fully in Part II (Chapter 5), where Axiom 1 will be verified to hold under the standard random-oracle model for the hash functions. For now, the take-away is structural: the Bloom filter

is a Bernoulli[bool] implementation, and Axiom 1 is the assumption that makes that claim non-trivial. (For a Bloom filter the FNR is degenerate at zero, since a member element always matches all k of its hash positions; the rates-as-random-variables framing of Axiom 1 here governs the per-element independence of FPR error events across distinct non-member queries, not the FNR.)

Example 2: Miller-Rabin's k-round bound requires Axiom 1

In Section 3.4 we derived the Miller–Rabin bound: after k independent rounds of witness testing, the probability that a composite number passes all k rounds (the k-round false-positive rate) is at most $(1/4)^k$. The bound rests on two facts: each round's false-positive probability is at most $1/4$, and the rounds are independent.

There is a subtle reframing required to fit Miller-Rabin under Axiom 1. The predicate being tested takes a *single* fixed input n (the integer in question), and randomness enters via the witness draw rather than via the input. To apply Axiom 1 cleanly, recast the test as a predicate $\tilde{P}\colon \{2, \ldots, n-2\} \to \{\text{composite}, \text{probably prime}\}$ on the witness pool: now distinct witnesses *are* distinct input elements, and Axiom 1's claim is that error events at distinct witnesses (the events "w_i is a strong liar for n") are independent. Whether or not this independence holds for any particular composite n depends on the structure of n's strong-liar set, a separate fact about Miller-Rabin's mathematical content.

With that reframing in place, the connection to Axiom 1 is clean. Each round tests a different witness w_i drawn uniformly at random from $\{2, \ldots, n-2\}$. Whether w_i happens to be a strong liar for n (the error event at w_i) must be independent of whether w_j is a strong liar (the error event at w_j), for $i \neq j$. The error events are the events that the witness fails to detect compositeness.

Under Axiom 1, the joint probability that all k witnesses are strong liars is the product of the individual probabilities:

$$\mathrm{P}\,(\text{all } k \text{ witnesses are strong liars}) \;\leq\; \prod_{i=1}^{k} \frac{1}{4} \;=\; \left(\frac{1}{4}\right)^{k}.$$

Without Axiom 1, the k error events could be positively correlated. The events "w_1 is a liar" and "w_2 is a liar" may be positively correlated, because both witnesses are sampled from the same composite n, which may itself happen to have an unusually large strong-liar set. Conditioning on

n, the witnesses are not independent of each other when the liar fraction is unknown. In that case, the product bound $(1/4)^k$ would underestimate the true false-positive probability. The correct expression would involve the joint distribution over all k witnesses, which does not simplify to a product without an independence assumption.

The Miller–Rabin analysis works because the witnesses are drawn uniformly and independently from a large set. In the worst case, at most $1/4$ of all candidates are strong liars; with independent uniform sampling, the fraction is driven down exponentially. This is Axiom 1 at work.

Classifier-verdict callback: a related independence failure

The classifier-verdict thread from Section 4.1 returns here, but with a deliberate caveat. Two correlated classifiers evaluating the *same* input is not literally an Axiom 1 failure: Axiom 1 governs distinct *inputs* to a single predicate, not distinct *predicates* on a single input. The example we work below is one step removed from Axiom 1, but it illustrates the same family of phenomena: when error events are correlated for any reason, the naive product formula underestimates the joint failure rate. The Axiom 1 case (correlated errors at distinct inputs to one predicate) is structurally identical.

Suppose two classifiers C_1 and C_2 are both trained on a shared dataset with overlapping features. Each classifier is a Bernoulli[bool] individually: on any given input x, C_1 has false-positive rate ε_1 and C_2 has false-positive rate ε_2.

Now consider their joint behavior on a single negative input $x \notin A$. If the errors were independent, the probability that both make a false positive would be:

$$\mathrm{P}\left(C_1(x) = 1,\, C_2(x) = 1 | x \notin A\right) \stackrel{?}{=} \varepsilon_1 \cdot \varepsilon_2.$$

But the two classifiers share training data, so their errors on x tend to be correlated.

Let $\rho \in [0, 1]$ be the correlation between the error indicators $E_x^{(1)}$ and $E_x^{(2)}$. The joint probability is

$$\mathrm{P}\left(E_x^{(1)} = 1,\, E_x^{(2)} = 1\right) \;=\; \varepsilon_1\varepsilon_2 \;+\; \rho\sqrt{\varepsilon_1(1-\varepsilon_1)\cdot\varepsilon_2(1-\varepsilon_2)}.$$

For a concrete example, let $\varepsilon_1 = \varepsilon_2 = 0.10$ and $\rho = 0.60$. The product bound predicts a joint false-positive rate of $0.10 \times 0.10 = 0.01$. The actual joint rate is

$$0.01{+}0.60{\times}\sqrt{0.10 \times 0.90 \times 0.10 \times 0.90} \;=\; 0.01{+}0.60{\times}0.09 \;=\; 0.01{+}0.054 \;=\; 0.064.$$

The inflation factor is $0.064/0.01 = 6.4$: the true joint false-positive rate is $6.4\times$ the naive product.

This is not a failure of the individual classifiers, which are each well-behaved Bernoulli[bool] objects; it is a failure of the independence assumption between them. The Axiom 1 violation has the same arithmetic; only the source of the correlation differs.

The twin lessons are these. First, Axiom 1 is a genuine modeling assumption, not an automatic consequence of each individual predicate being a Bernoulli[bool]: the individual marginals can be well calibrated while the joint distribution fails to factor. Second, correlated errors are worse than independent errors in a precise, quantifiable sense: the inflation factor grows with the correlation coefficient and with the individual error rates.

The Kronecker theorem proved in Section 4.4 makes this precise at the level of the joint confusion matrix: under Axiom 1, the matrix has a structured Kronecker-product form that encodes exactly the factored joint distribution in Proposition 4.2.1. Without Axiom 1, the matrix is unstructured, the parameter count explodes, and the elegant formulas of chapters 1–3 must be replaced by unwieldy joint distributions over all pairs and triples of query events.

4.3 Axiom 2: Conditional Independence of Block Error Rates

Section 4.2 gave the first independence axiom: error events at distinct inputs are mutually independent, given the objective set and the error rates. That axiom governs the *spatial* structure of errors, the relationship between different elements of the universal set queried simultaneously or in any order.

This section gives the second axiom. Axiom 2 governs how the random error rates at different blocks relate to each other: knowing the rate on one block tells you nothing about the rate on another. This is a *cross-block independence* claim, not a within-block or across-queries stability claim. Combined with Axiom 1, however, it implies a stability we will derive as a formal consequence in Proposition 4.3.1: per-block rates are constant across queries. We state the formal axiom first, then derive the stability consequence, then work through what fails when stability fails.

The two axioms are logically independent of each other. Axiom 1 alone does not imply Axiom 2, and Axiom 2 alone does not imply Axiom 1. Both are needed, and their conjunction is exactly what makes the Kronecker factorization theorem of Section 4.4 possible.

Partition blocks and model order

Before stating Axiom 2, we need the partition structure it ranges over. A full treatment appears in Section 2.4; we summarize here the minimum needed for the axiom statement.

Given an objective set $A \subseteq U$, the Bernoulli model partitions the universal set U into blocks $B_1, \ldots, B_n$ such that every element within a block has the same error rate. The *model order* is the number of blocks n. The simplest nontrivial case is order 2: the partition $\{B_1, B_2\} = \{A,\, A^c\}$ separates the positives from the negatives and assigns each group its own error rate.

For the Boolean universe $U = \{\top, \bot\}$ with objective set $A = \{\top\}$, model order 2 yields two blocks:

$$B_1 = \{\top\} = A,$$
$$B_2 = \{\bot\} = A^c.$$

Elements of B_1 have false-negative rate ω (they test positive correctly with probability τ, incorrectly with probability ω). Elements of B_2 have false-positive rate ε (they test negative correctly with probability η, incorrectly with probability ε). The channel matrix Q introduced in Section 2.1 captures exactly these two rates:

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix},$$

where row i describes the output distribution for an element in block B_i.

In general, an n-th order model has n blocks and a corresponding channel matrix with n distinct rows, one per block. Axiom 2 is a statement about those rows.

The formal statement

Axiom 2 (Conditional independence of block error rates)**.** *Let $\{\alpha_1, \ldots, \alpha_n\}$ be the random error rates for the respective partition blocks $B_1, \ldots, B_n$, conditional on the objective set A. The rates $\alpha_1, \ldots, \alpha_n$ are mutually independent:*

$$f(\alpha_1, \ldots, \alpha_n | A) \;=\; \prod_{i=1}^{n} f(\alpha_i | A).$$

In plain language: knowing the error rate on positives tells you nothing about the error rate on negatives, and vice versa. The block rates are independent of each other given the objective set. This applies to all n

blocks simultaneously: no block's rate gives information about any other block's rate.

A few clarifications help prevent common misreadings.

Rates as random variables. The rates α_i are random in the formal sense: they are the *expected* error frequencies, which may themselves vary across different realizations of the data structure or algorithm. A Bloom filter, for instance, has a false-positive rate that depends on the specific set of inserted elements and the hash function choices. Axiom 2 says that the resulting FPR and FNR are independent across blocks.

Stability, not constancy. Axiom 2 does not say the rates are the same for every element within a block (that is the identically-distributed clause of Axiom 1). It says the rates are jointly independent across blocks. The distinction matters: a system could have rates that vary from element to element within a block while still satisfying Axiom 2. The axioms play complementary roles; neither subsumes the other.

The order-2 specialization. In the second-order model, Axiom 2 reduces to a single statement: the random false-positive rate ε and the random false-negative rate ω are conditionally independent given A.[1] The joint density of the two rates factors:

$$f(\varepsilon, \omega | A) \;=\; f(\varepsilon | A) \cdot f(\omega | A).$$

This is the instance that arises in virtually every example in chapters 1–3.

Consequence: one channel matrix per block, stable across queries

Axiom 2 has a structural consequence for how we model repeated queries. Suppose we query an input $x \in B_i$ multiple times: first at step t, then again at step t', and so on. Axiom 2 says the error rate on query at step t does not influence the error rate on query at step t'. Combined with Axiom 1 (which says errors at distinct elements are independent), this means:

Proposition 4.3.1 (Stable channel matrices)**.** *Under Axiom 2, the channel matrix row corresponding to block B_i is the same for every query involving*

[1] The companion paper [Tow26a] specializes Axiom 2 in terms of (FPR, TPR) rather than (FPR, FNR). The two formulations are equivalent: TPR $= 1 -$ FNR is a deterministic transformation, so independence of (FPR, FNR) is the same as independence of (FPR, TPR).

an element of B_i, regardless of the order or history of prior queries. Formally, for any sequence of queries $(x_1, x_2, \ldots)$ with $x_t \in B_{i(t)}$, the conditional distribution

$$\mathrm{P}\left(\tilde{A}(x_t) = s | A,\ x_t \in B_{i(t)}\right)$$

is the same function of $i(t)$ and s for all t, independent of what happened at steps $1, \ldots, t-1$.

Proof. Fix block B_i and a sequence of queries. The distribution of the output at step t for $x_t \in B_i$ is determined by the rate α_i assigned to block B_i. By Axiom 2, α_i is independent of α_j for $j \neq i$. By Axiom 1, the error events at distinct elements are independent. Together, the conditional distribution of the output at step t is a function only of α_i and the block membership of x_t, not of any prior query outcomes.

By Axiom 1, every element of B_i uses the same rate α_i, drawn once at construction time for the data structure. Once that value is realized, the channel row for B_i is a fixed function of α_i at every subsequent query, regardless of the query index t. Marginalizing over the distribution of α_i then produces the same marginal channel row at every query. Because Axiom 1 also makes per-element error events conditionally independent given α_i, every query of an element from block B_i contributes the same row of Q. The Q matrix is therefore time-invariant across the query sequence. Since this holds for all t, the channel matrix row for block B_i is stable across the entire query sequence. □

This proposition is the precise sense in which "there is one channel matrix Q per input-block-pair." The matrix entry Q_{ij} gives the probability that an element of block B_i produces output j. That probability does not drift, does not warm up, and does not depend on how many prior queries have been processed. The channel is genuinely stationary within each block.

The practical consequence is large. Without this stability, we would need a separate channel description for each time step, and the joint distribution over a sequence of queries would require specifying an exponentially growing family of time-varying matrices. With Axiom 2, a single matrix Q suffices for the entire query sequence. This is what makes the Kronecker factorization of Section 4.4 possible: the m-query joint confusion matrix is the Kronecker product of m copies of the same Q, precisely because each query sees the same stable channel.

Worked illustration: the model-order-2 Boolean channel

Let us make Axiom 2's content concrete with the simplest nontrivial case, which will recur throughout the book: the Boolean universe with model order 2.

Set $U = \{\top, \bot\}$ and $A = \{\top\}$, giving two blocks: $B_1 = \{\top\}$ (positives) and $B_2 = \{\bot\}$ (negatives). The channel matrix is

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

Now consider a sequence of three queries: query 1 asks for $\top$, query 2 asks for $\bot$, query 3 asks for $\top$ again. Axiom 2 says the following.

- Query 1 involves $\top \in B_1$. It produces a positive response with probability τ and a negative response with probability ω. These probabilities come from row 1 of Q.
- Query 2 involves $\bot \in B_2$. It produces a positive response with probability ε and a negative response with probability η. These probabilities come from row 2 of Q. *The outcome of query 1 is irrelevant.*
- Query 3 involves $\top \in B_1$ again. It again uses row 1 of Q, with the *same* probabilities τ and ω as query 1. The fact that query 1 may have made an error (false negative) does not change the probability of error on query 3.

This is the channel's stationarity within blocks: the same row of Q applies whenever the same block is queried, no matter what came before.

To write the joint distribution over the three-query sequence $(x_1, x_2, x_3) = (\top, \bot, \top)$ with outcomes (s_1, s_2, s_3), Axioms 1 and 2 together give:

$$\begin{aligned} &\mathrm{P}\left(\tilde{A}(\top) = s_1,\ \tilde{A}(\bot) = s_2,\ \tilde{A}(\top) = s_3 | A\right) \\ &\quad = \; Q_{1,s_1} \cdot Q_{2,s_2} \cdot Q_{1,s_3}, \end{aligned}$$

where $Q_{i,s}$ denotes the (i,s)-entry of Q. The distribution factors as a product of independent channel evaluations, each determined by the block of the queried element. No interaction terms appear; no history from query 1 flows into query 3.

For a concrete numerical example, take $\varepsilon = 0.05$ and $\tau = 0.90$ (so $\omega = 0.10$ and $\eta = 0.95$). The probability that the three-query sequence produces outcomes $(+, -, +)$, that is, two true positives and one true negative, is:

$$Q_{1,+} \cdot Q_{2,-} \cdot Q_{1,+} \;=\; 0.90 \times 0.95 \times 0.90 \;=\; 0.7695.$$

The probability that query 2 produces a false positive while queries 1 and 3 produce true positives is:

$$Q_{1,+} \cdot Q_{2,+} \cdot Q_{1,+} \;=\; 0.90 \times 0.05 \times 0.90 \;=\; 0.0405.$$

Both numbers follow from reading the appropriate rows of Q and multiplying. No additional modeling is needed. The stationarity guaranteed by Axiom 2 is what makes this reading-and-multiplying procedure valid.

Contrast: a Markov-correlated channel violates Axiom 2

To see what Axiom 2 rules out, consider a channel that violates it explicitly. A *Markov-correlated channel* lets the error rate on each query depend on the outcome of the previous query. Concretely:

1. If the previous query produced a correct response, the current query uses a "base" false-positive rate ε_0.
2. If the previous query produced an error, the current query uses an elevated false-positive rate $\varepsilon_1 > \varepsilon_0$ (errors cluster together).

This is a plausible model for hardware memory with correlated faults, or for a sensor that drifts after one bad reading. But it violates Axiom 2.

To see why, consider block $B_2 = \{\perp\}$ (negatives). On query t, the error rate for block B_2 is not a fixed value ε: it is ε_0 if the previous query was correct, and ε_1 if the previous query was an error. The rate α_2 at step t is not independent of the rate at step $t-1$; they are coupled through the Markov chain. The joint density of the rates at different steps does not factor, and Axiom 2's independence condition fails.

Let us put numbers to this. Suppose the base rate is $\varepsilon_0 = 0.05$ and the elevated rate is $\varepsilon_1 = 0.30$. The channel starts in the correct state. Over two successive queries to $\perp$, the joint distribution of outcomes is:

Outcome pair	Probability	Axiom-2 product	Discrepancy
$(-,-)$	$0.95 \times 0.95 = 0.9025$	0.9025	none
$(-,+)$	$0.95 \times 0.05 = 0.0475$	0.0475	none
$(+,+)$	$0.05 \times 0.30 = 0.0150$	0.0025	$6\times$ inflation
$(+,-)$	$0.05 \times 0.70 = 0.0350$	0.0475	deflation

Axiom 2 predicts the joint probability of two consecutive false positives as $\varepsilon_0^2 = 0.05^2 = 0.0025$, since under Axiom 2 the per-block false-positive rate is a fixed constant ε_0 across queries (one Q matrix per block, stable).

The Markov channel produces 0.0150 instead, a factor of 6× higher. The discrepancy is exactly what Axiom 2 forbids: the rate after a previous false positive ($\varepsilon_1 = 0.30$) is six times the rate after a previous correct response ($\varepsilon_0 = 0.05$). The per-block rate is no longer a constant; it depends on history.

The discrepancy is not small. The Markov channel produces error clustering that is invisible to the factored model. If a practitioner were to apply the Kronecker factorization of Section 4.4 to this channel, they would underestimate the probability of back-to-back errors by a factor of six. In applications where error clustering is costly, this is not a tolerable approximation error.

The Markov channel is not a pathological construction: it models a real failure mode. What Axiom 2 asserts is that the Bernoulli framework, in its standard form, is designed for channels where this clustering does not occur. If the channel is known to be Markov-correlated, the model must be extended, for example by conditioning on the hidden state, or by treating the full trajectory as the object to be analyzed. The Bernoulli axioms are then either violated, requiring a richer model, or satisfied at the level of the extended state space, in which case the standard machinery applies after the appropriate state augmentation.

What Axiom 2 earns us

Axiom 1 and Axiom 2 are the full axiomatic foundation of the Bernoulli framework. They are minimal in the sense that neither alone is sufficient for the Kronecker factorization theorem: Axiom 1 gives spatial independence across elements, Axiom 2 gives temporal stability of rates across queries. Together they imply that the joint confusion matrix over any sequence of queries is a Kronecker product of single-query matrices, one factor per query (Section 4.4).

The parametric savings are substantial. Consider the order-2 Boolean predicate that runs throughout this chapter: a general channel on m joint queries with two blocks, and no independence assumption, requires $2(2^m - 1)$ free parameters: one for each joint outcome of m Boolean tests in each of the two rows, save normalization. Under Axioms 1 and 2, the Kronecker structure forces the parameter count down to $2m$: two parameters per query (one per block), with no cross-query parameters at all. For $m = 50$ queries on the order-2 Boolean channel, that is $2m = 100$ parameters under the axioms versus $2(2^{50} - 1) \approx 2.25 \times 10^{15}$ in the general case. The axioms are not merely philosophically satisfying; they are what makes the model

tractable.

One refinement is worth flagging now and pursuing in Section 4.4. The Kronecker structure forces the parameter count down to nm for the rate-conditional channel; the unconditional matrix achieves the same count when no block is queried twice (Corollary 4.4.1.1) or when rates are deterministic (the regime of all worked examples in this book). When two queries hit the same block under genuinely random rates, the unconditional joint involves higher moments of those rates and the simple per-query count no longer applies; Section 4.4 treats this distinction explicitly.

The Kronecker factorization theorem in Section 4.4 makes this precise and derives the parameter reduction as a corollary. For an n-th order model on m joint queries, the general case has $n(2^m - 1)$ free parameters; under the axioms, the count falls to nm.

We prove the Kronecker factorization theorem and its corollary on parameter reduction in Section 4.4. That section is where the full benefit of both axioms becomes visible.

4.4 The Kronecker Factorization Theorem

Two axioms, one payoff

We have named two axioms and shown what each rules out. Section 4.2 gave Axiom 1: error events at distinct inputs are mutually independent. Section 4.3 gave Axiom 2: per-block error rates are conditionally independent of each other, which implies that each block's channel row is stable across queries. Both sections foreshadowed a theorem that their conjunction enables. We now prove it.

The question is this. Given m queries on a Bernoulli predicate, each on a distinct element of the universal set—what does the joint confusion matrix look like? The joint matrix $Q^{(m)}$ is an $n^m \times 2^m$ object: rows are indexed by source-block tuples $(j_1, \ldots, j_m) \in \{1, \ldots, n\}^m$ (one row per ordered selection of source blocks across the m queries), and columns are indexed by binary output patterns $(k_1, \ldots, k_m) \in \{0, 1\}^m$. Without any structural assumption, specifying this matrix requires $n^m(2^m - 1)$ real parameters (one distribution over 2^m output patterns per source-block tuple). For $m = 50$ and $n = 2$, that is over 10^{30} free parameters.

A different, natural parameterization treats only the n distinct source blocks, not all n^m block-tuples, as the basic unit. Under this per-source-block convention the joint conditional distribution over 2^m output patterns is the same for every query from block B_j, giving n distributions of size

2^m and hence $n(2^m - 1)$ free parameters. The parameter counts in this section's corollary and in Section 4.3 are stated under this convention—and it is exactly the convention the axioms enforce, as the corollary explains.

Under Axioms 1 and 2, the per-source-block parameter count drops to nm: n free parameters per query (one per block row), with no cross-query interaction, summed over m queries. In the order-2 Boolean case this is $2m$. This is not an approximation; it is the exact structure forced by independence.

Notation. *The* Kronecker product *of two matrices P and Q, written $P \otimes Q$, is the block matrix formed by replacing each entry P_{ij} with the scaled submatrix $P_{ij}Q$. If P is $r \times s$ and Q is $p \times q$, then $P \otimes Q$ is $(rp) \times (sq)$. The Kronecker product is associative, so $P \otimes Q \otimes R$ is unambiguous without parentheses. We use $\otimes$ for the Kronecker product throughout this section and the rest of the book.*

Theorem 4.4.1 (Kronecker Factorization, conditional form)**.** *Let $\tilde{A}$ be an n-th order Bernoulli predicate satisfying Axiom 1 (Axiom 1) and Axiom 2 (Axiom 2). For any realization of the per-block rates $\alpha = (\alpha_1, \ldots, \alpha_n)$, let $Q_1^*(\alpha), \ldots, Q_m^*(\alpha)$ be the resulting per-query channel matrices, where $Q_i^*(\alpha)$ is the $n \times 2$ stochastic matrix whose (j,k)-entry is the probability, under the realized rate α_j, that a query of an element in block B_j produces output label k on the i-th query. Then the* rate-conditional *joint confusion matrix $Q^{(m)*}(\alpha)$ of m queries on m distinct input elements, whose $(j_1 \cdots j_m,\, k_1 \cdots k_m)$-entry is*

$$\mathrm{P}\left(\tilde{A}(x_1) = k_1, \ldots, \tilde{A}(x_m) = k_m | \alpha,\ x_\ell \in B_{j_\ell} \text{ for each } \ell\right),$$

factors as the Kronecker product

$$Q^{(m)*}(\alpha) \;=\; Q_1^*(\alpha) \otimes Q_2^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha).$$

Corollary 4.4.1.1 (Unconditional factorization, distinct-block queries)**.** *Under the hypotheses of Theorem 4.4.1, if the queried blocks $j_1, \ldots, j_m$ are pairwise distinct (each query hits a different block), the unconditional joint confusion matrix factors as*

$$Q^{(m)} \;=\; Q_1 \otimes Q_2 \otimes \cdots \otimes Q_m,$$

where $Q_\ell \coloneqq \mathrm{E}\left[Q_\ell^*(\alpha)\right]$ is the rate-marginalized per-query channel.

Before the proof, two notes on scope. **First**, the theorem applies to m queries on m *distinct* input elements. Repeated queries of the same

element introduce correlations that Axioms 1 and 2 do not address: the two error events share a single realization of the element's underlying error draw, making them perfectly correlated (in a deterministic implementation) rather than independent. For repeated-query behavior, including the effects of memoization and caching, see Chapter 16 (negative space).

Second, the theorem's primary claim is at the rate-conditional level: given any realization of the per-block rates α, the joint confusion matrix factors. The unconditional matrix factors only when the queried blocks are pairwise distinct (Corollary 4.4.1.1). When two queries hit the same block B_j, both factors involve the same random rate α_j, and the unconditional joint $\mathrm{E}\left[\alpha_j^2\right]$ exceeds the Kronecker prediction $(\mathrm{E}\left[\alpha_j\right])^2$ whenever α_j is non-degenerate (a Jensen gap). The conditional factorization holds always; the unconditional factorization holds when either the rates are realized as constants (the deterministic-implementation case used throughout chapters 1–3) or when no block is queried twice. In the running BSC thread below and in subsequent chapters, we work in the deterministic-rates regime, where $Q_\ell^* = Q_\ell$ and the conditional and unconditional forms coincide.

Proof. We build up the joint distribution by hand and read off the matrix structure.

Step 1: reduce to a joint probability calculation. Index the m query positions as $\ell = 1, \ldots, m$. Position ℓ queries element $x_\ell \in B_{j_\ell}$ and observes output k_ℓ. Fix a realization of the per-block rates $\alpha = (\alpha_1, \ldots, \alpha_n)$. The $(j_1 \cdots j_m, k_1 \cdots k_m)$-entry of the rate-conditional matrix $Q^{(m)*}(\alpha)$ is

$$Q^{(m)*}(\alpha)_{j_1 \cdots j_m,\, k_1 \cdots k_m} \;=\; \mathrm{P}\left(\tilde{A}(x_1) = k_1, \ldots, \tilde{A}(x_m) = k_m | \alpha,\; x_\ell \in B_{j_\ell},\; \ell = 1, \ldots, m\right). \tag{a}$$

We want to evaluate this probability and show it equals $\prod_{\ell=1}^{m} (Q_\ell^*(\alpha))_{j_\ell, k_\ell}$, which is precisely the $(j_1 \cdots j_m, k_1 \cdots k_m)$-entry of $Q_1^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha)$.

Step 2: apply Axiom 2 to fix the per-block rates. By Axiom 2 (Axiom 2), the random rate α_i on block B_i is independent of the random rate $\alpha_{i'}$ on block $B_{i'}$ for $i \neq i'$. By Proposition 4.3.1, once the rates are realized, the rate-conditional channel matrix $Q_\ell^*(\alpha)$ for query ℓ has its row j determined by α_j alone: row j gives the conditional output distribution for an element in block B_j under the realized rate α_j. Write $(Q_\ell^*(\alpha))_{j_\ell, k_\ell}$ for the (j_ℓ, k_ℓ)-entry of this random matrix, the rate-conditional probability that query ℓ from block B_{j_ℓ} returns label k_ℓ given the rate α_{j_ℓ}. Distinct queries on the same block share the same realization of α_{j_ℓ} and hence the same row of $Q_\ell^*(\alpha)$; this row does not depend on what happened at other query positions. When all

m queries share a single realization of the channel family (the common case for a stationary data structure), $Q_1^*(\alpha) = Q_2^*(\alpha) = \cdots = Q_m^*(\alpha) = Q^*(\alpha)$; the theorem's subscript ℓ flags per-query generality, not necessarily distinct matrices.

Step 3: apply Axiom 1 to factor the joint distribution. Conditional on the realized rates $\alpha = (\alpha_1, \ldots, \alpha_n)$, the outputs at positions $\ell = 1, \ldots, m$ are mutually independent, because each output is a deterministic function of the (conditionally independent) error event at x_ℓ. By Axiom 1 (Axiom 1), these error events are mutually independent given the rates and the objective set A. Therefore, conditional on the rates, the joint probability in (a) factors:

$$\begin{aligned} &\mathrm{P}\left(\tilde{A}(x_1) = k_1, \ldots, \tilde{A}(x_m) = k_m | \alpha, x_\ell \in B_{j_\ell}\right) \\ &\quad = \prod_{\ell=1}^{m} \mathrm{P}\left(\tilde{A}(x_\ell) = k_\ell | \alpha_{j_\ell},\, x_\ell \in B_{j_\ell}\right) \\ &\quad = \prod_{\ell=1}^{m} (Q_\ell^*(\alpha))_{j_\ell, k_\ell}. \end{aligned}$$

Step 4: read off the conditional matrix. The expression $\prod_{\ell=1}^{m} (Q_\ell^*)_{j_\ell, k_\ell}$ derived in Step 3 is, by definition, the $(j_1 \cdots j_m,\, k_1 \cdots k_m)$-entry of the Kronecker product $Q_1^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha)$. Since this holds for every index tuple, the rate-conditional joint matrix factors:

$$Q^{(m)*}(\alpha) \;=\; Q_1^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha). \tag{b}$$

This proves Theorem 4.4.1.

To obtain the unconditional version (Corollary 4.4.1.1) under the additional hypothesis that the queried blocks are pairwise distinct, take expectation over $\alpha = (\alpha_1, \ldots, \alpha_n)$:

$$\mathrm{E}\left[\prod_{\ell=1}^{m} (Q_\ell^*)_{j_\ell, k_\ell}\right] \;=\; \prod_{\ell=1}^{m} \mathrm{E}\left[(Q_\ell^*)_{j_\ell, k_\ell}\right] \;=\; \prod_{\ell=1}^{m} (Q_\ell)_{j_\ell, k_\ell}.$$

The first equality uses Axiom 2: when $j_1, \ldots, j_m$ are pairwise distinct, each factor depends on a distinct rate α_{j_ℓ}, and the cross-block independence guaranteed by Axiom 2 makes the factors mutually independent random variables. (When two queries hit the same block, this equality fails; see the second scope note above.) □

Honesty note. The parametric parsimony observation underlying this theorem appears as Remark 3.4 in the companion paper [Tow26a], which notes that Axiom 1 forces the joint confusion matrix to factor as a Kronecker product. The companion paper states this remark in a regime where rates are either deterministic or implicitly conditioned upon; under that reading the remark is correct, and the conditional form of Theorem 4.4.1 above makes the conditioning explicit. Axiom 2's role in this textbook is not to extend the conditional factorization to the unconditional case in general. That is impossible without further hypotheses, since same-block queries on random rates produce a Jensen gap $\mathrm{E}\left[\alpha_j^2\right] > (\mathrm{E}\left[\alpha_j\right])^2$. Axiom 2 instead extends the unconditional factorization to the special case of distinct-block queries (Corollary 4.4.1.1) and, more importantly, certifies the rate-stability that lets us speak of "the" channel matrix Q_ℓ for each query position. The constructive proof above and the explicit conditional/unconditional decomposition are new contributions of this textbook.

Corollary: the parameter count drop

Corollary 4.4.1.2 (Parameter reduction, conditional form)**.** *Under Axioms 1 and 2, the rate-conditional joint confusion matrix $Q^{(m)*}(\alpha)$ of an n-th order Bernoulli predicate queried m times requires at most nm parameters to specify, rather than the $n(2^m - 1)$ parameters required by a general (unconstrained) m-query channel. For the order-2 Boolean predicate this reduces to $2m$ parameters (one per query per block) rather than $2(2^m - 1)$. The same count applies to the unconditional matrix $Q^{(m)}$ when rates are realized as constants (the deterministic-rates regime of all worked examples in this book) or when no block is queried twice (Corollary 4.4.1.1).*

Proof. We count the free parameters in $Q^{(m)*}(\alpha) = Q_1^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha)$.

Constrained case. Under the axioms, $Q^{(m)*}(\alpha)$ factors as $Q_1^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha)$ (Theorem 4.4.1), so each per-query matrix $Q_\ell^*(\alpha)$ contributes its own parameters independently. Each $Q_\ell^*(\alpha)$ is an $n \times 2$ stochastic matrix: rows are indexed by the n source blocks, and columns by the two binary output labels. Each row sums to 1, leaving 1 free parameter per row. With n rows, each $Q_\ell^*(\alpha)$ has n free parameters. By Axiom 2 the per-block rates are independent across queries, so the m matrices share no parameters. Across m queries: nm free parameters in total.

In the order-2 Boolean case ($n = 2$), the two blocks are positives and negatives, parametrized by (τ, ε) per query, or equivalently by (ω, ε). That is 2 parameters per query, for $2m$ parameters in total across m queries.

General case. The full block-tuple matrix has n^m rows and 2^m columns; specifying it freely would require $n^m(2^m - 1)$ parameters. The more compact count uses the per-source-block convention introduced above: without the Kronecker structure but still working source-block by source-block, each of the n source blocks contributes one (unconstrained) conditional distribution over 2^m joint output patterns, requiring $2^m - 1$ free parameters after normalization. Total under this convention: $n(2^m - 1)$. This is the comparison point used here and in Section 4.3, because it reflects the reduction attributable to the axioms alone rather than conflating it with the block-tuple explosion.

Transfer to the unconditional matrix. When per-block rates are deterministic constants (so $Q_\ell^*(\alpha) = Q_\ell$ identically), the conditional and unconditional matrices coincide and the nm count applies to $Q^{(m)}$ directly. When rates are random but the queried blocks are pairwise distinct, Corollary 4.4.1.1 gives $Q^{(m)} = Q_1 \otimes \cdots \otimes Q_m$ with each Q_ℓ contributing n free parameters, again totalling nm. Both regimes cover every worked example in Part I.

Ratio.

$$\frac{n(2^m - 1)}{nm} \;=\; \frac{2^m - 1}{m},$$

which grows exponentially in m. At $m = 50$ and $n = 2$: general case $2(2^{50} - 1) \approx 2.25 \times 10^{15}$; constrained case $2 \times 50 = 100$. □

The corollary matches the closing summary of Section 4.3: the order-2 case requires $2m$ parameters under the axioms versus $2(2^m - 1)$ in the general case; the n-th order case requires nm versus $n(2^m - 1)$. The Kronecker structure, forced by the two axioms, is what produces this reduction.

Running thread: the BSC composed m times

The binary symmetric channel (BSC) introduced in Section 2.1 is the simplest nontrivial instance of the Kronecker theorem. A BSC with crossover probability ε has channel matrix

$$Q \;=\; \begin{pmatrix} 1 - \varepsilon & \varepsilon \\ \varepsilon & 1 - \varepsilon \end{pmatrix}.$$

Since the BSC is symmetric, ε is both the false-positive rate (an element outside A is reported as inside) and the false-negative rate (an element inside A is reported as outside). This is the symmetric special case of the order-2 Bernoulli model: a single crossover parameter ε suffices because the

channel obeys FPR = FNR = ε (the symmetry constraint), and the two rows $[1-\varepsilon,\ \varepsilon]$ and $[\varepsilon,\ 1-\varepsilon]$ are reversals of each other. The model order is 2 because there are two partition blocks (B_1 = positives, B_2 = negatives); the symmetry constraint reduces the two free parameters of the general order-2 model to one.

Now suppose we run m independent queries, each through the same BSC. We work in the deterministic-rates regime announced in the second scope note above, where $Q_\ell^* = Q$ for every query and the conditional matrix of Theorem 4.4.1 coincides with the unconditional one. The joint confusion matrix is then

$$Q^{(m)} \;=\; \underbrace{Q \otimes Q \otimes \cdots \otimes Q}_{m \text{ copies}} \;=\; Q^{\otimes m}.$$

Explicit computation for $m = 2$. Two queries, each through a BSC with crossover ε. The joint input lives in $\{B_1, B_1\} \cup \{B_1, B_2\} \cup \{B_2, B_1\} \cup \{B_2, B_2\}$, where we write B_1 for "in A" and B_2 for "not in A". The joint output lives in $\{(+,+), (+,-), (-,+), (-,-)\}$, where $+$ means "reported in A" and $-$ means "reported not in A".

By the Kronecker product definition, $Q \otimes Q$ is the 4×4 matrix formed by scaling each entry of Q by the full matrix Q:

$$Q^{\otimes 2} \;=\; Q \otimes Q \;=\; \begin{pmatrix} (1-\varepsilon)Q & \varepsilon Q \\ \varepsilon Q & (1-\varepsilon)Q \end{pmatrix}.$$

Expanding each block gives

$$Q^{\otimes 2} \;=\; \begin{pmatrix} (1-\varepsilon)^2 & (1-\varepsilon)\varepsilon & \varepsilon(1-\varepsilon) & \varepsilon^2 \\ (1-\varepsilon)\varepsilon & (1-\varepsilon)^2 & \varepsilon^2 & \varepsilon(1-\varepsilon) \\ \varepsilon(1-\varepsilon) & \varepsilon^2 & (1-\varepsilon)^2 & (1-\varepsilon)\varepsilon \\ \varepsilon^2 & \varepsilon(1-\varepsilon) & (1-\varepsilon)\varepsilon & (1-\varepsilon)^2 \end{pmatrix}.$$

Reading the matrix: the $(1,1)$-entry is the probability that two elements from block B_1 (both positives) are both reported positive, namely $(1-\varepsilon)^2$. The $(1,4)$-entry is the probability that two positives are both reported negative, namely ε^2. The $(2,2)$-entry—one positive, one negative; both reported positive—is $(1-\varepsilon)\varepsilon$: the positive is correctly reported positive $(1-\varepsilon)$ and the negative is falsely reported positive (ε).

For a numerical example, take $\varepsilon = 0.05$.

Joint input blocks	Joint output probabilities			
	$(+,+)$	$(+,-)$	$(-,+)$	$(-,-)$
(B_1, B_1)	0.9025	0.0475	0.0475	0.0025
(B_1, B_2)	0.0475	0.9025	0.0025	0.0475
(B_2, B_1)	0.0475	0.0025	0.9025	0.0475
(B_2, B_2)	0.0025	0.0475	0.0475	0.9025

The BSC thread that opened in Section 2.1 reaches its formal resolution here: m independent queries through the same BSC produce a joint channel whose confusion matrix is the m-fold Kronecker power $Q^{\otimes m}$. Each entry of $Q^{\otimes m}$ is a product of m individual BSC transition probabilities, each equal to ε or $1-\varepsilon$ according to whether that query position produced an error. The Kronecker structure is not a coincidence: it is exactly what Axioms 1 and 2 force.

A 2-query worked verification

We now verify Theorem 4.4.1 with a general order-2 channel to show that the BSC example is not special.

Let the predicate $\tilde{A}$ have false-positive rate ε and false-negative rate ω (so true-positive rate $\tau = 1-\omega$ and true-negative rate $\eta = 1-\varepsilon$). The per-query channel matrix is

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

where row 1 corresponds to positives ($x \in A$, block B_1) and row 2 corresponds to negatives ($x \notin A$, block B_2).

Suppose we query two elements $x_1 \in B_{j_1}$ and $x_2 \in B_{j_2}$ and observe outputs k_1 and k_2 respectively. For concreteness, take both queries to the same channel (both $Q_1 = Q_2 = Q$, since the rates are stationary by Proposition 4.3.1, which relies on both Axioms 1 and 2). As in the BSC thread, we work in the deterministic-rates regime, where the conditional matrix of Theorem 4.4.1 coincides with the unconditional one.

Theorem 4.4.1 predicts:

$$Q^{(2)} \;=\; Q \otimes Q.$$

Let us compute $Q \otimes Q$ directly and then verify each entry against its direct probability calculation.

The Kronecker product $Q \otimes Q$.

$$Q \otimes Q = \begin{pmatrix} \tau \cdot Q & \omega \cdot Q \\ \varepsilon \cdot Q & \eta \cdot Q \end{pmatrix}$$

$$= \begin{pmatrix} \tau^2 & \tau\omega & \omega\tau & \omega^2 \\ \tau\varepsilon & \tau\eta & \omega\varepsilon & \omega\eta \\ \varepsilon\tau & \varepsilon\omega & \eta\tau & \eta\omega \\ \varepsilon^2 & \varepsilon\eta & \eta\varepsilon & \eta^2 \end{pmatrix}.$$

Row order is $(B_1B_1, B_1B_2, B_2B_1, B_2B_2)$ and column order is $(++, +-, -+, --)$.

Direct probability calculation. We verify two entries by direct calculation.

Entry $(B_1B_1, ++)$: two positives, both reported positive. By Axiom 1, the events are independent:

$$\mathrm{P}\left(\tilde{A}(x_1) = +, \tilde{A}(x_2) = + | x_1 \in B_1, x_2 \in B_1\right) \;=\; \tau \cdot \tau \;=\; \tau^2.$$

This matches $(Q \otimes Q)_{11} = \tau^2$. Axiom 1 licenses the product; Axiom 2 ensures each τ factor is the same stable value.

Entry $(B_2B_1, -+)$: negative then positive, first reported negative (correct), second reported positive (correct).

$$\mathrm{P}\left(\tilde{A}(x_1) = -, \tilde{A}(x_2) = + | x_1 \in B_2, x_2 \in B_1\right) \;=\; \eta \cdot \tau.$$

This matches $(Q \otimes Q)_{33} = \eta\tau$ (row 3, column 3; note column 3 is the $(-, +)$ output pattern).

Entry $(B_1B_2, +-)$: positive then negative, both correctly reported. Block $B_1 = \{\top\}$ (positive), output $+$ is a true positive; block $B_2 = \{\bot\}$ (negative), output $-$ is a true negative.

$$\mathrm{P}\left(\tilde{A}(x_1) = +, \tilde{A}(x_2) = - | x_1 \in B_1, x_2 \in B_2\right) \;=\; \tau \cdot \eta.$$

This matches $(Q \otimes Q)_{22} = \tau\eta$. For the mixed true-positive / false-positive entry—output $(+, +)$ when blocks are (B_1, B_2), meaning the positive is correctly reported positive and the negative is falsely reported positive—we need column 1: $(Q \otimes Q)_{21} = \tau\varepsilon$, which equals $\mathrm{P}(+, +|B_1, B_2) = \tau \cdot \varepsilon$. Confirmed.

All entries are products of individual channel probabilities. This is exactly what Theorem 4.4.1 predicts: each row of $Q^{(2)}$ is the tensor product of the corresponding per-query rows, and the resulting matrix has 4 parameters (ε and ω per query, with both queries sharing the same rates by Axiom 2), not the $2(2^2 - 1) = 6$ that a general 2-query channel would need.

What the Kronecker structure buys for §3.1

The AND derivation in Section 3.1 computed the joint error rate of two noisy bits by treating the two error events as independent and multiplying probabilities. We can now see exactly which theorem that derivation was tacitly invoking.

The two-query joint channel for an AND gate is a special case of $Q^{(2)} = Q_1 \otimes Q_2$ where Q_1 and Q_2 are the per-input channel matrices for the two operands. The independence assumed in Section 3.1 was Axiom 1 (the two error events are independent) plus the stability of each operand's rates (Axiom 2). The Kronecker factorization theorem makes this precise: the joint distribution of the two AND operands' channel outputs factors, and the AND gate's output distribution is a deterministic function of that joint distribution. Nothing more was needed than what the two axioms provide.

The formal type is next

The Kronecker theorem is the formal payoff for chapters 1–4's accumulated independence assumptions. We now have a complete characterization of the joint confusion matrix under the Bernoulli framework's two axioms: it is a Kronecker product of per-query channel matrices, one factor per query, with nm parameters total for an n-th order predicate on m queries.

In Section 4.5, we turn this machinery into a formal definition of the Bernoulli[bool] type—the object that chapters 1–3 have been pointing at. The Kronecker theorem is what makes that definition tractable: it tells us that the formal type is parametrized by finitely many numbers (two, for the Boolean case), not by an exponentially large family of joint distributions.

4.5 The Bernoulli[bool] Type, Formally

We have arrived at the formal definition. Four sections of scaffolding have built up to this moment: Section 4.1 explained why axioms matter, Section 4.2 stated that error events at distinct inputs are mutually independent, Section 4.3 stated that per-block error rates are mutually independent across blocks, and Section 4.4 proved that those two axioms together force the joint confusion matrix to factor as a Kronecker product. The formal definition is not an assertion; it is a synthesis of what the preceding chapters have earned.

The definition

Definition 4.5.1 (Bernoulli[bool])**.** *A* Bernoulli[bool] instance *is a triple* (X, Y, Q) *where:*

1. $X = \{\top, \bot\}$ *is the* input domain *(the Boolean universe), with* $B_1 = \{\top\}$ *and* $B_2 = \{\bot\}$ *as the two partition blocks.*
2. $Y = \{\top, \bot\}$ *is the* output domain *(the observed report).*
3. Q *is a* 2×2 *row-stochastic matrix*

$$Q = \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} = \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

where row 1 governs elements in $B_1 = \{\top\}$ *(positives) and row 2 governs elements in* $B_2 = \{\bot\}$ *(negatives), with*

- $\tau \in [0, 1]$*: the true-positive rate (*$\mathrm{P}(\textit{reported } \top | \textit{true value } \top)$*),*
- $\omega = 1 - \tau$*: the false-negative rate (*$\mathrm{P}(\textit{reported } \bot | \textit{true value } \top)$*),*
- $\varepsilon \in [0, 1]$*: the false-positive rate (*$\mathrm{P}(\textit{reported } \top | \textit{true value } \bot)$*),*
- $\eta = 1 - \varepsilon$*: the true-negative rate (*$\mathrm{P}(\textit{reported } \bot | \textit{true value } \bot)$*).*

The triple satisfies Axiom 1 (Axiom 1: error events at distinct inputs are mutually independent) and Axiom 2 (Axiom 2: per-block error rates ε *and* ω *are conditionally independent of each other given the objective set).*

Notation. *We write* Bernoulli[bool] *for the* type*: the collection of all triples* (X, Y, Q) *satisfying the definition above. An individual predicate or data structure is an* instance *of* Bernoulli[bool] *when its behavior matches Definition 4.5.1. The two parameters* (ε, ω)*—or equivalently* (ε, τ)*— uniquely identify the channel matrix* Q *within the type.*

Several remarks on what the definition does and does not say.

Scope. This definition is for the Boolean special case: the input and output domains are both $\{\top, \bot\}$, giving a 2×2 channel matrix with two free parameters. The Bernoulli framework extends to arbitrary algebraic types—Bernoulli[T] for general T—but that extension is Part III material and does not appear here. For now, all instances are Boolean.

The role of the two axioms. The definition invokes both axioms explicitly because both are load-bearing. Axiom 1 (Axiom 1) is what licenses the

factored-product form of joint distributions: queries on distinct Boolean inputs produce independent error events. Without it, the channel matrix Q could not be read as a per-element description; correlation structure across inputs would require a far richer representation. Axiom 2 (Axiom 2) is what makes Q time-invariant: the false-positive rate ε and false-negative rate ω are jointly independent, so knowing one tells you nothing about the other, and neither varies with query history. Both axioms together are what makes Q a compact, two-parameter object sufficient to characterize an entire predicate's behavior—the content of Corollary 4.4.1.2.

The Q convention. Row 1 of Q corresponds to $B_1 = \{\top\}$ (positives); row 2 corresponds to $B_2 = \{\bot\}$ (negatives). Column 1 is the "report $\top$" output; column 2 is the "report $\bot$" output. This convention was introduced in Section 4.3 and used throughout Section 4.4; it is consistent across the entire book.

Biased-coin worked example

The biased-coin example has threaded through this book since Section 1.1. We introduced it there as one of the three faces of a noisy bit—a coin with bias p for heads, where the observation itself may be noisy. In Section 1.3 we looked at the Boolean universe qualitatively. We deferred the full joint-probability treatment to this section. Here it lands.

Setup. Let the coin have bias p: heads ($\top$) with probability p, tails ($\bot$) with probability $1-p$. The observation is noisy: it reports correctly for positives with probability $\tau = 1 - \omega$ and correctly for negatives with probability $\eta = 1 - \varepsilon$.

Example 1 (Biased coin as a Bernoulli[bool] **instance)** *Let $U = \{\top, \bot\}$, with $A = \{\top\}$ the objective set (the event "coin lands heads") and $A^c = \{\bot\}$ the complement (the event "coin lands tails"). Define:*

- *Coin bias: $p = \mathrm{P}(\textit{true value} = \top)$.*
- *Channel: false-positive rate ε (probability of reporting heads when the coin actually landed tails), false-negative rate ω (probability of reporting tails when the coin actually landed heads).*

The joint PMF over all four (true value, observed report) pairs is:[2]

[2]This formulation differs from the companion paper's Example 3.5, which gives the PMF over the realized approximate-set value rather than the joint distribution over true-vs-observed pairs; the two are equivalent but suit different purposes.

Outcome	*True value*	*Observed report*	*Probability*
True positive (TP)	$\top$	$\top$	$p\,(1-\omega) = p\,\tau$
False negative (FN)	$\top$	$\bot$	$p\,\omega$
False positive (FP)	$\bot$	$\top$	$(1-p)\,\varepsilon$
True negative (TN)	$\bot$	$\bot$	$(1-p)\,(1-\varepsilon) = (1-p)\,\eta$

Sanity check:

$$\begin{aligned} p\,\tau + p\,\omega + (1-p)\,\varepsilon + (1-p)\,\eta \;&=\; p(\tau+\omega) + (1-p)(\varepsilon+\eta) \\ &=\; p\cdot 1 + (1-p)\cdot 1 \;=\; 1. \quad \checkmark \end{aligned}$$

The channel matrix is:

$$Q \;=\; \begin{pmatrix} \tau & \omega \\ \varepsilon & \eta \end{pmatrix} \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

The triple $(\{\top,\bot\},\,\{\top,\bot\},\,Q)$ *is a* Bernoulli[bool] *instance whenever* ε *and* ω *are conditionally independent given the coin's objective set* $A = \{\top\}$*, which holds when the observation mechanism operates independently on each side of the partition. The prior* p *over the input is not part of the* Bernoulli[bool] *type itself—the type characterizes the* channel*, not the* source*—but* p *enters when we compute the joint or marginal distribution over outcomes.*

For a concrete instance, take $p = 0.6$*,* $\varepsilon = 0.05$*,* $\omega = 0.10$ *(so* $\tau = 0.90$*,* $\eta = 0.95$*). The four outcome probabilities are:*

Outcome	*Formula*	*Value*
TP	0.6×0.90	0.540
FN	0.6×0.10	0.060
FP	0.4×0.05	0.020
TN	0.4×0.95	0.380
Total		1.000

The channel matrix for this instance is:

$$Q \;=\; \begin{pmatrix} 0.90 & 0.10 \\ 0.05 & 0.95 \end{pmatrix}.$$

Axiom 1 is vacuously satisfied for this single-query example: there is only one input element, so "independence across distinct inputs" has no content. Axiom 1's operational role appears only with multi-query joint distributions (the setup of Theorem 4.4.1), where it justifies the factored joint matrix.

Here we are checking that the per-instance type structure does not violate Axiom 1, not that we exercise it.

Axiom 2 governs the joint distribution over random per-block rates; this concrete instance fixes a single realization $(\varepsilon, \omega) = (0.05, 0.10)$*. The choice of these two values without coupling between them is consistent with Axiom 2's cross-block independence. The full content of Axiom 2 emerges when we consider rates as random variables drawn from a joint distribution; see Proposition 4.3.1 and the closing summary of Section 4.3.*

This triple is a well-formed Bernoulli[bool] *instance.*

The biased-coin example is the simplest general (asymmetric) instance of the type. It has one free parameter in the source (p) and two in the channel (ε, ω). The source prior p controls how often each block is queried; the channel parameters control how reliably each query is reported. The formal Bernoulli[bool] type isolates the channel: it says exactly what can go wrong (and how often) for each kind of input, independently of how often that input arises.

Chapters 1 through 3 as instances

The formal definition gives us a vocabulary for what we have been doing all along. We can now look back at every example in chapters 1–3 and say, precisely, what type each object was.

Chapter 1: the noisy bit (Section 1.1). The three faces introduced in Section 1.1—biased coin, binary symmetric channel, classifier verdict—are all Bernoulli[bool] instances. The biased coin appears above as Example 1. The binary symmetric channel has $\varepsilon = \omega$ (symmetric noise), so its channel matrix is $Q = \left[\begin{smallmatrix} 1-\varepsilon & \varepsilon \\ \varepsilon & 1-\varepsilon \end{smallmatrix}\right]$, a special case of the general Q above. The classifier verdict has ε and ω determined by the classifier's decision boundary and the data distribution—arbitrary in general, but still a Bernoulli[bool] instance whenever Axioms 1 and 2 hold.

In Section 1.3 we walked through the Boolean universe qualitatively, deferring the full PMF treatment to here. That deferral has now paid off: Example 1 gives the PMF table, and Definition 4.5.1 gives the formal type.

Chapter 2: binary channels (Section 2.1, Section 2.2). The binary symmetric channel of Section 2.1 is exactly Bernoulli[bool] with Q symmetric: $\varepsilon = \omega$ reduces the two free parameters of the general Q to one. The formal Bernoulli[bool] type is strictly more general; the BSC lives as a one-parameter subfamily.

The asymmetric binary channel of Section 2.2 is Bernoulli[bool] with Q general: ε and ω may differ freely. The Z-channel (one-sided error) is the extreme case $\omega = 0$ or $\varepsilon = 0$. The Bloom filter, which reports false positives but never false negatives, is the $\omega = 0$ instance: a row-1 channel that always reports $\top$ correctly and a row-2 channel that reports $\top$ with probability $\varepsilon > 0$.

Chapter 3: AND, OR, NOT under noise (Section 3.1, Section 3.2, Section 3.3). The gate-level operations of chapter 3 are operations on Bernoulli[bool] instances that produce new Bernoulli[bool] instances.

AND: if $\tilde{b}_1$ has rates $(\varepsilon_1, \omega_1)$ and $\tilde{b}_2$ has rates $(\varepsilon_2, \omega_2)$, the AND of two noisy bits derived in Section 3.1 produces a new Bernoulli[bool] instance with closed-form ε_{AND} and ω_{AND}. The derivation in Section 3.1 implicitly invoked Axiom 1 (the two error events are independent) and Axiom 2 (the rates are stable across the joint computation). Now that both axioms are named, we can say it precisely: AND is a binary operation Bernoulli[bool] $\times$ Bernoulli[bool] $\to$ Bernoulli[bool].

OR: by De Morgan's duality, the OR of two Bernoulli[bool] instances is also a Bernoulli[bool] instance. Its rates can be derived from AND via De Morgan or computed directly from the truth table of the noisy OR (Section 3.2); either way, the operation has type signature Bernoulli[bool] $\times$ Bernoulli[bool] $\to$ Bernoulli[bool]. AND and OR together generate the rest of Boolean algebra under noise.

NOT: complement swaps the block labels, exchanging ε and ω. Let $\varepsilon' = \omega$ and $\omega' = \varepsilon$ denote the effective rates of the complemented instance. The result is a Bernoulli[bool] instance with

$$Q_{\text{NOT}} \;=\; \begin{pmatrix} 1-\omega' & \omega' \\ \varepsilon' & 1-\varepsilon' \end{pmatrix} \;=\; \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \omega & 1-\omega \end{pmatrix}.$$

Here the new instance's effective false-positive rate is the original ω and its effective false-negative rate is the original ε. NOT is a unary operation Bernoulli[bool] $\to$ Bernoulli[bool].

The composition theorem of Section 3.3—that error rates compose multiplicatively in series—is the special case of Theorem 4.4.1 (the Kronecker factorization) applied to chained Bernoulli[bool] instances. Each stage of the chain is one factor in the Kronecker product; the total joint distribution is the m-fold tensor $Q_1 \otimes \cdots \otimes Q_m$. This is not a coincidence: both rest on the independence structure that the two axioms supply. The composition theorem (matrix product, serial composition) and the Kronecker theorem (tensor product, parallel composition) are sister consequences of the same axiomatic foundation operating in different geometries.

What the formal definition earned

A pedagogy that opens with a formal definition and then works out examples is a different kind of book. This book chose the other order: examples first, patterns second, formal definition last. The benefit is that the definition arrives already supported by intuition—the reader knows what Bernoulli[bool] means before reading the words.

The cost of that ordering is the distance the reader must carry intuition before it is formalized. In Section 1.3, we described the Boolean universe qualitatively and said the full treatment would come later. That deferred a cognitive debt. This section retires it.

Chapters 1–3 built the intuition. Section 4.2 and Section 4.3 named the independence assumptions those chapters were silently invoking. Section 4.4 showed what those assumptions buy in terms of factorization and parameter reduction. Definition 4.5.1 collects all of that into a single, precise statement.

This is the atom-outward trajectory: intuition-first, formalism at the end of Part I rather than the beginning. The formal definition was not asserted; it was discovered, step by step, over four chapters. The reader who has followed those chapters does not need to be persuaded that the definition is the right one. They have seen it emerge.

In Section 4.6, we look ahead at where this definition leads.

4.6 What Comes Next

We have been building toward a single formal object—Bernoulli[bool] as defined in Section 4.5—and we have arrived. This is a good moment to take stock of where we stand and where Part II will take us.

What Part I delivered

Chapter 4 has completed Part I's formal work. The two axioms are now stated with precision (Sections 4.2 and 4.3), the Kronecker factorization theorem shows what those axioms license (Theorem 4.4.1), and Definition 4.5.1 gives Bernoulli[bool] a rigorous identity as the triple (X, Y, Q) under those axioms. Every example in chapters 1 through 3—the biased coin, the binary symmetric channel, the classifier verdict, AND and OR and NOT under noise, the k-round Miller-Rabin bound—is now recognizable as an instance or an operation on instances of this type. The definition was not asserted; it was earned.

Five Core Principles: status after Part I

The Bernoulli framework rests on five organizing principles. At the close of Part I, three are fully in view and two remain ahead.

Two Axioms **[Part I, complete**] Axiom 1 (element-wise independence: Section 4.2) and Axiom 2 (conditional independence of block error rates: Section 4.3) are both stated formally and proved to jointly imply the Kronecker factorization. They are the load-bearing assumptions throughout the rest of the book.

Composition Closure **[Part I, complete**] The composition theorem of Section 3.3 showed that chaining Bernoulli[bool] instances produces a new Bernoulli[bool] instance. AND, OR, and NOT are all closure-preserving operations. The Kronecker theorem of Theorem 4.4.1 is the tensor-product face of the same closure property: whether we compose instances serially (matrix product) or view them in parallel (tensor product), the type is closed. The Boolean universe is self-contained under the operations we care about.

Boolean Thread **[Part I, complete**] Part I chose $\mathsf{Bool} = \{\top, \bot\}$ as the input and output domain and stayed there. Two elements, two partition blocks, two error rates, a 2×2 channel matrix, two free parameters after Corollary 4.4.1.2 applies—the whole Part I story has Boolean structure at every level. This is not a limitation; it is the atom's defining feature. The Boolean thread runs from the biased coin in chapter 1 to the formal definition in chapter 4 without deviation. Part II lifts it to sets; Part III lifts it to maps.

Parametric Parsimony **[visible in Part I, completed in Part II**] The corollary to the Kronecker theorem (Corollary 4.4.1.2) showed the parameter count dropping from $n(2^m - 1)$ to $2m$ under the two axioms. For the Boolean case, that reduction is already dramatic. The full principle—why $2m$ parameters are not merely convenient but are, in an information-theoretic sense, the right number—requires the entropy-based lower bounds developed in Part II, chapter 8. The seed has been planted; the fruit arrives later.

Space-Accuracy Duality **[Part II, chapter 8**] This principle—that there is a fundamental trade-off between the space a Bernoulli structure consumes and the error rates it can achieve—has not yet appeared. Part I did not need it because we were analyzing a fixed Bernoulli[bool]

instance, not asking how small we can make one while meeting an accuracy target. Space-Accuracy Duality is the central organizing principle of Part II. It arrives in chapter 8.

From bits to sets

Part I gave us the atom. Part II asks: what happens when you build a set membership predicate from atoms?

The question is less abstract than it sounds. A set S over a universe U induces, for each element $u \in U$, a Boolean query: is $u \in S$? An *approximate* set replaces exact membership with a noisy channel: the predicate may report $u \in S$ when it should report $u \notin S$ (a false positive) or the reverse (a false negative). The per-element error events are—under Axiom 1—mutually independent. Each element's query is a Bernoulli[bool] instance. The approximate set as a whole is a structured collection of those instances, and its behavior is determined by how those instances are built and indexed.

The canonical first construction is the Bloom filter. A Bloom filter maintains a bit array of m bits and a family of k hash functions; it answers set-membership queries with zero false negatives and a tunable false-positive rate. Understanding the Bloom filter from the Bernoulli perspective—as a mechanism for manufacturing Bernoulli[bool] instances with $\omega = 0$ and a controlled ε—is the project of Chapter 5. The Bloom filter is not a special case we study because it is famous; it is the cleanest first construction because it makes the $\omega = 0$ boundary and the Space-Accuracy Duality both visible in one concrete object.

By the end of Chapter 5, the reader will see not just how the Bloom filter works but why its analysis is a consequence of the formal machinery we developed in Part I.

Part II then extends: chapter 6 studies how set operations (union, intersection, complement) propagate error rates; chapter 7 develops the classification measures that quantify approximate-set quality; and chapter 8 derives the information-theoretic lower bounds that make Space-Accuracy Duality precise. The Boolean thread does not disappear in Part II—it becomes the foundation on which more complex structures are built, one Bernoulli[bool] instance at a time.

Bibliographic Notes

The material in this chapter draws on three streams of literature.

The Kronecker product. The product that carries Leopold Kronecker's name appears in his 1866 work on bilinear forms, though its use in matrix algebra spread gradually through the following century. Kronecker himself was interested in the algebraic structure of determinants and quadratic forms; the representation-theoretic and probabilistic uses of the product came later. For a modern treatment of the Kronecker product's algebraic properties—including the mixed-product property, vectorization identities, and spectral structure—see Horn and Johnson [HJ91], whose Chapter 4 is the standard graduate reference. The formal theorem statement in Section 4.4 of this textbook—that under Axioms 1 and 2 the joint confusion matrix factors as a Kronecker product of per-query matrices—is a new contribution; see the honesty note at the end of Section 4.4 for a precise account of what is new and what descends from the companion papers.

Information-theoretic channels. Chapter 4's proof of the Kronecker factorization uses the language of channels and conditional distributions but does not develop information theory from scratch. Readers who want the theoretical background—binary symmetric channels, channel capacity, and the source-channel coding theorem—should consult Cover and Thomas [CT06]. Chapters 7 and 8 of that work cover binary channels at the depth Part I uses; Chapter 10 covers channel capacity. We borrow the channel-matrix vocabulary and the notion of a memoryless channel (which is, in the Bernoulli setting, precisely what Axioms 1 and 2 enforce), but we do not reproduce Cover and Thomas's development here.

The Bernoulli papers. The two-axiom framework that organizes this chapter comes from Towell [Tow26a]. Axioms 1 and 2, as formally stated in Sections 4.2 and 4.3, are reformulations of the axioms in that paper; the textbook's contribution is to separate them into named sections, give each its own worked examples, and prove the Kronecker factorization theorem as an explicit consequence rather than treating it as a remark. The companion paper presents the factorization in a regime where rates are deterministic or implicitly conditioned upon; the textbook makes the conditional and unconditional forms explicit (Theorem 4.4.1 and Corollary 4.4.1.1), noting that the unconditional form requires either deterministic rates or pairwise-distinct queried blocks. The model-order analysis—the idea that the partition of the input domain into blocks is what governs the parameter count—appears in Towell [Tow26f], which extends the Bernoulli framework from Boolean sets to approximate values over arbitrary algebraic types. The order-2 Boolean

case treated in this chapter is the simplest instance of that general theory.

This textbook reformulates and extends both papers. The formal theorem statement in Section 4.4, the full constructive proof, and the explicit connection between model order and the Kronecker product's parameter count are contributions of this text.

Exercises

1. ⋆**Axiom 1 in measure-theoretic notation.** Let $(\Omega, \mathcal{F}, \mathbb{P})$ be the underlying probability space, and for each $x \in U$ define the error event $E_x = \{\omega \in \Omega : \tilde{A}(x)(\omega) \neq \mathbf{1}_A(x)\} \in \mathcal{F}$. Reformulate Axiom 1 (Axiom 1) using this notation: write out the independence condition both pairwise and for an arbitrary finite family $\{x_1, \ldots, x_k\} \subseteq U$. Does the condition depend on the specific probability measure $\mathbb{P}$, or only on the sigma-algebra $\mathcal{F}$?

2. ⋆**Axiom 2 as a constraint on Q.** Restate Axiom 2 (Axiom 2) as a property of the channel matrix Q. Specifically: in the order-2 Boolean case, Q has two rows. Axiom 2 declares the random quantities governing those rows to be conditionally independent. Express this as a statement about the joint density $f(\varepsilon, \omega | A)$. What behavior does Axiom 2 *forbid* Q from exhibiting across a sequence of queries? (See Proposition 4.3.1 for the stability consequence derived in §4.3.)

3. ⋆**Computing a Kronecker product.** Let

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

   Compute $A \otimes B$ by hand. Verify that $A \otimes B$ is a 4×4 matrix. Then compute $B \otimes A$ and check whether $A \otimes B = B \otimes A$. (*Hint:* the Kronecker product is not in general commutative; the two products will differ, but both will be 4×4.)

4. ⋆⋆**Proving the parameter-count corollary.** Prove Corollary 4.4.1.2 by direct counting.

   (a) Without any independence assumption, an n-th order model on m joint queries has $n(2^m - 1)$ free parameters. Explain where this count comes from.

(b) Under Axioms 1 and 2, the Kronecker factorization (Theorem 4.4.1) forces the parameter count down to nm. Show why: how many free parameters does each per-query matrix Q_i contribute, and why do the m per-query matrices together total nm?

(c) For the order-2 Boolean channel ($n = 2$), compute the ratio $n(2^m - 1)/(nm)$ for $m = 10$ and $m = 50$. Comment on the magnitude of the reduction.

5. ⋆⋆**Verifying the Kronecker theorem by hand for $m = 2$.** Use the asymmetric channel from the biased-coin worked example (Example 1) with $\varepsilon = 0.05$ and $\omega = 0.10$, so $Q = \left(\begin{smallmatrix} 0.90 & 0.10 \\ 0.05 & 0.95 \end{smallmatrix}\right)$.

   (a) Compute $Q \otimes Q$ explicitly. Write out all sixteen entries of the resulting 4×4 matrix.

   (b) For the two-query input pair $(\top, \bot)$, use direct probability to compute $\mathrm{P}\left(\tilde{A}(\top) = s_1,\ \tilde{A}(\bot) = s_2 | A\right)$ for all four outcome pairs $(s_1, s_2) \in \{\top, \bot\}^2$.

   (c) Verify that the entries from (b) match the corresponding row of $Q \otimes Q$ from (a), confirming Theorem 4.4.1 for $m = 2$.

6. ⋆⋆**A Markov channel violates Axiom 1.** The Markov-correlated channel in §4.3 couples the error rate on each query to the outcome of the previous query: if the previous query produced an error, the current query uses elevated false-positive rate $\varepsilon_1 > \varepsilon_0$. Consider two consecutive queries on distinct elements $x, y \in B_2 = \{\bot\}$, with base rate $\varepsilon_0 = 0.05$ and elevated rate $\varepsilon_1 = 0.30$.

   (a) Compute $\mathrm{P}(E_x = 1,\ E_y = 1)$ under the Markov channel, where the channel starts in the correct state.

   (b) Compute $\mathrm{P}(E_x = 1) \cdot \mathrm{P}(E_y = 1)$ under the same parameters.

   (c) Show that these two quantities differ, thereby exhibiting a violation of Axiom 1 (Axiom 1).

   (*Note:* §4.3 frames this example as an Axiom 2 failure; here we recast it explicitly as an Axiom 1 failure. Both framings are valid; the Markov coupling violates both axioms simultaneously.)

7. ⋆⋆**The implicit Axiom-2 invocation in §3.1's AND derivation.** Re-read the derivation of AND under noise in Section 3.1. The derivation writes the joint probability that both inputs to an AND gate produce errors as a product of two individual error probabilities.

(a) Identify the step where Axiom 2 (Axiom 2) is invoked implicitly. Quote or describe the exact expression where the factoring occurs.

(b) Suppose Axiom 2 failed: the false-positive rate on the first input told you something about the false-positive rate on the second. Write an expression for the joint error probability in that case (in terms of the conditional density $f(\varepsilon_2|\varepsilon_1)$).

(c) Would the closed-form AND output rate from §3.1 survive if Axiom 2 failed? Explain in one or two sentences.

8. ⋆⋆**Constructing a** Bernoulli[bool] **from a biased coin.** Following Definition 4.5.1 and Example 1, construct a Bernoulli[bool] instance for a biased coin with bias $p = 0.7$, false-positive rate $\varepsilon = 0.10$, and false-negative rate $\omega = 0.05$.

(a) Write down the channel matrix Q and the joint PMF over all four (true value, observed report) pairs.

(b) Verify Axiom 1 (Axiom 1) in the following sense: the Boolean universe has only two elements ($\top$ and $\bot$), so state what elementwise independence means in this degenerate case and confirm it holds by construction.

(c) Verify Axiom 2 (Axiom 2) in the following sense: state the joint density condition $f(\varepsilon, \omega|A) = f(\varepsilon|A)\, f(\omega|A)$ and explain why it is satisfied when the observation mechanism acts independently on each partition block.

9. ⋆⋆⋆**Counterexample: Axiom 1 does not imply Axiom 2.** Construct a probability model that satisfies Axiom 1 (Axiom 1: error events at distinct elements are mutually independent) but violates Axiom 2 (Axiom 2: per-block error rates are conditionally independent across blocks).

(a) Describe the model explicitly: specify U, the partition blocks, and the joint distribution of error rates across blocks.

(b) Verify that Axiom 1 holds: for any fixed realization of the block rates, the per-element error events are independent.

(c) Show that Axiom 2 fails: the joint density of the block rates does not factor.

(*Hint:* consider a two-block model where the rates ε and ω are drawn from a bivariate distribution with nonzero correlation, such as a Dirich-

let or a copula, while the per-element draws given those rates remain independent.)

10. ⋆⋆⋆**Without Axiom 2, the channel-matrix view collapses.** Show that without Axiom 2 (Axiom 2), the joint distribution over m queries to an element in a fixed block B_i cannot in general be written as a product of m identical channel-matrix rows.

    (a) Give a concrete two-query example where the marginal per-query distribution is consistent with a channel matrix Q, yet the *joint* two-query distribution over outcomes for a single block B_i does not equal $(Q_{i,\cdot})^{\otimes 2}$. (You may use the Markov-correlated channel from §4.3 as your example.)

    (b) What mathematical object would replace the single matrix Q in a model without Axiom 2? Name or describe an appropriate generalization (e.g., a time-varying sequence of matrices, a hidden Markov model, or a conditional channel).

    (c) Explain in one paragraph why the Kronecker factorization theorem (Theorem 4.4.1) depends on Axiom 2 and not only on Axiom 1.

11. ⋆⋆⋆**Tightness of the parameter-count bound.** Corollary 4.4.1.2 states that under Axioms 1 and 2, an n-th order Bernoulli predicate on m queries requires nm free parameters. Argue that this bound is tight: there exist Bernoulli[bool] instances satisfying both axioms that *genuinely* require $2m$ parameters to specify (in the order-2 Boolean case).

    (a) Construct a family of Bernoulli[bool] instances parameterized by m independently chosen channel matrices $Q_1, Q_2, \ldots, Q_m$, each with its own $(\varepsilon_i, \omega_i)$. Verify that this family satisfies both axioms.

    (b) Show that no two distinct members of the family yield the same joint distribution over m queries, so distinct parameter tuples $(\varepsilon_1, \omega_1, \ldots, \varepsilon_m, \omega_m)$ produce distinct joint confusion matrices.

    (c) Conclude that the $2m$ parameters are not merely sufficient but necessary: no reparameterization can reduce the family to fewer than $2m$ free parameters without collapsing distinct members.

Part II

From Bits to Sets

Chapter 5

Approximate Sets and Predicates

After this chapter you will be able to:

- Define an approximate set as a function from a universe U to the noisy-bit atom of Part I, and recognize the membership predicate as a Bernoulli[bool] instance per element.
- Construct a Bloom filter from scratch and compute its empirical and theoretical false-positive rates.
- State the one-sided vs two-sided distinction for approximate sets and identify when each is appropriate.

5.1 From Atom to Set

Part I closed with a promise. Section 1.5 described the noisy bit as the framework's atom and previewed what Part II would do with it: take many of them, arrange them into a structure, and study the algebra of that structure. The single-Boolean object of chapters 1 through 4 would become a building block, and the natural building was a noisy bit per universe element. That object is the approximate set, and Part II opens with it.

Why is the construction inevitable? A classical set $A \subseteq U$ is characterized entirely by its membership predicate: a function $\mathbb{1}_A : U \to \{\top, \bot\}$ that returns $\top$ on the elements of A and $\bot$ everywhere else. Two sets are equal if and only if their predicates agree on every input. The predicate is the set; the set-builder notation $\{x : \mathbb{1}_A(x) = \top\}$ is just the predicate read backwards. Now replace the exact predicate with a noisy one. Each

query $x \mapsto \tilde{A}(x)$ returns a Boolean that is usually but not always the correct membership verdict. Each evaluation is a noisy bit, an instance of the formal type Bernoulli[bool] that Section 4.5 defined. An approximate set is the object whose membership predicate has been made noisy in this way, one query at a time.

The predicate framing

Let U be a fixed universe (finite or countably infinite) and let $A \subseteq U$ be the underlying *objective set* we intend to represent. An *approximate set* of A is a function

$$\mathsf{A}^{\pm} : U \longrightarrow \text{Bernoulli[bool]}$$

such that each evaluation $\mathsf{A}^{\pm}(x)$ is a Bernoulli[bool] instance. Per Definition 4.5.1, a Bernoulli[bool] instance is a triple (X, Y, Q) that produces a Boolean output when sampled; we write $\mathsf{A}^{\pm}(x) \in \{\top, \bot\}$ when referring to the sampled output of $\mathsf{A}^{\pm}$ at x. The notation is the bookkeeping: A is the truth, $\mathsf{A}^{\pm}$ is its noisy representation, and $\mathsf{A}^{\pm}(x) = \top$ is read as "the structure reports that x is a member of A." The structure may be a Bloom filter, a perfect hash filter, a learned classifier with a threshold, or any other implementation whose membership queries return Booleans. What makes it an approximate set, in the sense Part II develops, is that each per-element query is governed by a single Q-matrix from chapter 4: two rates of error, one rate for the positives and one rate for the negatives, and nothing else.

Two paragraphs are worth pausing on. First, the predicate framing is not a definitional choice; it is a recognition. The classical correspondence between a set and its indicator function is exact, and the approximate analogue is forced once we ask what a set looks like when each query may be wrong. The predicate is where the noise lives and there is nowhere else for it to live, because there is no other behavioral handle on a set besides the membership test. Second, the distinction between A and $\mathsf{A}^{\pm}$ is the distinction between truth and report. We never lose access to A at the level of the model: A is the objective set, and the rates of $\mathsf{A}^{\pm}$ are defined against it. What we lose, when we query the structure, is the ability to read A off directly. We see only the report, $\mathsf{A}^{\pm}(x)$, and the rates tell us how that report relates to the underlying truth.

The two error rates

Every query $\mathsf{A}^{\pm}(x)$ is one of two kinds, determined by whether x is in A. If $x \notin A$, then the correct verdict is $\bot$ and the structure errs only by

reporting $\top$. The false-positive rate of the approximate set is the conditional probability of that error:

$$\varepsilon \;=\; \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \top | x \notin A\right).$$

If $x \in A$, then the correct verdict is $\top$ and the structure errs only by reporting $\bot$. The false-negative rate is the conditional probability of that error:

$$\omega \;=\; \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \bot | x \in A\right).$$

These are the two rates that chapter 1 introduced for a single noisy bit (Section 1.1) and that chapter 4 collected into the channel matrix Q (Definition 4.5.1). The change in Part II is one of grouping rather than of substance. A noisy bit had one positive input and one negative input; the approximate set has $|A|$ positive inputs (the members) and $|U \setminus A|$ negative inputs (the non-members). The two-rate description does not grow: the same ε governs every query on a non-member, and the same ω governs every query on a member.

That collapse from many per-element rates to two per-set rates is not automatic. In principle every element $x \in U$ could carry its own rate $\varepsilon(x)$ or $\omega(x)$, and the predicate would need a function-valued description rather than a two-number one. What collapses the description is Axiom 2 of chapter 4 (Axiom 2, Section 4.3): the per-block rates are stable across the block. Membership splits the universe into two blocks, and Axiom 2 forces a single rate per block. Section 5.3 makes this precise. For now, the operational takeaway is that an approximate set is described, at the rate level, by the pair (ε, ω) alone, no matter how large $|U|$ or $|A|$ is.

The block structure

The universe U does not enter the rate description as a flat list of elements. It enters as a two-block partition: the in-set block $B_1 = A$ and the out-of-set block $B_2 = U \setminus A$. Membership induces the partition; the partition determines which rate applies to which query. This is exactly the model-order-two block structure of chapter 4: two blocks, two rate-rows in the channel matrix Q, one row per block. An approximate set is a Bernoulli[bool] instance whose input domain is U rather than $\{\top, \bot\}$, with the membership partition standing in for the Boolean partition of the atomic case. The matrix Q that governs each query is the same:

$$Q \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix},$$

with row 1 for the in-set block and row 2 for the out-of-set block. Section 5.3 writes this out formally and treats the case where the rates depend on x before Axiom 2 collapses them.

The payoff of the block structure is that the per-query description generalizes cleanly to many queries. Chapter 4 (Section 4.4, Theorem 4.4.1) showed that n independent queries on a Bernoulli[bool] instance produce a joint distribution that factors as the Kronecker product of the n per-query channels. The same theorem applies here, with the membership partition in place of the Boolean partition. The Kronecker factorization of n-query behavior also relies on Axiom 1 (Axiom 1), which we silently invoked above: error events at distinct elements of U are mutually independent. Axiom 1 lets us treat the queries as independent draws; Axiom 2 lets us assign the same per-block rate to every draw within a block. Together they give the Kronecker product structure of Theorem 4.4.1. A workload of n queries on $\mathsf{A}^{\pm}$, drawn from the in-set and out-of-set blocks in some specified ratio, has a joint behavior that factors exactly as the tensor product of the per-query matrices. The price of the full joint description is the two rates, not 2^n probabilities. The chapter 4 machinery does not need to be rebuilt for Part II; it transfers over wholesale, and Section 5.3 states the transfer precisely.

Where the chapter goes from here

The construction is so far an abstraction. We have said what an approximate set is, namely a function from a universe to the noisy-bit atom of Part I, and we have named the rates that describe it. We have not yet said how anyone would build one. The most widely deployed construction is the Bloom filter, and Section 5.2 develops it from scratch as the canonical instance of the predicate framing. The construction will have a structural false-negative rate of zero, a tunable false-positive rate, and a closed-form formula for the false-positive rate as a function of three integers (the bit-array size m, the hash count k, and the number of stored items n). The Bloom filter is the example chapters 6 through 8 keep returning to, so getting it on the table first matters.

Section 5.3 then develops the predicate algebra in general. The Bloom filter is one approximate set; the predicate framing covers every approximate set, and the chapter 4 axioms and Kronecker theorem apply uniformly across the family. Section 5.4 draws the line between approximate sets with one structural zero rate and those with two positive rates: a distinction that organizes chapters 6 and 15.

5.2 The Bloom Filter From Scratch

Section 5.1 introduced the approximate set as an abstraction: a function $\mathsf{A}^{\pm} : U \to \text{Bernoulli[bool]}$ whose per-element rates collapse to two per-set numbers. It said nothing about how such a function might be built in finite space. This section answers that. We will construct the Bloom filter from first principles as a bit array plus k hash functions, prove that its false-negative rate is exactly zero by inspection of the construction, derive a closed-form expression for its false-positive rate as a function of three integers, and find the choice of one of those integers that minimizes the rate. The Bloom filter is one specific realization of an approximate set, but it is the realization that chapters 6 through 8 keep returning to, and getting the construction fully on the table first repays itself many times over.

The construction

Fix a universe U, a positive integer m (the number of bits), and a positive integer k (the number of hash functions). The data structure consists of a bit array $b[1\,..\,m]$ of length m, initialized to all zeros, together with k independent hash functions

$$h_1, h_2, \ldots, h_k \;:\; U \longrightarrow \{1, 2, \ldots, m\}.$$

The independence assumed of the h_i is the strong assumption that makes the analysis below tractable: each h_i is treated as a uniform random function from U to $\{1, \ldots, m\}$, and the k functions are mutually independent. Real implementations approximate this with families like double hashing over SHA-256 output, and the divergence between the theoretical and empirical behaviour is small enough that we ignore it through this section.

Definition 5.2.1 (Bloom filter)**.** *The* Bloom filter *of parameters (m, k) is the data structure $(b, h_1, \ldots, h_k)$ together with the two operations:*

- INSERT(x)*: for $i = 1, \ldots, k$, set $b[h_i(x)] \leftarrow 1$.*
- QUERY(x)*: return $\top$ if $b[h_i(x)] = 1$ for every $i \in \{1, \ldots, k\}$; return $\bot$ otherwise.*

After inserting the n elements of a set $A \subseteq U$, the predicate $x \mapsto$ QUERY(x) defines the membership report of the approximate set $\mathsf{A}^{\pm}$.

Algorithm 2 states the two operations in pseudocode. Insert sets k bits. Query reads k bits and returns the conjunction.

Algorithm 2: Bloom filter insert and query

```
1 procedure INSERT(x)
2 |   for i ← 1 to k do
3 |   |   b[h_i(x)] ← 1;
4 |   end
5 function QUERY(x)
6 |   for i ← 1 to k do
7 |   |   if b[h_i(x)] = 0 then return ⊥;
8 |   end
9 |   return ⊤;
```

Both operations run in $O(k)$ time, independent of $|A|$ or $|U|$. The data structure uses exactly m bits of storage, also independent of $|A|$ or $|U|$. The bit array is the only state, and it is never read except via QUERY.

A notation note on n

The Bloom-filter literature universally writes (m, k, n) for the parameter triple, where m is the bit count, k is the hash count, and n is the number of items inserted (the cardinality of A). We adopt that convention here. The symbol n in this section is the set size, not the workload size or the number of independent queries. Chapter 1's n for sample size and chapter 4's n for query count are different objects. Where the context could be ambiguous (especially in chapter 6 once we start composing Bloom filters and querying many items against the composition), we write n_{items} for the set size and n_{queries} for the workload size. Inside the present section the meaning is unambiguous and we write n alone.

Why the false-negative rate is exactly zero

Suppose $x \in A$ has been inserted into the filter. By definition of INSERT, the bits $b[h_1(x)], b[h_2(x)], \ldots, b[h_k(x)]$ were all set to 1 at the time of insertion. Later insertions of other elements can only set additional bits to 1; nothing in the construction ever resets a bit to 0. Therefore at every subsequent time, the bits $b[h_1(x)], \ldots, b[h_k(x)]$ all remain 1, and QUERY(x) reads k ones and returns $\top$.

The argument is structural rather than probabilistic. It does not rely on the choice of hash family, on the cardinality of A, on the bit count m,

or on the workload distribution. It relies only on the fact that INSERT sets bits to 1 and no operation resets them. For every inserted item and every parameter setting,

$$\omega \;=\; \mathrm{P}\left(\textsc{Query}(x) = \bot \mid x \in A\right) \;=\; 0.$$

A Bloom filter is therefore *one-sided*: its only failure mode is the false positive. Section 5.4 will give the one-sided property a name and contrast it with two-sided approximate sets. For now, the point to register is that one of the two error rates of an approximate set, in this construction, has been driven to zero by construction rather than by parameter tuning. The remaining question is the size of the other rate.

The false-positive rate

Consider a query for an element $x \notin A$. The query returns $\top$ if and only if the bits $b[h_1(x)], \ldots, b[h_k(x)]$ are all 1. These k bits are positions in the same bit array that the n insertions populated. Whether the bits are 1 depends on whether the kn hash values produced during insertion ever landed on the same k positions that the query now reads.

The natural quantity to compute first is the probability that a *specific* bit, say $b[j]$ for some fixed j, is still 0 after the n insertions. Each insertion contributes k hash values, each uniformly distributed on $\{1, \ldots, m\}$. The probability that one hash value misses position j is $1 - 1/m$. The probability that all kn hash values miss position j, assuming they are independent, is

$$\mathrm{P}\left(b[j] = 0 \text{ after } n \text{ inserts}\right) \;=\; \left(1 - \frac{1}{m}\right)^{kn} \;\approx\; e^{-kn/m}.$$

The approximation uses $(1 - 1/m)^{kn} = \exp(kn \ln(1 - 1/m))$ and $\ln(1 - 1/m) \approx -1/m$ for large m. The error in the exponent is on the order of $1/m^2$ per insertion, so the substitution is excellent in any regime where m is at all large. The complementary probability is

$$\mathrm{P}\left(b[j] = 1 \text{ after } n \text{ inserts}\right) \;\approx\; 1 - e^{-kn/m}.$$

Call this probability p. Each bit of the array is, at the end of the insertion phase, a Bernoulli random variable with mean p. The bits are identically distributed and the mean p controls how full the array is. The product kn/m is the load: it counts the average number of hash hits per bit.

From bit occupancy to false-positive rate. A query for $x \notin A$ reads k bits at positions $h_1(x), \ldots, h_k(x)$. Each position is uniformly distributed in $\{1, \ldots, m\}$, and each bit at that position has marginal probability p of being 1. The query returns $\top$ if and only if all k read bits are 1. If the k bit-read events were mutually independent, the joint probability would be p^k, and we would conclude

$$\varepsilon \;\approx\; \left(1 - e^{-kn/m}\right)^k. \tag{5.1}$$

On the independence assumption. The k bits read during the query are positions in the same shared bit array, and the same n insertions populated the array. The events "$b[h_i(x)] = 1$" for $i = 1, \ldots, k$ are therefore not strictly independent. Knowing that one queried bit is 1 shifts the posterior on which positions were hit by the kn insertion-time hashes, and that in turn shifts the marginals on the remaining $k - 1$ queried bits. The dependence is positive (one bit being 1 raises the probability that the others are too) and small. Bloom [Blo70] already noted the approximation, and subsequent analyses have quantified the deviation rigorously: the approximation (5.1) is an under-estimate, and the true FPR is bounded by a slightly larger expression involving binomial coefficients. For sparse filters (small load kn/m) the deviation is negligible; for the practical regime $kn/m \approx \ln 2$ that we are about to derive as optimal, the deviation is below a percent of the rate itself, and we do not lose any practical accuracy by adopting (5.1) as the working formula. We flag the assumption explicitly so that the reader is aware of the gap, and we proceed with (5.1) for the optimization that follows. The exercises revisit the issue with a small worked counter-example (Item 7).

Optimal k for fixed m and n

The bit count m is the space budget. The set size n is given by the application. The hash count k is the one design knob the implementer controls. The natural question is: what choice of k minimizes the FPR (5.1) for fixed m and n?

Let $f(k) = (1 - e^{-kn/m})^k$. We minimize over $k > 0$, treating k as a continuous variable; the optimal integer k is then read off as the nearest integer to the continuous optimum. Taking logarithms,

$$\ln f(k) \;=\; k \ln(1 - e^{-kn/m}).$$

Differentiating with respect to k,

$$\frac{d}{dk}\ln f(k) \;=\; \ln(1-e^{-kn/m}) \;+\; k\cdot\frac{(n/m)e^{-kn/m}}{1-e^{-kn/m}}.$$

Setting the derivative to zero and writing $u = e^{-kn/m}$ for brevity,

$$\ln(1-u) \;+\; \frac{kn}{m}\cdot\frac{u}{1-u} \;=\; 0.$$

The quantity kn/m is exactly $-\ln u$, so the equation becomes

$$\ln(1-u) \;-\; (\ln u)\cdot\frac{u}{1-u} \;=\; 0.$$

Multiply through by $(1-u)$:

$$(1-u)\ln(1-u) \;-\; u\ln u \;=\; 0,$$

that is, $(1-u)\ln(1-u) = u\ln u$. By inspection, $u = 1/2$ solves this equation: both sides equal $(1/2)\ln(1/2) = -(\ln 2)/2$. This is a minimum of f, not a maximum or saddle, by the following boundary argument. The base $1-e^{-kn/m}$ lies in $(0,1)$ for every $k>0$, so $f(k)\in(0,1)$ everywhere in the interior. At the boundaries, $f\to 1$: as $k\to 0^+$, $\ln f(k) = k\ln(1-e^{-kn/m}) \approx k\ln(kn/m)\to 0$; as $k\to\infty$, $1-e^{-kn/m}\to 1$ fast enough that $k\ln(1-e^{-kn/m})\approx -ke^{-kn/m}\to 0$. So f is a smooth function on $(0,\infty)$ that sits strictly below 1 in the interior and tends to 1 at both endpoints; it therefore attains a global minimum at some interior critical point. Differentiating the FOC and substituting $u = 1/2$ gives

$$\left.\frac{d^2\ln f}{dk^2}\right|_{k=k^*} \;=\; 2\,(n/m)\,(1-\ln 2) \;>\; 0,$$

so $k^* = (m/n)\ln 2$ is a local minimum, and by direct check the FOC has $u = 1/2$ as its only solution in $(0,1)$ (see Item 5). The local minimum is therefore the global one.

Substituting back, $u = e^{-kn/m} = 1/2$ gives $kn/m = \ln 2$, hence

$$k^* \;=\; \frac{m}{n}\ln 2. \tag{5.2}$$

At this k^* each bit has probability exactly $1/2$ of being 1 (half the array is occupied) and the FPR formula evaluates to

$$\varepsilon^* \;\approx\; \left(\frac{1}{2}\right)^{k^*} \;=\; 2^{-(m/n)\ln 2} \;=\; (0.6185\ldots)^{m/n}. \tag{5.3}$$

The optimal Bloom filter spends m/n bits per stored item and pays roughly 0.62 for every additional bit per item by reducing the FPR by that factor. The half-full bit array at the optimum is a striking fact: the filter is most accurate not when the bits are sparse but when exactly half of them are set.

Theorem 5.2.1 (Optimal k for the Bloom filter)**.** *For fixed bit count m and item count n, the false-positive rate of a Bloom filter under the approximation* (5.1) *is minimized at*

$$k^* \;=\; \frac{m}{n}\ln 2,$$

at which the bit array is half-occupied in expectation and the false-positive rate satisfies

$$\varepsilon^* \;\approx\; \left(\frac{1}{2}\right)^{k^*} \;=\; 2^{-(m/n)\ln 2}.$$

The bit-per-item ratio m/n is the right way to think about a Bloom filter's space budget, and Theorem 5.2.1 says the FPR decays exponentially in that ratio with base $1/2$ (after the optimal k is chosen). Chapter 8 will compare this $0.6185^{m/n}$ scaling to the information-theoretic lower bound and will find a factor of about 1.44 between the two: the standard Bloom filter is within a constant of optimal but does not reach it. Chapter 14 closes the gap.

A worked example

Take $m = 10000$ bits and $n = 1000$ items, a comfortable 10 bits per item. Then $m/n = 10$ and

$$k^* \;=\; 10\ln 2 \;\approx\; 6.93.$$

The nearest integer is $k = 7$, which we adopt. The optimum FPR is

$$\varepsilon^* \;\approx\; (1/2)^7 \;=\; 1/128 \;\approx\; 0.0078,$$

or about one false positive in every 128 negative queries.

To see that $k = 7$ is genuinely near the minimum and not just our rounded approximation, evaluate (5.1) at $k = 5, 6, 7, 8, 9$ with $m = 10000, n = 1000$:

k	$1 - e^{-kn/m}$	$\varepsilon \approx (1 - e^{-kn/m})^k$
5	0.3935	0.00943
6	0.4512	0.00844
7	0.5034	0.00819
8	0.5507	0.00846
9	0.5934	0.00913

The FPR curve is shallow near the minimum: the difference between $k = 6, 7, 8$ is in the third significant digit. Practical implementations often pick $k = 7$ for $m/n = 10$ and rarely bother to fine-tune. The corresponding occupancy at $k = 7$ is $1 - e^{-7/10} = 1 - 0.4966 = 0.5034$, just over half the bits set, which matches the half-full prediction of Theorem 5.2.1 up to the rounding from $k^* = 6.93$ to $k = 7$.

Two sanity checks anchor the formula. First, the FPR depends only on m and n through the ratio m/n once k is optimized. Doubling both m and n leaves the optimal FPR unchanged and doubles the storage; the bit-per-item budget is what matters. Second, the FPR is indeed sensitive to k away from the optimum: a Bloom filter built with $k = 1$ and $m/n = 10$ has FPR $1-e^{-0.1} \approx 0.0952$, about 12 times the optimal rate; a Bloom filter built with $k = 20$ has FPR $(1 - e^{-2})^{20} \approx 0.0546$, about 7 times the optimal rate. Both off-optimum choices are markedly worse than $k = 7$, and the curve is shallow only in a small neighborhood of the optimum. The optimum sits where two opposing pressures balance. Increasing k widens the conjunction the query must satisfy, which on its own would drive the FPR down; but it also fills the array faster, so each individual bit-check is more likely $\top$, driving the FPR up. Decreasing k has the mirror effect: each bit-check is less likely $\top$, but the conjunction is narrower. The minimum at $k^* = (m/n) \ln 2$ is where these two forces cancel: half the array is occupied at the optimum, and the per-bit and per-query terms exactly trade off.

Looking ahead

We have constructed one approximate set in closed form. The Bloom filter has $\omega = 0$ by construction, $\varepsilon \approx (1 - e^{-kn/m})^k$ in the bit-independence approximation, and an optimal $k^* = (m/n) \ln 2$ at which the FPR decays as $0.6185^{m/n}$ in the bit-per-item budget. Every later chapter of Part II will use this filter as a running example.

The construction also stands as one instance of the abstraction Section 5.1 developed. The same predicate framing, the same per-block rate structure, the same chapter 4 axioms apply to the Bloom filter and to any other approximate set. Section 5.3 develops the predicate algebra in generality: the channel matrix per query, the block-stability under Axiom 2, and the Kronecker factorization over multiple queries. The Bloom filter is one row of that general theory. Section 5.4 then takes up the special structure of this row, the one-sided property $\omega = 0$, and contrasts it with the two-sided constructions Part II will also encounter in chapters to come.

5.3 The Predicate Algebra

Section 5.1 introduced the approximate set as a function $\mathsf{A}^{\pm} : U \to \text{Bernoulli[bool]}$ and described its rates informally. Section 5.2 built one specific instance, the Bloom filter, and showed how the rates of that construction come out. Both sections leaned on the Bernoulli[bool] machinery of Chapter 4 without saying precisely how the lift to a set was structured at the matrix level. This section closes that gap. We restate the predicate $\mathsf{A}^{\pm}(x)$ as a Bernoulli[bool] instance per element, write down the channel matrix in its most general per-element form, show that Axiom 2 collapses the per-element rates to two per-block rates, and then invoke the Kronecker factorization of Theorem 4.4.1 to factor the joint distribution over many queries. The corollary is the chapter's first formal payoff: an approximate set's full probabilistic description, no matter how large the universe, requires exactly two parameters.

The predicate as a Bernoulli[bool] instance

Fix a universe U and an objective set $A \subseteq U$. For every $x \in U$, the query $\mathsf{A}^{\pm}(x)$ produces a Boolean output, and the rates of that output depend on whether x is inside or outside A. In the most general per-element setting, each x carries its own per-element rate. If $x \in A$, the false-negative rate at x is $\omega(x) = \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \bot | x \in A\right)$, and the structure correctly reports $\top$ with probability $1 - \omega(x)$. If $x \notin A$, the false-positive rate at x is $\varepsilon(x) = \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \top | x \notin A\right)$, and the structure correctly reports $\bot$ with probability $1 - \varepsilon(x)$.

The four conditional probabilities assemble into a 2×2 channel matrix attached to the query at x:

$$Q_x \;=\; \begin{pmatrix} 1-\omega(x) & \omega(x) \\ \varepsilon(x) & 1-\varepsilon(x) \end{pmatrix}.$$

Row 1 corresponds to the in-set case (truth $\top$, that is, $x \in A$); row 2 corresponds to the out-of-set case (truth $\bot$, that is, $x \notin A$). Column 1 is the report $\top$; column 2 is the report $\bot$. The convention matches the Bernoulli[bool] definition of Definition 4.5.1: row 1 governs positives, row 2 governs negatives, and each row sums to 1. The matrix Q_x is the Bernoulli[bool] instance that describes the per-query behavior of $\mathsf{A}^{\pm}$ at x. There is one such instance per universe element, and in this most general formulation the family $\{Q_x : x \in U\}$ may carry $2|U|$ free parameters, one false-positive rate and one false-negative rate per element, clamped only by the obvious constraint that an in-set element has no false-positive rate and

an out-of-set element has no false-negative rate. Without further structure, this is the cost of describing $\mathsf{A}^{\pm}$ at the rate level: a parameter count that scales with the universe, not a constant. The next subsection shows that the cost collapses.

Axiom 2 collapses per-element rates to per-block rates

Axiom 2 of chapter 4 (Axiom 2, Section 4.3) asserts that the per-block error rates are conditionally independent of each other given the objective set. For an approximate set, the two blocks are the in-set block $B_1 = A$ and the out-of-set block $B_2 = U \setminus A$, and Axiom 2 forces a stronger condition along with the independence: the per-element rates inside each block agree. The reason is that Axiom 2's statement is at the block level, not the element level. The axiom assigns one random rate per block; once that rate is realized, every element of the block inherits it. Element x inside block B_i does not get its own draw separate from y inside the same B_i; both share the block's realized rate. Proposition 4.3.1 of chapter 4 already recorded this: once the rates are realized, the per-element channel matrices stabilize, with row i of every per-element matrix taking the realized rate of block B_i. The proposition below states the corresponding fact for the predicate algebra of an approximate set.

Proposition 5.3.1 (Block-stability of per-element rates)**.** *Let* $\mathsf{A}^{\pm} : U \to$ Bernoulli[bool] *be an approximate set satisfying Axiom 2 (Axiom 2) over the partition* $\{B_1, B_2\} = \{A, U \setminus A\}$*. Then there exist two scalar rates* $\varepsilon, \omega \in [0, 1]$ *such that*

$$\omega(x) \;=\; \omega \quad \textit{for all } x \in A, \qquad \varepsilon(x) \;=\; \varepsilon \quad \textit{for all } x \in U \setminus A,$$

and the per-element channel matrix is the same for every query:

$$Q_x \;=\; Q \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

The rate description of $\mathsf{A}^{\pm}$ *collapses from* $2|U|$ *parameters to two:* (ε, ω)*.*

Proof. By Axiom 2, each block B_i carries a single random rate α_i. Proposition 4.3.1 of chapter 4 then says that once the rates are realized, the per-element channel matrix Q_x depends on x only through the block B_i to which x belongs, with row i of Q_x determined by α_i. For the two-block partition $\{A, U \setminus A\}$, this gives two scalars: ω for row 1 (governing every $x \in A$) and ε for row 2 (governing every $x \in U \setminus A$). The remaining row

entries follow from the stochasticity of Q_x. Since the row identified by an element's block determines its rate, the matrix Q_x is the same for every x in the same block; and since the rows correspond exactly to the two blocks, $Q_x = Q$ is the same matrix for every $x \in U$. □

Throughout Part II we work in the deterministic-rates regime of Chapter 4: the per-block rates ε and ω are fixed parameters of $\mathsf{A}^{\pm}$, not random variables. Under this regime $Q^*_\ell = Q_\ell$ and the conditional and unconditional Kronecker forms of Theorem 4.4.1 coincide, so we may apply the factorization directly to joint behavior over many queries without re-establishing the conditional structure each time. With this assumption in hand we suppress the per-element subscript on Q_x and write a single Q for the approximate set's channel matrix. What varies across queries is the row that the query reads, not the matrix itself.

The joint distribution over m queries factors via Kronecker

When the Bloom filter of Section 5.2 is queried on a workload of m items, the joint behavior over those m queries is what determines the workload-level error rate; the single-query analysis of Section 5.2 was only half the story. Suppose a workload issues m independent queries on $\mathsf{A}^{\pm}$, one on each of m distinct elements $x_1, \ldots, x_m \in U$. Each element x_ℓ belongs to one of the two blocks; call its block index $j_\ell \in \{1, 2\}$. Each query produces a Boolean output $k_\ell \in \{\top, \bot\}$, with conditional probability $(Q)_{j_\ell, k_\ell}$ as prescribed by the per-query matrix.

The joint distribution of $(k_1, \ldots, k_m)$ given the source-block tuple $(j_1, \ldots, j_m)$ is the object Theorem 4.4.1 of chapter 4 governs. Its hypotheses are exactly the two axioms of the Bernoulli framework: Axiom 1 (Axiom 1) gives the mutual independence of error events across distinct elements, and Axiom 2 (Axiom 2) gives the block-stability of the rates that Proposition 5.3.1 just established. Since every per-query matrix is the same Q, the theorem yields the m-fold Kronecker power:

$$Q^{(m)} \;=\; \underbrace{Q \otimes Q \otimes \cdots \otimes Q}_{m \text{ copies}} \;=\; Q^{\otimes m}.$$

Corollary 5.3.0.1 (Kronecker factorization for approximate sets)**.** *Let* $\mathsf{A}^{\pm} : U \to$ Bernoulli[bool] *be an approximate set satisfying Axioms 1 and 2 over the partition* $\{A, U \setminus A\}$*, and assume the deterministic-rates regime announced above. The joint confusion matrix* $Q^{(m)}$ *of* m *queries on* m *distinct*

elements of U factors as the m-fold Kronecker power $Q^{\otimes m}$ of the common per-query channel matrix

$$Q \;=\; \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

The full joint probabilistic description of $\mathsf{A}^{\pm}$ requires only the two parameters (ε, ω), independently of $|U|$ and $|A|$.

Proof. Proposition 5.3.1 makes the per-element matrix Q_x equal to a single matrix Q parameterized by (ε, ω). Theorem 4.4.1 then factors the joint matrix as $Q^{(m)} = Q_1 \otimes \cdots \otimes Q_m$; the deterministic-rates assumption makes the conditional and unconditional forms coincide, and every factor equals Q, so $Q^{(m)} = Q^{\otimes m}$. The two scalars (ε, ω) that fix Q fix the entire joint matrix. □

Corollary 5.3.0.1 is the chapter's first formal payoff. The informal claim of Section 5.1, that the rate description does not grow with $|U|$, has now been earned at the joint level: two scalars fix Q, Q fixes the Kronecker power, and the Kronecker power fixes the joint behavior of arbitrarily many queries. The parametric-parsimony observation of Chapter 4 carries over without dilution.

A worked example

Let us see the factorization concretely. Take $U = \{1, 2, \ldots, 10\}$, $A = \{1, 2, 3\}$, and an approximate set $\mathsf{A}^{\pm}$ with $\varepsilon = 0.1$ and $\omega = 0.05$. Block $B_1 = A$ has three in-set elements and $B_2 = U \setminus A$ has seven out-of-set elements. The per-query channel matrix is

$$Q \;=\; \begin{pmatrix} 1-0.05 & 0.05 \\ 0.1 & 1-0.1 \end{pmatrix} \;=\; \begin{pmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{pmatrix}.$$

Now consider a workload of four queries: two on specific in-set elements (say $x_1 = 1$ and $x_2 = 2$, both in A) and two on specific out-of-set elements (say $x_3 = 4$ and $x_4 = 5$, both in $U \setminus A$). Suppose we want the probability that the structure reports $\top$ on all four queries. The two in-set queries are correct reports (true positives), and the two out-of-set queries are wrong reports (false positives).

By Corollary 5.3.0.1, the joint confusion matrix is $Q^{(4)} = Q^{\otimes 4}$, and the probability of the specific source-block tuple (B_1, B_1, B_2, B_2) producing the

specific output pattern $(\top, \top, \top, \top)$ is the product of per-query entries from Q:

$$\begin{aligned}
&\mathrm{P}\left(\mathsf{A}^{\pm}(x_1) = \top, \mathsf{A}^{\pm}(x_2) = \top, \mathsf{A}^{\pm}(x_3) = \top, \mathsf{A}^{\pm}(x_4) = \top | x_1, x_2 \in A,\ x_3, x_4 \notin A\right) \\
&\quad = Q_{1,1} \cdot Q_{1,1} \cdot Q_{2,1} \cdot Q_{2,1} \\
&\quad = (1-\omega)^2 \cdot \varepsilon^2 \\
&\quad = (0.95)^2 \cdot (0.10)^2 \\
&\quad = 0.9025 \cdot 0.01 \\
&\quad = 0.009025.
\end{aligned}$$

The two true-positive factors $(0.95)^2 = 0.9025$ come from the two in-set queries reading row 1 of Q, column 1; the two false-positive factors $(0.10)^2 = 0.01$ come from the two out-of-set queries reading row 2 of Q, column 1. Each factor is one cell of the per-query matrix, and the Kronecker product structure puts them together multiplicatively because the queries are mutually independent (Axiom 1) and share the same per-block rates across the workload (Axiom 2).

The same calculation would proceed identically for any other query pattern. The probability of reporting $\bot$ on the two in-set queries and $\top$ on the two out-of-set queries, for instance, would be $\omega^2 \cdot \varepsilon^2 = 0.0025 \cdot 0.01 = 0.000025$; the probability of reporting $\top$ on the two in-set queries and $\bot$ on the two out-of-set queries would be $(1-\omega)^2 \cdot (1-\varepsilon)^2 = 0.9025 \cdot 0.81 = 0.731025$, the most likely single outcome and the one in which every query is reported correctly. Across all $2^4 = 16$ output patterns, the probabilities sum to one, and each is a product of four per-query entries selected from Q according to the block of the query and the report of the output.

The point is the structure, not the numbers. Doubling the universe to 20 elements while keeping the same rates leaves this query pattern's probability unchanged; doubling the workload to 40 queries grows the Kronecker product to 40 copies of Q, but each entry is still a product of two-rate factors chosen one per query.

Looking ahead

The Bloom filter of Section 5.2 is one Bernoulli[bool] instance in this family: its Q has $\omega = 0$ structurally and ε tuned by the bit count, hash count, and item count, so the worked example above applies directly once the Bloom filter's ε is substituted for the generic 0.1. Any other approximate set construction yields a different Q but the same algebra: per-query matrix

in row-stochastic form, joint matrix as Kronecker product, two-parameter description of the full probabilistic behavior. The one-sided versus two-sided distinction of Section 5.4 is the next slice: what happens when one of the two parameters is structurally zero, as it is for the Bloom filter, and how that case contrasts with the two-positive-rate setting that arises elsewhere.

5.4 One-Sided vs Two-Sided Approximate Sets

The Bloom filter of Section 5.2 has a striking property: one of its two error rates is exactly zero, not by parameter choice but by construction. The bit array can only accumulate ones, so an inserted item is never reported missing; the other rate is positive and tunable. This is one shape an approximate set can take. A different construction might force the false-positive rate to zero on a designated set instead, leaving the false-negative rate positive; a third might allow both rates to be positive. The present section gives the three shapes their names and connects the distinction to the algebraic structure that chapter 6 will develop.

Definition

Definition 5.4.1 (One-sided and two-sided approximate sets)**.** *Let* $\mathsf{A}^{\pm} : U \to \text{Bernoulli[bool]}$ *be an approximate set of* $A \subseteq U$ *with per-block rates* (ε, ω) *as in Section 5.3. We say* $\mathsf{A}^{\pm}$ *is*

- one-sided in FNR *(or* FNR-zero*) if* $\omega = 0$ *and* $\varepsilon > 0$*;*
- one-sided in FPR *(or* FPR-zero*) if* $\varepsilon = 0$ *and* $\omega > 0$*;*
- two-sided *if both* $\varepsilon > 0$ *and* $\omega > 0$*.*

The Bloom filter of Definition 5.2.1 is one-sided in FNR. The perfect hash filter of Chapter 15 is one-sided in FPR on its design domain.

The names track the rate that is zero, not the one that is positive. The Z-channel of Definition 2.2.2 is the per-query analog; the present section's one-sided property is its per-set lift. A two-sided approximate set sits in the interior of the rate square $[0, 1]^2$; a one-sided set sits on one of the two coordinate axes. The corner $\varepsilon = \omega = 0$ is the exact set, not an approximate set at all, and is excluded from Definition 5.4.1.

Why FNR-zero is the natural default for insertion-only structures

The Bloom filter's FNR-zero property was structural, not designed. The insert operation sets bits to 1, and no operation in the chosen insertion-only API ever resets them; a subsequent query for an inserted item reads bits that were set, so the query returns $\top$ regardless of what else has happened since. The argument depended only on the monotonicity of the bit array under that API. Insertions go up, never down. Any data structure built on the same monotone-insert primitive shares the property: counting sketches keyed by hash, summary structures over append-only logs, and any representation of an insertion-only bit-set query (read what was written) are one-sided in FNR for the same structural reason. (Counting Bloom filters, taken up below, restore deletions by storing small integer counters in place of bits, paying for that change of API with $\omega > 0$.)

The asymmetry runs the other way. Forcing $\varepsilon = 0$ on a designated set, while keeping $\omega > 0$ elsewhere, requires the construction to know which queries are guaranteed inside and which are not. A deterministic check against the underlying set would do this, but at the cost of storing the set itself, which destroys the space saving an approximate set was supposed to provide. The constructive path to an FPR-zero structure must be more clever. Chapter 15's perfect hash filter (Chapter 15) computes a perfect hash for a fixed A at construction time and stores only a fingerprint per element; a query for $x \in A$ lands on its fingerprint and is reported correctly with probability 1, while a query for $x \notin A$ may still collide. The set has to be known up front, and updates are expensive. Those costs buy the opposite asymmetry: certainty in the positive direction rather than the negative.

The trade-off: safety versus space

One-sided approximate sets are valuable because one direction of the answer is exact. If a Bloom filter reports $\bot$, the queried element is definitely not in the underlying set; downstream logic can act on the certain answer without further checks. An FPR-zero filter has the dual safety: a $\top$ report is certain, only $\bot$ needs follow-up. Many systems are built around exactly this asymmetric trust. Web caches use a Bloom filter to skip lookups for keys known to be absent; the certain $\bot$ avoids a slow disk read. Spell-checkers flag possibly misspelled words; the certain $\top$ on a known word avoids a distracting underline. In every such setting, the one-sided structure substitutes for an exact set precisely because the certain direction is the one the application

needs.

Two-sided sets give up the certain direction and recover space. Counting Bloom filters store small integer counters instead of bits to support deletions, paying for that with a nonzero ω. Cardinality estimators such as HyperLogLog and generic sublinear sketches are typically two-sided on derived predicates. The trade is honest: dropping one side of the asymmetry frees the construction to use space more aggressively.

A bit-budget comparison makes the size of the trade visible. By the optimal-k formula of Section 5.2, a Bloom filter at $\varepsilon = 0.01$ and $\omega = 0$ requires $m \approx 1.44\, n \log_2(1/\varepsilon) \approx 9.6\, n$ bits, since $\log_2(100)/\ln 2 \approx 9.58$. Chapter 8 will show that the information-theoretic floor for an FNR-zero approximate set at the same ε is $n \log_2(1/\varepsilon) = n \log_2 100 \approx 6.64\, n$ bits per element, so the Bloom filter pays a $9.6/6.64 \approx 1.44\times$ overhead above the floor that any FNR-zero construction must respect. That fixed overhead is the standard literature's $\log_2 e \approx 1.44$ constant, and chapter 14's Bernoulli Hash Function is the construction that closes the gap. A two-sided set is not bound by the same one-sided floor at ε, because it can split its error budget across both directions. Splitting the budget can purchase space below the FNR-zero floor, but it spends the certain $\perp$ that the Bloom filter buys with its extra bits. The qualitative shape is what matters here. One-sidedness is paid for in bits, and a construction that gives up the certain direction can in principle spend those bits elsewhere.

The perfect filter line

The two-dimensional rate plane $(\varepsilon, \omega) \in [0,1]^2$ parameterizes every approximate set. The one-sided sets occupy the two coordinate axes: the FNR-zero family on $\{\omega = 0,\, 0 < \varepsilon \leq 1\}$ and the FPR-zero family on $\{\varepsilon = 0,\, 0 < \omega \leq 1\}$. We call the union of these two segments the *perfect filter line*. The name is the convention of the underlying paper family; the word "line" is appropriate because each segment is a one-dimensional locus inside the two-dimensional plane. A one-sided approximate set has one degree of rate-freedom, not two. Two-sided sets fill the interior of the unit square; the exact set is the origin.

The perfect filter line is not a passive labeling. Chapter 6 (Chapter 6) will derive the union, intersection, and complement of two Bernoulli sets and show that

- the union of two FNR-zero sets is FNR-zero;
- the intersection of two FNR-zero sets is FNR-zero;

- the complement swaps the two axes, sending an FNR-zero set to an FPR-zero set and vice versa.

The FPR-zero side is in general not closed under the same operations: the union of two FPR-zero sets is FPR-zero only when both are exact on the same designated set, a fragile condition. The asymmetry parallels the construction-side asymmetry: FNR-zero is the natural default in many places, FPR-zero requires more careful design. Section 3.5 of chapter 3 already previewed the union-as-bit-OR identity for FNR-zero Bloom filters as a special case of the composition theorem; chapter 6 generalizes the calculation and makes the perfect filter line into an algebraic constraint.

Two further chapters take up the line as their concern. Chapter 14 (Chapter 14) develops the Bernoulli Hash Function and shows that within the FNR-zero family the bit-per-item cost is bounded below by an entropy formula, and the BHF achieves the bound. Chapter 15 (Chapter 15) takes the FPR-zero side seriously and constructs the perfect hash filter: an approximate set with $\varepsilon = 0$ on a designated set A and $\varepsilon > 0$ outside. Each chapter is one half of the same picture, sitting on one half of the perfect filter line.

Looking ahead

We have separated the approximate sets of Part II into three rate shapes: FNR-zero (the Bloom filter and its kin), FPR-zero (the perfect hash filter and its kin), and two-sided (counting filters and sketches). The shapes are not interchangeable. Each carries different safety guarantees, different space costs, and different behavior under algebraic operations on sets. Section 5.5 surveys the trajectory of the rest of the chapter and of Part II as a whole: the composition algebra of chapter 6, the classification measures of chapter 7, the entropy bound of chapter 8, and the space-optimal constructions of chapter 14 that close the gap the standard Bloom filter leaves on the table.

5.5 What Comes Next

The chapter has built one abstraction and worked one example. Section 5.1 lifted the noisy bit of Part I to a function $\mathsf{A}^{\pm} : U \to \text{Bernoulli[bool]}$, the approximate set. Section 5.2 built the Bloom filter as the canonical realization of that abstraction and pinned down its rates. Section 5.3 placed the abstraction inside chapter 4's formal machinery: every query is a Bernoulli[bool]

instance, every n-query workload is a Kronecker product of per-block channels, and the per-set rates (ε, ω) are the only parameters that survive Axiom 2. Section 5.4 separated the shapes an approximate set can take and pointed at the perfect filter line. What remains to do in Part II is the algebra and the bounds that the abstraction underwrites. Four chapters follow, each picking up one strand from the present chapter and carrying it to its proper conclusion.

Chapter 6: the algebra of sets

Chapter 6 takes up set operations. Approximate sets have unions, intersections, complements, and differences; each operation has rates that follow from the rates of its inputs. The calculation is not new machinery; it is the composition theorem of Section 3.3 applied to the per-element predicates described in Section 5.3. The union of two Bloom filters by bit-OR gives an FPR of $1 - (1 - \varepsilon_1)(1 - \varepsilon_2)$, the intersection gives a cleaner $\varepsilon_1\varepsilon_2$, and the complement swaps the two coordinate axes. The chapter then asks which subfamilies of approximate sets are closed under these operations. The FNR-zero family of Section 5.4 is closed under union and intersection (one-sidedness is preserved in the same direction), but the complement maps it to the FPR-zero family on the opposite axis. The perfect filter line is a subalgebra under some operations and not others, and the asymmetry foreshadows chapter 15's constructive treatment of the FPR-zero side.

Chapter 7: classification measures

Chapter 7 measures quality. The two rates (ε, ω) are a complete intrinsic description, but practitioners often want derived quantities: positive predictive value, negative predictive value, recall, F1. Each is a function of the two rates and the workload's prevalence of in-set elements, and each depends on the workload in a way the rates themselves do not. A Bloom filter with $\varepsilon = 0.01$ achieves a PPV near 0.99 on a workload of mostly true positives and a PPV near 0.5 on a workload of mostly out-of-set queries. The chapter develops the closed forms, derives sampling distributions and confidence intervals for the estimators, and retires the deferral Section 1.4 flagged: the Wald interval was a placeholder, and the Wilson, Clopper-Pearson, and Jeffreys alternatives that handle the boundary properly are stated and applied. The chapter that began with a sawtooth in chapter 1's notebook ends here with the proper estimator vocabulary in place.

Chapter 8: information-theoretic bounds

Chapter 8 bounds the space. Any FNR-zero approximate set that achieves false-positive rate ε must consume at least $-\log_2 \varepsilon$ bits per stored element. The proof runs through Shannon entropy and the source-coding theorem; the statement is sharp. The standard Bloom filter of Section 5.2 achieves $\log_2 e \cdot \log_2(1/\varepsilon) \approx 1.44 \log_2(1/\varepsilon)$ bits per element under Theorem 5.2.1, which is roughly 44% above the floor. That 44% overhead is exactly the gap Section 5.4 named in its bit-budget comparison. The chapter's job is to make the gap precise. The construction that closes it is chapter 14's, not chapter 8's.

Chapter 14: the construction that closes the gap

Chapter 14 is the constructive payoff of the bound. The Bernoulli Hash Function (BHF) is an approximate set whose bit-per-item cost achieves the entropy lower bound up to lower-order terms. The 44% overhead the standard Bloom filter pays is the right kind of unfinished business: a gap that is real, that is calculable, and that admits a closed-form remedy. The BHF construction is one of the climactic results of the book; chapters 5 through 8 build the language in which the result can be both stated and appreciated, and chapter 14 then delivers it.

Chapter 15 is the FPR-zero counterpart on the other half of the perfect filter line. It takes the chapter 6 algebraic constraint seriously and constructs a representation that achieves $\varepsilon = 0$ on a designated set, paying the dual price the asymmetry of Section 5.4 predicts.

Closing

The bibliographic notes of Section 5.2's parent chapter, drafted next, name the antecedents of what we have done: Bloom's 1970 paper, the survey literature that grew up around it, and the Bernoulli-set papers that recast the data structure as an instance of a predicate algebra. The four chapters previewed above develop that algebra into a working theory of approximate-set operations, classification measures, information-theoretic bounds, and space-optimal constructions. Each picks up one of the strands the present chapter has set down; together they make the approximate set a first-class object of study.

Bibliographic Notes

Bloom's original paper. The data structure at the center of Section 5.2 is due to Bloom [Blo70], in a four-page *Communications of the ACM* note from 1970. Bloom's framing was operational rather than probabilistic: he was concerned with hyphenation dictionaries for a typesetting application, and the question was how much memory could be saved by accepting a small rate of mishyphenated words in exchange for not having to store the dictionary itself. The analysis in that paper already contains the optimal-k derivation that Theorem 5.2.1 reproduces in modern notation, including the $k = (m/n) \ln 2$ result and the resulting per-bit false-positive rate $(1/2)^k$ at the optimum. What this textbook adds is not the construction or its analysis but the framing: the Bloom filter as a particular construction of a function $U \to$ Bernoulli[bool], rather than as a clever bit-array trick for set membership. The two readings agree on every numerical prediction.

Survey and textbook context. For an entry into the broader literature on approximate-set data structures, two references are standard. Mitzenmacher and Upfal [MU17] devotes Chapter 5 to hashing, Bloom filters, and the balls-and-bins analysis that underlies both; the proof of the optimal-k result in that text follows the same calculus argument used in Section 5.2, though without the Bernoulli[bool] framing. Broder and Mitzenmacher [BM04] collects applications of Bloom filters across networking, distributed systems, and database query processing, organized by the kind of approximate-set operation each application requires. The survey is the natural follow-up reading for anyone who wants to see how widely the Bloom filter's $\omega = 0$ property gets used in practice; it also documents the variants (counting filters, scalable filters, compressed filters) that the present chapter mentions only in passing.

The set-as-predicate viewpoint. The framing of an approximate set as a function $U \to$ Bernoulli[bool], rather than as a data structure that happens to have an error rate, is the contribution of Towell [Tow26a]. Earlier literature, including Bloom [Blo70] and the broad sweep of Broder and Mitzenmacher [BM04], treats approximate sets as *data structures with error rates*: an array of bits plus a membership test, characterized by a single false-positive probability. The Bernoulli-sets reading inverts that emphasis. The object of study is the function itself, the per-query Boolean output is a Bernoulli[bool] instance in the sense of Towell [Tow26f], and the data structure is one of many possible realizations. Sections 5.1 and 5.3 develop

this framing as a continuation of Part I's atom-and-axiom construction; the algebra of Section 5.3 is the direct lift of the order-2 Bernoulli[bool] algebra of chapter 4 to the per-element setting.

One-sided and perfect-filter lineage. The one-sided / two-sided dichotomy of Section 5.4 has two ancestors in the literature. The FNR-zero shape is the Bloom filter's structural property, named as such in Bloom [Blo70] but not isolated as a category until the broader approximate-set literature accumulated multiple constructions sharing it. The FPR-zero shape—a *perfect filter* in the sense of Definition 5.4.1, on a designated set—has its constructive roots in the perfect-hashing literature opened by Carter and Wegman [CW79]. Carter and Wegman's universal hash families and the perfect hashing constructions that followed are what makes a fingerprint-per-element scheme realizable in the first place: without a perfect hash on A, the fingerprint table cannot guarantee $\varepsilon = 0$ on $x \in A$. Chapter 15's perfect hash filter (Chapter 15) is the modern descendant of that line. The chapter's exposition treats the Bernoulli-set characterization first, then the construction; the historical order is the opposite.

Forward bridge: chapters 14 and 15. Both chapter 14 (Chapter 14) and chapter 15 (Chapter 15) build on a single foundational manuscript: Towell [Tow26h], which derives the space-optimal Bernoulli hash filter (BHF) for both the unrestricted and FPR-zero-on-A regimes and proves the entropy lower bound that makes "optimal" precise. The construction in that paper realizes the FPR-zero shape of Definition 5.4.1 explicitly and shows that the Bloom filter is, on the relevant axis, off-optimal by a constant factor that is well understood. Readers who want the punch line of the chapters that follow—why *optimal* can be said precisely, and what the cost of optimality is in construction time and update support—will find it in that manuscript.

Exercises

1. ⋆ **Predicate-to-rates.** Let $U = \{1, 2, \ldots, 10\}$ and $A = \{2, 5, 7\}$. An approximate set $\mathsf{A}^{\pm}$ is queried once on every element of U and produces the following table of reports:

x	1	2	3	4	5	6	7	8	9	10
$\mathsf{A}^{\pm}(x)$	$\bot$	$\top$	$\top$	$\bot$	$\top$	$\bot$	$\bot$	$\bot$	$\top$	$\bot$

(a) Identify which queries hit in-set elements and which hit out-of-set elements; tabulate the four outcome counts (TP, FP, FN, TN) using A as ground truth.

(b) Compute the empirical false-positive rate $\hat{\varepsilon}$ as the fraction of out-of-set queries that report $\top$, and the empirical false-negative rate $\hat{\omega}$ as the fraction of in-set queries that report $\bot$. Be explicit about the denominators.

(c) State in one sentence why these two empirical rates are the natural sample-based estimators of the per-block parameters (ε, ω) that Section 5.3 attaches to $\mathsf{A}^{\pm}$.

2. $\star$ **Bloom filter parameters.** A Bloom filter is built with $m = 8000$ bits, $n = 1000$ inserted elements, and $k = 2$ hash functions.

(a) Compute ε from Equation (5.1), that is, $\varepsilon \approx (1 - e^{-kn/m})^k$ with the given (m, n, k). Report at least three significant figures.

(b) For the same (m, n), compute the optimal hash count $k^* = (m/n) \ln 2$ from Theorem 5.2.1. Round to the nearest integer if necessary and report both the real-valued optimum and the integer used.

(c) Evaluate (5.1) at $k = k^*$ and compare to the rate from (a). By what factor does the optimal k reduce the false-positive rate at this (m, n)?

3. $\star$ **One-sided check.** For each of the following data structures, state whether it is one-sided in ω ($\omega = 0$, $\varepsilon > 0$), one-sided in ε ($\varepsilon = 0$, $\omega > 0$), two-sided (both rates positive), or exact (both rates zero). Give a one-sentence justification per item drawn from the construction.

(a) A Bloom filter (Definition 5.2.1).

(b) An exact set-membership test on a hash table that stores every element of A verbatim.

(c) A cardinality estimator such as HyperLogLog used as a membership oracle by thresholding the estimated cardinality.

4. $\star\star$ **Notebook-tied Bloom filter empirics.** Open the chapter 5 companion notebook `python/notebooks/ch05-approximate-sets.ipynb`. Build a Bloom filter with $m = 10000$ bits and $n = 1000$ inserted random strings, and vary the hash count over $k \in \{1, 2, 3, \ldots, 15\}$.

(a) For each k, estimate the empirical false-positive rate $\hat{\varepsilon}(k)$ by querying a large pool of strings known to be absent from the inserted set and recording the fraction reported as $\top$.

(b) On the same axes, plot $\hat{\varepsilon}(k)$ and the theoretical curve $(1-e^{-kn/m})^k$ from Equation (5.1). Describe in one sentence how closely the two curves agree across the range of k.

(c) Read off the integer k that minimizes $\hat{\varepsilon}$ and compare to the optimum $k^* = (m/n)\ln 2$ predicted by Theorem 5.2.1. Do the empirical and theoretical optima agree to within the one-bin resolution of your sweep?

5. ⋆⋆ **Uniqueness of the optimal k.** The proof of Theorem 5.2.1 in Section 5.2 reduced the first-order optimality condition to

$$(1-u)\ln(1-u) \;-\; u\ln u \;=\; 0, \qquad u \in (0,1),$$

under the change of variable $u = e^{-kn/m}$, and observed by inspection that $u = 1/2$ solves it. The proof then claimed that $u = 1/2$ is the *only* solution in $(0,1)$, leaving the uniqueness to this exercise.

(a) Define $\phi(u) = (1-u)\ln(1-u) - u\ln u$ on $(0,1)$. Verify the symmetry $\phi(1-u) = -\phi(u)$, and confirm that $\phi(u) \to 0$ as $u \to 0^+$ and as $u \to 1^-$. Conclude that $\phi(1/2) = 0$ without further calculation.

(b) Compute $\phi'(u)$ and verify that $\phi'(u) = -\ln(1-u) - \ln u - 2$. Show that ϕ' attains its minimum at $u = 1/2$, where $\phi'(1/2) = 2\ln 2 - 2 < 0$, and tends to $+\infty$ at both endpoints. Conclude that ϕ' changes sign exactly twice on $(0,1)$, once on $(0,1/2)$ and once on $(1/2,1)$.

(c) Use the sign pattern of (b) and the boundary behavior of (a) to conclude that ϕ is positive throughout $(0,1/2)$ and negative throughout $(1/2,1)$, with the unique zero located at $u = 1/2$.

(d) Transfer the conclusion back to k via the change of variable $u = e^{-kn/m}$. Confirm that $k^* = (m/n)\ln 2$ is the unique interior critical point of $f(k) = (1-e^{-kn/m})^k$ on $(0,\infty)$, and combine with the second-derivative check in Section 5.2 to conclude that it is the global minimum.

6. ⋆⋆ **Predicate algebra: joint probability via Kronecker.** Let U have $|U| = 6$ elements and $A \subseteq U$ with $|A| = 2$. An approximate set

$\mathsf{A}^{\pm}$ has per-block rates $\varepsilon = 0.1$ and $\omega = 0.05$ in the deterministic-rates regime of Section 5.3.

(a) Fix three specific out-of-set elements $x_1, x_2, x_3 \in U \setminus A$. Compute the probability that $\mathsf{A}^{\pm}$ reports $\top$ on each of x_1, x_2, x_3 (three simultaneous false positives), assuming Axioms 1 and 2 are in force.

(b) Write the same probability as a single entry of the Kronecker power $Q^{\otimes 3}$, identifying which row and column of the per-query matrix $Q = \left(\begin{smallmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{smallmatrix}\right)$ each of the three factors reads.

(c) Justify the factorization in one or two sentences by naming the role of Theorem 4.4.1 (which gives the joint matrix as a Kronecker product) and Corollary 5.3.0.1 (which specializes the product to the two-parameter approximate-set setting).

7. ⋆⋆ **Bit-check independence counter-example.** The derivation of $\varepsilon \approx (1 - e^{-kn/m})^k$ in Section 5.2 treats the k bit-check events as independent. Equation (5.1) is therefore an approximation; the exact false-positive rate is in general different. This exercise constructs the smallest worked counter-example.

(a) Take $m = 4$ bits, $n = 1$ inserted element, $k = 2$ independent hash functions, each uniform over $\{1, 2, 3, 4\}$. Enumerate all $4 \times 4 = 16$ assignments of $(h_1(\text{inserted}), h_2(\text{inserted}))$ with equal probability $1/16$; for each assignment, write out which bits are set.

(b) Now consider a query on a fresh out-of-set element y. Its two hashes $(h_1(y), h_2(y))$ are also uniform over $\{1, \ldots, 4\}^2$, independent of $h_i(\text{inserted})$. Compute the exact probability that both $b[h_1(y)] = 1$ and $b[h_2(y)] = 1$, that is, the exact false-positive rate, by averaging over all $16 \times 16 = 256$ joint hash assignments.

(c) Compare the exact rate from (b) to the approximation (5.1), $\varepsilon \approx (1 - e^{-2 \cdot 1/4})^2 = (1 - e^{-1/2})^2 \approx 0.155$. State in one or two sentences why the two values differ and identify the source of the correlation: if $h_1(y) = h_2(y)$, the two "independent" bit-check events collapse into one.

8. ⋆⋆ **Two-sided vs one-sided space trade-off via the entropy floor.** Section 5.4 previews chapter 8's entropy floor: any FNR-zero approximate set at false-positive rate ε requires at least $-\log_2 \varepsilon$ bits per stored element. The standard Bloom filter, at its optimal k^* (Theorem 5.2.1), pays $\approx 1.44 \cdot \log_2(1/\varepsilon)$ bits per element.

(a) Compute the entropy floor $-\log_2 \varepsilon$ in bits per element for the three target rates $\varepsilon \in \{0.1, 0.01, 0.001\}$. Tabulate to one decimal place.

(b) Compute the Bloom filter's actual bit-per-element cost $1.44 \cdot \log_2(1/\varepsilon)$ for the same three targets.

(c) Report the absolute overhead in bits per element (Bloom cost minus entropy floor) at each target. Interpret in one sentence: as ε shrinks, does the absolute overhead grow, shrink, or stay constant, and what does that mean for the practical cost of one-sidedness at very low false-positive rates?

9. $\star\star\star$ **Counting Bloom filter sketch.** The standard Bloom filter (Definition 5.2.1) supports no deletions: clearing a bit on DELETE(x) would clear bits shared by other inserted elements and break their queries. A *counting Bloom filter* resolves this by replacing each bit with a small unsigned integer counter (typically 4 bits); INSERT increments the k chosen counters and DELETE decrements them; QUERY returns $\top$ if all k counters are positive.

   (a) Argue that DELETE is correct *provided* no counter ever overflows during INSERT and no counter is ever decremented for an element that was not previously inserted. Both conditions can fail in practice; identify the failure mode of each.

   (b) Sketch why a counting Bloom filter is in general two-sided rather than one-sided: counter overflow (saturation at the maximum value) and the use of small counters together introduce a positive false-negative rate. Counter overflow caps the count, so subsequent decrements may zero a counter that should still be positive, causing a query on an inserted element to return $\bot$.

   (c) Compare the space cost. A b-bit counter Bloom filter occupies bm bits, compared with m bits for the standard filter. For the typical choice $b = 4$, this is a $4\times$ overhead. Comment in one or two sentences on the trade-off: the counting filter buys an additional API (DELETE) at the price of $4\times$ space, structural two-sidedness, and a small overflow-induced loss.

10. $\star\star\star$ **Block-stability under non-uniform priors.** Proposition 5.3.1 of Section 5.3 invokes Axiom 2 to collapse the per-element rates $\varepsilon(x), \omega(x)$ to per-block rates ε, ω. The collapse depends on the assumption that

the random rate attached to each block is shared by every element in that block. Construct a scenario where this assumption fails.

(a) Take a learned classifier (a noisy approximate-membership oracle, in our framework) whose accuracy varies across the out-of-set block: queries on out-of-set elements with very low prior probability of confusion are answered correctly with rate $1 - \varepsilon_{\text{easy}} = 0.99$, while queries on near-the-boundary out-of-set elements achieve only $1 - \varepsilon_{\text{hard}} = 0.70$. Describe how this construction gives a $\varepsilon(x)$ that genuinely depends on x within the out-of-set block $B_2 = U \setminus A$.

(b) Argue informally that the construction violates the spirit of Axiom 2 even though the classifier itself is deterministic. (*Hint:* Axiom 2 says the per-block rate is a single random variable shared by every block member; here the per-element rates are distinct, so no single block rate fits.)

(c) State the immediate consequence: the Kronecker factorization of Corollary 5.3.0.1 no longer applies in its two-parameter form; the joint distribution over m queries must instead carry per-element rates and is no longer universe-size-independent. Briefly identify which workloads make the gap visible and which mask it (*hint:* a workload that samples uniformly from B_2 averages over the heterogeneity and looks like a single-rate workload; a workload that adversarially selects near-boundary out-of-set elements exposes the gap).

11. $\star\star\star$ **Forward to chapter 6: union FPR via the composition theorem.** Chapter 6 will derive the false-positive rate of the union of two approximate sets $\mathsf{A}_1^{\pm} \cup \mathsf{A}_2^{\pm}$ from the rates of the constituents. For two FNR-zero approximate sets with FPRs $\varepsilon_1, \varepsilon_2$, both representing the same underlying set A on the same universe, and assuming independent error events, the union (implemented by bit-OR on Bloom filters with disjoint hash families) has FPR

$$\varepsilon_{\text{union}} \;=\; 1 - (1 - \varepsilon_1)(1 - \varepsilon_2).$$

(a) Derive this expression from Theorem 3.3.1 of chapter 3. (*Hint:* for a fixed out-of-set element x, the union reports $\top$ if and only if at least one of the two constituent queries reports $\top$. The complement is that both report $\bot$, which under independence factors as $(1 - \varepsilon_1)(1 - \varepsilon_2)$.)

(b) Verify the formula on a numerical example: take $\varepsilon_1 = 0.10$ and $\varepsilon_2 = 0.05$ and compute $\varepsilon_{\text{union}}$. Confirm that the result lies between $\max(\varepsilon_1, \varepsilon_2)$ and $\varepsilon_1 + \varepsilon_2$, with the second-order term $\varepsilon_1\varepsilon_2$ accounting for the gap.

(c) Describe a concrete situation where the independence assumption breaks and the formula no longer applies. For instance, the two filters share the same hash family over the same universe; then the bit-OR event on the same out-of-set element x is perfectly positively correlated across the two filters whenever both filters set the same bits, and the union FPR collapses to a single-filter FPR rather than the composition-theorem prediction. State the qualitative effect on the union rate (over-estimate or under-estimate) and identify which structural property of the construction the formula presupposes.

Chapter 6

Composition of Sets

After this chapter you will be able to:

- Derive the error rates of $\cup, \cap, \setminus, \complement$ for approximate sets by applying the composition theorem of Theorem 3.3.1 to the predicate framing of Chapter 5.
- Recognize the cancellation phenomena that let composed sets escape the perfect-filter line.
- State the closure properties of the perfect-filter line and name which set operations preserve one-sidedness.

6.1 Union of Approximate Sets

Chapter 5 framed an approximate set $\mathsf{A}^{\pm}$ as a noisy membership predicate over a universe U, with two per-block error rates (ε, ω). The Bloom filter of Definition 5.2.1 was the canonical instance: a structurally one-sided approximate set with $\omega = 0$ and a tunable ε. The present section takes the first compositional step. Given two approximate sets $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ over the same universe, we ask what their union is and what error rates the union inherits. The answer is the composition theorem of Theorem 3.3.1 applied to the OR gate of Section 3.2, lifted from the per-bit level to the per-set level.

Definition

A classical union $A \cup B$ is characterized by its membership predicate: $\mathbb{1}_{A \cup B}(x) = \mathbb{1}_A(x) \vee \mathbb{1}_B(x)$. The set-builder reading is the same: $x \in A \cup B$ if and only if

$x \in A$ or $x \in B$. The membership test is a Boolean OR of two simpler membership tests. The predicate framing of Section 5.1 lifts this characterization directly to approximate sets.

Definition 6.1.1 (Union of approximate sets)**.** *Let* $\mathsf{A}^{\pm}, \mathsf{B}^{\pm} : U \to$ Bernoulli[bool] *be two approximate sets over the same universe* U*. Their* union $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ *is the approximate set whose membership predicate is the Boolean OR of the two operand predicates: for every* $x \in U$*,*

$$(\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm})(x) \;=\; \mathsf{A}^{\pm}(x) \;\vee\; \mathsf{B}^{\pm}(x).$$

The union approximates the objective set $A \cup B$*, where* A *and* B *are the underlying objective sets that* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *represent.*

The definition is structural and it makes no commitment to a particular implementation. To evaluate the union's predicate at x, query $\mathsf{A}^{\pm}(x)$ and $\mathsf{B}^{\pm}(x)$, then take the OR of the two Boolean outputs. The union is therefore well-defined as soon as the two operand predicates are, and its rates will follow from the rates of the operands together with the OR-gate analysis already done in Chapter 3.

Two realizations on Bloom filters: predicate-OR and bit-OR-Bloom

Definition 6.1.1 commits to a predicate-level operation: evaluate $\mathsf{A}^{\pm}(x)$, evaluate $\mathsf{B}^{\pm}(x)$, take the Boolean OR of the two outputs. We call this realization *predicate-OR*. It keeps the two filters as separate objects and asks two queries per lookup. The proposition derived below (Proposition 6.1.1) and its formula $1-(1-\varepsilon_A)(1-\varepsilon_B)$ describe this construction and only this one.

When both operands are Bloom filters with the same parameters, a tempting operational shortcut presents itself. Suppose $\mathsf{A}^{\pm}$ is the Bloom filter $(b_1, h_1, \ldots, h_k)$ for a set A and $\mathsf{B}^{\pm}$ is the Bloom filter $(b_2, h_1, \ldots, h_k)$ for a set B, using the same bit-array size m and the same hash family $h_1, \ldots, h_k$. The bit-wise OR $b_1 \vee b_2$ is itself a length-m bit array, and one can query the combined bitmap with a single k-position lookup. Call this realization *bit-OR-Bloom*: one bitmap, one query, one returned verdict. It is a different operation from predicate-OR, and it gives a different FPR.

Both realizations preserve FNR-zero on FNR-zero operands. For bit-OR-Bloom: every element of A has its k bits set in b_1, hence in $b_1 \vee b_2$, and symmetrically for B, so every member of $A \cup B$ is reported $\top$ with certainty.

For predicate-OR: each operand correctly reports $\top$ on its inserted items, and the OR of any two outputs containing a $\top$ is $\top$.

The two realizations diverge on false positives. Bit-OR-Bloom is exactly a Bloom filter with parameters (m, k) holding the union of inserted items, so its FPR is

$$\varepsilon_{\text{bit-OR}} \approx \left(1 - e^{-k(n_A+n_B)/m}\right)^k, \tag{6.1}$$

the chapter 5 Bloom-filter formula evaluated at the combined item count $n_A + n_B$. This is materially different from the predicate-OR rate $1 - (1 - \varepsilon_A)(1 - \varepsilon_B)$. At $m = 5000$, $k = 7$, $n_A = n_B = 500$, the per-filter rate is $\varepsilon \approx 0.0082$; predicate-OR gives ≈ 0.0163 while (6.1) gives ≈ 0.137, an order of magnitude higher. The chapter notebook measures both and recovers each formula within sampling noise.

The gap is structural. Predicate-OR asks $(\forall i : b_1[h_i(x)] = 1) \vee (\forall i : b_2[h_i(x)] = 1)$: one filter must have *all* k bits set for the query to pass. Bit-OR-Bloom asks $\forall i : (b_1[h_i(x)] = 1 \vee b_2[h_i(x)] = 1)$: at every queried position, *either* filter's bit suffices, and the responsible filter may differ position by position. Queries that fail predicate-OR can pass bit-OR-Bloom by combining bits from different sources, so bit-OR-Bloom is the more permissive operation and carries the larger FPR.

Predicate-OR is the union of approximate sets in the sense of Definition 6.1.1: the lift of $\vee$ at the predicate level, inheriting its rates from the composition theorem of Chapter 3. Bit-OR-Bloom is a different data-structure choice, the choice of inserting both sets into a single Bloom filter to save one bitmap and one k-position lookup per query, paid for by the higher FPR of (6.1). Bit-OR-Bloom also requires the two operand filters to share parameters and hash family; bit-wise OR between filters with different hash families produces a bit array whose query behavior does not correspond to any clean set-level operation. Predicate-OR is well-defined regardless. Exercises in this chapter and the chapter notebook revisit these distinctions.

False-positive rate

Proposition 6.1.1 (Union rate formulas)**.** *Let* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *be approximate sets of* $A, B \subseteq U$ *with per-block rates* $(\varepsilon_A, \omega_A)$ *and* $(\varepsilon_B, \omega_B)$ *respectively. Assume the error events of* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *at every query point* $x \in U$ *are mutually independent. Then the union* $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ *has false-positive rate*

$$\varepsilon_{\cup} = 1 - (1 - \varepsilon_A)(1 - \varepsilon_B). \tag{6.2}$$

The false-negative rate $\omega_\cup$ depends on the prior distribution over the three regions $A \setminus B$, $B \setminus A$, $A \cap B$ of the objective union $A \cup B$; for a query distribution that draws x from each region with respective weights w_1, w_2, w_3 summing to one,

$$\omega_\cup \;=\; w_1\omega_A(1-\varepsilon_B) \;+\; w_2\omega_B(1-\varepsilon_A) \;+\; w_3\omega_A\omega_B. \tag{6.3}$$

We prove the two parts in turn.

FPR derivation. Fix a query point $x \notin A \cup B$. Since x is outside both operand sets, the correct verdict for the union at x is $\bot$, and the only way for $(\mathsf{A}^\pm \cup \mathsf{B}^\pm)(x) = \top$ to occur is for at least one of the two operand predicates to falsely report $\top$ at x. The two events $\mathsf{A}^\pm(x) = \top$ (given $x \notin A$) and $\mathsf{B}^\pm(x) = \top$ (given $x \notin B$) are exactly the per-operand false-positive events at x, with conditional probabilities ε_A and ε_B respectively. By the cross-set independence assumption, the two events are independent. We compute the union false-positive event as the complement of the joint non-event:

$$\begin{aligned}
\mathrm{P}\left((\mathsf{A}^\pm \cup \mathsf{B}^\pm)(x) = \top | x \notin A \cup B\right) &= 1 - \mathrm{P}\left(\mathsf{A}^\pm(x) = \bot,\, \mathsf{B}^\pm(x) = \bot | x \notin A,\, x \notin B\right) \\
&= 1 - \mathrm{P}\left(\mathsf{A}^\pm(x) = \bot | x \notin A\right)\, \mathrm{P}\left(\mathsf{B}^\pm(x) = \bot | x \notin B\right) \\
&= 1 - (1-\varepsilon_A)(1-\varepsilon_B).
\end{aligned}$$

The result holds for every $x \notin A \cup B$ with the same probability, so the union's false-positive rate is $\varepsilon_\cup = 1 - (1-\varepsilon_A)(1-\varepsilon_B)$, as claimed.

FNR derivation. A false negative of the union at $x \in A \cup B$ requires both operand predicates to report $\bot$. Whether each report is itself a false negative or a true negative depends on which of the three regions x comes from. Partition the objective union into the disjoint blocks

$$A \cup B \;=\; (A \setminus B) \sqcup (B \setminus A) \sqcup (A \cap B).$$

For $x \in A \setminus B$ we have $x \in A$ and $x \notin B$, so $\mathsf{A}^\pm(x) = \bot$ is a false negative of $\mathsf{A}^\pm$ with probability ω_A, while $\mathsf{B}^\pm(x) = \bot$ is a true negative of $\mathsf{B}^\pm$ with probability $1 - \varepsilon_B$. By independence the joint probability that both predicates report $\bot$ on this region is $\omega_A(1-\varepsilon_B)$. The region $B \setminus A$ contributes symmetrically $\omega_B(1-\varepsilon_A)$. For $x \in A \cap B$ both operand predicates must miss their respective member, contributing $\omega_A\omega_B$ by independence. Weighting the three regions by the prior w_1, w_2, w_3 on $A \cup B$ and applying the law of total probability yields (6.3).

The role of the prior. The union false-positive rate $\varepsilon_\cup$ is a closed-form polynomial in the two operand FPRs alone; no prior is required because every negative of $A \cup B$ falls into one and the same region (the complement of the union). The union false-negative rate, in contrast, mixes three different per-region rates, and the closed form depends on how often a query lands in each region. The weights w_i are determined by the cardinalities $|A \setminus B|, |B \setminus A|, |A \cap B|$ together with the prior over $A \cup B$. When the operand sets are disjoint the third region vanishes and the formula simplifies; when the operands coincide the first two regions vanish and only the $\omega_A \omega_B$ term survives. The general expression is the *marginal* rate over the union; chapter 7's classification measures will revisit it for confidence-interval purposes.

Worked example: union of two Bloom filters

Let $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ be two Bloom filters over the same universe with the same parameters, with operand rates

$$\varepsilon_A = 0.01, \qquad \omega_A = 0, \qquad \varepsilon_B = 0.02, \qquad \omega_B = 0.$$

Both operands are FNR-zero, in the sense of Definition 5.4.1: they sit on the FNR-zero axis of the perfect filter line. The union has

$$\varepsilon_\cup \;=\; 1 - (1 - 0.01)(1 - 0.02) \;=\; 1 - 0.99 \times 0.98 \;=\; 1 - 0.9702 \;=\; 0.0298.$$

The union FNR is $\omega_\cup = 0$, by direct inspection of (6.3) with $\omega_A = \omega_B = 0$: every term in the weighted sum vanishes. Both realizations of the union preserve FNR-zero: predicate-OR because the OR of two correct $\top$'s is $\top$, and bit-OR-Bloom because every member of either operand set still has all k bits set in the OR'd array. The arithmetic above, however, is the predicate-OR FPR: it composes the two operand rates via $1 - (1 - \varepsilon_A)(1 - \varepsilon_B)$, the rate of the Boolean OR of two independent operand verdicts. The bit-OR-Bloom realization would give a different (larger) number through (6.1), since merging two filters into one bitmap inflates the load.

The increase from 0.02 (the worse operand) to 0.0298 is the price of the OR composition; the multiplicative second-order term $\varepsilon_A \varepsilon_B = 0.0002$ is small relative to the additive part, so for small operand rates the union FPR is well-approximated by the sum $\varepsilon_A + \varepsilon_B$.

Connection to the composition theorem

(6.2) is the composition theorem of Theorem 3.3.1 applied to the OR gate of Section 3.2 with $k = 2$ stages and $\eta_i = \varepsilon_i$. Chapter 3 §3.2 already analyzed

the OR-under-noise event at the per-bit level: the false positive of OR is the failure to read a $\perp$ from at least one input, and the probability of that joint failure factors into the product of per-input correctness probabilities $1 - \varepsilon_i$. Section 3.5 previewed the lift to sets; the present section completes it. The Boolean operation OR is the same at both levels and the algebra is the same; what changes is the interpretation of the rates. At the bit level ε_i is the per-bit crossover of an asymmetric channel; at the set level it is the per-block rate of an approximate set. The composition theorem governs both.

The connection is the central pedagogical claim of chapters 6 and later: every set operation has an underlying Boolean gate, every gate has its noise propagation rule from Chapter 3, and the set-level rate algebra is the Boolean noise algebra reread at the set scale. The remaining sections of this chapter walk through that correspondence for intersection, complement, and difference, then identify the regimes in which the composed rates leave the perfect filter line.

What comes next

Section 6.2 takes up intersection, complement, and difference. Intersection is the AND gate at the set level, with the dual structure of the union formulas: multiplicative FPR and an additive-style FNR. Complement is the NOT gate and swaps the two rates without further work. Set difference is intersection with the complemented second operand, and its rates fall out by composing the two formulas. Section 6.3 then turns to the regimes in which composition escapes the perfect filter line, when a chain of operations forces both rates positive even though each operand sat on an axis, and Section 6.4 restates the line as the formal subalgebra it really is.

6.2 Intersection, Complement, Difference

The union of Section 6.1 was the first compositional step. The Boolean algebra of classical sets has three other operations to account for: intersection, complement, and difference. Each lifts to approximate sets in the same way the union did. The membership predicate of the composed set is a Boolean combination of the operand predicates, and the rates of the composed set follow from the corresponding gate analysis of Chapter 3 together with the cross-set independence assumption already invoked in §6.1. This section takes the three operations in turn, derives the rate formulas, collects them

in a worked numerical example, and signposts the cancellation phenomena that §6.3 will scrutinize.

Intersection

A classical intersection $A \cap B$ has the AND of the two operand membership predicates: $\mathbb{1}_{A\cap B}(x) = \mathbb{1}_A(x) \wedge \mathbb{1}_B(x)$. The predicate framing carries over.

Definition 6.2.1 (Intersection of approximate sets)**.** *Let* $\mathsf{A}^{\pm}, \mathsf{B}^{\pm} : U \to$ Bernoulli[bool] *be two approximate sets over the same universe* U*. Their* intersection $\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm}$ *is the approximate set whose membership predicate is the Boolean AND of the two operand predicates: for every* $x \in U$*,*

$$(\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm})(x) \;=\; \mathsf{A}^{\pm}(x) \wedge \mathsf{B}^{\pm}(x).$$

The intersection approximates the objective set $A \cap B$*, where* A *and* B *are the objective sets that* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *represent.*

The operational realization for two Bloom filters with the same parameters and the same hash family is the bit-wise AND of the two bit arrays, in the same spirit as the bit-wise OR for union. The bit-AND construction has a subtlety we will flag at the end of this subsection; the predicate-level definition above is unambiguous and is the one the rate algebra operates on.

Proposition 6.2.1 (Intersection rate formulas)**.** *Let* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *be approximate sets of* $A, B \subseteq U$ *with per-block rates* $(\varepsilon_A, \omega_A)$ *and* $(\varepsilon_B, \omega_B)$ *respectively. Assume the error events of the two operands at every query point* x *are mutually independent (the same cross-set independence assumption as Proposition 6.1.1). Then the intersection* $\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm}$ *has false-negative rate*

$$\omega_{\cap} \;=\; 1 - (1 - \omega_A)(1 - \omega_B), \tag{6.4}$$

and false-positive rate

$$\varepsilon_{\cap} \;=\; \varepsilon_A \cdot \varepsilon_B \tag{6.5}$$

on negatives at which the two operand sets agree (the bulk regime; see below).

FNR derivation. Fix a query point $x \in A \cap B$. Both operand predicates should report $\top$. A false negative of the intersection occurs when at least one of the two reports $\bot$, that is, when the AND fails to be $\top$. We compute the false-negative event as the complement of the joint correct-report event:

$$\begin{aligned}
\mathrm{P}\left((\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm})(x) = \bot | x \in A \cap B\right) &= 1 - \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \top,\, \mathsf{B}^{\pm}(x) = \top | x \in A,\, x \in B\right) \\
&= 1 - \mathrm{P}\left(\mathsf{A}^{\pm}(x) = \top | x \in A\right)\, \mathrm{P}\left(\mathsf{B}^{\pm}(x) = \top | x \in B\right) \\
&= 1 - (1 - \omega_A)(1 - \omega_B).
\end{aligned}$$

The probability is the same for every $x \in A \cap B$, so the intersection's false-negative rate is (6.4). The formula is the dual of the union FPR (6.2) with ω in place of ε, exactly as De Morgan duality leads us to expect.

FPR derivation, simplified. A false positive of the intersection requires both operand predicates to report $\top$ at an $x \notin A \cap B$. For $x \notin A$ and $x \notin B$ simultaneously, the two reports are false positives of the operands with conditional probabilities ε_A and ε_B, so by cross-set independence the joint event has probability $\varepsilon_A \cdot \varepsilon_B$, giving (6.5). The full intersection FPR mixes this contribution with two boundary regions where x lies in exactly one operand set; the resulting weighted average is bounded above by $\varepsilon_A + \varepsilon_B$ and below by $\varepsilon_A \cdot \varepsilon_B$, and the simplified form is exact when the two operand sets coincide on the negatives of the intersection. We adopt (6.5) as the working formula for the intersection FPR; the full mixture appears in the Proposition 6.1.1-style derivation that the companion composition paper records.

The multiplicative shape matters. The intersection FPR is the *product* of the operand FPRs, not a sum. For small operand rates the contrast with the union is sharp: two filters at $\varepsilon \approx 0.01$ give a union FPR near 0.02 but an intersection FPR near 0.0001. Intersection drives false positives exponentially toward zero with the number of independent filters; union inflates them roughly linearly. The dual fact for false negatives is that intersection's FNR adds in the same inclusion-exclusion sense the union's FPR did, while union's FNR is multiplicative across the regions where both operands could miss.

Complement

Complement is the simplest of the three operations to analyze.

Definition 6.2.2 (Complement of an approximate set)**.** *Let* $\mathsf{A}^{\pm} : U \to$ Bernoulli[bool] *be an approximate set of* $A \subseteq U$*. Its* complement $\complement\mathsf{A}^{\pm}$ *is the approximate set whose membership predicate is the Boolean NOT of* $\mathsf{A}^{\pm}$*'s predicate:* $(\complement\mathsf{A}^{\pm})(x) = \neg\mathsf{A}^{\pm}(x)$*. The complement approximates the classical complement* $\complement A = U \setminus A$*.*

Proposition 6.2.2 (Complement-swap of rates)**.** *If* $\mathsf{A}^{\pm}$ *has per-block rates* $(\varepsilon_A, \omega_A)$*, then its complement* $\complement\mathsf{A}^{\pm}$ *has rates*

$$\varepsilon_{\complement} = \omega_A, \qquad \omega_{\complement} = \varepsilon_A. \tag{6.6}$$

The result is the set-level reading of the NOT-under-noise calculation already done in Section 3.2 at the per-bit level, lifted from the noisy bit to the noisy predicate. We give the two-line proof.

A false positive of $\complement\mathsf{A}^{\pm}$ is the event that the truth says negative and the report says positive on the complement side: $x \in \complement A$ is false (so $x \in A$) while $\complement\mathsf{A}^{\pm}(x) = \top$ (so $\mathsf{A}^{\pm}(x) = \bot$). The combined event is $\mathsf{A}^{\pm}(x) = \bot$ given $x \in A$, which is a false negative of $\mathsf{A}^{\pm}$ at rate ω_A. So $\varepsilon_{\complement} = \omega_A$. Dually, a false negative of $\complement\mathsf{A}^{\pm}$ is the event $x \in \complement A$ true (so $x \notin A$) while $\complement\mathsf{A}^{\pm}(x) = \bot$ (so $\mathsf{A}^{\pm}(x) = \top$). The combined event is $\mathsf{A}^{\pm}(x) = \top$ given $x \notin A$, which is a false positive of $\mathsf{A}^{\pm}$ at rate ε_A. So $\omega_{\complement} = \varepsilon_A$. The two rates swap, as (6.6) records.

The same result was proven at the noisy-bit level in Section 3.2. The lifting to the set level uses the predicate framing of Chapter 5 and nothing else. NOT is its own inverse: applying complement twice returns the original rates, $(\varepsilon_A, \omega_A) \mapsto (\omega_A, \varepsilon_A) \mapsto (\varepsilon_A, \omega_A)$. The complement operation is an involution on the rate plane $[0,1]^2$, exchanging the two coordinate axes.

Difference

Set difference is intersection with the complemented second operand.

Definition 6.2.3 (Difference of approximate sets)**.** *For approximate sets $\mathsf{A}^{\pm}, \mathsf{B}^{\pm}$ over U, the* difference $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ *is the approximate set whose membership predicate is $\mathsf{A}^{\pm}(x) \wedge \neg\mathsf{B}^{\pm}(x)$; equivalently, $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm} = \mathsf{A}^{\pm} \cap \complement\mathsf{B}^{\pm}$. It approximates the classical difference $A \setminus B = A \cap \complement B$.*

Proposition 6.2.3 (Difference rate formulas)**.** *Under the same cross-set independence as Proposition 6.2.1, the difference $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ has false-positive rate*

$$\varepsilon_{\setminus} \;=\; \varepsilon_A \cdot \omega_B \tag{6.7}$$

on the bulk negative region, and false-negative rate

$$\omega_{\setminus} \;=\; 1 - (1 - \omega_A)(1 - \varepsilon_B). \tag{6.8}$$

The proof is compositional. The complement of $\mathsf{B}^{\pm}$ has rates $(\varepsilon_B^{\complement}, \omega_B^{\complement}) = (\omega_B, \varepsilon_B)$ by Proposition 6.2.2. Substitute these into the intersection formulas of Proposition 6.2.1 applied to $\mathsf{A}^{\pm}$ and $\complement\mathsf{B}^{\pm}$:

$$\varepsilon_{\setminus} \;=\; \varepsilon_A \cdot \varepsilon_B^{\complement} \;=\; \varepsilon_A \cdot \omega_B,$$

which is (6.7), and

$$\omega_{\setminus} \;=\; 1 - (1 - \omega_A)(1 - \omega_B^{\complement}) \;=\; 1 - (1 - \omega_A)(1 - \varepsilon_B),$$

which is (6.8).

The formulas mix the two operands' rates in a characteristic way: the difference FPR pairs ε_A with ω_B (the swapped complement-side FPR), and the difference FNR pairs ω_A with ε_B (the swapped complement-side FNR). Both operands' *both* rates appear in the answer; even an FNR-zero $\mathsf{A}^{\pm}$ inherits false negatives from the difference when $\mathsf{B}^{\pm}$ has a nonzero FPR, because elements of $A \setminus B$ that are false positives of $\mathsf{B}^{\pm}$ are wrongly excluded.

Worked numerical example

Let $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ be two Bloom filters over the same universe with the same parameters and operand rates

$$\varepsilon_A = 0.01, \qquad \omega_A = 0, \qquad \varepsilon_B = 0.02, \qquad \omega_B = 0.$$

Both operands are FNR-zero (Definition 5.4.1), so they sit on the FNR-zero axis of the perfect filter line. The intersection has

$$\varepsilon_{\cap} \;=\; \varepsilon_A \cdot \varepsilon_B \;=\; 0.01 \times 0.02 \;=\; 0.0002, \qquad \omega_{\cap} \;=\; 1 - (1-0)(1-0) \;=\; 0.$$

The intersection of two FNR-zero Bloom filters is itself FNR-zero, and its false-positive rate is smaller than either operand's by an order of magnitude (an absolute drop of nearly a factor of 50 relative to the worse operand). The contrast with the union's $\varepsilon_{\cup} = 0.0298$ of Section 6.1 is striking: composing two filters multiplicatively reduces the false-positive rate, while composing them disjunctively inflates it. The two operations are exact duals of one another, and the choice of which to use is dictated by what one wants the composed filter to assert.

The complement of $\mathsf{A}^{\pm}$ has rates $(\varepsilon_{\complement}, \omega_{\complement}) = (\omega_A, \varepsilon_A) = (0, 0.01)$. The complement has moved $\mathsf{A}^{\pm}$ off the FNR-zero axis and onto the FPR-zero axis: what was a one-sided false-positive machine is now a one-sided false-negative machine. The next section returns to this geometric observation.

The difference $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ has

$$\varepsilon_{\setminus} \;=\; \varepsilon_A \cdot \omega_B \;=\; 0.01 \times 0 \;=\; 0, \qquad \omega_{\setminus} \;=\; 1 - (1-0)(1-0.02) \;=\; 0.02.$$

The difference of two FNR-zero filters has dropped its false-positive rate to zero (because $\omega_B = 0$ kills the FPR product) and picked up a false-negative rate equal to ε_B. The structural reason is that $\mathsf{B}^{\pm}$'s false positives become misclassified exclusions in $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$: an element of $A \setminus B$ that $\mathsf{B}^{\pm}$ erroneously reports as a member of B will be filtered out by the difference even though it should be kept. The difference exemplifies the cancellation phenomena §6.3 will scrutinize.

A caveat on bit-AND of Bloom filters

The intersection of Definition 6.2.1 is a predicate-level operation: query $\mathsf{A}^{\pm}$, query $\mathsf{B}^{\pm}$, AND the two Boolean outputs. For two Bloom filters with the same parameters and the same hash family, this predicate intersection is the canonical definition and the rate formulas of Proposition 6.2.1 apply to it. The operational shortcut of bit-wise AND of the two underlying bit arrays is a different construction. The bit-AND of two Bloom filters gives a bit array whose query behaviour reports $\top$ when the AND of the bit arrays has 1 at every queried position; this is the same as both filters individually reporting $\top$, so on items inserted into *both* sets the bit-AND query matches the predicate AND. On items inserted into one set only, however, the bit-AND filter can still report $\top$ when both bits happen to be set in the AND'd array (one from the legitimate insertion, one from a coincidental hash collision of an insertion of the other set), and the resulting filter is not in general a Bloom filter for $A \cap B$. The bit-AND is an operational shortcut that coincides with the predicate intersection in some regimes and not others; the chapter notebook explores the discrepancy empirically, and we will not rely on bit-AND in the rate analyses to come.

What comes next

The four operations of §6.1 and §6.2 (union, intersection, complement, difference) suffice to express every Boolean combination of approximate sets, with closed-form rate formulas derived from the composition theorem of Theorem 3.3.1 applied to the corresponding Boolean gate. Section 6.3 takes up the phenomena that the difference example already foreshadowed: composition can move an operand off the perfect filter line, can swap which side of the line a one-sided set sits on, and can introduce both rates as nonzero even when each input had one rate set to zero by construction. Section 6.4 then restates the line itself as a formal subalgebra: the FNR-zero axis is closed under union and intersection, the FPR-zero axis is closed dually, and the two axes together form a non-convex set whose closure properties this chapter's operations either preserve or break.

6.3 Cancellation and the Loss of One-Sidedness

The operand rates of Section 6.1 and Section 6.2 were drawn from one-sided approximate sets sitting on the perfect filter line of Definition 5.4.1. The union of two FNR-zero Bloom filters stayed FNR-zero; the intersection of

two FNR-zero Bloom filters stayed FNR-zero; the worked examples landed back on the same axis the operands came from. This section asks the natural follow-up question. Are the operations of chapter 6 always axis-preserving in this way, or can a chain of operations take an operand off the line entirely? The answer is that composition can both move a one-sided set to the other axis and force both rates positive at once. The phenomenon is unavoidable once the complement operator enters the picture and intersection involves a mixture of axes, and §6.4 will reread it as a closure question about the perfect filter line.

Complement swaps which axis

The simplest escape from a single axis is the complement operation. Proposition 6.2.2 stated the swap at the rate level: if $\mathsf{A}^{\pm}$ has rates $(\varepsilon_A, \omega_A)$, then $\complement\mathsf{A}^{\pm}$ has rates $(\varepsilon_{\complement}, \omega_{\complement}) = (\omega_A, \varepsilon_A)$. For a one-sided operand the consequences are immediate.

Proposition 6.3.1 (Complement swaps the perfect-filter axis)**.** *Let $\mathsf{A}^{\pm}$ be a one-sided approximate set in the sense of Definition 5.4.1. Then $\complement\mathsf{A}^{\pm}$ is also one-sided, but on the other axis: if $\mathsf{A}^{\pm}$ is FNR-zero then $\complement\mathsf{A}^{\pm}$ is FPR-zero, and if $\mathsf{A}^{\pm}$ is FPR-zero then $\complement\mathsf{A}^{\pm}$ is FNR-zero.*

Proof. By Proposition 6.2.2 the complement sends $(\varepsilon_A, \omega_A)$ to $(\omega_A, \varepsilon_A)$. If $\mathsf{A}^{\pm}$ is FNR-zero then $\omega_A = 0 < \varepsilon_A$, so the swap produces $(\varepsilon_{\complement}, \omega_{\complement}) = (0, \varepsilon_A)$ with $\varepsilon_{\complement} = 0 < \omega_{\complement}$: the complement is FPR-zero. The FPR-zero case is symmetric. □

Geometrically the rate plane has two coordinate axes and the complement operator exchanges them. A Bloom filter with $\varepsilon = 0.01$ and $\omega = 0$ sits at the point $(0.01, 0)$ on the FNR-zero axis. Its complement sits at the point $(0, 0.01)$ on the FPR-zero axis. The two structures encode the same information in dual senses. The Bloom filter is certain about absence and uncertain about presence, and its complement is certain about presence and uncertain about absence. A single application of complement walks across the rate plane from one axis to the other; nothing in between is visited because the operation is a discrete reflection rather than a continuous deformation.

Worked example: complement of a Bloom union

The first compound escape combines union and complement. Let $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ be two FNR-zero Bloom filters with operand rates

$$\varepsilon_A = 0.01, \qquad \omega_A = 0, \qquad \varepsilon_B = 0.02, \qquad \omega_B = 0,$$

the same numerical example as Section 6.1 and Section 6.2. The union $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ inherits both rates from Proposition 6.1.1: it is FNR-zero with

$$\varepsilon_{\cup} \;=\; 1 - (1 - 0.01)(1 - 0.02) \;=\; 0.0298, \qquad \omega_{\cup} \;=\; 0.$$

Now form the complement $\complement(\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm})$ and apply Proposition 6.2.2. The complement swaps the union's two rates:

$$\varepsilon' \;=\; \omega_{\cup} \;=\; 0, \qquad \omega' \;=\; \varepsilon_{\cup} \;=\; 0.0298.$$

The original two operands sat on the FNR-zero axis. The union stayed on that axis. The complement of the union moved to the FPR-zero axis. A two-step chain (union, then complement) has walked a one-sided approximate set from one half of the perfect filter line to the other. The composed structure is still one-sided, and still has only one degree of rate-freedom, but the certain direction has flipped. Where the union answered "is x in A or in B?" and reported $\bot$ with certainty, the complement of the union answers "is x outside both A and B?" and reports $\top$ with certainty. The rate algebra has done the bookkeeping for us; the qualitative content of the structure has changed.

Worked example: intersection with a complemented operand

The second compound escape composes intersection with a single complement and produces both rates positive at once. Let $\mathsf{A}^{\pm}$ again be FNR-zero with $\varepsilon_A = 0.01, \omega_A = 0$, and let $\mathsf{B}^{\pm}$ be FNR-zero with $\varepsilon_B = 0.02, \omega_B = 0$. The complement $\complement\mathsf{B}^{\pm}$ has rates $(0, 0.02)$ by Proposition 6.2.2; it sits on the FPR-zero axis. The mixed intersection $\mathsf{A}^{\pm} \cap \complement\mathsf{B}^{\pm}$ asks both operands to report $\top$ on points of $A \cap \complement B = A \setminus B$; this is the difference of Definition 6.2.3, which the intersection-with-complement decomposition makes explicit.

Apply Proposition 6.2.1 to the two operands $\mathsf{A}^{\pm}$ with rates $(\varepsilon_A, \omega_A) = (0.01, 0)$ and $\complement\mathsf{B}^{\pm}$ with rates $(\omega_B, \varepsilon_B) = (0, 0.02)$:

$$\varepsilon_{\cap} \;=\; \varepsilon_A \cdot \omega_B \;=\; 0.01 \times 0 \;=\; 0,$$

$$\omega_{\cap} \;=\; 1 - (1 - \omega_A)(1 - \varepsilon_B) \;=\; 1 - (1 - 0)(1 - 0.02) \;=\; 0.02.$$

The composition has landed at $(0, 0.02)$ on the FPR-zero axis in this specific arithmetic; both numbers came out as integer-times-zero products of the FNR-zero structure of the operands. The point is not that the result is generic; the point is that the moment one operand's FNR is genuinely

positive, both output rates also become positive. Take the same composition with $\varepsilon_A = 0.01$, $\omega_A = 0$ and $\complement\mathsf{B}^{\pm}$ having rates $(\varepsilon_B^{\complement}, \omega_B^{\complement}) = (\omega_B, \varepsilon_B) = (0.005, 0.02)$ (coming from a two-sided $\mathsf{B}^{\pm}$ with rates $(0.02, 0.005)$):

$$\varepsilon_{\cap} \;=\; \varepsilon_A \cdot \varepsilon_B^{\complement} \;=\; 0.01 \times 0.005 \;=\; 5 \times 10^{-5},$$

$$\omega_{\cap} \;=\; 1 - (1 - \omega_A)(1 - \omega_B^{\complement}) \;=\; 1 - 1 \times 0.98 \;=\; 0.02.$$

Both rates are now strictly positive. The composition has left both axes and landed in the two-sided interior of the rate plane. The operands did not need to be far from the line for this to happen; even an arbitrarily small leak in the FNR-zero structure of one operand produces a two-sided composed set.

Proposition 6.3.2 (Mixed-axis intersection escapes the line)**.** *Let $\mathsf{A}^{\pm}$ be FNR-zero with $\varepsilon_A > 0$ and let $\mathsf{B}^{\pm}$ be FPR-zero with $\omega_B > 0$. Then $\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm}$ generically has both rates strictly positive: $\varepsilon_{\cap} = 0$ and $\omega_{\cap} = \omega_B > 0$ in one boundary regime, while a two-sided $\mathsf{B}^{\pm}$ with any positive ε_B in place of the strict FPR-zero gives $\varepsilon_{\cap} = \varepsilon_A \varepsilon_B > 0$ and $\omega_{\cap} = 1 - (1 - \omega_B) > 0$. The line is not preserved by this composition.*

Proof. Substitute the operand rates into Proposition 6.2.1. The boundary case has $\varepsilon_B = 0$, giving $\varepsilon_{\cap} = \varepsilon_A \cdot 0 = 0$ and $\omega_{\cap} = 1 - (1 - 0)(1 - \omega_B) = \omega_B$. The non-boundary case has $\varepsilon_B > 0$, giving the two positive expressions above. In neither case does the result fall on the FNR-zero or FPR-zero axis of the operands. □

The escape, viewed geometrically

The rate plane $[0, 1]^2$ has two distinguished one-dimensional loci: the FNR-zero axis (the segment with $\omega = 0$ and $\varepsilon > 0$) and the FPR-zero axis (the segment with $\varepsilon = 0$ and $\omega > 0$). Their union is the perfect filter line. Three classes of compositional behavior arise.

Axis-preserving operations. Union of two FNR-zero sets stays FNR-zero. Intersection of two FNR-zero sets stays FNR-zero. Symmetric statements hold on the FPR-zero side. These are the compositions in which both operands sit on the same axis and the rate algebra returns a result on the same axis. The operations of Proposition 6.1.1 and Proposition 6.2.1 drive false positives and false negatives toward zero on the operand's axis without ever "leaking" the other rate.

Axis-swapping operations. Complement of a one-sided set is one-sided on the other axis (Proposition 6.3.1). The composition does not stay on

either axis individually but stays on the line as a set: it walks across the line, from one half to the other, in a single discrete step.

Axis-leaving operations. Intersection of an FNR-zero set with an FPR-zero set is generically two-sided (Proposition 6.3.2). Difference $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$, being intersection with the complemented second operand, is two-sided unless one of the operands has both rates zero or both operands sit on the same axis after the complement. These are the compositions that leave the line entirely; the composed rate lives in the interior of the rate square, with both directions of report now uncertain.

The three classes exhaust the qualitative behavior of the chapter 6 operations on the perfect filter line. The line is closed under axis- preserving operations, mapped to itself by axis-swapping operations, and exited by axis-leaving operations. Section 6.4 will state this classification as a closure proposition; for now it is enough that we have identified the qualitative regimes and connected each one to a specific composition.

Why this matters

The cancellation phenomena are not abstract. Three downstream chapters depend on whether a composed set sits on the perfect filter line.

Chapter 7 develops PPV and NPV from the FPR and FNR. A one-sided structure simplifies the formulas: an FNR-zero structure has NPV $= 1$ on its domain, and the confidence-interval treatment reduces to the one rate that is nonzero. A two-sided structure exercises the full machinery, with posterior weight on both directions of report and intervals on both rates.

Chapter 14 develops the Bernoulli Hash Function and the entropy-floor argument for FNR-zero approximate sets. The information-theoretic lower bound is $-\log_2 \varepsilon$ bits per element at false-positive rate ε, and the BHF achieves this floor up to a small additive constant. A compositional chain that leaves the axis puts the result outside the regime the bound measures; the resulting two-sided structure would need a separate analysis to assess space-optimality.

Chapter 15 develops the perfect hash filter and achieves FPR-zero on a designated input set A, paying with a construction-time perfect-hash computation. The chapter operates entirely on the FPR-zero axis. Composing a perfect hash filter with a Bloom filter via mixed-axis intersection produces a two-sided structure by Proposition 6.3.2, and the certain $\top$ answer is lost.

The pattern across chapters 7, 14, and 15 is the same. Each chapter is the constructive realization of one half of the perfect filter line at the lowest space cost, and each pays a real penalty when a compositional chain forces

the result off the line. Composition discipline is the price of staying near the floor.

What comes next

Section 6.4 states the perfect filter line as a formal definition and collects the closure results of this section into a coherent algebraic statement. The FNR-zero axis is closed under union and intersection but not under complement; the FPR-zero axis is closed dually; the union of the two axes is closed under no single operation but is mapped to itself by axis-swapping operations and exited by mixed-axis intersection. The section organizes the classification, and chapters 7, 14, and 15 then take up the consequences of staying on the line or leaving it.

6.4 The Perfect Filter Line

Section 5.4 named the perfect filter line as the union of the two coordinate axes of the rate plane and previewed its relevance to chapter 6. The composition formulas of Section 6.1 and Section 6.2, together with the cancellation classification of Section 6.3, have given us everything required to read the line as an algebraic object: a distinguished subset of $[0,1]^2$ that some operations preserve, some move within, and some leave. This section collects the closure results into a single statement, makes the geometric structure of the line explicit, and points forward to the two chapters that take each half of the line as their constructive target.

Definition

Definition 6.4.1 (The perfect filter line)**.** *The* perfect filter line *is the subset of the rate plane* $[0,1]^2$ *consisting of points with at least one coordinate equal to zero. Explicitly, the line is the union of two coordinate axes:*

$$\mathcal{L} \;=\; \mathcal{L}_\omega \cup \mathcal{L}_\varepsilon,$$

where the FNR-zero axis *is* $\mathcal{L}_\omega = \{(\varepsilon, 0) : 0 < \varepsilon \leq 1\}$ *and the* FPR-zero axis *is* $\mathcal{L}_\varepsilon = \{(0, \omega) : 0 < \omega \leq 1\}$*. An approximate set* lies on the line *when its rate point* (ε, ω) *belongs to* $\mathcal{L}$*.*

The two axes meet only at the origin $(0,0)$, which is the exact-set point and is excluded from $\mathcal{L}$ by the strict inequalities in the definition. An exact set is not an approximate set, in the sense that one of its two rates is not

merely zero but unmeasured: the construction does not face the trade-offs the rate plane parameterizes. The perfect filter line is the set of approximate sets that achieve certainty in exactly one direction of report; its origin neighbor achieves certainty in both, at the cost of the space savings that motivated approximate sets in the first place.

Closure properties of the FNR-zero axis

The composition results of the previous sections specialize cleanly when one or both operands sit on $\mathcal{L}_\omega$. We collect the four cases.

Proposition 6.4.1 (FNR-zero axis closure)**.** *Let* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *be FNR-zero approximate sets, so* $\omega_A = \omega_B = 0$ *and the operand rate points lie on* $\mathcal{L}_\omega$*. Then:*

1. $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ *is FNR-zero, with* $\omega_\cup = 0$ *and* $\varepsilon_\cup = 1 - (1 - \varepsilon_A)(1 - \varepsilon_B)$*.*
2. $\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm}$ *is FNR-zero, with* $\omega_\cap = 0$ *and* $\varepsilon_\cap = \varepsilon_A \varepsilon_B$*.*
3. $\complement\mathsf{A}^{\pm}$ *is FPR-zero, with rates* $(0, \varepsilon_A)$*; the complement maps the FNR-zero axis to the FPR-zero axis pointwise.*
4. $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ *is FNR-zero when* $\mathsf{B}^{\pm}$ *is in addition FPR-zero (so* $\varepsilon_B = 0$ *also), giving rates* $(0, \omega_B \cdot 0$ *terms vanish); otherwise the difference generically leaves the line.*

In short, $\mathcal{L}_\omega$ *is closed under union and intersection, mapped to* $\mathcal{L}_\varepsilon$ *by complement, and only closed under difference under a strong condition on the second operand.*

Proof. Parts (1) and (2) substitute $\omega_A = \omega_B = 0$ into Proposition 6.1.1 and Proposition 6.2.1. The union FNR formula (6.3) of Section 6.1 becomes $w_1 \cdot 0 \cdot (1 - \varepsilon_B) + w_2 \cdot 0 \cdot (1 - \varepsilon_A) + w_3 \cdot 0 \cdot 0 = 0$, and the intersection FNR formula (6.4) becomes $1 - (1 - 0)(1 - 0) = 0$. The two FPR formulas follow without modification because they do not involve the FNRs of the operands.

Part (3) is the special case of Proposition 6.2.2 with $\omega_A = 0$. The swap sends $(\varepsilon_A, 0)$ to $(0, \varepsilon_A)$, which is a point on $\mathcal{L}_\varepsilon$.

Part (4) substitutes the operand rates into Proposition 6.2.3. With $\omega_A = \omega_B = 0$ the difference FNR formula becomes $\omega_\setminus = 1 - (1 - 0)(1 - \varepsilon_B) = \varepsilon_B$, which is positive whenever $\varepsilon_B > 0$, so the difference leaves $\mathcal{L}_\omega$. Imposing $\varepsilon_B = 0$ in addition makes $\mathsf{B}^{\pm}$ a perfect filter on $\mathcal{L}_\varepsilon$, the difference FNR collapses to 0, and $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ reduces to an exact-set test. □

Union and intersection are the two operations that close $\mathcal{L}_\omega$ on its own; complement closes the line $\mathcal{L}$ as a whole by exchanging the two axes; and difference is conditionally closed, requiring the second operand to be a perfect filter on the opposite axis. The cancellation analysis of Section 6.3 already recorded the geometric reason difference fails: it is intersection with a complemented operand, and the intermediate complement walks the second operand across the line.

Closure properties of the FPR-zero axis

The symmetric statement for $\mathcal{L}_\varepsilon$ follows from $\mathcal{L}_\omega$ by applying the complement-swap to both operands of any composition and unswapping the result. We record the companion proposition without re-deriving each case.

Proposition 6.4.2 (FPR-zero axis closure)**.** *Let* $\mathsf{A}^{\pm}$ *and* $\mathsf{B}^{\pm}$ *be FPR-zero approximate sets, so* $\varepsilon_A = \varepsilon_B = 0$*. Then* $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ *has* $\varepsilon_\cup = 0$*, and* $\mathsf{A}^{\pm} \cap \mathsf{B}^{\pm}$ *has* $\varepsilon_\cap = 0$*; both compositions are FPR-zero. The complement of an FPR-zero set is FNR-zero, mapping* $\mathcal{L}_\varepsilon$ *to* $\mathcal{L}_\omega$ *pointwise. The difference* $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ *is FPR-zero only under a strong condition on the second operand and is otherwise two-sided.*

The proof is by complement-swap from Proposition 6.4.1: any FPR-zero set is the complement of an FNR-zero set, the rate-plane algebra is symmetric under coordinate exchange, and the conclusions transpose accordingly.

The algebraic shape of the line

The line $\mathcal{L}$ is a non-convex subset of $[0,1]^2$: the midpoint of any segment from a point $(\varepsilon, 0)$ on $\mathcal{L}_\omega$ to a point $(0, \omega)$ on $\mathcal{L}_\varepsilon$ is $(\varepsilon/2, \omega/2)$, which lies in the interior of the rate square and not on either axis. The two axes meet only at the excluded origin, so $\mathcal{L}$ is a wedge of two open segments rather than a connected curve; it has two "arms" meeting at the corner the definition excises. The shape matters because no convex combination of operations preserves the line: mixing one operand from each axis produces a point off both, which is exactly the conclusion Proposition 6.3.2 reached in Section 6.3.

This is the formal sense in which the perfect filter line is a *subalgebra* of the rate plane: closed under a specific subset of the chapter 6 operations, exited by the remaining operations, and internally structured into two halves that the complement operator swaps. Same-axis operations compose freely without leaving the line; cross-axis operations require care, because their

composed structure is no longer a perfect filter and the chapter 14 and chapter 15 space guarantees do not apply.

Forward to chapters 14 and 15

Two later chapters take the two halves of the line as their constructive concern. The forward references close the picture of why the line matters for the rest of Part II.

Chapter 14 develops the Bernoulli Hash Function, the constructive realization of an approximate set on $\mathcal{L}_\omega$ at the lowest possible space cost. The information-theoretic floor of chapter 8 gives the minimum number of bits per element required to represent an FNR-zero approximate set at false-positive rate ε; the BHF achieves the floor up to a small additive constant, closing the $\log_2 e \approx 1.44$ multiplicative overhead the standard Bloom filter of chapter 5 leaves above the floor on the same axis. The closure results of this section then govern composed BHFs: union and intersection of BHFs remain on $\mathcal{L}_\omega$ at rates the chapter 6 formulas predict, so the entropy floor extends to the composition.

Chapter 15 develops the perfect hash filter, the constructive realization on $\mathcal{L}_\varepsilon$ for a designated input set A. A perfect-hash computation at build time stores a fingerprint per element; a query for $x \in A$ lands on its fingerprint and is reported correctly with probability one, while a query for $x \notin A$ may still hash to a stored fingerprint. The construction sits on the FPR-zero axis and stays there under same-axis union and intersection by Proposition 6.4.2. Mixed-axis composition with a BHF, by Proposition 6.3.2, produces a two-sided structure off the line and the FPR-zero certainty is lost; the designer must respect the closure constraints to retain the constructive guarantees of either chapter.

6.5 What Comes Next

The chapter has fixed the algebra. Section 6.1 and Section 6.2 gave closed-form rates for union, intersection, complement, and difference of approximate sets. Section 6.3 classified the compositions that escape the perfect filter line. Section 6.4 collected the closure results into a single subalgebra statement and named the two halves of the line. What we have not yet done is connect those rates to the two questions that motivate the rest of Part II: how good is an approximate set when measured against a workload, and how small can one be made? The next two chapters take up each question in turn.

Chapter 7: rates feed into classification measures

Chapter 7 takes the per-block rates (ε, ω) this chapter computed and turns them into the workload-dependent quality measures practitioners actually report: positive predictive value, negative predictive value, F_1, and the recall-precision pair. Each is a function of (ε, ω) and the prevalence of in-set elements in the query stream. The composition formulas of this chapter feed those derivations directly. A union of two FNR-zero Bloom filters at output FPR $\varepsilon_\cup$, by Proposition 6.4.1, is itself FNR-zero; chapter 7's PPV expression in $\varepsilon_\cup$ and the workload prevalence then gives the PPV for the composed structure with no additional work.

Chapter 7 also retires the deferral Section 1.4 flagged in Part I. The Wald confidence interval for a binomial rate was a working placeholder, and the chapter introduces the Wilson, Clopper-Pearson, and Jeffreys alternatives that handle the boundary at $\varepsilon = 0$ or $\varepsilon = 1$ properly. The estimator vocabulary chapter 1 left incomplete is finished there.

Chapter 8: rates have a space-cost analog

Chapter 8 bounds how compact an approximate set can be. Every operation of this chapter has a space-cost analog: the entropy of an FNR-zero set with false-positive rate ε is $-\log_2 \varepsilon$ bits per stored element, the entropy of a composed structure is the entropy of its rate point, and the chapter 6 closure results on the FNR-zero axis then bound the bits per element a composition can possibly cost. The floor governs compression along the axis; Chapter 14 achieves it constructively with the Bernoulli Hash Function, closing the standard Bloom filter's $\approx 44\%$ overhead. The composition theorem of chapter 3 returns in chapter 8 dressed as the bit-budget arithmetic for chained approximate sets, and the chapter 6 formulas this section closes are the operations the bit budget applies to.

Closing

The bibliographic notes that follow name the antecedents of the set-operation framework: the Bernoulli composition manuscript that collected the closed-form rates, Mitzenmacher and Upfal for the survey context of Bloom-filter union and intersection, and the chapter 3 treatment of De Morgan under noise that the complement-swap result specializes. Chapter 7 then opens the second half of Part II.

Bibliographic Notes

The composition framework. The rate-propagation formulas of Sections 6.1 and 6.2 are the set-level instances of a more general framework developed in Towell [Tow26d]. That manuscript proves the rate-propagation laws for the full algebra of approximate sets—union, intersection, complement, difference, and the symmetric difference—under independence of the constituent Bernoulli[bool] outputs, and catalogues which compound operations preserve and which destroy one-sidedness. The chapter's contribution is to derive each rule constructively from the predicate algebra of Section 5.3 and the AND/OR/NOT rules of Section 3.2, rather than stating the formulas as a fait accompli. The composition manuscript treats the rules at the level of joint (ε, ω) pairs; chapter 6 unpacks the proofs and connects each rule back to its Boolean-algebra ancestor in chapter 3.

Bloom filter union and intersection. The well-known fact that two Bloom filters of the same length and the same hash family admit a bitwise union (logical OR of the bit arrays) and a bitwise intersection (logical AND) is folklore by now, but the analysis of what the resulting filter *computes*—and at what error rate—is treated systematically in Mitzenmacher and Upfal [MU17], Chapter 5, and surveyed across applications in Broder and Mitzenmacher [BM04]. The union case is clean: the bitwise OR of two filters for sets A and B is exactly a filter for $A \cup B$, with $\omega = 0$ preserved and a slightly elevated ε that chapter 6's Proposition 6.1.1 accounts for. The intersection case is famously subtle: the bitwise AND of two filters does *not* in general produce a filter for $A \cap B$, because the bit pattern of an AND filter admits ghost elements not in either input. Chapter 6 sidesteps the bitwise-AND trap by treating intersection at the predicate level instead, which gives the correct rate without committing to a specific representation. Readers who want the bitwise variant and its diagnostic issues should consult the survey directly.

De Morgan under noise. The complement-swap result of Proposition 6.2.2—that complementing an approximate set exchanges its FPR and FNR—is the set-level shadow of the Boolean-algebra rule worked out in Section 3.2: NOT swaps the two error rates of a noisy bit. Classical De Morgan reads $\complement(A \cup B) = \complement A \cap \complement B$, an identity between sets. Under noise the identity at the set level still holds, but the error rates of the two sides traverse the rate plane along different paths: one side touches the FNR-zero axis if both inputs are FNR-zero, the other side touches the FPR-zero axis after the

swap. The "different paths, same destination" phenomenon is what makes the perfect-filter-line classification of Section 6.4 informative: it tells you which compound operations stay on a one-sided axis and which mixed operations are forced into the interior. This chapter's treatment is the set-algebra rephrasing of the noisy-bit De Morgan worked through in chapter 3.

Perfect-filter literature and the bridge forward. The perfect-filter line of Section 6.4—the locus of one-sided sets in the rate plane (Section 5.4)—is where chapter 6 lands its main classification result. The line itself is a chapter 5 construct, but the question of *which* compound operations preserve membership on the line is the chapter 6 contribution. The constructive side of the story, how to actually build a set whose rates sit exactly on the FPR-zero axis, has its roots in the perfect-hashing literature opened by Carter and Wegman [CW79] and is taken up in full in chapter 15 (Chapter 15), which builds on Towell [Tow26h] for the space-optimal construction. The Bloom filter of Bloom [Blo70] occupies the opposite axis; it has been the canonical one-sided approximate set since 1970, and its algebraic behaviour under union and intersection is exactly what this chapter's closure analysis formalizes. Chapter 6 sits between the predicate-algebra reading of chapter 5 and the constructive perfect-filter machinery of chapter 15: the algebra of operations, divorced from any specific representation.

Exercises

1. ⋆ **Union rates for two FNR-zero Bloom filters.** Two FNR-zero Bloom filters on the same universe with the same parameters have operand rates $\varepsilon_1 = 0.05, \omega_1 = 0$ and $\varepsilon_2 = 0.10, \omega_2 = 0$.

   (a) Apply Equation (6.2) of Proposition 6.1.1 to compute the union false-positive rate $\varepsilon_\cup$. Report to four decimal places.

   (b) Apply Equation (6.3) to compute the union false-negative rate $\omega_\cup$, noting that every term in the weighted sum carries at least one operand ω as a factor.

   (c) Confirm that the inequality $\max(\varepsilon_1, \varepsilon_2) \le \varepsilon_\cup \le \varepsilon_1 + \varepsilon_2$ holds for the numbers you computed, and name the second-order term that accounts for the gap between $\varepsilon_\cup$ and $\varepsilon_1 + \varepsilon_2$.

2. ⋆ **Intersection rates for the same two filters.** Take the same operand rates as Item 1: $\varepsilon_1 = 0.05, \omega_1 = 0, \varepsilon_2 = 0.10, \omega_2 = 0$.

(a) Apply Proposition 6.2.1 to compute the intersection false-positive rate $\varepsilon_{\cap}$. Report to four decimal places.

(b) Compute the intersection false-negative rate $\omega_{\cap}$. State why $\omega_{\cap} = 0$ when both operands are FNR-zero.

(c) Compare the magnitudes of $\varepsilon_{\cap}$ and $\varepsilon_{\cup}$ from Item 1. By what factor does intersection reduce the false-positive rate relative to the smaller operand ε_1, and by what factor does union inflate it?

3. $\star$ **Complement-swap verification.** An approximate set $\mathsf{A}^{\pm}$ has per-block rates $\varepsilon = 0.03$ and $\omega = 0.07$.

(a) Apply Proposition 6.2.2 to write the rates $(\varepsilon_{\complement}, \omega_{\complement})$ of the complement $\complement\mathsf{A}^{\pm}$.

(b) Apply complement a second time. Verify that the twice-complemented rates return to $(0.03, 0.07)$, confirming that the complement operation is an involution on the rate plane.

(c) In one sentence, name the per-bit precedent for this swap that chapter 3 §3.2 already established at the noisy-bit level.

4. $\star\star$ **Notebook-tied set-op verification.** Open the chapter 6 companion notebook `python/notebooks/ch06-set-composition.ipynb`. Build two Bloom filters with $m = 5000$ bits, $n = 500$ inserted random strings each, and $k = 7$ hash functions, drawn from disjoint inserted sets S_1 and S_2 so that the operand rates approximate $\varepsilon_1 \approx \varepsilon_2 \approx 0.01$ with $\omega_1 = \omega_2 = 0$.

(a) Construct the union by bit-wise OR of the two underlying bit arrays. Query a pool of 10000 random strings known to be absent from both inserted sets, and record the empirical false-positive rate $\hat{\varepsilon}_{\cup}$.

(b) Compute the closed-form prediction $1 - (1 - \hat{\varepsilon}_1)(1 - \hat{\varepsilon}_2)$ from Equation (6.2), using empirical operand rates from the notebook in place of the targets, and compare to $\hat{\varepsilon}_{\cup}$. State in one sentence how close the empirical and closed-form values agree.

(c) Repeat the construction with the operand sets S_1 and S_2 chosen to overlap by 250 elements. Does the empirical union FPR still match the formula, and if so, why does the formula not depend on the cardinality of $S_1 \cap S_2$? Refer back to the FPR derivation in Proposition 6.1.1.

5. ⋆⋆ **Difference algebra.** Proposition 6.2.3 gave the difference rate formulas $\varepsilon_{\setminus} = \varepsilon_A \cdot \omega_B$ and $\omega_{\setminus} = 1 - (1 - \omega_A)(1 - \varepsilon_B)$ on the bulk region. Specialize these to two boundary cases.

   (a) Take $\mathsf{B}^{\pm}$ to be a perfect filter on the FPR-zero axis, so $\varepsilon_B = 0$ and $\omega_B \geq 0$. Substitute into the difference FPR formula and confirm that $\varepsilon_{\setminus} = \varepsilon_A \cdot \omega_B$ is the only surviving term. Substitute into the difference FNR formula and show that $\omega_{\setminus} = \omega_A$, that is, $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$ inherits its FNR from the first operand alone.

   (b) Now swap: take $\mathsf{B}^{\pm}$ to be FNR-zero, so $\omega_B = 0$ and $\varepsilon_B \geq 0$. Substitute into the difference FPR formula and confirm $\varepsilon_{\setminus} = 0$. Then substitute into the difference FNR formula and derive $\omega_{\setminus} = 1 - (1 - \omega_A)(1 - \varepsilon_B)$. In one sentence, explain the mechanism: false positives of $\mathsf{B}^{\pm}$ become wrongful exclusions in $\mathsf{A}^{\pm} \setminus \mathsf{B}^{\pm}$, and therefore false negatives of the difference.

   (c) Combine the two boundary readings to argue that the difference operation is FPR-preserving when the second operand is FNR-zero, and FNR-preserving when the second operand is a perfect filter on the FPR-zero axis.

6. ⋆⋆ **De Morgan duality under noise.** The classical De Morgan laws assert $\complement(A \cup B) = \complement A \cap \complement B$ and $\complement(A \cap B) = \complement A \cup \complement B$. This exercise verifies that the rate algebra of chapter 6 preserves the first of these identities under the cross-set independence assumption of Proposition 6.1.1.

   (a) Compute the rates of $\complement(\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm})$ by first applying Proposition 6.1.1 to get the union rates and then applying Proposition 6.2.2. Write the resulting (ε, ω) as a pair of expressions in the operand rates.

   (b) Compute the rates of $\complement\mathsf{A}^{\pm} \cap \complement\mathsf{B}^{\pm}$ by first applying Proposition 6.2.2 to each operand and then applying Proposition 6.2.1 to the complements.

   (c) Compare the two answers and confirm that the FPR expressions agree and the FNR expressions agree. The De Morgan duality holds for the rate algebra whenever the cross-set independence assumption is in force for both compositions.

7. ⋆⋆ **Independence sensitivity.** The FPR derivation in the proof of Proposition 6.1.1 assumed that the two operand error events at a query point x are mutually independent. This exercise constructs a small example where the assumption fails and shows what happens to the formula.

(a) Let $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ be two Bloom filters that share *exactly the same* hash family $h_1, \ldots, h_k$, that store the same set S, and that are derived from the same bit array. Argue that the two operand predicates are not merely correlated but identical: $\mathsf{A}^{\pm}(x) = \mathsf{B}^{\pm}(x)$ for every $x \in U$.

(b) Compute the actual union FPR for this construction. Since the two predicates are identical, the OR of their outputs agrees with the single-filter output, and the union FPR equals the common operand FPR ε. Compare to the formula prediction $1 - (1 - \varepsilon)(1 - \varepsilon) = 2\varepsilon - \varepsilon^2$ and describe the direction of the discrepancy.

(c) In one sentence, identify which structural property of the construction the formula presupposes, and which paragraph of Section 6.1 flags the requirement that the bit-wise OR construction needs distinct (or at least uncorrelated) hash families to match the rate prediction.

8. $\star\star$ **Intersection escape from the line.** Proposition 6.4.1 stated that intersection of two FNR-zero sets stays FNR-zero, but the same need not hold when one operand is FNR-zero and the other is FPR-zero.

(a) Let $\mathsf{A}^{\pm}$ be FNR-zero with $\varepsilon_A > 0$ and $\omega_A = 0$, and let $\mathsf{B}^{\pm}$ be FPR-zero with $\varepsilon_B = 0$ and $\omega_B > 0$. Substitute these operand rates into Proposition 6.2.1 and derive the intersection rates as expressions in ε_A and ω_B.

(b) Verify that $\varepsilon_\cap = 0$ in this regime, and that $\omega_\cap = \omega_B > 0$. The intersection has migrated to the FPR-zero axis: starting from one operand on each axis, the composition has landed on a single axis but on the FPR-zero one, not the FNR-zero one of $\mathsf{A}^{\pm}$.

(c) Now perturb the construction: let $\mathsf{B}^{\pm}$ instead be two-sided with $\varepsilon_B > 0$ and $\omega_B > 0$, leaving $\mathsf{A}^{\pm}$ FNR-zero. Substitute again and show that both rates of the intersection become strictly positive. In one sentence, name the geometric phenomenon (the composed point now lies in the interior of the rate square) and refer the reader to Definition 6.4.1 for the line whose escape this exercise concretely exhibits.

9. $\star\star\star$ **Three-way union via the composition theorem.** Extend Proposition 6.1.1 to three operands. Let $\mathsf{A}^{\pm}, \mathsf{B}^{\pm}, \mathsf{C}^{\pm}$ be three approximate sets with per-operand FPRs $\varepsilon_A, \varepsilon_B, \varepsilon_C$, all pairwise independent at every query point.

(a) Compute $\varepsilon_\cup$ for $\mathsf{A}^\pm \cup \mathsf{B}^\pm \cup \mathsf{C}^\pm$ by chained pairwise union: first apply Equation (6.2) to $\mathsf{A}^\pm \cup \mathsf{B}^\pm$, then apply it again to the result composed with $\mathsf{C}^\pm$. Simplify the nested expression to a single product form.

(b) Compute $\varepsilon_\cup$ directly from independence as in the proof of Proposition 6.1.1: a false positive of the three-way union at an x outside all three sets requires at least one of the three operand predicates to report $\top$, so the union FPR is 1 minus the joint probability that all three report $\bot$. Use independence to factor the joint into a product.

(c) Confirm that (a) and (b) agree. Then argue that the result is the composition theorem Theorem 3.3.1 of chapter 3 §3.3 instantiated at $k = 3$, that is, Equation (3.7) with $\eta_i = \varepsilon_i$. The same product formula governs every $k \geq 2$ by induction.

10. ⋆ ⋆ ⋆ **Perfect filter as algebraic inverse.** A perfect filter on a designated input set A is an approximate set with $\varepsilon = 0$ on A, paying with a positive ω on the complement domain. Proposition 6.4.1 part (4) suggested that pairing such a filter with an FNR-zero filter can give an exact-set test on a constrained domain. This exercise sketches the construction.

(a) Let $\mathsf{A}^\pm_+$ be a perfect filter for A with rates $(0, \omega_+)$ on its designed domain, and let $\mathsf{A}^\pm_-$ be an FNR-zero filter for A with rates $(\varepsilon_-, 0)$. Construct the composed predicate $\mathsf{A}^\pm_+(x) \wedge \mathsf{A}^\pm_-(x)$ and apply Proposition 6.2.1 to compute its rates. Confirm $\varepsilon = \varepsilon_- \cdot 0 = 0$ and $\omega = 1 - (1 - \omega_+)(1 - 0) = \omega_+$.

(b) State the condition under which $\omega_+ = 0$ also. The perfect filter has zero FNR on a subset of its designed domain (its "acceptance set"); if that subset coincides with A itself, the composition has both rates zero on A, that is, an exact-set test.

(c) Describe the architectural reading: the FNR-zero filter serves as a fast first-pass classifier (eliminating most non-members with one cheap query), and the perfect filter serves as a precise second-pass classifier (verifying the remaining candidates with certainty on the designed domain). Sketch why this two-pass composition is the algorithmic skeleton of the perfect hash filter that chapter 15 will develop in detail.

11. ⋆ ⋆ ⋆ **Entropy floor on the union operation.** Chapter 8 will prove

that any FNR-zero approximate set at false-positive rate ε requires at least $-\log_2 \varepsilon$ bits per stored element to represent, and chapter 14 will exhibit the Bernoulli Hash Function construction that achieves this floor. This exercise applies the floor to the union of two FNR-zero sets and shows that no clever implementation of union can beat it.

(a) Let $\mathsf{A}^{\pm}$ and $\mathsf{B}^{\pm}$ be two FNR-zero approximate sets for $A, B \subseteq U$, with operand FPRs $\varepsilon_A, \varepsilon_B$. By Proposition 6.4.1 part (1) the union $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ is FNR-zero with FPR $\varepsilon_\cup = 1 - (1 - \varepsilon_A)(1 - \varepsilon_B)$. Apply the chapter 8 floor (anticipated) to the union: any representation of $\mathsf{A}^{\pm} \cup \mathsf{B}^{\pm}$ as a single FNR-zero approximate set of the objective set $A \cup B$ requires at least $|A \cup B| \cdot (-\log_2 \varepsilon_\cup)$ bits.

(b) Argue that the chapter 6 composition formulas do not cheat the floor. The composed structure stores no more than the two operands together (its bit array, in the Bloom case, is the OR of the two operand bit arrays and so occupies m bits, the same as each operand). However, $\varepsilon_\cup > \max(\varepsilon_A, \varepsilon_B)$, so the floor for the composed structure permits a correspondingly looser bound; the bit budget allocated by each operand at its own floor is sufficient (and asymptotically tight, by chapter 14) for the composed FPR.

(c) In one or two sentences, draw the structural conclusion that motivates chapter 14. The union operation does not save bits beyond what the operands already paid for; constructive realizations that achieve the floor on each operand also achieve the floor on their union, modulo the small inflation $\varepsilon_\cup > \max(\varepsilon_A, \varepsilon_B)$ that the union FPR formula records. Chapter 14 develops the constructive realization (the BHF) that saturates this regime.

Chapter 7

Classification Measures

After this chapter you will be able to:

- Define PPV, NPV, recall, and F1 as functions of ε, ω, and workload prevalence; recognize the prevalence-dependence of precision and the prevalence-independence of ε, ω.
- Estimate any of these quantities from samples with proper confidence intervals (Wald, Wilson, Clopper-Pearson, Jeffreys); apply the delta method to derived measures.
- Apply the framework to Bloom filter diagnostics: given an observed false-positive count on a workload, recover a CI for the underlying ε and use it to bound PPV at the workload's prevalence.

7.1 The Setup Revisited

Section 1.4 stated the problem and stopped just short of the theory. An approximate set arrives with two unknown rates: a rate ε at which it reports $\top$ on inputs whose truth is $\bot$, and a rate ω at which it reports $\bot$ on inputs whose truth is $\top$. The classifier was treated as a black box: query it on n labelled inputs, split them into the n_0 trials whose truth is $\bot$ and the $n_1 = n - n_0$ trials whose truth is $\top$, count the false positives and false negatives, and report the empirical proportions $\hat{\varepsilon} = \mathrm{FP}/n_0$ and $\hat{\omega} = \mathrm{FN}/n_1$. Part II now writes that classifier as an approximate set $\mathsf{A}^{\pm} : U \to \mathrm{Bernoulli}[\mathrm{bool}]$, a noisy membership predicate over a universe U, but the estimation problem is unchanged. Chapter 7 closes the loop.

What chapter 1 left deferred

Section 1.4 reported the binomial form of the estimators, stated their unbiasedness and variance without proof, and gave the Wald confidence interval with a warning attached. It promised the full theory to chapter 7. Four items were named:

(i) Estimator properties. Section 1.4 claimed $\hat{\varepsilon} = \mathrm{FP}/n_0$ is unbiased for ε with variance $\varepsilon(1-\varepsilon)/n_0$. It did not prove either claim. It did not identify $\hat{\varepsilon}$ as the maximum-likelihood estimator, and it did not show that the estimator's variance equals the Cramer-Rao lower bound. All four facts (bias, variance, MLE identity, efficiency) are stated and proved in Section 7.2.

(ii) Proper confidence intervals. The Wald interval $\hat{\varepsilon} \pm z_{\alpha/2}\sqrt{\hat{\varepsilon}(1-\hat{\varepsilon})/n_0}$ is asymptotically correct but inadequate in the small-n_0 regime where the cost of a single observation matters. Three standard alternatives exist (Wilson, Clopper-Pearson, Jeffreys), each with a different motivation and a different operating characteristic. Section 7.3 derives each and recommends a default.

(iii) Sawtooth under-coverage near the boundaries. The chapter 1 notebook visualized the Wald interval failing at small n_0 when ε is near 0 or 1: empirical coverage drops below the nominal $1-\alpha$ in a jagged, oscillating pattern that worsens as the rate approaches a boundary. The pattern has a name (sawtooth coverage) and a known cause (discrete binomial counts viewed through a continuous Normal approximation). The same section diagnoses the failure and shows which of the alternative intervals suppress it.

(iv) Derived measures. The rates ε and ω are intrinsic to a classifier; precision, the positive predictive value (PPV), the negative predictive value (NPV), and the F1 score are not. Each is a function of ε, ω, and the prevalence π of $\top$-inputs in the workload. Section 7.4 writes each derived measure in those three variables and uses the delta method to turn the binomial CIs of Section 7.3 into CIs for the derived quantities.

The chapter's roadmap

Section 7.2 retires item (i): four short propositions establish that $\hat{\varepsilon}$ is unbiased, has the stated variance, equals the MLE, and achieves the Cramer-Rao lower bound. Section 7.3 retires items (ii) and (iii): recap the Wald form, diagnose its sawtooth, define the Wilson, Clopper-Pearson, and Jeffreys intervals, and compare their coverage on the small-n_0 regime where Wald fails. Section 7.4 retires item (iv): defines PPV, NPV, recall, and F1 as functions of $(\varepsilon, \omega, \pi)$, applies the delta method, and works the prevalence-dependence of PPV as the central caveat. Section 7.5 closes by running the full toolkit

on a deployed Bloom filter with a measured workload, producing a CI for the filter's PPV and answering a diagnostic question (reduce the false-positive rate, or accept the current prevalence?).

Notation refresh

The classification rates introduced in Section 1.2 and used throughout chapters 1 to 6 are written via the standard `alex.sty` macros:

ε (`\fprate`) false-positive rate. The chapter-1 letter was ε.

ω (`\fnrate`) false-negative rate, written ω.

τ (`\tprate`) true-positive rate $= 1 - \omega$, written τ. Also called *recall* or *sensitivity*.

η (`\tnrate`) true-negative rate $= 1 - \varepsilon$, written η. Also called *specificity*.

The hatted forms $\hat{\varepsilon}, \hat{\omega}, \hat{\tau}, \hat{\eta}$ are the empirical estimators of those rates. Sample sizes are written n_0 for the count of trials with truth $\bot$ and n_1 for the count of trials with truth $\top$, so that $\hat{\varepsilon} = \mathrm{FP}/n_0$ and $\hat{\omega} = \mathrm{FN}/n_1$. The Normal-quantile symbol $z_{\alpha/2}$ denotes the upper-$\alpha/2$ critical value of the standard Normal distribution and is used in the Wald CI of Section 1.4 and the Wilson CI of Section 7.3. Throughout this chapter we also use the workload prevalence $\pi = \Pr[X = \top]$ for the fraction of $\top$-inputs in the inference sample; this is the variable on which PPV, NPV, and F1 depend and on which ε, ω do not.

7.2 The Binomial Estimator: Properties

Section 7.1 named four items the chapter 1 hook left deferred. Item (i) is the present section's target: the binomial estimator $\hat{\varepsilon} = \mathrm{FP}/n_0$ stated without proof in Section 1.4 is unbiased for ε, has the variance $\varepsilon(1-\varepsilon)/n_0$ claimed there, equals the maximum-likelihood estimator, and attains the Cramer-Rao lower bound. Four short propositions, each with a one-paragraph proof, settle these claims.

Setup

Fix the truth-$\bot$ slice of the sample. Of the n trials of Section 1.4, the n_0 trials whose truth is $\bot$ are the only ones a false positive can come from. For each such trial $i \in \{1, \ldots, n_0\}$, let $X_i = 1$ if the noisy bit reports $\top$ and

$X_i = 0$ otherwise. Axiom 1 of the Bernoulli model (element-wise independence) makes the X_i mutually independent and identically distributed, each Bernoulli with success probability ε:

$$X_i \;\sim\; \text{Bernoulli}(\varepsilon), \qquad X_1, \ldots, X_{n_0} \text{ independent.}$$

The false-positive count is their sum,

$$\text{FP} \;=\; \sum_{i=1}^{n_0} X_i \;\sim\; \text{Binomial}(n_0, \varepsilon),$$

recovering the Binomial form already stated in Section 1.4. The estimator under study is the row-conditional empirical proportion

$$\hat{\varepsilon} \;=\; \frac{\text{FP}}{n_0} \;=\; \frac{1}{n_0} \sum_{i=1}^{n_0} X_i.$$

The arguments for $\hat{\omega} = \text{FN}/n_1$ on the truth-$\top$ slice are formally identical and are not repeated; everything in this section transfers by renaming $\varepsilon \to \omega$ and $n_0 \to n_1$.

Unbiasedness

Proposition 7.2.1 (Unbiasedness)**.** *The empirical estimator $\hat{\varepsilon} = \text{FP}/n_0$ is unbiased for ε:*

$$\mathbb{E}[\hat{\varepsilon}] \;=\; \varepsilon.$$

Proof. Linearity of expectation, applied to the sum $\text{FP} = \sum_{i=1}^{n_0} X_i$, gives

$$\mathbb{E}[\text{FP}] \;=\; \sum_{i=1}^{n_0} \mathbb{E}[X_i] \;=\; \sum_{i=1}^{n_0} \varepsilon \;=\; n_0 \varepsilon,$$

because each $X_i \sim \text{Bernoulli}(\varepsilon)$ has expectation ε. Dividing by the deterministic constant n_0,

$$\mathbb{E}[\hat{\varepsilon}] \;=\; \mathbb{E}\left[\frac{\text{FP}}{n_0}\right] \;=\; \frac{\mathbb{E}[\text{FP}]}{n_0} \;=\; \frac{n_0 \varepsilon}{n_0} \;=\; \varepsilon,$$

as claimed. □

The bias of the estimator is zero at every $\varepsilon \in [0, 1]$ and every sample size $n_0 \geq 1$, not merely in the limit. This is the first piece of the §1.4 deferral retired.

Variance

Proposition 7.2.2 (Variance)**.** *The empirical estimator* $\hat{\varepsilon} = \mathrm{FP}/n_0$ *has variance*

$$\mathrm{Var}(\hat{\varepsilon}) \;=\; \frac{\varepsilon(1-\varepsilon)}{n_0}.$$

Proof. The textbook variance of a Binomial random variable is $\mathrm{Var}(\mathrm{Binomial}(n,p)) = np(1-p)$; we recover it from the Bernoulli decomposition for completeness. Each X_i takes value 1 with probability ε and 0 with probability $1-\varepsilon$, so

$$\mathrm{Var}(X_i) \;=\; \mathbb{E}[X_i^2] - \mathbb{E}[X_i]^2 \;=\; \varepsilon - \varepsilon^2 \;=\; \varepsilon(1-\varepsilon).$$

By Axiom 1 the X_i are mutually independent, so the variance of their sum is the sum of their variances,

$$\mathrm{Var}(\mathrm{FP}) \;=\; \sum_{i=1}^{n_0} \mathrm{Var}(X_i) \;=\; n_0\,\varepsilon(1-\varepsilon).$$

The scaling rule $\mathrm{Var}(cZ) = c^2\mathrm{Var}(Z)$ for a constant c then gives

$$\mathrm{Var}(\hat{\varepsilon}) \;=\; \mathrm{Var}\left(\frac{\mathrm{FP}}{n_0}\right) \;=\; \frac{1}{n_0^2}\mathrm{Var}(\mathrm{FP}) \;=\; \frac{n_0\,\varepsilon(1-\varepsilon)}{n_0^2} \;=\; \frac{\varepsilon(1-\varepsilon)}{n_0},$$

which is the formula stated. □

Two consequences are worth naming. First, the variance shrinks as $1/n_0$: doubling the truth-$\perp$ sample halves the variance, and the standard error shrinks as $1/\sqrt{n_0}$. Second, the numerator $\varepsilon(1-\varepsilon)$ peaks at $\varepsilon = 1/2$ and vanishes at the boundaries $\varepsilon \in \{0,1\}$. The estimator is hardest to pin down when the classifier is maximally noisy and easiest when it is deterministic, a fact that returns in Section 7.3 as the reason Wald intervals deform near the boundary.

Maximum likelihood

Proposition 7.2.3 (The MLE is the empirical estimator)**.** *The maximum-likelihood estimator of* ε *from the binomial observation* $\mathrm{FP} \sim \mathrm{Binomial}(n_0, \varepsilon)$ *equals the row-conditional empirical proportion:*

$$\hat{\varepsilon}_{\mathrm{MLE}} \;=\; \frac{\mathrm{FP}}{n_0} \;=\; \hat{\varepsilon}.$$

Proof. The likelihood of the observed count FP under the model Binomial(n_0, ε) is

$$L(\varepsilon) \;=\; \binom{n_0}{\mathrm{FP}} \varepsilon^{\mathrm{FP}} \, (1-\varepsilon)^{n_0 - \mathrm{FP}}.$$

The binomial coefficient does not depend on ε and contributes an additive constant to the log-likelihood:

$$\log L(\varepsilon) \;=\; \log \binom{n_0}{\mathrm{FP}} + \mathrm{FP} \log \varepsilon + (n_0 - \mathrm{FP}) \log(1-\varepsilon).$$

Differentiating in ε over the open interval $(0,1)$ gives

$$\frac{\partial \log L}{\partial \varepsilon} \;=\; \frac{\mathrm{FP}}{\varepsilon} - \frac{n_0 - \mathrm{FP}}{1-\varepsilon}.$$

The first-order condition $\partial \log L / \partial \varepsilon = 0$ becomes

$$\mathrm{FP}\,(1-\varepsilon) \;=\; (n_0 - \mathrm{FP})\,\varepsilon,$$

which simplifies to $\mathrm{FP} - \mathrm{FP}\varepsilon = n_0\varepsilon - \mathrm{FP}\varepsilon$, then to $\mathrm{FP} = n_0\varepsilon$, and finally to $\varepsilon = \mathrm{FP}/n_0$. The second derivative

$$\frac{\partial^2 \log L}{\partial \varepsilon^2} \;=\; -\frac{\mathrm{FP}}{\varepsilon^2} - \frac{n_0 - \mathrm{FP}}{(1-\varepsilon)^2}$$

is strictly negative on $(0,1)$ for any $\mathrm{FP} \in \{1, \dots, n_0 - 1\}$, so the critical point is the unique interior maximum. The boundary cases $\mathrm{FP} = 0$ and $\mathrm{FP} = n_0$ assign $\hat{\varepsilon}_{\mathrm{MLE}} = 0$ and $\hat{\varepsilon}_{\mathrm{MLE}} = 1$ respectively, again matching FP/n_0. Hence $\hat{\varepsilon}_{\mathrm{MLE}} = \mathrm{FP}/n_0 = \hat{\varepsilon}$. □

The MLE identity says the empirical estimator was not just convenient: it is what likelihood theory asks for under the binomial model. Method-of-moments and maximum-likelihood agree on this problem; in more general settings they need not, as Exercise 9 of this chapter illustrates.

Fisher information and the Cramer-Rao bound

Proposition 7.2.4 (Fisher information and Cramer-Rao bound)**.** *The Fisher information of ε in the* Binomial(n_0, ε) *model is*

$$I(\varepsilon) \;=\; \frac{n_0}{\varepsilon(1-\varepsilon)}.$$

Consequently, every unbiased estimator T of ε satisfies the Cramer-Rao lower bound

$$\mathrm{Var}(T) \;\geq\; \frac{1}{I(\varepsilon)} \;=\; \frac{\varepsilon(1-\varepsilon)}{n_0}.$$

Proof. The Fisher information for a regular parametric model with log-likelihood $\log L$ is $I(\varepsilon) = -\mathbb{E}[\partial^2 \log L / \partial \varepsilon^2]$. The second derivative was computed in the proof of Proposition 7.2.3:

$$\frac{\partial^2 \log L}{\partial \varepsilon^2} \;=\; -\frac{\mathrm{FP}}{\varepsilon^2} - \frac{n_0 - \mathrm{FP}}{(1-\varepsilon)^2}.$$

Taking expectations under $\mathrm{FP} \sim \mathrm{Binomial}(n_0, \varepsilon)$ uses $\mathbb{E}[\mathrm{FP}] = n_0 \varepsilon$ and $\mathbb{E}[n_0 - \mathrm{FP}] = n_0(1-\varepsilon)$:

$$\mathbb{E}\left[\frac{\partial^2 \log L}{\partial \varepsilon^2}\right] \;=\; -\frac{n_0 \varepsilon}{\varepsilon^2} - \frac{n_0(1-\varepsilon)}{(1-\varepsilon)^2} \;=\; -\frac{n_0}{\varepsilon} - \frac{n_0}{1-\varepsilon}.$$

Combining the two terms over the common denominator $\varepsilon(1-\varepsilon)$,

$$-\mathbb{E}\left[\frac{\partial^2 \log L}{\partial \varepsilon^2}\right] \;=\; \frac{n_0(1-\varepsilon) + n_0 \varepsilon}{\varepsilon(1-\varepsilon)} \;=\; \frac{n_0}{\varepsilon(1-\varepsilon)} \;=\; I(\varepsilon).$$

The Binomial family is regular (the support $\{0, 1, \ldots, n_0\}$ does not depend on ε, the log-likelihood is twice differentiable in ε on $(0,1)$, and the standard interchange-of-order conditions hold), so the Cramer-Rao inequality applies and the bound $\mathrm{Var}(T) \geq 1/I(\varepsilon) = \varepsilon(1-\varepsilon)/n_0$ follows by direct substitution. □

Efficiency

Proposition 7.2.5 (Efficiency)**.** *The empirical estimator $\hat{\varepsilon} = \mathrm{FP}/n_0$ attains the Cramer-Rao lower bound and is therefore an efficient unbiased estimator of ε.*

Proof. Proposition 7.2.1 establishes that $\hat{\varepsilon}$ is unbiased. Proposition 7.2.2 computes $\mathrm{Var}(\hat{\varepsilon}) = \varepsilon(1-\varepsilon)/n_0$. Proposition 7.2.4 computes the Cramer-Rao bound as $1/I(\varepsilon) = \varepsilon(1-\varepsilon)/n_0$. The two quantities are equal, so $\hat{\varepsilon}$ attains the bound. □

What has been retired

The four propositions above establish the four claims the chapter 1 hook deferred for item (i): $\hat{\varepsilon} = \mathrm{FP}/n_0$ is unbiased, has variance $\varepsilon(1-\varepsilon)/n_0$, equals the maximum-likelihood estimator, and attains the Cramer-Rao lower bound of $\varepsilon(1-\varepsilon)/n_0$. No unbiased estimator of ε can do better in mean-squared error within the binomial model: any candidate that reports a lower variance

is biased and is trading bias for variance. Item (i) of the §1.4 deferral is now retired. What remains is the question of how to turn the point estimate into a confidence statement that holds at every n_0 and every ε, not merely asymptotically. Section 7.3 takes up that question and retires items (ii) and (iii).

7.3 Confidence Intervals Properly

Section 7.2 fixed the point estimate $\hat{\varepsilon} = \mathrm{FP}/n_0$ as the unbiased, efficient maximum-likelihood estimator of ε. A point estimate is not a confidence statement. The Wald interval given in Section 1.4 attaches Normal-approximation error bars to the point estimate and stops there. The chapter 1 notebook showed the attached error bars failing in exactly the regime a practitioner most needs them: small n_0, ε near 0 or 1. This section diagnoses the failure rigorously, then derives three alternatives (Wilson, Clopper-Pearson, Jeffreys), compares their coverage on the small-n_0 regime, and recommends a default.

The Wald interval

The Wald interval is the central-limit-theorem interval read off the sampling distribution of the point estimate, with the unknown ε in the variance replaced by its empirical value.

Definition 7.3.1 (Wald confidence interval)**.** *At confidence level $1-\alpha$, the Wald interval for ε is*

$$\hat{\varepsilon} \;\pm\; z_{\alpha/2}\sqrt{\frac{\hat{\varepsilon}(1-\hat{\varepsilon})}{n_0}},$$

where $z_{\alpha/2}$ is the upper-$\alpha/2$ quantile of the standard Normal distribution and $\hat{\varepsilon} = \mathrm{FP}/n_0$ is the point estimate from Section 7.2.

The derivation is the standard one. The central limit theorem applied to the i.i.d. sum $\mathrm{FP} = \sum X_i$ of Section 7.2 gives an asymptotic Normal approximation $\hat{\varepsilon} \sim \mathrm{Normal}(\varepsilon, \varepsilon(1-\varepsilon)/n_0)$ as $n_0 \to \infty$. Substituting $\hat{\varepsilon}$ for ε inside the variance (the plug-in step) and reading off the $1-\alpha$ two-sided quantile interval of the resulting Normal yields the formula. The interval is correct in the limit $n_0 \to \infty$ for every fixed $\varepsilon \in (0,1)$. The trouble is that practitioners work at finite n_0.

The under-coverage problem

The chapter 1 notebook ran M independent replications of the classifier at sample size n_0, constructed the Wald interval at each, and tallied the fraction containing the true ε. At $n_0 = 100$ and $\varepsilon = 0.05$ the empirical coverage settled near 0.92 rather than the nominal 0.95. The gap widened as ε moved closer to 0. Two distinct failure modes are at work.

(a) The boundary problem. The Wald formula is a symmetric interval centered at $\hat{\varepsilon}$, which lives in $[0, 1]$. The radius $z_{\alpha/2}\sqrt{\hat{\varepsilon}(1-\hat{\varepsilon})/n_0}$ is positive and depends on $\hat{\varepsilon}$, not on ε. When $\hat{\varepsilon}$ is small and the radius is comparable to it, the lower endpoint $\hat{\varepsilon} - z_{\alpha/2}\sqrt{\hat{\varepsilon}(1-\hat{\varepsilon})/n_0}$ can fall below 0; symmetrically the upper endpoint can rise above 1 when $\hat{\varepsilon}$ is near 1. Both cases assign positive probability mass to impossible values of ε. The interval has to be clipped to $[0, 1]$ by hand, which is not a property the construction provides.

(b) The skew problem. The binomial distribution Binomial(n_0, ε) is symmetric only at $\varepsilon = 1/2$. Near $\varepsilon = 0$ it is right-skewed: the mass piles up at small counts with a long right tail. Near $\varepsilon = 1$ it is left-skewed. The Normal approximation is symmetric and cannot capture the skew. At small n_0 and ε near a boundary, the central limit theorem has not yet overcome the binomial skew, and the symmetric Wald interval miscalibrates: it places too much of its tail probability on the side away from the boundary and too little on the side toward it. The empirical coverage drops below nominal.

There is a third pattern, visible in the chapter 1 notebook sweep of n_0 at fixed $\varepsilon = 0.05$. The empirical coverage as a function of n_0 is not monotone: it rises and dips in a jagged pattern as n_0 grows. This is the *sawtooth*. The cause is discreteness. The Wald interval is built on a continuous Normal approximation, but the underlying FP takes only the integer values $0, 1, \ldots, n_0$. As n_0 increments by one, the boundary at which the interval just barely covers or just barely misses the true ε shifts in a discrete jump, and the empirical coverage hops between values close to but rarely exactly at nominal. The pattern is a real feature of every CI built on a discrete distribution; the Wald interval is unusual in how pronounced the sawtooth is, because of failure modes (a) and (b) above.

The three alternatives in this section address the failures in different ways: Wilson fixes the boundary problem by inverting a different statistic, Clopper-Pearson sidesteps the Normal approximation entirely by inverting the exact binomial CDF, and Jeffreys does the Bayesian thing and reports a posterior credible interval.

The Wilson interval

The Wald construction plugs $\hat{\varepsilon}$ into the variance and inverts the resulting Normal-form statistic. Wilson inverts a different statistic, the score statistic, which uses the true ε (not its estimate) in the variance.

The score statistic for the binomial mean is

$$Z(\varepsilon) \;=\; \frac{\hat{\varepsilon} - \varepsilon}{\sqrt{\varepsilon(1-\varepsilon)/n_0}}.$$

Under the central limit theorem, $Z(\varepsilon) \to \text{Normal}(0,1)$ as $n_0 \to \infty$ when ε is the true parameter, so the confidence set at level $1-\alpha$ is the set of ε satisfying $|Z(\varepsilon)| \le z_{\alpha/2}$. Squaring both sides and clearing the denominator gives a quadratic inequality in ε:

$$(\hat{\varepsilon} - \varepsilon)^2 \;\le\; z_{\alpha/2}^2 \, \frac{\varepsilon(1-\varepsilon)}{n_0}.$$

Writing $z = z_{\alpha/2}$ for brevity and expanding,

$$\hat{\varepsilon}^2 - 2\hat{\varepsilon}\varepsilon + \varepsilon^2 \;\le\; \frac{z^2}{n_0}\varepsilon - \frac{z^2}{n_0}\varepsilon^2,$$

which rearranges to

$$\varepsilon^2 \left(1 + \frac{z^2}{n_0}\right) - 2\varepsilon \left(\hat{\varepsilon} + \frac{z^2}{2n_0}\right) + \hat{\varepsilon}^2 \;\le\; 0.$$

This is a downward-closed quadratic in ε, so its solution set is the interval between the two roots. Applying the quadratic formula and simplifying gives the form below.

Definition 7.3.2 (Wilson confidence interval)**.** *At confidence level $1-\alpha$, with $z = z_{\alpha/2}$, the Wilson interval for ε is*

$$\frac{\hat{\varepsilon} + z^2/(2n_0)}{1 + z^2/n_0} \;\pm\; \frac{z}{1 + z^2/n_0} \sqrt{\frac{\hat{\varepsilon}(1-\hat{\varepsilon})}{n_0} + \frac{z^2}{4n_0^2}}.$$

The center of the Wilson interval is not $\hat{\varepsilon}$ but a shrunken version of it pulled toward $1/2$ by the factor $z^2/(2n_0)$ in the numerator and the factor $1 + z^2/n_0$ in the denominator. The radius likewise has a correction $z^2/(4n_0^2)$ inside the square root that keeps the interval well-defined even at $\hat{\varepsilon} \in \{0, 1\}$. Both adjustments matter. At $\hat{\varepsilon} = 0$ the Wald interval collapses to the

degenerate point $[0, 0]$; the Wilson interval gives $[0, z^2/(n_0 + z^2)]$, a non-empty interval whose upper endpoint admits the possibility that the true rate is small but positive. The boundary problem of (a) is gone: both endpoints lie in $[0, 1]$ for every $\hat{\varepsilon}$ and every n_0, because the defining inequality has the true ε in the variance and is a proper quadratic in ε. The skew problem of (b) is partially addressed: the Wilson center is shifted toward $1/2$, which is the direction the symmetric Wald interval got wrong near the boundaries.

The Clopper-Pearson interval

Wilson still relies on a Normal approximation, even if a sharper one. Clopper-Pearson abandons the approximation and works with the exact binomial CDF. The construction is the test-inversion construction in its cleanest form: for each candidate ε, test whether the observed FP is consistent with $\text{Binomial}(n_0, \varepsilon)$ at level α, and include ε in the interval iff the test fails to reject.

The lower endpoint L is the smallest ε such that $\Pr[\text{Binomial}(n_0, \varepsilon) \geq \text{FP}] > \alpha/2$; equivalently, L is the largest ε such that $\Pr[\text{Binomial}(n_0, \varepsilon) \geq \text{FP}] \leq \alpha/2$. The upper endpoint U is symmetric: the smallest ε such that $\Pr[\text{Binomial}(n_0, \varepsilon) \leq \text{FP}] \leq \alpha/2$.

These endpoints have closed forms via the Beta distribution, because the binomial CDF and the Beta CDF are related by the identity

$$\Pr[\text{Binomial}(n_0, \varepsilon) \geq k] \;=\; \Pr[\text{Beta}(k, n_0 - k + 1) \leq \varepsilon].$$

The endpoints are therefore Beta quantiles.

Definition 7.3.3 (Clopper-Pearson confidence interval)**.** *At confidence level $1 - \alpha$, the Clopper-Pearson interval for ε has endpoints*

$$L \;=\; F^{-1}_{\text{Beta}(\text{FP},\, n_0 - \text{FP} + 1)}(\alpha/2), \qquad U \;=\; F^{-1}_{\text{Beta}(\text{FP}+1,\, n_0 - \text{FP})}(1 - \alpha/2),$$

where $F^{-1}_{\text{Beta}(a,b)}$ denotes the inverse CDF of the $\text{Beta}(a, b)$ *distribution. At the boundaries: if* $\text{FP} = 0$ *set* $L = 0$*; if* $\text{FP} = n_0$ *set* $U = 1$*.*

The interval is called *exact* because the construction uses the exact binomial CDF rather than a Normal approximation. The label is slightly misleading: the coverage probability is guaranteed to be at least $1 - \alpha$ at every ε, but is strictly greater than $1 - \alpha$ at most ε because the binomial is discrete. The interval over-covers, sometimes substantially, especially at small n_0. Conservative coverage is the price of the guarantee.

The Jeffreys interval

The Bayesian alternative reports a credible interval from a posterior distribution over ε. The choice of prior matters; the Jeffreys prior is the one whose density is proportional to the square root of the Fisher information,

$$\pi_J(\varepsilon) \;\propto\; \sqrt{I(\varepsilon)} \;=\; \sqrt{\frac{n_0}{\varepsilon(1-\varepsilon)}} \;\propto\; \varepsilon^{-1/2}(1-\varepsilon)^{-1/2}.$$

This is the kernel of the $\mathrm{Beta}(1/2, 1/2)$ distribution, which serves as a non-informative reference prior with a frequentist calibration property: the resulting credible interval has approximate $1-\alpha$ coverage to higher order than the Wald interval. Under this prior the posterior given the observed count FP is $\mathrm{Beta}(\mathrm{FP} + 1/2, n_0 - \mathrm{FP} + 1/2)$ by conjugacy.

Definition 7.3.4 (Jeffreys credible interval)**.** *At credibility level $1-\alpha$, the Jeffreys interval for ε is the central $1-\alpha$ quantile interval of the posterior* $\mathrm{Beta}(\mathrm{FP} + 1/2, n_0 - \mathrm{FP} + 1/2)$*:*

$$\big[\, F^{-1}_{\mathrm{Beta}(\mathrm{FP}+1/2,\, n_0-\mathrm{FP}+1/2)}(\alpha/2),\; F^{-1}_{\mathrm{Beta}(\mathrm{FP}+1/2,\, n_0-\mathrm{FP}+1/2)}(1-\alpha/2) \,\big].$$

The Jeffreys interval is structurally close to Clopper-Pearson but uses different Beta parameters (the half-integer shift) and does not treat the endpoints asymmetrically. At $\mathrm{FP} = 0$ the posterior is $\mathrm{Beta}(1/2, n_0 + 1/2)$, which has a well-defined lower quantile strictly above 0; the interval is non-degenerate at the boundaries. The frequentist coverage of the Jeffreys interval is close to nominal at small n_0 without the heavy conservatism of Clopper-Pearson.

Comparison

The four intervals can be ranked along several axes: empirical coverage at a fixed (n_0, ε), behavior at the boundary $\hat{\varepsilon} = 0$, and structural symmetry about the point estimate. Table 7.1 summarizes.

The coverage entries for the small-n_0 row are not constants: each oscillates along a sawtooth as n_0 varies, by the discreteness argument above. The numbers report the average coverage over a small window of n_0 values near 100, smoothing the sawtooth. The Wilson and Jeffreys sawteeth are mild and stay close to nominal. The Clopper-Pearson sawtooth dips down only to the guarantee floor $1-\alpha = 0.95$ and is otherwise above; the over-coverage is visible in the strict inequality. The Wald sawtooth dips below nominal repeatedly and is the only one of the four whose minimum is far enough below nominal to mislead at the precision a careful practitioner would report.

	Wald	Wilson	Clopper-Pearson	Jeffreys
Coverage at $n_0 = 100$, $\varepsilon = 0.05$	≈ 0.92	≈ 0.94	≥ 0.95	≈ 0.945
Coverage at $n_0 = 1000$, $\varepsilon = 0.05$	≈ 0.95	≈ 0.95	≥ 0.95	≈ 0.95
Interval at $\hat{\varepsilon} = 0$	$[0, 0]$	$[0,\ z^2/(n_0 + z^2)]$	$[0,\ U_{\text{CP}}]$	$[L_J,\ U_J],\ L_J > 0$
Symmetric about $\hat{\varepsilon}$	yes	no	no	no
Endpoints in $[0, 1]$ by construction	no	yes	yes	yes
Approximation type	Normal	Normal (score)	exact binomial	Bayesian posterior

Table 7.1: Comparison of four binomial confidence intervals at nominal $1 - \alpha = 0.95$. Coverage numbers are read from the chapter 1 notebook sweep at $\varepsilon = 0.05$ for the Wald row; Wilson, Clopper-Pearson, and Jeffreys numbers are standard results from Brown, Cai, and DasGupta (2001) at the same parameters. The small-n_0 row shows the Wald under-coverage diagnosed above and the substantial recovery from each of the three alternatives. The large-n_0 row confirms that all four intervals agree asymptotically. The boundary row shows that Wald collapses to a degenerate point at $\hat{\varepsilon} = 0$ while the three alternatives give non-trivial intervals that admit small positive values of ε as compatible with the data.

Recommendation

The Wilson interval is the default for binomial confidence intervals in this book and is what Section 7.4 feeds into the delta-method derivations. It corrects the Wald boundary failure without the conservatism of Clopper-Pearson and is easy to implement from a Normal-quantile table. Jeffreys is the Bayesian alternative when a credible-interval interpretation is wanted; its frequentist coverage is also excellent. Clopper-Pearson is the conservative-safe choice when an absolute lower bound on coverage is required (in a regulatory or worst-case setting) and a slightly wider interval is acceptable. Wald is acceptable only when n_0 is large and $\hat{\varepsilon}$ is far from 0 and 1, the regime in which all four intervals agree; using Wald outside that regime invites the under-coverage of Table 7.1.

What has been retired

Items (ii) and (iii) of the §1.4 deferral named in Section 7.1 are now retired. The Wilson, Clopper-Pearson, and Jeffreys intervals fix the boundary failure of the Wald interval and substantially recover the lost coverage in the small-n_0 regime; the sawtooth pattern observed in the chapter 1 notebook is now attributed to its proper cause (binomial discreteness viewed through

a continuous-distribution lens) and shown to be mild for the three alternatives. Section 7.4 takes up item (iv): the derived measures (PPV, NPV, F1) and the delta-method translation of binomial CIs on ε, ω into CIs on the derived quantities.

7.4 Derived Measures

Sections 7.2 and 7.3 retired the first three items of the §1.4 deferral. The fourth item is the derived measures. Precision, recall, F_1, and the two predictive values (PPV and NPV) appear throughout binary classification and information retrieval. Section 1.2 introduced precision and recall informally and pointed out that precision depends on a quantity the noisy bit itself does not control: the workload's prevalence of $\top$. The present section makes that observation precise. Each derived measure is written as a function of $(\varepsilon, \omega, \pi)$, where $\pi = \Pr[X = \top]$ is the prevalence. Estimators for each are read off the same contingency table whose row counts gave $\hat{\varepsilon}, \hat{\omega}$ in Section 7.2; their variances and confidence intervals follow from the delta method.

Definitions in terms of $(\varepsilon, \omega, \pi)$

Fix a noisy bit (equivalently, an approximate set $\mathsf{A}^{\pm} : U \to \text{Bernoulli[bool]}$) with rates ε, ω. Apply it to a workload of n inputs in which each input's truth is drawn independently from a Bernoulli distribution with success probability $\pi = \Pr[X = \top]$. The expected counts on the four cells of the contingency table are then

$$\mathbb{E}[\text{TP}] \;=\; n\pi(1-\omega), \qquad \mathbb{E}[\text{FN}] \;=\; n\pi\omega,$$

$$\mathbb{E}[\text{FP}] \;=\; n(1-\pi)\varepsilon, \qquad \mathbb{E}[\text{TN}] \;=\; n(1-\pi)(1-\varepsilon).$$

The first row of the table sums to $n\pi$ (the row total for truth $\top$); the second row sums to $n(1-\pi)$. The four derived measures are ratios of these expectations.

Recall is the row-conditional rate already named in Section 1.2:

$$\text{recall} \;=\; \tau \;=\; 1-\omega.$$

The denominator is the truth-$\top$ row total $n\pi$ and the numerator is the diagonal $n\pi(1-\omega)$, so the π cancels. Recall is a property of the noisy bit alone.

Definition 7.4.1 (Positive predictive value)**.** *The positive predictive value of a noisy bit at workload prevalence π is*

$$\mathrm{PPV} \;=\; \text{precision} \;=\; \frac{\pi(1-\omega)}{\pi(1-\omega)+(1-\pi)\varepsilon}.$$

The numerator is $\mathbb{E}[\mathrm{TP}]/n$ and the denominator is the column total $\mathbb{E}[\mathrm{TP}+\mathrm{FP}]/n$. The π does not cancel. PPV is precision under the crosswalk of Section 1.2, and the definition makes the workload's share of $\top$ explicit.

Definition 7.4.2 (Negative predictive value)**.** *The negative predictive value of a noisy bit at workload prevalence π is*

$$\mathrm{NPV} \;=\; \frac{(1-\pi)(1-\varepsilon)}{(1-\pi)(1-\varepsilon)+\pi\omega}.$$

NPV is the analogue of PPV for the other column: the fraction of $\perp$-reports that are correct. Like PPV, it depends on π. The numerator is $\mathbb{E}[\mathrm{TN}]/n$ and the denominator is the $\perp$-output column total $\mathbb{E}[\mathrm{TN}+\mathrm{FN}]/n$.

Definition 7.4.3 (F_1-score)**.** *The F_1-score of a noisy bit at workload prevalence π is the harmonic mean of precision and recall,*

$$F_1 \;=\; \frac{2\mathrm{precision}\cdot\mathrm{recall}}{\mathrm{precision}+\mathrm{recall}}.$$

Substituting the formulas for precision and recall gives

$$F_1 \;=\; \frac{2\pi(1-\omega)}{2\pi(1-\omega)+(1-\pi)\varepsilon+\pi\omega} \;=\; \frac{2\pi(1-\omega)}{\pi(1-\omega)+\pi+(1-\pi)\varepsilon},$$

where the second equality uses $\pi(1-\omega)+\pi\omega=\pi$ to collapse the denominator. F_1 is a balanced summary of precision and recall; like precision, it depends on the workload.

Each of these is the ratio of two binomial-mean quantities and is a deterministic function of (ε,ω,π). Replace the parameters by their estimates and the same formulas give point estimators $\widehat{\mathrm{recall}}, \widehat{\mathrm{PPV}}, \widehat{\mathrm{NPV}}, \widehat{F_1}$. The next subsection takes up what happens to the ratios as π varies; the subsection after returns to estimators and their distributions.

Prevalence dependence

Recall does not depend on π. PPV does. Fix ε, ω and read Definition 7.4.1 as a function of $\pi \in [0, 1]$:

$$\mathrm{PPV}(\pi) \;=\; \frac{\pi(1-\omega)}{\pi(1-\omega) + (1-\pi)\varepsilon}.$$

At $\pi = 0$ the numerator is 0 and the denominator is $\varepsilon \geq 0$, giving $\mathrm{PPV}(0) = 0$ when $\varepsilon > 0$. A noisy bit on a workload with no genuine positives reports only false positives; every $\top$-report is wrong. At $\pi = 1$ the denominator becomes the numerator and $\mathrm{PPV}(1) = 1$: a workload with no genuine negatives cannot produce a false positive, so every $\top$-report is correct. Between the endpoints the function is monotone increasing, and its rate of change near $\pi = 0$ is steep when ε is appreciable. NPV runs the same story in reverse: $\mathrm{NPV}(0) = 1$ and $\mathrm{NPV}(1) = 0$ for $\omega > 0$.

The point is the framing. The pair (ε, ω) is intrinsic to the classifier: it describes how the noisy bit responds to $\bot$-inputs and to $\top$-inputs, conditional on each. Precision is intrinsic to the classifier and the workload together: change the mix of inputs and precision changes, even though ε and ω have not moved. A classifier vendor who reports only precision is, implicitly, reporting only what their evaluation workload happened to look like. A vendor who reports (ε, ω) has reported the noisy bit. The book uses (ε, ω) as the primary description for exactly this reason.

PPV from samples

The estimator analogous to $\hat{\varepsilon}, \hat{\omega}$ for PPV is the empirical proportion read off the column total of the contingency table:

$$\widehat{\mathrm{PPV}} \;=\; \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}.$$

On the workload of n inputs with random truth, TP is binomial on the truth-$\top$ row with parameters $(n_\top, \tau)$ where $n_\top = n\pi$ is itself random if π is treated as unknown. Conditioning on $n_\top, n_\bot$ removes that source of variability; then $\mathrm{TP} \sim \mathrm{Binomial}(n_\top, \tau)$ and $\mathrm{FP} \sim \mathrm{Binomial}(n_\bot, \varepsilon)$, and the two counts are independent by Axiom 1. The estimator $\widehat{\mathrm{PPV}}$ is the ratio of two independent binomial counts.

Two consequences follow. First, $\widehat{\mathrm{PPV}}$ is not unbiased for PPV. The expectation of a ratio is not the ratio of expectations; Jensen's inequality applied to the convex function $x \mapsto a/(a + x)$ in the denominator gives

$\mathbb{E}[\widehat{\mathrm{PPV}}] \neq \mathrm{PPV}$ in general. The bias is small when the means $\mathbb{E}[\mathrm{TP}], \mathbb{E}[\mathrm{FP}]$ are both large (the central-limit regime) and $\widehat{\mathrm{PPV}}$ is a consistent estimator of PPV as $n \to \infty$. Second, $\widehat{\mathrm{PPV}}$ has variance and a sampling distribution, both inherited from the binomial counts. Reading them off directly is awkward because of the ratio; the delta method is the standard tool for the job.

The delta method

The delta method approximates the distribution of a smooth function of an asymptotically Normal estimator by a Normal whose variance is read off the derivative of the function.

Theorem 7.4.1 (Delta method)**.** *Suppose $\hat{\theta}$ is an estimator that is asymptotically Normal,*

$$\sqrt{n}(\hat{\theta} - \theta) \xrightarrow{d} \mathrm{Normal}(0, \sigma^2(\theta)),$$

and suppose g is a function differentiable at θ with $g'(\theta) \neq 0$. Then

$$\sqrt{n}(g(\hat{\theta}) - g(\theta)) \xrightarrow{d} \mathrm{Normal}\big(0,\, [g'(\theta)]^2 \sigma^2(\theta)\big).$$

The theorem is a one-line consequence of the first-order Taylor expansion $g(\hat{\theta}) \approx g(\theta) + g'(\theta)(\hat{\theta} - \theta)$ combined with Slutsky's lemma; a proof appears in Casella and Berger [CB02]. The generalization to multivariate $\hat{\theta}$ replaces $g'(\theta)$ with the gradient $\nabla g(\theta)$ and $\sigma^2(\theta)$ with a covariance matrix $\Sigma(\theta)$; the asymptotic variance of $g(\hat{\theta})$ becomes the quadratic form $\nabla g(\theta)^\top \Sigma(\theta) \nabla g(\theta)$.

Proposition 7.4.1 (Delta-method variance of $\widehat{\mathrm{PPV}}$)**.** *With $\bar{t} = n_\top \tau$, $\bar{f} = n_\bot \varepsilon$, $\sigma_t^2 = n_\top \tau(1-\tau)$, and $\sigma_f^2 = n_\bot \varepsilon(1-\varepsilon)$, the first-order delta-method approximation to the variance of $\widehat{\mathrm{PPV}}$ is*

$$\mathrm{Var}\,(\widehat{[]\,\mathrm{PPV}}) \;\approx\; \frac{\bar{f}^{\,2}\sigma_t^2 + \bar{t}^{\,2}\sigma_f^2}{(\bar{t}+\bar{f})^4}.$$

The derivation is a direct application of Theorem 7.4.1 in its bivariate form. Let $g(t, f) = t/(t+f)$ be the PPV function of two arguments. Its gradient is

$$\nabla g(t,f) \;=\; \frac{1}{(t+f)^2}\begin{bmatrix} f \\ -t \end{bmatrix}.$$

The covariance matrix of $(\mathrm{TP}, \mathrm{FP})$ is $\mathrm{diag}(\sigma_t^2, \sigma_f^2)$ because the two counts are independent. Evaluating ∇g at the mean $(\bar{t}, \bar{f})$ and computing the quadratic form gives

$$\mathrm{Var}\,(\widehat{[]\,\mathrm{PPV}}) \;\approx\; \nabla g^{\top}\,\mathrm{diag}(\sigma_t^2, \sigma_f^2)\,\nabla g \;=\; \frac{\bar{f}^{\,2}\sigma_t^2 + \bar{t}^{\,2}\sigma_f^2}{(\bar{t}+\bar{f})^4},$$

which is the stated formula. Substituting the parameter values $\bar{t} = n\pi(1-\omega)$, $\bar{f} = n(1-\pi)\varepsilon$, $\sigma_t^2 = n\pi(1-\omega)\omega$, and $\sigma_f^2 = n(1-\pi)\varepsilon(1-\varepsilon)$ writes the variance as an explicit function of $(\varepsilon, \omega, \pi, n)$.

Delta-method CI for PPV

A Wald-style $1-\alpha$ confidence interval for PPV follows from Proposition 7.4.1 by substituting the empirical estimates $\hat{\varepsilon}, \hat{\omega}, \hat{\pi}$ into the variance formula and pairing the result with the empirical $\widehat{\mathrm{PPV}}$:

$$\widehat{\mathrm{PPV}} \;\pm\; z_{\alpha/2}\sqrt{\widehat{\mathrm{Var}}\,(\widehat{[]\,\mathrm{PPV}})},$$

where the hat over the variance indicates plug-in evaluation. This inherits the boundary and skew problems of the Wald interval for ε in Section 7.3: when $\widehat{\mathrm{PPV}}$ is near 0 or 1 the symmetric interval can extend past the unit interval, and at small $n_\top$ or $n_\perp$ the Normal approximation breaks down. The remedy is the same: a Wilson- or Clopper-Pearson-style interval on the binomial count TP conditioned on TP + FP (which is binomial with parameter PPV when $n_\top, n_\perp$ are conditioned upon), or a bootstrap interval resampling the contingency table. The bootstrap is attractive when the analytic formula is unwieldy; the score-based Wilson interval is attractive when computation is cheap. The choice is mostly a matter of taste once n is large enough that all three agree, and a matter of correctness near the boundaries.

F_1 via the delta method

The same calculation runs for F_1. Writing $F_1 = 2\mathrm{TP}/(\mathrm{TP} + \mathrm{FP} + n_\top)$ (the $\mathrm{FN} = n_\top - \mathrm{TP}$ substitution turns the harmonic mean into a function of TP and FP only) and applying Theorem 7.4.1 to the gradient of $h(t, f) = 2t/(t + f + n_\top)$ at the means gives the first-order delta-method variance

$$\mathrm{Var}\,(\widehat{[]\,F_1}) \;\approx\; \frac{4(\bar{f}+n_\top)^2\sigma_t^2 + 4\bar{t}^{\,2}\sigma_f^2}{(\bar{t}+\bar{f}+n_\top)^4}.$$

The extra $n_\top$ in the coefficient of σ_t^2 is the only structural difference from the PPV variance: it adds the column total to the gradient component along TP.

Example 2 (F_1 with delta-method CI) *Take $\varepsilon = 0.05$, $\omega = 0.10$, $\pi = 0.1$, $n = 1000$. Conditioning on $n_\top = n\pi = 100$ and $n_\perp = n(1-\pi) = 900$, the point estimate is*

$$\text{recall} = 1-0.10 = 0.90, \qquad \text{precision} = \frac{0.1 \cdot 0.9}{0.1 \cdot 0.9 + 0.9 \cdot 0.05} = \frac{0.09}{0.135} = \frac{2}{3} \approx 0.667,$$

$$F_1 = \frac{2 \cdot (2/3) \cdot 0.9}{(2/3) + 0.9} = \frac{1.2}{1.5667} \approx 0.766.$$

The expected counts are $\bar{t} = 90$, $\bar{f} = 45$, with variances $\sigma_t^2 = 100 \cdot 0.9 \cdot 0.1 = 9$ and $\sigma_f^2 = 900 \cdot 0.05 \cdot 0.95 = 42.75$. The denominator is $s = 90+45+100 = 235$, so

$$\text{Var}\,(\lceil\rfloor\, F_1] \;\approx\; \frac{4 \cdot 145^2 \cdot 9 + 4 \cdot 90^2 \cdot 42.75}{235^4} \;=\; \frac{756900 + 1385100}{3{,}049{,}800{,}625} \;\approx\; 7.02{\times}10^{-4},$$

giving a standard deviation of about 0.0265. The 95% Wald-style confidence interval at $z_{0.025} = 1.96$ is therefore

$$F_1 \;\in\; 0.766 \pm 1.96 \cdot 0.0265 \;\approx\; 0.766 \pm 0.052 \;\approx\; [0.714, 0.818].$$

The interval is narrower than the analogous PPV interval at the same parameters because the $n_\top$ in the denominator of F_1 acts as a stabilizer: a small change in TP *or* FP *moves F_1 less than it moves* PPV*, and the variance is correspondingly smaller.*

Bloom filter PPV under workload

The dependence of PPV on prevalence is sharp enough to drive a concrete example. Consider a Bloom filter (Definition 5.2.1 and section 5.2) tuned to one-sided error: $\omega = 0$ (no false negatives, by the filter's construction) and $\varepsilon = 0.01$ (typical of the $m/n \approx 10$, $k = 7$ regime). The same filter is now deployed on two workloads.

Workload A: rare $\top$. Suppose the workload has prevalence $\pi = 0.01$. By Definition 7.4.1,

$$\text{PPV} \;=\; \frac{0.01 \cdot 1}{0.01 \cdot 1 + 0.99 \cdot 0.01} \;=\; \frac{0.01}{0.0199} \;\approx\; 0.503.$$

About half of all positive reports the filter produces are false positives. The filter is correct on every $\top$ in the workload (by $\omega = 0$) and wrong on 1% of the much larger $\bot$ population; in the aggregate the wrongs roughly match the rights at the column total.

Workload B: common $\top$. The same filter on a workload with $\pi = 0.5$ gives

$$\text{PPV} \;=\; \frac{0.5 \cdot 1}{0.5 \cdot 1 + 0.5 \cdot 0.01} \;=\; \frac{0.5}{0.505} \;\approx\; 0.990.$$

About one in a hundred positive reports is wrong.

Same filter, same (ε, ω), two workloads, PPV moving by nearly a factor of two. A claim like "the Bloom filter has precision 0.99" is workload-dependent and uninformative without the workload. The corresponding claim about (ε, ω) is workload-free. Section 7.5 works out the diagnostic inversion: given an observed false-positive count on a workload of known prevalence, recover a confidence interval for the underlying ε and translate it back into a confidence interval for PPV at that prevalence. The forward direction (parameters to PPV) is straightforward; the backward direction (observation to PPV with uncertainty) is where the chapter-7 toolkit pays off.

What has been retired

Item (iv) of the §1.4 deferral is now retired. PPV, NPV, recall, and F_1 are defined in terms of $(\varepsilon, \omega, \pi)$; estimators read off the contingency table inherit variances from the binomial counts via the delta method; the prevalence-dependence of PPV is recorded as the central caveat that separates intrinsic classifier properties from joint classifier-and-workload properties. Section 7.5 closes the chapter with one worked diagnostic application using the entire toolkit.

7.5 Worked Application: Bloom Filter Diagnostics

The chapter-7 toolkit is now complete: Section 7.2 proved the empirical-proportion estimator is unbiased and efficient; Section 7.3 replaced the Wald interval with Wilson as the default; Section 7.4 carried both into PPV via the delta method. The remaining task is to use all three on a single deployment problem and watch the diagnostic question fall out of the numbers. The scenario below is a spam filter implemented as a Bloom filter, the running

example of Section 5.2; the deployment question is whether the filter is precise enough on the workload at hand, and if not, what to change.

Scenario

A spam filter is implemented as a Bloom filter (Definition 5.2.1 and section 5.2) with parameters $(m, k, n) = (50000, 7, 5000)$. The bit-to-item ratio $m/n = 10$ and $k = 7$ put the filter in the optimal regime of Theorem 5.2.1. Applying (5.1) with $kn/m = 0.7$,

$$\varepsilon \;\approx\; \left(1 - e^{-0.7}\right)^7 \;\approx\; (0.5034)^7 \;\approx\; 0.0082.$$

The construction guarantees $\omega = 0$: no spam message is missed. A workload of $N = 10000$ messages arrives, a mix of spam and not-spam in proportions the deployment does not control. The pair $(\varepsilon, \omega) = (0.0082, 0)$ describes the filter; the workload's prevalence is a separate quantity that does not enter the filter at all. The PPV at deployment is a joint property of both, as Section 7.4 insisted. The deployment question is what fraction of $\top$-flagged messages are actually spam, and what to do if that fraction is not high enough for the operational requirement.

Empirical observation

Ground truth is available on this workload (the operator labels a sample, or downstream feedback eventually arrives). Of the $N = 10000$ messages, $n_\top = 1000$ are spam and $n_\bot = 9000$ are not. The filter reports $\top$ on every spam message by $\omega = 0$ and on $\mathrm{FP} = 92$ of the non-spam. The empirical false-positive rate is therefore

$$\hat{\varepsilon} \;=\; \frac{\mathrm{FP}}{n_\bot} \;=\; \frac{92}{9000} \;\approx\; 0.01022,$$

about twenty-five percent above the theoretical 0.0082. The Wilson 95% confidence interval (Definition 7.3.2) at $z = 1.96$ has center

$$\frac{0.01022 + 1.96^2/18000}{1 + 1.96^2/9000} \;\approx\; \frac{0.01044}{1.000427} \;\approx\; 0.01043$$

and half-width

$$\frac{1.96}{1.000427}\sqrt{\frac{0.01022 \cdot 0.98978}{9000} + \frac{1.96^2}{4 \cdot 9000^2}} \;\approx\; 1.959\sqrt{1.124 \times 10^{-6} + 1.186 \times 10^{-8}} \;\approx\; 0.00209,$$

giving the interval $[0.00834, 0.01252]$. The theoretical $\varepsilon = 0.0082$ sits just outside the lower endpoint. The discrepancy is real and not an artifact of small samples: at $n_\perp = 9000$ the Wilson interval is narrow enough to detect a 25% deviation, and the theoretical value falls below the lower endpoint of the interval by roughly one standard error. Probable causes include real-world hash collisions that mildly violate the uniform-hash assumption behind (5.1), or an item count slightly above the design $n = 5000$ pushing the filter into a less favourable regime. The diagnostic point is that the deployment ε is an empirical quantity to be measured, not a design parameter to be trusted on faith. We carry $\hat{\varepsilon}$ forward in the calculations below; the theoretical value 0.0082 is the designer's promise, and the empirical 0.0102 is what the workload actually sees.

PPV computation

The workload's prevalence point estimate is $\hat{\pi} = n_\top / N = 0.1$. Combining this with $\hat{\varepsilon}$ and $\hat{\omega} = 0$ in Definition 7.4.1 gives

$$\widehat{\mathrm{PPV}} \;=\; \frac{0.1 \cdot 1}{0.1 \cdot 1 + 0.9 \cdot 0.01022} \;=\; \frac{0.1}{0.10920} \;\approx\; 0.9158.$$

About 92% of $\top$-flagged messages are actually spam. A delta-method confidence interval follows from Proposition 7.4.1. Conditioning on $n_\top, n_\perp$ and substituting $\bar{t} = n_\top \cdot 1 = 1000$, $\bar{f} = n_\perp \hat{\varepsilon} = 92$, $\sigma_t^2 = n_\top \cdot 1 \cdot 0 = 0$, and $\sigma_f^2 = n_\perp \hat{\varepsilon}(1 - \hat{\varepsilon}) \approx 91.06$,

$$\mathrm{Var}\,(\widehat{\mathrm{PPV}}) \;\approx\; \frac{1000^2 \cdot 91.06}{(1000 + 92)^4} \;\approx\; 6.40 \times 10^{-5},$$

so the standard error is about 0.00800 and the Wald-style delta-method 95% interval is

$$\widehat{\mathrm{PPV}} \;\pm\; 1.96 \cdot 0.00800 \;\approx\; 0.9158 \pm 0.0157 \;\approx\; [0.900, 0.932].$$

The interval is informative: PPV is bounded above by about 0.93 at this workload's prevalence and at this filter's measured ε, with 95% confidence on the sampling distribution. The boundary-failure concern of Section 7.3 does not apply here because $\widehat{\mathrm{PPV}}$ is far from 0 and 1, and at the conditional sample sizes $n_\top = 1000, n_\perp = 9000$ the Normal approximation underlying the delta method is comfortable. The variance formula is also simplified by $\omega = 0$: the σ_t^2 term vanishes and the entire uncertainty in PPV comes from the false-positive count. Were $\omega > 0$, the recall would itself be a random quantity and both terms in Proposition 7.4.1 would contribute.

Diagnostic question

Suppose the operational requirement is PPV > 0.95: at most one $\top$-report in twenty should be wrong. The point estimate 0.9158 falls short. The interval $[0.900, 0.932]$ excludes 0.95. The deployment needs a fix.

Definition 7.4.1 has only three knobs: ε, ω, and π. The filter has $\omega = 0$ already. Two routes are open.

Route A: re-tune. Lower ε by enlarging the bit array or shifting k. To match PPV $= 0.95$ at $\pi = 0.1$,

$$0.95 \;=\; \frac{0.1}{0.1 + 0.9\varepsilon} \quad\Longleftrightarrow\quad \varepsilon \;=\; \frac{0.1 \cdot 0.05}{0.9 \cdot 0.95} \;\approx\; 0.00585.$$

At $\varepsilon = 0.005$ the formula gives PPV $= 0.1/(0.1 + 0.9 \cdot 0.005) \approx 0.9569$, comfortably above the target. Reaching $\varepsilon = 0.005$ from 0.01022 by (5.1) costs about 30% more bits at fixed k, or a small re-balancing of k at fixed m. The 30% space cost is a one-time hardware decision and buys an ε that survives the worst-case workload.

Route B: wait. Keep the filter as-is and bound the acceptable workload. At fixed $\varepsilon = 0.01022$, the prevalence required for PPV $= 0.95$ is

$$0.95 \;=\; \frac{\pi}{\pi + (1 - \pi) \cdot 0.01022} \quad\Longleftrightarrow\quad \pi \;\approx\; 0.163.$$

At $\pi = 0.2$ the formula gives PPV $= 0.2/(0.2 + 0.8 \cdot 0.01022) \approx 0.9607$, also above the target. Doing nothing structural is acceptable when the spam rate naturally rises above about one in six; below that, the target is missed. This route trades capital cost for the assumption that the workload mix will shift in the desired direction, which the deployment does not get to choose. It is the right route when prevalence is expected to climb for independent reasons, and the wrong route when prevalence might stay below the threshold indefinitely.

The choice is operational rather than statistical. Route A spends storage to lower ε; Route B accepts a residual 8% false share until the workload's prevalence rises on its own. Both calculations used the same three formulas: (5.1) for the filter's ε, the Wilson interval for its empirical estimate, and Definition 7.4.1 for the translation into precision at workload prevalence. What the chapter-7 toolkit added beyond the chapter-5 design formulas is the honest accounting of uncertainty. A point estimate of PPV alone, 0.9158, would have suggested the target 0.95 was nearly met and a small tweak would suffice. The delta-method interval $[0.900, 0.932]$ shows that the data rule out 0.95 at 95% confidence: a small tweak would not have sufficed.

The diagnostic question reduces to one division, run forward and backward, but the conclusion that a fix is needed at all rests on the interval, not the point.

Bibliographic Notes

The classification-measures manuscript. The treatment of PPV, NPV, recall, and F1 as random variables that are functions of $(\varepsilon, \omega, \pi)$, together with the delta-method derivations of their variances and Wald-style confidence intervals, follows Towell [Tow26c]. That manuscript is the primary source for Section 7.4: the PPV and NPV expressions, the prevalence-dependence diagnostic, the bivariate delta-method computation that yields Proposition 7.4.1, and the F_1 derivation are all taken from it. The chapter's contribution beyond the manuscript is the pedagogical scaffolding — the explicit retirement of the four items deferred in Section 1.4, the worked Bloom filter scenario of Section 7.5, and the connection between the manuscript's machinery and the Part II approximate-set framework via the predicate $\mathsf{A}^{\pm} : U \to \text{Bernoulli[bool]}$.

Wilson's score interval. The interval of Definition 7.3.2 is due to Wilson [Wil27], who derived it by inverting a different statistic from the one that gives the Wald form: instead of fixing $\varepsilon = \hat{\varepsilon}$ in the variance and asking which true values are consistent with the observed deviation, Wilson keeps ε as a free parameter inside the variance and solves the resulting quadratic in ε. The construction is a special case of test inversion (set-of-parameter values not rejected by a level-α test), but applied to the score statistic rather than the Wald statistic. The 1927 paper frames the result in the language of the "law of succession" and was written before the modern frequentist-CI vocabulary had stabilized; later texts standardized the procedure and gave it the name now in common use. Wilson's interval is the default modern recommendation for the binomial proportion when n_0 is small or the rate is near a boundary, and it is the recommendation this chapter inherits.

The Clopper-Pearson exact interval. The construction of Definition 7.3.3 is due to Clopper and Pearson [CP34], whose paper introduces the test-inversion idea in its cleanest form. For each candidate ε, run a two-sided binomial test at level α; the interval is the set of ε values that the test fails to reject. The construction is sometimes called the exact interval because it uses the exact binomial CDF rather than a Normal approximation, but

as the chapter notes, the discreteness of the binomial forces the coverage strictly above $1 - \alpha$ at most values of ε. The over-coverage is the price of the guarantee and is precisely what one wants in safety-critical applications. The Jeffreys interval of Definition 7.3.4 predates the modern Bayesian terminology under which it is now derived; its calibration property (matching frequentist coverage to higher order than Wald) is a result in higher-order asymptotic theory and is treated in standard texts on asymptotic inference.

The delta method. The asymptotic-Normality result for smooth functions of an asymptotically Normal estimator (Theorem 7.4.1) is a classical consequence of the first-order Taylor expansion and Slutsky's lemma. A proof in the form most directly usable for the chapter's PPV and F_1 derivations appears in Casella and Berger [CB02], Chapter 5, alongside the univariate and multivariate versions, the regularity conditions, and worked examples. The chapter cites that text for the proof and applies the result without re-deriving it. Readers who want a more thorough development of asymptotic theory, including the higher-order corrections that explain why Wilson and Jeffreys outperform Wald in the small-n_0 regime, should consult a standard graduate text on large-sample inference.

Diagnostic testing and the prevalence question. The prevalence-dependence of PPV and NPV at fixed (ε, ω) — the central caveat of Section 7.4 — is the same phenomenon that drives the medical diagnostic-testing literature referenced in Section 1.5: the bilingual conversation between epidemiology, signal detection, and information retrieval over which marginal to condition on and which derived rate gets the spotlight. A test with high sensitivity and high specificity can still have low PPV when the disease is rare, because PPV is the classifier-plus-workload combination and not the classifier alone. Chapter 1's bibliographic notes (Section 1.5) record the contingency-table tradition that this chapter inherits; chapter 7's contribution is to make the dependence explicit in formula and to give the reader a proper CI for any of the derived quantities at any specified prevalence.

Exercises

1. $\star$ **Compute PPV.** A classifier has rates $\varepsilon = 0.05$ and $\omega = 0.10$, and is deployed on three workloads whose prevalences are $\pi \in \{0.01, 0.1, 0.5\}$.

   (a) Substitute each prevalence into Definition 7.4.1 and compute the positive predictive value. Report to four decimal places.

(b) Comment on the magnitude of PPV at the three prevalences. The classifier's (ε, ω) has not moved across the three workloads; only π has.

(c) In one sentence, name which of the three workloads makes the classifier appear most precise and which makes it appear least precise, and identify the §7.4 paragraph that flags this prevalence-dependence as the central caveat for reporting precision in isolation.

2. ⋆ **Compute F_1.** The same classifier $\varepsilon = 0.05, \omega = 0.10$ is deployed on the balanced workload $\pi = 0.5$.

(a) Compute recall $= 1 - \omega$.

(b) Compute precision = PPV at $\pi = 0.5$ by Definition 7.4.1.

(c) Substitute the two values into Definition 7.4.3 to compute F_1. Report to four decimal places.

3. ⋆ **Wilson interval evaluation.** A classifier was run on $n_0 = 100$ truth-$\perp$ trials and produced FP $= 10$ false positives.

(a) Compute the empirical rate $\hat{\varepsilon} = \mathrm{FP}/n_0$.

(b) Apply Definition 7.3.2 at $z_{0.025} = 1.96$ to compute the Wilson 95% confidence interval for ε. Report the center, the half-width, and the resulting interval endpoints to four decimal places.

(c) Compare to the Wald interval Definition 7.3.1 at the same data: by how much does the Wilson center differ from $\hat{\varepsilon}$, and in which direction is it shifted? Refer back to §7.3 for the score-statistic shift toward 1/2.

4. ⋆⋆ **Notebook-tied CI comparison.** Open the chapter 7 companion notebook `python/notebooks/ch07-classification-measures.ipynb`.

(a) For each combination $n_0 \in \{20, 100, 500\}$ and $\varepsilon \in \{0.01, 0.05, 0.5\}$, simulate $M = 10000$ independent draws of FP $\sim$ Binomial(n_0, ε), construct the four intervals of Definitions 7.3.1 to 7.3.4 at each draw, and tabulate the empirical coverage (fraction of intervals containing the true ε).

(b) From the resulting 3×3 table for each interval, identify the regime (n_0, ε) where Wald coverage falls farthest below the nominal 0.95.

(c) In one sentence, state which of the three alternative intervals recovers most of the lost coverage in that regime and which has the highest coverage overall. Refer to Definitions 7.3.2 to 7.3.4 by name.

5. ⋆⋆ **Delta method for log-odds.** The log-odds transformation $g(\varepsilon) = \log[\varepsilon/(1-\varepsilon)]$ maps the binomial parameter from $(0, 1)$ to the entire real line and is the standard re-parametrization in logistic regression.

   (a) Differentiate g and write $g'(\varepsilon)$ as a function of ε. Simplify the two-term derivative to a single rational expression.

   (b) Apply Theorem 7.4.1 with $\sigma^2(\varepsilon) = \varepsilon(1-\varepsilon)/n_0$ (the variance of $\hat{\varepsilon}$ from Proposition 7.2.2). Derive $\mathrm{Var}\,([])\, g(\hat{\varepsilon})]$ in terms of ε and n_0, simplifying as far as possible.

   (c) In one sentence, identify the geometric feature of the result that makes the log-odds scale attractive for confidence intervals near the boundaries $\varepsilon \to 0$ or $\varepsilon \to 1$: the variance on the log-odds scale changes smoothly across the unit interval, whereas the variance of $\hat{\varepsilon}$ itself goes to zero at the endpoints.

6. ⋆⋆ **Prevalence-dependence visualization.** Fix the symmetric classifier $\varepsilon = 0.01, \omega = 0.01$.

   (a) Plot or sketch $\mathrm{PPV}(\pi)$ from Definition 7.4.1 as a function of $\pi \in [0, 1]$. Identify the value of π at which PPV is minimized, the value at which it is maximized, and the corresponding extremal values of PPV.

   (b) Plot or sketch $\mathrm{NPV}(\pi)$ from Definition 7.4.2 as a function of $\pi \in [0, 1]$. Identify the value of π at which NPV is minimized and the value at which it is maximized.

   (c) In one sentence, describe the qualitative symmetry between the two curves: PPV rises from 0 at $\pi = 0$ to 1 at $\pi = 1$, and NPV falls from 1 at $\pi = 0$ to 0 at $\pi = 1$, because PPV is a column-conditional rate on the positive-output column and NPV is its mirror on the negative-output column.

7. ⋆⋆ **Sample-size formula for PPV CI.** Suppose the workload prevalence π is known and the goal is a Wald-style 95% confidence interval for PPV whose half-width is at most h.

(a) Starting from Proposition 7.4.1 and the Wald form $\widehat{\mathrm{PPV}} \pm 1.96\sqrt{\mathrm{Var}\,([])\,\widehat{\mathrm{PPV}}]}$, set the half-width to h and solve for the total sample size n. Use $n_\top = n\pi$ and $n_\perp = n(1-\pi)$ in the variance formula.

(b) State the resulting formula for n as a function of $\varepsilon, \omega, \pi, h$. Comment in one sentence on how n scales with h (the standard $1/h^2$ scaling shared by every Wald-style sample-size calculation) and on whether n grows or shrinks as $\pi \to 0$ at fixed (ε, ω, h).

(c) Substitute the numbers $\varepsilon = 0.05, \omega = 0.10, \pi = 0.1, h = 0.01$ and report the required n. Round up to the nearest integer.

8. ⋆⋆ **Bloom filter PPV with uncertain prevalence.** A Bloom filter has known $\varepsilon = 0.005$ and $\omega = 0$. It is deployed on a workload whose prevalence is estimated as $\hat{\pi} = 0.05$ from a Bernoulli sample of 1000 messages. The Wilson 95% CI for π is approximately $[0.038, 0.066]$.

(a) Apply Definition 7.4.1 at the point estimates $\varepsilon = 0.005, \omega = 0, \hat{\pi} = 0.05$ to compute the point estimate $\widehat{\mathrm{PPV}}$.

(b) Treat PPV as a function of two random quantities, $\hat{\varepsilon}$ and $\hat{\pi}$, with $\hat{\varepsilon}$ holding its deployed value 0.005 and $\hat{\pi}$ taking its empirical value with the stated Wilson CI. Apply the bivariate Theorem 7.4.1 (or substitute the Wilson endpoints directly into Definition 7.4.1 as a quick approximation) to propagate the prevalence uncertainty into a CI for PPV.

(c) Report the resulting PPV interval to three decimal places. In one sentence, compare its width to the width that would be obtained by treating $\hat{\pi}$ as known: the second source of uncertainty has now entered, and the CI is correspondingly wider than the Proposition 7.4.1-based CI computed at fixed π.

9. ⋆⋆⋆ **MLE vs method-of-moments.** Proposition 7.2.3 identified $\hat{\varepsilon} = \mathrm{FP}/n_0$ as the maximum-likelihood estimator. The method of moments offers an alternative recipe: equate sample moments to population moments and solve.

(a) The first-moment method-of-moments estimator equates $\mathbb{E}[\mathrm{FP}] = n_0\varepsilon$ to the observed FP and solves $\hat{\varepsilon}_{\mathrm{MoM},1} = \mathrm{FP}/n_0$. Confirm that this coincides with the MLE of Proposition 7.2.3.

(b) A second-moment method-of-moments estimator equates $\mathbb{E}[\mathrm{FP}^2] = n_0\varepsilon(1-\varepsilon) + n_0^2\varepsilon^2$ to FP^2 and solves the resulting quadratic for ε.

Carry out the algebra and confirm that the positive root reduces to FP/n_0 in the binomial model (the two recipes coincide because the Bernoulli/binomial family has $\mathrm{Var}(X) = p(1-p)$ fully determined by the mean).

(c) Construct a small example where the two recipes disagree. Take a two-parameter model such as $\mathrm{Beta}(a, b)$ with a, b both unknown: the first-moment equation alone is one equation in two unknowns, so the moment hierarchy must include the second moment to identify both parameters. State in one sentence the moral that the MLE = first-moment-MoM coincidence is special to one-parameter exponential families of which the binomial is the canonical example.

10. ⋆⋆⋆ **Coverage of the delta-method CI.** The §7.4 delta-method CI for $\widehat{\mathrm{PPV}}$ inherits its calibration from the underlying Wald CIs on $\hat{\varepsilon}$ and $\hat{\omega}$ via Proposition 7.4.1.

(a) Sketch why the empirical coverage of the delta-method PPV CI can drop below nominal $1 - \alpha$ when the prevalence π is small. Identify the term in Proposition 7.4.1 that becomes the dominant source of variability in the small-π regime, and argue that this term is the false-positive count FP on the truth-$\perp$ slice, whose own Wald CI is unreliable at small ε by §7.3.

(b) Connect the failure to the §7.3 Wald sawtooth: the PPV CI is a ratio with a small denominator $\bar{t} + \bar{f}$ when π is small and the per-event ε is also small, so a discrete jump in FP moves $\widehat{\mathrm{PPV}}$ by a substantial fraction of the CI half-width, replicating the sawtooth in a derived quantity.

(c) In one or two sentences, propose a remedy: replace the Wald-style delta-method CI with a Wilson-style score interval on the column-conditional binomial $\mathrm{TP} \mid \mathrm{TP} + \mathrm{FP} \sim \mathrm{Binomial}(\mathrm{TP} + \mathrm{FP}, \mathrm{PPV})$, which inherits the boundary correction of Definition 7.3.2 and stays inside $[0, 1]$ by construction.

11. ⋆⋆⋆ F_1 **vs precision-recall tradeoff.** The harmonic mean $F_1 = 2\mathrm{precision} \cdot \mathrm{recall}/(\mathrm{precision} + \mathrm{recall})$ of Definition 7.4.3 treats precision and recall on equal footing, but the resulting iso-F_1 contours in the $(\mathrm{precision}, \mathrm{recall})$ plane have geometric structure worth naming.

(a) Fix the target $F_1 = 0.9$. Solve $0.9 = 2\mathrm{precision} \cdot \mathrm{recall}/(\mathrm{precision} + \mathrm{recall})$ for recall as a function of precision. Verify that the result-

ing expression is well-defined precisely when precision > 0.45 (so that the denominator stays positive and recall $\in (0, 1]$).

(b) Plot or sketch the curve in the (precision, recall) unit square. Identify its symmetry under interchange of the two axes (the curve is invariant under precision $\leftrightarrow$ recall, a direct consequence of the symmetry of the harmonic mean in its two arguments) and the point at which precision = recall on the curve.

(c) Argue that F_1 weights precision and recall equally, in the precise sense that $\partial F_1/\partial\text{precision} = \partial F_1/\partial\text{recall}$ at the symmetric point precision = recall. Then argue informally that any weighted generalization $F_\beta = (1 + \beta^2)\text{precision} \cdot \text{recall}/(\beta^2\text{precision} + \text{recall})$ with $\beta \neq 1$ would break this symmetry, reweighting the two component rates by β^2 on the precision side.

Chapter 8

Information-Theoretic Bounds

After this chapter you will be able to:

- State Shannon entropy $H(X)$ and channel capacity for the BSC; relate them to the noisy-bit object of Part I.
- Derive the $-\log_2 \varepsilon$ bits-per-element lower bound for any FNR-zero approximate set with false-positive rate ε.
- Quantify the standard Bloom filter's $\approx 1.44\times$ space overhead relative to the entropy floor, and identify the construction (chapter 14's BHF) that closes the gap.

8.1 Shannon Entropy Refresher

The chapter's two headline results, the BSC capacity in Section 8.2 and the entropy floor for approximate sets in Section 8.3, both rest on a small slice of Shannon's information theory: the entropy of a random variable, the special form of that entropy for a single binary outcome, and the mutual information between two random variables. This section collects the three pieces and fixes notation. Cover and Thomas [CT06] is the standard depth reference; nothing here is derived from first principles.

Entropy of a discrete random variable

For a discrete random variable X taking values in a finite alphabet $\mathcal{X}$ with distribution $\Pr[X = x]$, the *Shannon entropy* of X is

$$H(X) \;=\; -\sum_{x \in \mathcal{X}} \Pr[X = x] \, \log_2 \Pr[X = x], \tag{8.1}$$

with the convention $0 \log_2 0 = 0$. The base-2 logarithm sets the unit to *bits*. The operational reading is that $H(X)$ is the average number of yes/no questions an optimal binary search needs to identify a draw of X: at one extreme, a constant random variable has $H(X) = 0$ (no questions needed); at the other extreme, a uniform distribution over $|\mathcal{X}|$ outcomes has $H(X) = \log_2 |\mathcal{X}|$ bits (a balanced binary search). Real-world distributions sit somewhere between. The quantity $H(X)$ is also the tight lower bound on the expected length of any prefix-free binary code for X, by Shannon's source-coding theorem, and that operational reading is what Section 8.3 uses.

The binary entropy function

The single most-used special case is a Bernoulli random variable. For $X \sim \text{Bernoulli}(p)$ with $\Pr[X = 1] = p$ and $\Pr[X = 0] = 1 - p$, equation (8.1) collapses to a function of p alone:

$$H(p) \;=\; -p \log_2 p \;-\; (1 - p) \log_2 (1 - p), \qquad p \in [0, 1]. \tag{8.2}$$

We overload the symbol H: the form $H(X)$ takes a random variable and the form $H(p)$ takes a scalar $p \in [0, 1]$; context disambiguates. The function $H(p)$ has three properties used repeatedly in this chapter and in chapter 14:

1. *Concave on* $[0, 1]$. The second derivative $H''(p) = -1/(p(1 - p) \ln 2)$ is negative on $(0, 1)$, so H is strictly concave. Jensen-style averaging arguments apply.

2. *Maximum* 1 *at* $p = 1/2$. Setting $H'(p) = 0$ gives $\log_2((1 - p)/p) = 0$, so $p = 1/2$, where $H(1/2) = 1$ bit. A fair coin is the most uncertain single bit.

3. *Zero at the boundaries.* $H(0) = H(1) = 0$ by the $0 \log_2 0 = 0$ convention. A deterministic bit carries no information.

The function is symmetric about $p = 1/2$, since $H(p) = H(1-p)$ swaps the two terms. Numerically, $H(0.01) \approx 0.081$ bit, $H(0.1) \approx 0.469$ bit, $H(0.5) = 1$ bit. The quantity that matters in Section 8.2 is $H(\varepsilon)$ where ε is the BSC crossover rate; the quantity that matters in Section 8.3 is the related quantity $-\log_2 \varepsilon$, which is the entropy of an indicator that an event of probability ε occurs (the dominant term of $H(\varepsilon)$ as $\varepsilon \to 0$).

Mutual information

When two random variables X and Y are jointly distributed, the *mutual information*

$$I(X;Y) \;=\; H(X) - H(X \mid Y) \;=\; H(Y) - H(Y \mid X) \tag{8.3}$$

measures how much observing Y reduces uncertainty about X, or equivalently how much observing X reduces uncertainty about Y. The two forms in (8.3) are equal: $I(X;Y)$ is symmetric in its two arguments, and the chapter uses both orientations as convenience dictates. Here $H(X \mid Y) = \sum_y \Pr[Y=y]\, H(X \mid Y=y)$ is the conditional entropy of X given Y, the average residual uncertainty about X after Y is known. An equivalent form writes the mutual information directly from the joint distribution,

$$I(X;Y) \;=\; \sum_{x,y} \Pr[X=x, Y=y] \, \log_2 \frac{\Pr[X=x, Y=y]}{\Pr[X=x]\,\Pr[Y=y]}, \tag{8.4}$$

showing that $I(X;Y)$ is a Kullback-Leibler divergence between the joint distribution and the product of marginals, and so always nonnegative. Equality $I(X;Y) = 0$ holds iff X and Y are independent: the joint factorizes, the ratio inside the logarithm is uniformly 1, and observing one of the pair tells you nothing about the other. At the opposite extreme, if X is a function of Y, then $H(X \mid Y) = 0$ and $I(X;Y) = H(X)$: the observation of Y pins down X completely, and the reduction equals all of X's prior uncertainty. Between those two endpoints, $I(X;Y)$ interpolates smoothly and is the central object that Section 8.2 optimizes.

For a noisy channel with input X and output Y governed by a fixed conditional distribution $\Pr[Y \mid X]$, the mutual information $I(X;Y)$ depends on the input distribution $\Pr[X]$. The *channel capacity* is the maximum:

$$C \;=\; \max_{\Pr[X]} I(X;Y). \tag{8.5}$$

This is the most information that a single use of the channel can convey, optimized over what the sender is allowed to choose (the distribution on

inputs). Section 8.2 works out (8.5) explicitly for the BSC and arrives at the compact form $C = 1 - H(\varepsilon)$.

The chapter's plan for these tools

Equations (8.1), (8.2), (8.3), and (8.5) are the only information-theoretic objects this chapter needs. Section 8.2 applies them to the BSC of Section 2.1 and derives the capacity $C = 1 - H(\varepsilon)$, giving the chapter-2 channel its quantitative information-theoretic summary and recovering the familiar endpoints (one bit per use of a noiseless channel, zero bits per use of a maximally noisy channel). Section 8.3 then applies the source-coding reading of $H(X)$ to the bit-string representation of an approximate set: the result is a per-element entropy floor of $-\log_2 \varepsilon$ bits for any approximate set with zero false-negative rate and false-positive rate at most ε. The reading is direct. An approximate-set representation is a binary code for some random object, and any such code is bound below by the entropy of what it encodes; the floor is that entropy made explicit. Section 8.4 compares the standard Bloom filter's actual cost to that floor and recovers the literature's $\log_2 e \approx 1.443$ overhead constant. Throughout, the role of (8.2) is to bridge a probability into the bit budget that probability implies, and the role of (8.5) is to bridge a channel into the per-use information bound it sets.

8.2 Channel Capacity in the BSC

Recall: the BSC

The binary symmetric channel of Section 2.1 takes a single Boolean input $X \in \{\top, \bot\}$ and produces a Boolean output $Y \in \{\top, \bot\}$. Definition 2.1.1 fixes a single parameter $\varepsilon \in (0, 1)$ that governs the channel: each transmitted bit is reproduced faithfully with probability $1 - \varepsilon$ and flipped to the opposite value with probability ε, the symmetry being that the flip probability does not depend on which value was sent. The channel matrix

$$Q = \begin{pmatrix} 1 - \varepsilon & \varepsilon \\ \varepsilon & 1 - \varepsilon \end{pmatrix}$$

records this: rows indexed by the input, columns by the output, both diagonal entries equal to $1 - \varepsilon$ (correct transmission) and both off-diagonal entries equal to ε (flip). The BSC is the common abstraction behind chapter 1's biased coin, noisy communication line, and binary classifier; capacity is its quantitative information-theoretic summary.

Capacity, specialized to the BSC

The channel capacity defined in (8.5) of Section 8.1 is

$$C \;=\; \max_{P_X} I(X;Y),$$

the largest mutual information achievable by choosing the input distribution P_X on $\{\top,\bot\}$ freely while the conditional distribution $\Pr[Y \mid X]$ is held fixed by the channel matrix Q. The maximization has a single scalar free parameter: write $P_X(\top) = q \in [0,1]$ and $P_X(\bot) = 1-q$. Symmetry of the BSC will collapse the optimum to $q = 1/2$, the uniform input. The argument is short enough to write out in full.

Derivation

We compute $I(X;Y) = H(Y) - H(Y \mid X)$, the second of the two equivalent forms in (8.3), because $H(Y \mid X)$ turns out to be independent of the input distribution and the optimization then reduces to maximizing $H(Y)$.

Step 1: $H(Y \mid X) = H(\varepsilon)$. Given input $X = \top$, the output Y is a Bernoulli random variable: $Y = \top$ with probability $1-\varepsilon$ (correct transmission) and $Y = \bot$ with probability ε (flip). Its entropy, by (8.2), is $H(\varepsilon)$. The same calculation with $X = \bot$ yields the same $H(\varepsilon)$, because the two rows of Q are permutations of each other. Averaging over the input distribution then gives

$$H(Y \mid X) \;=\; q \cdot H(\varepsilon) + (1-q) \cdot H(\varepsilon) \;=\; H(\varepsilon),$$

a constant independent of q. The conditional entropy is the intrinsic flip uncertainty of the channel and has nothing to do with what the sender chooses to put in.

Step 2: $H(Y) \le 1$, with equality at $q = 1/2$. The output marginal probability $\Pr[Y = \top]$ is

$$\Pr[Y = \top] \;=\; q(1-\varepsilon) + (1-q)\varepsilon \;=\; \varepsilon + q(1-2\varepsilon),$$

which at $q = 1/2$ equals $\varepsilon + \frac{1}{2}(1-2\varepsilon) = \frac{1}{2}$. The output is uniform on $\{\top,\bot\}$, and its entropy is the maximum value of the binary entropy function, $H(1/2) = 1$ bit. For any other q, $\Pr[Y = \top]$ is some value in $[0,1]$ different from $\frac{1}{2}$, and $H(Y) < 1$ by strict concavity of H with maximum at $1/2$.

Step 3: combine. The mutual information is

$$I(X;Y) \;=\; H(Y) - H(Y \mid X) \;=\; H\big(\varepsilon + q(1-2\varepsilon)\big) - H(\varepsilon),$$

with the second term constant. Maximizing over q pushes the first term to its ceiling of 1, attained at $q = 1/2$. The capacity is the maximum:

$$C \;=\; 1 - H(\varepsilon).$$

Proposition 8.2.1 (BSC capacity)**.** *The capacity of the binary symmetric channel of Definition 2.1.1 with crossover probability $\varepsilon \in (0,1)$ is*

$$C \;=\; 1 - H(\varepsilon) \;=\; 1 + \varepsilon \log_2 \varepsilon + (1-\varepsilon)\log_2(1-\varepsilon) \quad \textit{bits per channel use,} \tag{8.6}$$

attained by the uniform input distribution $P_X(\top) = P_X(\bot) = 1/2$.

A single number summarizes the BSC's quantitative information content. Once ε is fixed, the channel admits at most $C(\varepsilon)$ bits per use, no matter what coding scheme the sender devises. Shannon's noisy-channel coding theorem then promises that this ceiling is asymptotically achievable: for any rate $R < C$ and any $\varepsilon > 0$, there exist block codes of large enough length whose decoding error rate is below ε. The BSC capacity $1 - H(\varepsilon)$ is thus both an upper bound on what is possible and the asymptotic rate that good codes actually achieve.

Reading the result

The closed form $C = 1 - H(\varepsilon)$ has a sharp shape worth fixing in mind, because §8.3 uses the same binary entropy function in a structurally similar role for approximate sets.

$\varepsilon = 0$**: capacity** 1**.** A noiseless channel transmits one bit per use without error. Setting $\varepsilon = 0$ in (8.6) gives $H(0) = 0$, so $C = 1 - 0 = 1$. The output equals the input deterministically; one bit goes in, one bit comes out.

$\varepsilon = 1/2$**: capacity** 0**.** A maximally noisy channel transmits zero bits per use. Setting $\varepsilon = 1/2$ gives $H(1/2) = 1$, so $C = 1 - 1 = 0$. The output is a fair coin flip, statistically independent of the input, and no amount of clever coding extracts information from it.

$\varepsilon = 1$**: capacity** 1 **again.** A channel that always flips the bit is just as informative as a noiseless one: the receiver applies the deterministic relabeling $\top \leftrightarrow \bot$ and recovers the input perfectly. The formula reflects this: $H(1) = 0$, so $C = 1$. The function $C(\varepsilon) = 1 - H(\varepsilon)$ is symmetric about $\varepsilon = 1/2$, mirroring the symmetry of the binary entropy function.

Intermediate values. At $\varepsilon = 0.01$, $H(0.01) \approx 0.081$, so $C \approx 0.919$ bits per use: a 1%-error channel delivers roughly 92% of one bit per transmission, a small but real penalty. At $\varepsilon = 0.1$, $C \approx 1 - 0.469 = 0.531$

bits per use, slightly more than half a bit. The shape of C on $[0, 1/2]$ is steep near $\varepsilon = 0$ (the first bit of crossover noise costs little) and shallow near $\varepsilon = 1/2$ (the last bits of noise cost almost everything). The asymmetry has a direct operational reading. A channel with 1% crossover is almost as good as a noiseless one for practical coding purposes; a channel with 40% crossover, by contrast, has $C \approx 0.029$ bits per use, and getting any useful throughput out of it requires a very long block code spreading each source bit over many channel uses. The chapter 8 notebook in `python/notebooks/ch08-information-bounds.ipynb` plots the full curve and tabulates representative values.

From channel capacity to representation cost

Proposition 8.2.1 is the canonical first application of Section 8.1's tools and the quantitative completion of chapter 2's BSC. The same machinery underlies a different but related question that occupies the rest of this chapter: how many bits are needed to *represent* an approximate set with target false-positive rate ε? Shannon's source-coding theorem, mentioned briefly in Section 8.1, gives the relevant form. Any prefix-free binary encoding of a random source emitting n symbols of entropy H each requires on average at least nH bits. Section 8.3 applies this lower bound to the bit string that an approximate-set data structure stores: the bits in that string have a constrained distribution governed by ε, and the resulting per-element entropy gives a floor of $-\log_2 \varepsilon$ bits per stored element for any FNR-zero representation. The BSC capacity derived here and the entropy floor derived there are two facets of the same source-coding principle, with the role of the binary entropy function H holding throughout.

8.3 The Entropy Floor for Approximate Sets

The Bloom filter of Definition 5.2.1 solves a concrete engineering problem with a concrete bit budget, and chapter 5 already hinted that the budget is not minimal: a footnote in Section 5.4 placed Bloom at roughly 9.6 bits per item at $\varepsilon = 0.01$ and contrasted that with a floor near 6.64 bits. The present section turns that contrast into a theorem. For any approximate set with false-negative rate zero and false-positive rate at most ε, every representation, including ones nobody has invented yet, must store at least $\log_2(1/\varepsilon)$ bits per element on average. The Bloom filter pays a constant factor above this floor; chapter 14 constructs a representation that meets it.

Setup

Fix a finite universe U with $|U| = N$ and a target set $A \subseteq U$ with $|A| = n$. By an *approximate-set representation* we mean a pair (R, query) in which $R \in \{0,1\}^L$ is a bit string of length L, and $\text{query}(R, \cdot) : U \to \{\top, \bot\}$ is a deterministic algorithm that reads R and any candidate $x \in U$ and returns a membership verdict. The bit-string length L is the cost we want to bound below, and the parameter pair (ε, ω) that chapter 5 used to characterize an approximate set translates here into constraints on the verdict the algorithm produces. Following Definition 5.4.1, we restrict attention to FNR-zero representations, the family that contains the Bloom filter:

FNR-zero. For every $x \in A$, $\text{query}(R, x) = \top$. The representation never misses an inserted element.

FPR at most ε. When x is sampled uniformly at random from $U \setminus A$, the probability that $\text{query}(R, x) = \top$ is at most ε. Equivalently, the number of non-members that the algorithm accepts is at most $\varepsilon\,(N - n)$.

The two conditions together pin down the structure of the *accepted set*

$$A_R \;\coloneqq\; \{\, x \in U \;:\; \text{query}(R, x) = \top \,\}.$$

FNR-zero forces $A \subseteq A_R$: every true element is accepted. The FPR bound forces $|A_R \setminus A| \le \varepsilon(N - n)$: among the $N - n$ non-members, at most a ε fraction is accepted. So

$$|A_R| \;\le\; n + \varepsilon(N - n) \;\approx\; n + \varepsilon N \quad \text{for } N \gg n. \tag{8.7}$$

The approximate set $\mathsf{A}^{\pm}$ is determined by A_R, since the verdict on x depends on R only through which side of A_R the query lands on. The bit string R is whatever encoding the data structure happens to use; the accepted set A_R is the representation-independent object that does the work. We will count both, and the relation between the two is what gives the floor.

The theorem

Theorem 8.3.1 (Entropy floor for FNR-zero approximate sets)**.** *Let U be a universe of size N and let $\mathcal{A}_n = \{A \subseteq U : |A| = n\}$ be the family of n-element subsets. Suppose (R, query) is an FNR-zero approximate-set representation scheme with bit-string length L that, for every $A \in \mathcal{A}_n$, produces a representation $R(A)$ whose accepted set A_R satisfies $A \subseteq A_R$ and the FPR*

bound $|A_R \setminus A| \leq \varepsilon(N - n)$. *Then in the asymptotic regime* $N \gg n$ *and* $\varepsilon \in (0, 1)$ *bounded away from* 1*,*

$$L \;\geq\; n \log_2(1/\varepsilon) \;-\; o(n), \tag{8.8}$$

where $o(n)$ *is sublinear in* n*. Equivalently, the bit cost per stored element is bounded below by*

$$L/n \;\geq\; -\log_2 \varepsilon \;+\; o(1).$$

The dominant term $n \log_2(1/\varepsilon)$ is the floor; the sublinear correction collects finite-universe boundary effects and is negligible at the scales for which approximate sets are deployed. The constant has no parameters to tune: it is a feature of the information being represented, not of the algorithm representing it.

Proof sketch

The argument is a counting argument in two steps. The first counts how many distinct A_R accepting sets a single representation R can describe; the second counts how many A's of size n the scheme must cover. The bit budget L is then forced by the pigeonhole principle.

Step 1: each A_R covers many A's. Fix the accepted set A_R produced by a particular representation R. By FNR-zero we know that $A \subseteq A_R$, and by (8.7) we know that $|A_R| \leq n + \varepsilon N$. The true set A is therefore some n-element subset of A_R: any of

$$\binom{|A_R|}{n}$$

choices is consistent with the verdict that R delivers. To phrase this from the algorithm's point of view: query$(R, \cdot)$ cannot distinguish among the $\binom{|A_R|}{n}$ possible true sets A that all sit inside A_R. They are all consistent with what the algorithm reports. The representation R implicitly describes an *equivalence class* of true sets, not a single A.

Step 2: every A needs a representation. The total number of n-element subsets of U is $\binom{N}{n}$. For every such A, the scheme must produce some bit string $R(A) \in \{0, 1\}^L$ that, when paired with query, delivers an accepting set A_R containing A and satisfying the FPR bound. Two distinct true sets A_1, A_2 may share the same bit string R, but only if both are contained in the common A_R; otherwise some query would have to be answered differently and the representation could not work for both. The number of distinct

L-bit strings is at most 2^L, and each string accounts for at most $\binom{|A_R|}{n}$ true sets. So

$$2^L \cdot \max_{A_R} \binom{|A_R|}{n} \;\geq\; \binom{N}{n}. \tag{8.9}$$

Inequality (8.9) is the pigeonhole step.

Step 3: solve for L. Take logarithms in (8.9), use $|A_R| \leq n + \varepsilon N$ in the binomial on the left, and use Stirling's approximation for both binomials:

$$L \geq \log_2 \binom{N}{n} \;-\; \log_2 \binom{n + \varepsilon N}{n}.$$

The first term is $\log_2 \binom{N}{n} = n \log_2(N/n) + O(n)$ for $N \gg n$. The second term is $\log_2 \binom{n+\varepsilon N}{n} = n \log_2\big((n+\varepsilon N)/n\big) + O(n) = n \log_2\big(1+\varepsilon N/n\big) + O(n)$, which in the regime $\varepsilon N \gg n$ simplifies to $n \log_2(\varepsilon N/n) + O(n)$. Subtracting,

$$L \;\geq\; n \log_2(N/n) \;-\; n \log_2(\varepsilon N/n) \;-\; O(n) \;=\; n \log_2(1/\varepsilon) \;-\; O(n).$$

The $O(n)$ slack absorbs the $\log_2$-of-Stirling polynomial corrections and is sublinear when divided by n, so $L/n \geq -\log_2 \varepsilon + o(1)$.

The asymptotic regime. The simplification $\varepsilon N \gg n$ used in step 3 is the regime in which ε is fixed and N grows large relative to n, which is precisely the deployment regime for approximate sets: a target set of n items in a much larger universe, with ε the designer's chosen error tolerance. In that regime the floor is sharp. Outside it (for instance, when U is barely larger than A and $\varepsilon N \approx n$), the bound holds in the rougher form (8.9) but the per-element interpretation $-\log_2 \varepsilon$ degrades into a more elaborate expression. The exercises return to this regime question. The full asymptotic analysis, including the $o(n)$ control, is in the `towell-bernoulli-entropy` paper, which works through the Stirling expansion in detail and identifies the sublinear correction explicitly. □

Form of the bound

The pleasing thing about (8.8) is its lack of parameters. The per-element cost is a function of the one quantity the user controls, and the function is simple: a logarithm of the inverse error rate. The bit budget for a representation at target ε scales as $-\log_2 \varepsilon$ bits per element regardless of how the representation is built. Numerical values at standard error tolerances make the scale concrete:

ε	$-\log_2 \varepsilon$ (bits/element)	bits for $n = 10^6$
10^{-1}	≈ 3.32	≈ 0.42 MB
10^{-2}	≈ 6.64	≈ 0.83 MB
10^{-3}	≈ 9.97	≈ 1.25 MB
10^{-6}	≈ 19.93	≈ 2.49 MB
10^{-9}	≈ 29.90	≈ 3.74 MB

A million-element approximate set at one-in-a-billion false-positive rate cannot fit in less than about 3.74 megabytes, no matter what clever encoding it uses, so long as it is FNR-zero. The number doubles roughly every order of magnitude in $1/\varepsilon$, because each extra factor of 10 in error tolerance buys about $\log_2 10 \approx 3.32$ extra bits per element. The size of the universe N does not appear in the leading term. Only the target set size n and the error rate ε matter. That is striking, since one might have expected a universe-dependent term akin to $\log_2 \binom{N}{n}$ (the cost of identifying A exactly within U). The point of an approximate set is that the FPR budget buys back exactly the universe-dependent excess, leaving the residual $n \log_2(1/\varepsilon)$.

Why the floor is in ε, not in ω

Theorem 8.3.1 is stated for the FNR-zero family of Definition 5.4.1. The reader might reasonably ask why the floor takes the form $-\log_2 \varepsilon$ and not the symmetric $-\log_2 \varepsilon - \log_2 \omega$ or some average of the two. The short answer is that the floor is asymmetric because the constraint is asymmetric. The proof's pigeonhole step (8.9) uses the FNR-zero condition to force $A \subseteq A_R$, and that containment is what makes the count $\binom{|A_R|}{n}$ the right measure of equivalence-class size. If the FNR were also positive, the true set A would not need to sit inside A_R at all: some true elements could be misclassified, and the equivalence classes that representations describe would broaden in both directions. Spreading the error budget across both rates breaks the rigid $A \subseteq A_R$ containment that step 1 of the proof rests on. The bound (8.8) relaxes.

Stated as a theorem we do not prove here: a two-sided approximate set with rates (ε, ω) can in principle be represented in fewer than $n \log_2(1/\varepsilon)$ bits, because the false negatives let the representation get away with describing a smaller equivalence class on average. The exact two-sided bound is the binary-symmetric analog of the present floor: roughly $n\big(1 - H(\omega)\big) \log_2(1/\varepsilon)$ bits per element, with H the binary entropy function of (8.2) and the $H(\omega)$ term discounting the cost by the amount the false negatives blur the identity of A. We omit the derivation. The qualitative reading is enough for the present chapter: the FNR-zero family pays a strictly positive premium

relative to the two-sided rates because it has given up the option of trading some false negatives for fewer bits. Chapter 14 discusses the two-sided question in greater depth.

The constructive question

Theorem 8.3.1 is a lower bound. Like all lower bounds, it forbids: no FNR-zero scheme can do better than $-\log_2 \varepsilon$ bits per element. The question it begs is whether the bound is tight. Is there an FNR-zero approximate set that actually achieves $L/n = -\log_2 \varepsilon$, or do all constructions pay a strictly positive overhead? The standard Bloom filter pays a factor of about 1.44, which Section 8.4 derives in the next section. Chapter 14 (Chapter 14) constructs the Bernoulli Hash Function, a representation that achieves the floor asymptotically. The combination of Theorem 8.3.1 and the chapter 14 construction is a tight characterization of the FNR-zero family: the floor is at $-\log_2 \varepsilon$, and there is a known scheme that hits it. Until chapter 14 lands, the floor is a target; Section 8.4 measures how far the standard Bloom filter sits above it and asks what structural choice of the Bloom design is responsible for the overhead.

8.4 The Bloom Filter's Gap

Theorem 8.3.1 put a floor under every FNR-zero approximate set: at least $-\log_2 \varepsilon$ bits per element, no matter what the encoding looks like. The standard Bloom filter is FNR-zero by construction (Definition 5.2.1), so it is in the family the theorem governs. The question this section settles is how far above the floor the standard construction sits. The answer is a single constant, $\log_2 e \approx 1.443$, and the constant is the same at every ε. Bloom pays a fixed 44% markup on the entropy floor and never closes it; chapter 14 builds a different representation that does.

Setup

Recall the result of Theorem 5.2.1. A Bloom filter parameterized by bit count m, hash count k, and insertion count n achieves its minimum false-positive rate at $k^* = (m/n)\ln 2$, the choice that leaves the bit array half-occupied in expectation. At that k^* the optimal rate is

$$\varepsilon^* \;=\; 2^{-(m/n)\ln 2}. \tag{8.10}$$

The design parameter the engineer controls is the bit-per-element ratio m/n. The error rate ε^* that comes out is a fixed function of that ratio. Larger m/n buys a smaller ε^*, and the trade is exponential.

Bits per element at a target FPR

The engineering question is usually posed the other way around. A designer chooses a target false-positive rate ε and asks how many bits per element the filter needs. Solving (8.10) for m/n at $\varepsilon^* = \varepsilon$ inverts the relation. Take logarithms base 2 of both sides:

$$\log_2 \varepsilon \;=\; -(m/n)\ln 2,$$

which rearranges to

$$\frac{m}{n} \;=\; \frac{\log_2(1/\varepsilon)}{\ln 2} \;=\; \log_2(1/\varepsilon)\cdot \log_2 e. \tag{8.11}$$

The second equality uses the identity $1/\ln 2 = \log_2 e$. So the bit budget per element for a standard Bloom filter at target ε is a constant multiple, $\log_2 e$, of the quantity $\log_2(1/\varepsilon)$, which is the entropy-floor cost from Theorem 8.3.1.

Comparing to the floor

The floor of (8.8) reads, in the same units as (8.11),

$$\frac{m_{\min}}{n} \;\geq\; \log_2(1/\varepsilon).$$

Dividing the Bloom expression by the floor gives the ratio

$$\frac{m/n}{m_{\min}/n} \;=\; \log_2 e \;=\; \frac{1}{\ln 2} \;\approx\; 1.4427.$$

The ratio is independent of ε: every Bloom filter, regardless of the target error rate, pays the same $\log_2 e$ multiplier on the floor.

Proposition 8.4.1 (Bloom filter overhead)**.** *At every target false-positive rate $\varepsilon \in (0,1)$, the optimal standard Bloom filter (Theorem 5.2.1) uses*

$$\frac{m}{n} \;=\; \log_2(1/\varepsilon)\cdot \log_2 e$$

bits per element, which is $\log_2 e \approx 1.4427$ times the information-theoretic floor of Theorem 8.3.1. The relative overhead is a constant, independent of ε.

The number $\log_2 e$ has surfaced before, in the chapter 5 derivation of k^*: the optimal hash count $(m/n)\ln 2$ is the one that produces a half-occupied bitmap, and $\ln 2$ is precisely $1/\log_2 e$. The same constant that pins down the optimal hash count also pins down the gap to the floor. The two facts are dual statements about the same construction: half-occupancy is information-theoretically the wrong target, but it is the best target available to a representation built from k independent hashes and a flat bit array.

The 44% overhead in absolute terms

The constant 1.44 understates how much storage the gap costs at small ε. Multiplying a fixed factor onto a function that grows like $\log_2(1/\varepsilon)$ produces an absolute overhead that also grows, and at the error tolerances people actually deploy the overhead is several bits per element. The table makes this concrete:

ε	floor (bits/el)	Bloom (bits/el)	gap (bits/el)
10^{-2}	6.64	9.59	2.95
10^{-3}	9.97	14.38	4.41
10^{-6}	19.93	28.76	8.83
10^{-9}	29.90	43.14	13.25

At a one-percent false-positive rate the standard Bloom filter pays about three extra bits per element on top of the floor. At a one-in-a-million rate it pays about nine. The relative overhead is fixed at 44%, but the absolute overhead grows linearly in $\log_2(1/\varepsilon)$ along with the floor itself. Doubling the bit budget (in floor-equivalent terms) doubles the absolute waste. For a million-element approximate set at $\varepsilon = 10^{-6}$, the gap between Bloom and the floor is about 8.83 megabits, or 1.10 megabytes per million items, that the standard construction simply cannot recover.

A more sympathetic reading is also available. The Bloom filter pays a constant factor, not a growing one, and a 44% markup on an already-small quantity is acceptable in many deployment settings. The information-theoretic floor is a target, not a deadline; the Bloom filter is competitive even though it is suboptimal. The point of Proposition 8.4.1 is to quantify the slack, not to condemn the construction. Bloom filters dominate the approximate-set ecosystem for a reason. The question we now turn to is what part of the Bloom design forces the slack and what kind of construction would avoid it.

Why the gap is structural

The Bloom filter pays its $\log_2 e$ overhead because of three design choices that compose into the standard recipe:

- k *independent hash functions* write into a single shared bit array.
- The *shared bit array is flat*: bit j is independent of bit j' in the encoding's structure, and the only correlation between bits comes from the random hash collisions during insertion.
- The *query is a conjunction of bit reads*: QUERY(x) accepts iff every one of the k bits read at positions $h_1(x), \ldots, h_k(x)$ is 1.

The analysis of Theorem 5.2.1 treats the k bits read at query time as independent Bernoulli(p) variables with $p = 1 - e^{-kn/m}$. The product p^k is the FPR formula (5.1), and the optimum of p^k over k for fixed m/n sits at $p = 1/2$, where the bit array is half-occupied. A half-occupied array carries one bit of Shannon entropy per array bit, which is the maximum a binary variable can carry. The representation is therefore information-dense at the bit level. But the entropy floor is set by the membership-test problem, not by the information density of the encoding, and the two coincide only when the encoding's bits are tuned to the problem they encode rather than to a generic balance condition. The Bloom filter optimizes the wrong thing. It maximizes per-bit entropy and then takes the kth power of that, getting a clean closed form and a clean optimum but spending bits on the encoding's internal balance rather than on the discrimination it was supposed to perform.

A representation that achieves the floor must abandon at least one of these three design choices. The choice chapter 14 abandons is the second: the flat shared bit array. The Bernoulli Hash Function (BHF) of Chapter 14 replaces the flat bit array with a *perfect-hash-based encoding*, in which each inserted element gets a deterministic location chosen by a perfect hash, and the encoding stores a short fingerprint at that location. The fingerprint length is exactly $\log_2(1/\varepsilon)$ bits, matching the floor; the perfect hash ensures that collisions in the fingerprint table are themselves the only source of false positives, so the per-element cost is dominated by the fingerprint and the $\log_2 e$ overhead disappears. The detailed construction is chapter 14's business. The point to register here is that the gap is not a deficiency of analysis. It comes from a specific structural choice in the Bloom design, and a different structural choice can in principle close the gap.

What §8.5 takes up

Section 8.5 sketches the rest of the route from the present chapter to the constructive payoff in chapter 14: Part III generalizes from sets to maps and to arbitrary algebraic types, and Part IV builds the constructions, BHF among them, that close the gap quantified by Proposition 8.4.1.

8.5 What Comes Next

Part II began with the approximate set as a noisy bit per universe element, the object Chapter 5 built up from the binary symmetric channel of Part I. From that atom the part developed three things: the algebra of approximate sets under union, intersection, and complement (Chapter 6); the estimation discipline for false-positive and false-negative rates and the derived diagnostic measures that follow from them (Chapter 7); and the information-theoretic floor that pins the per-element space cost of any FNR-zero representation (this chapter). The approximate set as an atomic object is now fully described. Its rates are estimable, its compositions are predictable, its space cost is bounded below by a number that depends only on the false-positive rate, and the standard Bloom filter sits a known $\log_2 e$ above that number. Part II is done.

Part III generalizes from sets to maps. Chapter 9 lifts the type Bernoulli[bool], the noisy bit, to Bernoulli[T] for arbitrary algebraic types T. The model-order ceiling of 2 that the binary channel imposed in Part I lifts accordingly: an approximate map into a k-element output type is characterized by a $k \times k$ channel matrix, not a single (ε, ω) pair. Chapters 10 through 13 then develop, in order, sketches (count-min, hyperloglog, and the family of streaming estimators), randomized algorithms whose acceptance probability is itself the object of interest, approximate relations as the lifting of approximate sets from one column to two, and the formal algebraic-type framework that ties the type constructors of Bernoulli[T] to the error propagation rules Part II derived for sets. Maps generalize sets in the same way that channel matrices generalize the BSC.

Part IV builds the constructive payoff. Chapter 14 constructs the Bernoulli Hash Function, the approximate set that achieves the entropy floor of Theorem 8.3.1 asymptotically. The construction abandons the flat shared bit array of the standard Bloom design (the structural choice Section 8.4 identified as responsible for the $\log_2 e$ overhead) in favor of a perfect-hash-based fingerprint encoding. Chapter 15 constructs the counterpart on the other one-sided axis, the FPR-zero family of perfect hash filters, which give exact

membership at the cost of a controlled false-negative rate. Together these two chapters close out the constructive question that Part II's lower bound leaves open.

Part V draws the boundary. Chapter 16 catalogues the systems that are *not* Bernoulli, the cases in which the independent-error model breaks down: errors that are correlated across inputs, rates that drift in time, adversarial query distributions that defeat the uniform-sampling assumption underlying FPR estimation. The catalogue is itself useful, since it tells the practitioner when the machinery of Parts II through IV applies and when it does not. Chapters 17 and 18 then exploit the model's edges in a different way, turning to cipher maps and encrypted search: settings in which the controlled error of a Bernoulli map serves as a cryptographic resource rather than a deployment constraint. The thread that runs through all five parts is the same: an atom of controlled error, its algebra, its floor, its optimal constructions, and the boundary that fences off what the atom cannot describe.

Bibliographic Notes

The entropy manuscript. The entropy floor of Theorem 8.3.1 and the standard-Bloom overhead calculation of Proposition 8.4.1 both follow Towell [Tow26e]. That manuscript is the primary source for Section 8.3 and Section 8.4: the counting argument that converts the FNR-zero approximate-set specification into a lower bound on representation length, the asymptotic form $-n \log_2 \varepsilon$, and the comparison to the $\log_2 e$ overhead of the standard Bloom filter are all taken from it. The chapter's contribution beyond the manuscript is the pedagogical scaffolding: the entropy refresher of Section 8.1 written for readers who have not seen $\mathcal{H}(()X)$ before, the BSC bridge in Section 8.2 that ties the floor to Part I's running channel example, and the explicit forward pointers to chapter 14's construction that closes the gap.

Universal hashing and the counting argument. The counting-argument shape of the proof sketch in Section 8.3 owes its structure to the universal hashing program of Carter and Wegman [CW79], who showed that one can reason about the worst-case behavior of randomized hash-based data structures by counting collisions across an explicitly chosen family of hash functions rather than by appeal to the random-oracle model. The same combinatorial style of bookkeeping is what makes Theorem 8.3.1 go through: for

every representation R of length L, count the accepting sets A_R that R can describe; for every true set A that has to be represented at FPR $\leq \varepsilon$, count the candidate R's that work. The ratio is the bound. The Carter-Wegman framework is the prior art that makes this kind of argument standard rather than ad hoc.

Entropy and channel capacity. The information-theoretic background of Section 8.1 and Section 8.2 is standard textbook material; the depth reference for readers who want more than the working summary given here is Cover and Thomas [CT06], whose first three chapters cover $\mathcal{H}(()X)$, mutual information $I(X;Y)$, the binary symmetric channel, channel capacity, and the source-coding theorem. The BSC capacity result $C = 1 - \mathcal{H}(()\varepsilon)$ that Section 8.2 sketches is treated rigorously there, as is the source-coding interpretation that underwrites Theorem 8.3.1: any encoding of a structured object with inherent entropy rate h bits per symbol needs at least h bits per symbol on average. The entropy floor is the specialization of that generic statement to the FNR-zero approximate-set object whose inherent rate is $-\log_2 \varepsilon$ bits per element. Shannon's original 1948 paper [Sha48] remains the cleanest short presentation of the source-coding idea for readers who prefer to go to the source.

Compressed Bloom filters. Mitzenmacher [Mit02] observed that the standard Bloom filter operating at the optimal $k = (m/n) \ln 2$ produces a bit array in which roughly half the bits are 1 and half are 0. The array's empirical entropy is therefore close to 1 bit per array bit, and arithmetic coding of the array recovers a representation whose length matches the entropy floor of Theorem 8.3.1 rather than the $\log_2 e$-inflated cost of Proposition 8.4.1. The compressed Bloom filter trades the constant-time random-access query of the bitmap for the smaller stored size, which is appropriate when the filter is transmitted across a network (the original setting of the 2002 paper) but inappropriate when the filter is queried in place on disk or in cache. Mitzenmacher's paper is the right entry point for readers who want to understand how much of the 1.44 gap can be closed without abandoning the bitmap structure, and it sets up the contrast with chapter 14's BHF, which closes the gap by abandoning the bitmap entirely in favor of a perfect-hash-based representation whose query is again constant time. The Mitzenmacher and Upfal text [MU17] cited in earlier chapters is a useful companion for the probabilistic background.

Exercises

1. ⋆ **Entropy of a biased coin.** Compute the binary entropy $H(p)$ from Equation (8.2) for $p \in \{0.01, 0.5, 0.99\}$. Report each to four decimal places. In one sentence, name the symmetry $H(p) = H(1-p)$ that makes the $p = 0.01$ and $p = 0.99$ values agree, and identify the maximum of H on $[0, 1]$.

2. ⋆ **BSC capacity computation.** For $\varepsilon \in \{0.01, 0.1, 0.5\}$, compute the BSC channel capacity $C = 1 - H(\varepsilon)$ from Proposition 8.2.1. Report each to four decimal places. In one sentence, identify the regime (ε near 0 vs ε near $1/2$) where the channel carries close to a full bit per use vs where it carries essentially nothing.

3. ⋆ **Entropy floor evaluation.** For each target false-positive rate $\varepsilon \in \{0.1, 0.01, 0.001\}$, compute the entropy floor $-\log_2 \varepsilon$ in bits per element from Theorem 8.3.1. Report each to four decimal places. In one sentence, describe the linear-in-$-\log_2 \varepsilon$ scaling of the floor and contrast it with the linear-in-$1/\varepsilon$ scaling one might naively expect from the rate alone.

4. ⋆⋆ **Notebook-tied gap plot.** Open the chapter 8 companion notebook `python/notebooks/ch08-information-bounds.ipynb`.

   (a) For $\varepsilon \in [10^{-5}, 10^{-1}]$ (log-spaced, at least 25 points), compute the entropy floor $-\log_2 \varepsilon$ bits per element and the standard Bloom cost $-\log_2 \varepsilon \cdot \log_2 e$ bits per element from Proposition 8.4.1.

   (b) Plot both as line plots on a log-ε axis. Visually confirm that the two lines are parallel on this scale (they differ by a constant additive offset in bits per element).

   (c) Plot the ratio (Bloom cost) / (floor) vs ε. Verify the ratio is constant at ≈ 1.443 across the entire range, matching $\log_2 e = 1/\ln 2$ exactly. In one sentence, state which feature of the standard Bloom filter (the k independent hash functions of Theorem 5.2.1) produces this constant overhead.

5. ⋆⋆ **Mutual information of the BSC.** The BSC of Definition 2.1.1 has input $X \in \{\top, \bot\}$ with distribution P_X, output $Y \in \{\top, \bot\}$, and crossover ε. Use Equation (8.3) as $I(X;Y) = H(Y) - H(Y|X)$.

(a) Show that $H(Y|X) = H(\varepsilon)$ for every P_X (the conditional output uncertainty is just the channel's flip uncertainty, independent of the input distribution).

(b) Compute the marginal output distribution as $P_Y(\top) = P_X(\top)(1-\varepsilon) + P_X(\bot)\varepsilon$, and substitute to obtain $I(X;Y) = H(P_X(\top)(1-\varepsilon) + P_X(\bot)\varepsilon) - H(\varepsilon)$.

(c) Maximize $I(X;Y)$ over P_X by maximizing $H(Y)$ in the first term. Since H on a binary alphabet is maximized at $1/2$, the optimum is P_X uniform, which produces P_Y uniform and $H(Y) = 1$. Recover the BSC capacity $C = 1 - H(\varepsilon)$ of Proposition 8.2.1.

6. ⋆⋆ **The 1.44 constant.** The constant $\log_2 e \approx 1.4427$ appears in Proposition 8.4.1 as the ratio of standard Bloom cost to entropy floor.

(a) Starting from the change-of-base identity $\log_2 e = \ln e / \ln 2 = 1/\ln 2$, compute $\log_2 e$ to four decimal places using $\ln 2 \approx 0.6931$.

(b) Verify the equivalent form $\log_2(1/\varepsilon) \cdot \log_2 e = \ln(1/\varepsilon)/(\ln 2)^2$ by substituting $\log_2 x = \ln x / \ln 2$ on both factors. State why this quadratic-in-$1/\ln 2$ form is the natural one for the Bloom filter's bits-per-element formula (one factor of $1/\ln 2$ comes from the optimal-k scaling, a second from the conversion of natural to binary logarithms).

(c) In one sentence, state the geometric meaning of $\log_2 e$: it is the number of base-2 digits required to represent one nat of information.

7. ⋆⋆ **Entropy floor for sparse universes.** Theorem 8.3.1 is stated in the asymptotic regime $N \gg n$, where $N = |U|$ is the universe size and $n = |A|$ is the set size.

(a) Suppose instead that N is comparable to n, say $N = 10n$. Recompute the counting argument of Section 8.3: how many false positives can a representation accept before the FPR ceiling ε is breached? Does the dominant term in the bound $|A_R| \leq n + \varepsilon(N - n)$ change as N approaches n?

(b) Discuss the regime where the asymptotic form $-n \log_2 \varepsilon$ is accurate (large N, small ε) vs where finite-N corrections matter (small N/n ratio, or ε close to the trivial ceiling $1 - n/N$).

(c) In one sentence, identify the trivial representation that achieves $\varepsilon = 1 - n/N$ at zero space cost (always return $\top$), and argue that the entropy floor is informative only below this trivial ceiling.

8. ⋆⋆ **Two-sided floor.** Theorem 8.3.1 is stated for FNR-zero approximate sets. A two-sided approximate set permits both false positives at rate ε and false negatives at rate ω.

   (a) Sketch (without proof) why a two-sided set can have a lower space cost than $-n \log_2 \varepsilon$ at the same maximum error. Hint: the two-sided set splits its error budget across both directions, so each side can be allowed a larger rate than the one-sided ε.

   (b) Suppose the maximum-error budget is $\varepsilon = \delta$ for the one-sided case and $\max(\varepsilon, \omega) = \delta$ for the two-sided case. Argue informally that the two-sided set need only achieve $\varepsilon = \omega = \delta$ (each saturating its own ceiling), so the relevant rate in the bound is now δ on each side rather than δ concentrated on the FPR axis alone.

   (c) In one sentence, state the qualitative consequence: a two-sided set with $\varepsilon = \omega$ can sit below the one-sided floor at the same max-error budget, at the cost of abandoning the FNR-zero guarantee of Theorem 8.3.1.

9. ⋆⋆⋆ **The floor as a counting argument.** Theorem 8.3.1 is proved in Section 8.3 via a counting argument.

   (a) Reproduce the proof sketch in your own words. Begin with: a representation R of length L bits induces a query algorithm that accepts a superset $A_R \supseteq A$; FNR-zero forces the inclusion; FPR $\leq \varepsilon$ caps $|A_R \setminus A|$.

   (b) Identify the load-bearing step: the pigeonhole bound on the number of distinct A that a single R can encode. State the inequality $2^L \cdot (\text{accepting sets per } R) \geq \binom{N}{n}$ and explain in one sentence why each of the $\binom{N}{n}$ possible truth-sets A must be encoded by at least one R.

   (c) Take the binary logarithm of the resulting inequality and apply Stirling's approximation to $\binom{N}{n}$ to recover $L \geq n \log_2(1/\varepsilon)$ asymptotically. Identify which step of the derivation discards the additive lower-order terms.

10. ⋆⋆⋆ **Compressed Bloom filter.** [Mit02] showed that arithmetic compression of a standard Bloom filter's bit array can recover much of the $\log_2 e$ overhead identified in Proposition 8.4.1.

    (a) Sketch the argument: under the optimal k of Theorem 5.2.1, the bitmap is tuned so that approximately half its bits are 1

and half are 0 at insertion saturation. Conclude that the per-bit entropy of the bitmap is $H(1/2) = 1$ bit, so the bitmap is already informationally dense per stored bit.

(b) Argue that the $\log_2 e$ overhead comes from a different place: not from per-bit redundancy in the bitmap, but from the choice of bitmap-with-k-independent-hashes as the underlying representation. Arithmetic compression eliminates the per-bit redundancy that DOES exist when the bitmap is loaded below the half-full optimum (e.g., during construction or with a non-optimal k).

(c) In one sentence, identify why full elimination of the gap still requires abandoning the bitmap-with-k-hashes structure: the standard Bloom representation is wasteful at the structural level, not merely at the per-bit-encoding level, and only a different representation (chapter 14's BHF) can close the gap to the floor of Theorem 8.3.1.

11. ⋆⋆⋆ **Entropy floor for the union.** Definition 6.1.1 of §6.1 derived the FPR of the union $\tilde{A} \cup \tilde{B}$ of two approximate sets. The §8.3 floor applies to the union itself as an FNR-zero approximate set.

(a) Apply Theorem 8.3.1 to the union: the bits required to represent $A \cup B$ approximately at FPR $\varepsilon_\cup$ are at least $|A \cup B| \cdot \log_2(1/\varepsilon_\cup)$.

(b) Compare to the cost of representing A and B separately at FPRs $\varepsilon_A, \varepsilon_B$ and combining their query algorithms by disjunction. The separate cost is at least $|A| \log_2(1/\varepsilon_A) + |B| \log_2(1/\varepsilon_B)$ bits; the combined query has FPR $\leq \varepsilon_A + \varepsilon_B - \varepsilon_A\varepsilon_B$ (and equals $\varepsilon_A + \varepsilon_B$ when both are small). When A and B are disjoint and $\varepsilon_A = \varepsilon_B = \varepsilon_\cup/2$ (so the combined rate matches the union's $\varepsilon_\cup$), show that the two costs agree to leading order: $(|A|+|B|)\log_2(2/\varepsilon_\cup) = |A \cup B|(\log_2(1/\varepsilon_\cup) + 1)$, exceeding the union floor by exactly one bit per element.

(c) Identify when union representation is strictly more efficient: when A and B overlap heavily ($|A \cup B| \ll |A| + |B|$), the union floor scales with $|A \cup B|$, while the separate-then-combine cost scales with $|A|+|B|$. State in one sentence the practical guideline: store the union directly when membership in A vs B is not separately required by downstream queries.

Part III

From Sets to Maps

Chapter 9

Approximate Functions and n-ary Channels

After this chapter you will be able to:

- Define Bernoulli[T] for a finite-enumeration type T, and write the $K \times L$ channel matrix Q at model-order K.
- State and apply the n-ary Kronecker factorization theorem, which generalizes chapter 4 §4.4 from model-order 2 to general K.
- Compute channel capacity for an n-ary channel, generalizing the BSC capacity $C = 1 - H(\varepsilon)$ of chapter 8 §8.2.

9.1 From Approximate Sets to Approximate Functions

Recall: an approximate set is a function with a Boolean codomain

Section 5.1 opened Part II by observing that a classical set $A \subseteq U$ is nothing more than its membership predicate, a function $\mathbb{1}_A : U \to \{\top, \bot\}$ that returns $\top$ on the elements of A and $\bot$ elsewhere. The approximate analogue replaces the exact predicate with a noisy one: each evaluation $\mathsf{A}^{\pm}(x)$ is an instance of the noisy-bit type Bernoulli[bool], and the per-element rates collapse, under the chapter 4 axioms, to a single pair (ε, ω). The set as predicate is the framing that all of Chapter 5 inherits, and the framing pinned the codomain to $\{\top, \bot\}$. Part III asks what happens when the codomain is larger.

What if the codomain is bigger?

The predicate framing is not special to Booleans. Nothing in it relies on the codomain having exactly two values; what it relies on is that each evaluation is a draw from a distribution on the codomain, and that the per-query distribution depends only on which input block the query falls in. Replacing $\{\top, \bot\}$ with any other type T gives the *approximate function*,

$$\tilde{f} : U \longrightarrow T,$$

whose evaluation at x is a noisy T-valued report. Each evaluation $\tilde{f}(x)$ is an instance of the lifted type Bernoulli[T], the T-valued analogue of the noisy bit of Part I. The lifting is mechanical: a Bernoulli[T] instance has the same triple structure as a Bernoulli[bool] instance, except that the output alphabet has $|T|$ values rather than two, and the per-block rate is a distribution on T rather than a single error probability.

The framing covers a wide range of computational objects. For $T =$ bool, the codomain is $\{\top, \bot\}$ and the approximate function is exactly the membership predicate of an approximate set; Part II is a special case. For $T = \{1, 2, \ldots, K\}$, the codomain is a finite enumeration, and $\tilde{f}$ is an n-ary classifier whose output is one of K labels: the natural object for a tagger whose inputs are images and whose outputs name a category, or for a router whose inputs are queries and whose outputs name a shard. For $T = \mathbb{N}$, the codomain is the natural numbers, and $\tilde{f}$ is a counter or estimator: the natural object for a sketch that maps a multi-set to its approximate cardinality, or for a frequency estimator that maps a stream item to its approximate count. In each case the mathematical structure is the same: a function from a fixed input domain to a typed codomain, with the per-evaluation distribution governed by which block of inputs the query belongs to.

The shift is not just an enlargement of the codomain; it is a change in how the rates are described. A Boolean codomain admits two rates, ε for false positives and ω for false negatives, and the pair fully specifies the per-block behavior. A larger codomain admits more. With $|T| = K$ output values, each input block can map to any of K outputs, and the per-block behavior is a distribution on the K outputs rather than a single error probability. The two-by-two channel matrix of Section 5.1 becomes a $K \times L$ matrix where K counts input blocks and $L = |T|$ counts output values, with rows summing to one and the (i, j) entry giving the probability that the i-th block produces the j-th output. The matrix is the rate description for an approximate function in the same way that the pair (ε, ω) was for an approximate set: it

is the complete summary of per-query behavior, collapsed by the chapter 4 axioms from a potentially much larger parameter space.

Two natural examples

A 3-class classifier. Take $T = \{\text{cat}, \text{dog}, \text{bird}\}$ and let $\tilde{f}$ be a classifier whose input is an image and whose output is one of the three labels. The input domain U partitions into three blocks by true class: the cats, the dogs, and the birds. The per-block behavior is a distribution on the three labels, recorded as a 3×3 confusion matrix:

$$Q = \begin{pmatrix} Q_{\text{cat,cat}} & Q_{\text{cat,dog}} & Q_{\text{cat,bird}} \\ Q_{\text{dog,cat}} & Q_{\text{dog,dog}} & Q_{\text{dog,bird}} \\ Q_{\text{bird,cat}} & Q_{\text{bird,dog}} & Q_{\text{bird,bird}} \end{pmatrix}.$$

Row i records what the classifier does on inputs whose true class is i. The diagonal entries are the per-class accuracies; the off-diagonal entries are the confusion rates. A high-quality classifier has diagonal entries close to 1; a degenerate one that always says "cat" has its first column equal to one and the rest zero. The two-rate description of an approximate set generalizes to the nine-entry description of a 3-class classifier, with six free parameters once the row-sum constraints are imposed. The structure that Part II built for two-by-two membership channels is the model-order-2 specialization of the structure this chapter builds for $K \times L$ confusion matrices.

A counter. Take $T = \mathbb{N}$ and let $\tilde{f}$ be a counter whose input is a multi-set query and whose output is an estimate $\hat{n}$ of the true count n. The input domain partitions by true count: the block B_n consists of all multi-sets whose true count is n. The per-block behavior is a distribution on $\mathbb{N}$, recording the probability that the counter reports $\hat{n}$ when the truth is n. The channel matrix has countably many rows and countably many columns, but the structure is the same: row n is a distribution on $\mathbb{N}$ centered (one hopes) near n, with a tail width that records the counter's precision. Errors are not symmetric: the counter can both over-count and under-count, and the rate description records both directions of error as a function of the true count. For practical sketches (CountMin in chapter 10 and HyperLogLog in chapter 10's later treatment), the row distributions are concentrated, with most mass on $\hat{n}$ values close to n and an analytically tractable tail. The two-rate description of an approximate set generalizes to a row distribution per block, with the precision of the counter set by the tail width of the row.

What changes structurally

The lift from Part II's predicate framing to Part III's typed framing preserves more than it changes. The chapter 4 axioms transfer unchanged: Axiom 1 still requires that error events at distinct inputs be mutually independent, and Axiom 2 still requires that the per-block rate be stable across the block. What changes is the size of the per-block rate. With a Boolean codomain, the per-block rate is a single number (a Bernoulli parameter); with a T-valued codomain, the per-block rate is a distribution on T. The channel matrix grows from 2×2 to $K \times L$ accordingly, and the parameter count grows from 2 to $K(L-1)$, since each of the K rows of an L-column row-stochastic matrix has $L-1$ free entries.

Two chapter-level results scale with the change. Theorem 4.4.1 of Section 4.4 showed that the joint distribution of n independent queries on a Bernoulli[bool] instance factors as the Kronecker product of the n per-query two-by-two matrices. The same factorization carries over, with the per-query $K \times L$ matrices in place of the two-by-two ones, and Section 9.3 proves the general form. Similarly, Proposition 8.2.1 of Section 8.2 gave the closed-form capacity $C = 1 - H(\varepsilon)$ for the binary symmetric channel. The closed form is specific to the binary case, but the framework, capacity as the maximum mutual information over input distributions, generalizes to arbitrary $K \times L$ channels, and Section 9.4 develops the generalization (with closed forms for the symmetric sub-family, and numerical recipes for the rest).

Where the chapter goes from here

The rest of the chapter formalizes the generalization in three steps. Section 9.2 gives the formal definition of an n-ary channel as a $K \times L$ row-stochastic matrix, fixes the notation (K uppercase for model-order, distinguishing it from the lowercase k used for Bloom-filter hash counts and Miller-Rabin rounds), and works the constraints out from the chapter 4 axioms. Section 9.3 states and proves the n-ary Kronecker theorem, the headline result of the chapter, which lifts Theorem 4.4.1 from the binary case to general K. Section 9.4 lifts Proposition 8.2.1 from the BSC to general $K \times L$ channels, with closed forms where they exist and pointers to Blahut-Arimoto where they do not. The chapter closes with a bridge to chapter 10's sketches, each of which is an approximate function with a different choice of codomain T and a correspondingly different channel matrix.

9.2 N-ary Channels

Section 9.1 established the type-lifting picture: an approximate function $\tilde{f} : U \to T$ produces a noisy T-valued report at every input, and the per-block behavior is a distribution on T rather than the single Bernoulli parameter that sufficed for the Boolean case. This section converts that picture into a formal definition. The object is a row-stochastic matrix; the parameter count and the axioms transfer directly from chapter 4, with the binary 2 replaced by the model-order K on the input side and the output cardinality L on the output side. The rest of Part III builds on the matrix defined here.

The formal definition

The setup mirrors Definition 4.5.1: an input domain, an output domain, and a channel matrix that records the per-block output distribution. What changes is the size of the matrix.

Definition 9.2.1 (n-ary channel)**.** *An* n-ary channel *is a triple* (X, Y, Q) *where:*

1. X *is the* input domain*, partitioned by the finest compatible partition (Definition 2.4.1) into* K *blocks* $B_1, B_2, \ldots, B_K$*. The integer* K *is the* model order *of the channel.*

2. Y *is the* output domain*, with* $|Y| = L$*. The elements of* Y *are indexed* $1, 2, \ldots, L$*.*

3. Q *is a* $K \times L$ row-stochastic matrix*: its entries satisfy* $Q_{ij} \geq 0$ *for every* $i \in \{1, \ldots, K\}$ *and* $j \in \{1, \ldots, L\}$*, and every row sums to one,*

$$\sum_{j=1}^{L} Q_{ij} \;=\; 1 \qquad \text{for each } i \in \{1, \ldots, K\}.$$

The entry Q_{ij} *is the probability that the channel reports output* j *given that the input lies in block* B_i*:*

$$Q_{ij} \;=\; \mathrm{P}\left(\mathit{output} = j \,|\, \mathit{input} \in B_i\right).$$

The triple satisfies Axiom 1 (Axiom 1: error events at distinct inputs are mutually independent) and Axiom 2 (Axiom 2: the per-block output distributions are conditionally independent of one another given the objective function).

The triple (X, Y, Q) is the n-ary analogue of the Boolean triple of Definition 4.5.1. Setting $K = L = 2$ recovers that definition exactly, with the two rows of Q governing the two blocks $B_1 = \{\top\}$ and $B_2 = \{\bot\}$. Allowing K or L to exceed 2 produces a strictly larger family of channels, of which the Boolean case is the smallest non-trivial member.

Three components of the definition deserve highlighting. First, the input partition is the finest compatible partition in the sense of Definition 2.4.1: two inputs go in the same block precisely when their output distributions agree. This is the same convention as chapter 2; the model order K is a property of the channel itself, not a description of the input domain's set-theoretic structure. Second, the rows are indexed by input blocks and the columns by output values, with row i giving the distribution conditional on input block B_i. The convention matches the 2×2 orientation of chapter 4 (row 1 for $\{\top\}$, row 2 for $\{\bot\}$) and generalizes without surprise to the $K \times L$ case. Third, the row-stochastic constraint $\sum_j Q_{ij} = 1$ is the only constraint the entries satisfy beyond non-negativity. Every row is a probability distribution on the output alphabet, and that distribution is arbitrary up to summing to one. The shape of the matrix is wider when L is larger and taller when K is larger; non-square shapes ($K \neq L$) are the rule rather than the exception, and a $K = 5, L = 2$ channel (say, a five-block classifier whose output is binary accept/reject) is just as much an n-ary channel as the square cases.

Notation: K uppercase, k lowercase

Throughout Part III, the symbol K (uppercase) denotes the model-order of the input partition, as fixed by Definition 9.2.1. The symbol k (lowercase) is reserved for the two unrelated counts that have run through Parts I and II: the number of hash functions in a Bloom filter (Section 5.2) and the number of independent witness draws in a round of Miller-Rabin primality testing (Section 3.4). The two roles are unrelated, and the case distinction (uppercase versus lowercase) is the one device that keeps them apart.

The collision is real, not contrived. A Bloom filter has model order $K = 2$ (the input domain partitions into "in the set" and "not in the set") and hash count k (the per-element bookkeeping parameter that controls space and the false-positive rate). Miller-Rabin has model order $K = 2$ (the input partitions into "prime" and "composite") and round count k (the independent witnesses combined per query). In both cases the model-order K is fixed by the problem (it is binary), and the lowercase k is a design knob that the implementer chooses. Treating them as a single symbol would

collapse two genuinely different roles and obscure the fact that $K = 2$ is structural while k is tunable. Beyond Part II, where $K = 2$ is implicit throughout and the lowercase k shoulders the narrative weight, Part III makes K explicit and lets it vary, while the lowercase k continues to denote per-construction tuning parameters in the chapters that introduce them.

The same case discipline applies to L. The output cardinality is the uppercase L; any lowercase l that appears in derivations is a per-stage index or summation variable and is local in scope. The convention is the same one used in chapter 4 for Q and in chapter 8 for capacity: structural parameters of the channel are uppercase, indices and tuning knobs are lowercase.

Examples

The definition specializes to every channel that has appeared so far in the book, plus the new n-ary cases that motivate Part III. We work through four examples: two binary, one three-way, and one with a countably infinite input and output domain.

Example 1: $K = 2$, $L = 2$, symmetric. The simplest n-ary channel is the binary symmetric channel of Definition 2.1.1. The input partitions into $B_1 = \{\top\}$ and $B_2 = \{\bot\}$, both with the same flip probability ε, and the matrix is

$$Q_{\text{BSC}} = \begin{pmatrix} 1-\varepsilon & \varepsilon \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

The model order is $K = 2$ by partition count under the discrete partition, with the symmetry constraint ε shared between the rows; the output cardinality is $L = 2$; and the matrix is square. The definition specializes exactly: Q_{BSC} is a 2×2 row-stochastic matrix and the axioms transfer directly from Definition 4.5.1.

Example 2: $K = 2$, $L = 2$, asymmetric. Dropping the symmetry constraint recovers the asymmetric binary channel of Definition 2.2.1:

$$Q_{\text{asym}} = \begin{pmatrix} 1-\omega & \omega \\ \varepsilon & 1-\varepsilon \end{pmatrix}.$$

Row 1 governs $B_1 = \{\top\}$ with false-negative rate ω; row 2 governs $B_2 = \{\bot\}$ with false-positive rate ε. The matrix is again 2×2, but the two rows now have independently chosen probabilities, and the channel is fully specified by the pair (ε, ω). The Z-channel is the specialization $\omega = 0$ (or $\varepsilon = 0$): a one-sided asymmetric channel still living in the $K = L = 2$ family.

Example 3: $K = 3$, $L = 3$, a 3-class classifier. A confusion matrix is exactly an n-ary channel. Suppose $\tilde{f}$ classifies an input image as one of three labels: cat, dog, or bird. The input domain partitions into three blocks by true class, $B_1 = \text{cats}$, $B_2 = \text{dogs}$, $B_3 = \text{birds}$, and the output alphabet is also $\{\text{cat}, \text{dog}, \text{bird}\}$. A representative channel with 90% per-class accuracy and the off-diagonal mass split evenly between the two competing labels is

$$Q = \begin{pmatrix} 0.90 & 0.05 & 0.05 \\ 0.05 & 0.90 & 0.05 \\ 0.05 & 0.05 & 0.90 \end{pmatrix}.$$

Each row sums to 1; the diagonal entries are the per-class accuracies; the off-diagonal entries are the per-class confusion rates. The matrix has six free parameters once row sums are imposed ($3 \times (3 - 1) = 6$). The example above is constrained beyond the row-sum requirement (each row has only one free parameter, the diagonal 0.90), but a generic 3-class classifier uses all six degrees of freedom and produces a non-symmetric confusion matrix in which different true classes have different error profiles.

Example 4: $K = \infty$, $L = \infty$, a counter. Sketches preview. Let $\tilde{f}$ be a frequency estimator whose input is a stream item and whose output is an integer estimate $\hat{n}$ of that item's true count n. The input domain partitions into the blocks $B_n = \{x : \text{true count of } x = n\}$ for $n = 0, 1, 2, \ldots$, giving $K = \infty$. The output alphabet is $\mathbb{N}$, so $L = \infty$. The channel matrix is infinite in both dimensions, but row n is still a probability distribution on $\mathbb{N}$ (typically concentrated near n with a tractable tail). Writing the matrix out is impractical, but the structure transfers: Q_{nm} is the probability that the estimator reports m when the true count is n, and every row sums to one. Concrete sketches realize this object with specific tail shapes (Count-Min gives a one-sided tail, HyperLogLog gives a relative-error tail), and chapter 10 builds out the constructions in detail.

Axiom transfer from chapter 4

The two chapter 4 axioms transfer to the n-ary setting without modification. Both axioms are stated in chapter 4 for the Boolean case, but neither statement is binary in any essential way: they are claims about independence, and independence is indifferent to alphabet size.

Axiom 1 (Axiom 1) lifts unchanged. The Boolean form of the axiom asserts that error events at distinct inputs are mutually independent: the

probability that the channel errs on input x_1 is independent of the probability that it errs on x_2, for any pair of distinct inputs. The same statement applies verbatim in the n-ary case, with "errs" replaced by "produces output j": the events $\{\text{output on } x_1 = j_1\}$ and $\{\text{output on } x_2 = j_2\}$ are independent for all $x_1 \neq x_2$ and all $j_1, j_2 \in Y$. The Boolean case is the $L = 2$ instance of the same condition. Nothing in the proof of Axiom 1 (or in any of its uses) depended on $L = 2$, and the proof template carries forward.

Axiom 2 (Axiom 2) lifts unchanged. The Boolean form asserts that the per-block rates ε and ω are conditionally independent of each other given the objective set. In the n-ary case, each block B_i has a per-block output distribution rather than a single rate, but the independence claim is the same: the per-block distributions $Q_{1\cdot}, Q_{2\cdot}, \ldots, Q_{K\cdot}$ (the K rows of Q viewed as probability distributions on Y) are mutually conditionally independent given the objective function. The Boolean case is the $K = 2$ instance, with the two row distributions parametrized by (ε) and (ω) respectively. The generalization to K rows is dimensional, not conceptual.

The conjunction of Axiom 1 and Axiom 2 is what makes the channel matrix Q a sufficient summary of the n-ary channel's behavior, in the same sense that the pair (ε, ω) was sufficient for the Boolean case. Section 9.3 converts the independence claim into a factorization theorem for joint queries.

Parameter count

A $K \times L$ row-stochastic matrix has $K \cdot L$ entries, but the K row-sum constraints (one per row) eliminate one degree of freedom per row. The free-parameter count is therefore

$$K \cdot L - K \;=\; K(L-1).$$

Each row of L probabilities is a probability distribution on a finite set, and a distribution on an L-element set has $L - 1$ free parameters. Multiplying by the K rows gives the total. The formula is the n-ary generalization of the Boolean parameter count of Corollary 4.4.1.2: at $K = L = 2$, it reduces to $2 \cdot (2 - 1) = 2$, the familiar (ε, ω) pair.

The growth rate. The parameter count grows linearly in both K and L (a $K = 5, L = 3$ channel has $5 \cdot 2 = 10$ parameters; a $K = 10, L = 4$ channel has $10 \cdot 3 = 30$). For practical specification, the growth is gentle: a 10-class classifier with model order $K = 10$ is specified by 90 numbers (90% of the 100 entries of Q). The contrast with the unconstrained joint-distribution

parameter count is sharp. A general joint distribution over the same $K \times L$ input-output pairs has $K \cdot L - 1$ parameters in total (just one normalization constraint over the whole table). The Axiom 1 and Axiom 2 reduction from chapter 4 cuts this to $K(L-1)$ by imposing per-row normalization, exactly as in the Boolean case.

Constraints from problem structure. Many channels carry additional structural constraints that further reduce the parameter count below $K(L-1)$. Example 1's BSC imposes the symmetry constraint ε shared between the two rows, dropping the count from 2 to 1. Example 3's 3-class classifier with uniform off-diagonal mass (and a common diagonal) imposes additional constraints that reduce the six row-sum-free parameters to a single accuracy value 0.90. Strongly symmetric n-ary channels (Section 9.4) impose that the rows of Q be permutations of each other and that the columns be permutations as well, reducing the count to a single error probability. The $K(L-1)$ count is the upper bound; specific channels can be much more compact under their own symmetries.

Where the chapter goes next

The chapter now has the formal object that Section 9.1 previewed: a $K \times L$ row-stochastic matrix Q, satisfying the two chapter 4 axioms, with $K(L-1)$ free parameters in general. The next two sections lift the two structural results of Parts I and II to the n-ary setting. Section 9.3 generalizes Corollary 4.4.1.2: the joint distribution over m queries on an n-ary channel factors as the m-fold Kronecker product of the per-query channel matrices, with parameter count growing as $mK(L-1)$ rather than exponentially in m. Section 9.4 generalizes the BSC capacity result: capacity is still the maximum mutual information $I(X;Y)$ over input distributions on the K blocks, with closed forms for the symmetric sub-family and numerical recipes (Blahut-Arimoto) for the general case. Both results have the same shape as their Boolean ancestors; the change is dimensional, and the proofs are mostly bookkeeping once the n-ary Kronecker structure is in hand.

9.3 The n-ary Kronecker Theorem

Section 9.2 closed with the promise that the joint distribution over m queries on an n-ary channel would factor in the same way as for the Boolean case of chapter 4. This section delivers the promise. The statement, the proof, the corollary, and the worked example all mirror Section 4.4; the only change is

dimensional, with the binary output alphabet replaced by an output alphabet of size L and the binary input partition replaced by one of size K.

Recalling the chapter 4 theorem

At model-order 2, chapter 4 proved that the joint confusion matrix of m queries on a Bernoulli predicate factors as a Kronecker product of the m per-query channel matrices. The headline statement is Theorem 4.4.1: under Axiom 1 (error events at distinct inputs are mutually independent, Axiom 1) and Axiom 2 (per-block rates are mutually independent across blocks, Axiom 2), the rate-conditional joint matrix factors as

$$Q^{(m)*}(\alpha) \;=\; Q_1^*(\alpha) \otimes Q_2^*(\alpha) \otimes \cdots \otimes Q_m^*(\alpha).$$

Corollary 4.4.1.1 promotes this conditional statement to an unconditional one when the queried blocks are pairwise distinct, or, more importantly for the worked examples of Parts I and II, when the per-block rates are deterministic constants. In the deterministic-rates regime, $Q_\ell^* = Q_\ell$ for every query position ℓ, and the conditional and unconditional matrices coincide. Plan 5's §5.3 review of Part II confirmed that every Bloom filter and Miller-Rabin computation in the book operates in this regime, and Part III continues the convention. From here on we drop the conditional star and write Q for the (deterministic) per-query channel matrix.

The structural payoff of the chapter 4 theorem was the parameter-count collapse of Corollary 4.4.1.2: the joint matrix $Q^{(m)}$ requires nm free parameters rather than the $n(2^m - 1)$ that an unconstrained m-query channel would demand. The exponent 2^m became a linear $2m$ because the two axioms force a Kronecker structure that no general joint distribution shares. The n-ary generalization preserves this collapse, with the binary 2 replaced by the output cardinality L.

The headline theorem

Fix an n-ary channel (X, Y, Q) in the sense of Definition 9.2.1: input domain X partitioned into K blocks $B_1, \ldots, B_K$, output domain Y of size L, and channel matrix Q a $K \times L$ row-stochastic matrix whose entries Q_{ij} record $\mathrm{P}(Y = j | X \in B_i)$. The axioms transfer from chapter 4 without modification, as already established in Section 9.2. We now query the channel m times.

Theorem 9.3.1 (n-ary Kronecker factorization)**.** *Let (X, Y, Q) be an n-ary channel of model-order K and output cardinality L, satisfying Axiom 1*

(Axiom 1) and Axiom 2 (Axiom 2), and working in the deterministic-rates regime of chapter 4. Let $X_1, \ldots, X_m$ be m inputs queried in order, with $X_i \in B_{j_i}$ for each $i = 1, \ldots, m$, and let $Y_1, \ldots, Y_m$ be the corresponding outputs. Then the joint distribution of outputs given the input blocks factors as

$$\mathrm{P}\left(Y_1, \ldots, Y_m | X_1 \in B_{j_1}, \ldots, X_m \in B_{j_m}\right) \;=\; \prod_{i=1}^{m} Q_{j_i, Y_i}.$$

Equivalently, the $K^m \times L^m$ joint channel matrix $Q^{(m)}$ whose $(j_1 \cdots j_m,\, y_1 \cdots y_m)$-entry is the probability above factors as the m-fold Kronecker product

$$Q^{(m)} \;=\; Q \otimes Q \otimes \cdots \otimes Q \;=\; Q^{\otimes m}.$$

The statement is the n-ary version of Corollary 4.4.1.1 once the conditional form Theorem 4.4.1 is specialized to deterministic rates. Each per-query channel matrix is the same $K \times L$ matrix Q, so the m-fold Kronecker product is a Kronecker power. Different queries on inputs from the same block draw from the same row of Q, and the independence forced by Axiom 1 makes the joint probability the product of m row-column lookups in Q. The factor for query i is the (j_i, Y_i) entry: pick row j_i for the input block, read column Y_i for the observed output, and multiply across queries.

Two notes on scope. First, the statement is in the deterministic-rates regime of chapter 4, the regime that every worked example in Parts I and II inhabited and that Part III continues to inhabit by default. In the more general random-rates setting, the same conditional-then-unconditional structure of Theorem 4.4.1 applies, with the same Jensen-gap caveat for repeated-block queries. The deterministic case is the case we use, and keeping the statement focused on it keeps the bookkeeping clean. Second, the queries are on distinct input elements; repeated queries on the same element produce perfectly correlated outputs in any deterministic implementation, just as in chapter 4. Repeated queries on *different* elements that happen to lie in the same block $B_{j_i} = B_{j_{i'}}$ are still independent under Axiom 1: the events depend on different elements' draws, and only the row of Q they consult is shared.

Proof sketch

The proof recycles the chapter 4 argument with one cosmetic change: each row of Q has L entries instead of two. The four steps below mirror Theorem 4.4.1's proof step for step.

Step 1: reduce to a joint probability calculation. Index the queries by $i = 1, \ldots, m$. Query i inputs element $X_i \in B_{j_i}$ and observes output $Y_i \in \{1, \ldots, L\}$. The $(j_1 \cdots j_m,\, y_1 \cdots y_m)$-entry of $Q^{(m)}$ is

$$Q^{(m)}_{j_1 \cdots j_m,\, y_1 \cdots y_m} \;=\; \mathrm{P}\left(Y_1 = y_1, \ldots, Y_m = y_m | X_i \in B_{j_i} \text{ for each } i\right).$$

We aim to show this equals $\prod_{i=1}^{m} Q_{j_i, y_i}$, which is the corresponding entry of $Q^{\otimes m}$ by the definition of the Kronecker product.

Step 2: Axiom 2 makes each per-query row stable. The deterministic-rates regime means each per-block output distribution is a fixed row of Q, not a random draw from a distribution over rows. Axiom 2 (Axiom 2) guarantees that the rows of Q are jointly meaningful: the distribution of outputs given $X_i \in B_{j_i}$ is row j_i of Q, the same row regardless of which other queries are made or what their input blocks are. In the chapter 4 proof this was the role of the proposition that rates remain stable across queries. In the n-ary case the statement is identical, with rows of Q in place of the (ε, ω) pair.

Step 3: Axiom 1 factors the joint distribution. The outputs $Y_1, \ldots, Y_m$ are deterministic functions of the error events at $X_1, \ldots, X_m$. Axiom 1 (Axiom 1) asserts that these error events are mutually independent across distinct inputs, and the $L = 2$ restriction in the original Boolean statement plays no role in the independence claim itself. Independence of error events implies independence of outputs, so

$$\begin{aligned}
&\mathrm{P}\left(Y_1 = y_1, \ldots, Y_m = y_m | X_i \in B_{j_i} \text{ for each } i\right) \\
&\quad= \prod_{i=1}^{m} \mathrm{P}\left(Y_i = y_i | X_i \in B_{j_i}\right) \\
&\quad= \prod_{i=1}^{m} Q_{j_i, y_i}.
\end{aligned}$$

The first equality is independence; the second is the definition of Q_{j_i, y_i} as the probability that the channel reports output y_i given input block B_{j_i}.

Step 4: read off the matrix. The product $\prod_{i=1}^{m} Q_{j_i, y_i}$ is, by the definition of the Kronecker product, the $(j_1 \cdots j_m,\, y_1 \cdots y_m)$-entry of $Q \otimes \cdots \otimes Q = Q^{\otimes m}$. Since the identification holds for every index tuple, the joint matrix factors:

$$Q^{(m)} \;=\; Q^{\otimes m}. \quad \square$$

The conditions used. The deterministic-rates regime plus Axiom 1 plus Axiom 2 is exactly the n-ary version of what the chapter 4 proof of Theorem 4.4.1 required. Axiom 1 supplies factorization across queries; Axiom 2 supplies stability of each per-block row across queries; the deterministic-rates regime collapses the rate-conditional and unconditional statements into one. The only binary-specific step in chapter 4 was reading the two entries of each row as (ε, ω) or (τ, ω) pairs; the n-ary proof reads each row as a length-L probability vector and the rest is the same calculation.

Parameter count

Corollary 9.3.1.1 (Parameter count for joint n-ary queries)**.** *Under the hypotheses of Theorem 9.3.1, the joint channel matrix $Q^{(m)}$ of m queries on an n-ary channel of model-order K and output cardinality L is fully specified by the single $K \times L$ matrix Q, which carries $K(L-1)$ free parameters. Adding queries does not add free parameters; it adds only the specification of which input block each query draws from. The general m-query joint channel, by contrast, would have $K(L^m - 1)$ free parameters per source-block convention, growing exponentially in m.*

Proof. The Kronecker product $Q^{\otimes m}$ is determined entirely by the single matrix Q. Each query position contributes one row index j_i (the input block) and one column index y_i (the observed output), and the joint entry is the product of m entries of Q read off in the order given by these indices. The free parameters in Q are exactly $K(L-1)$, as established by the parameter count of Section 9.2 (each of the K rows is a probability distribution on L outputs, hence $L-1$ free parameters per row). Adding a query position $m+1$ multiplies the joint matrix shape from $K^m \times L^m$ to $K^{m+1} \times L^{m+1}$, but the underlying parameters do not change: the new factor in the Kronecker product is still Q. The exponential parameter cost of the unconstrained m-query channel is

$$K(L^m - 1),$$

one probability distribution on L^m joint outputs per source block, minus the normalization constraint per block. The ratio $\frac{K(L^m-1)}{K(L-1)} = \frac{L^m-1}{L-1}$ grows exponentially in m for fixed $L \geq 2$. □

This corollary is the direct generalization of Corollary 4.4.1.2. At $K = L = 2$, the parameter count $K(L-1) = 2$ recovers the familiar (ε, ω) pair of the Boolean case, and the unconstrained count $K(L^m - 1) = 2(2^m - 1)$ recovers the exponential reference point of chapter 4. The growth in m is

the same: linear under the axioms, exponential without them. The growth in L is new and benign: the unconstrained joint scales as L^m, but each row of the per-query channel scales only as L, so a 10-output channel queried 50 times is specified by $K \cdot 9$ numbers rather than $K \cdot (10^{50} - 1)$. The reduction is even more dramatic at larger L than it was at $L = 2$, which is part of why the n-ary generalization is worth stating explicitly.

Worked example: the 3-class classifier

We compute a joint probability with the 3-class classifier of Section 9.2, Example 3. The channel has $K = L = 3$ and matrix

$$Q = \begin{pmatrix} 0.9 & 0.05 & 0.05 \\ 0.05 & 0.9 & 0.05 \\ 0.05 & 0.05 & 0.9 \end{pmatrix}.$$

Row i governs inputs in block B_i (the inputs whose true class is i), column j records the probability that the classifier outputs class j, and the diagonal entries are the per-class accuracies of 90%. The matrix has $K(L-1) = 3 \cdot 2 = 6$ free parameters in general; this particular Q is constrained beyond that, with a single diagonal value and uniform off-diagonal mass, but the structure of $Q^{\otimes m}$ does not care about those extra constraints.

Consider three queries with inputs

$$(X_1, X_2, X_3) \in (B_1, B_2, B_1)$$

and observed outputs

$$(Y_1, Y_2, Y_3) = (1, 2, 1).$$

Each query is correctly classified: X_1 is a true class-1 input that the classifier reports as class 1; X_2 is a true class-2 input reported as class 2; X_3 is another true class-1 input reported as class 1. The block sequence (B_1, B_2, B_1) has B_1 appearing twice; in the random-rates setting of chapter 4 this would trigger the Jensen-gap caveat, but in the deterministic-rates regime both class-1 queries draw from the same fixed row of Q and the products work out cleanly.

Theorem 9.3.1 gives the joint probability as

$$\begin{aligned} \mathrm{P}\left(Y_1 = 1, Y_2 = 2, Y_3 = 1 | X_1 \in B_1, X_2 \in B_2, X_3 \in B_1\right) &= Q_{1,1} \cdot Q_{2,2} \cdot Q_{1,1} \\ &= 0.9 \cdot 0.9 \cdot 0.9 \\ &= 0.729. \end{aligned}$$

Three independent queries each correctly classified with probability 0.9 produces a joint correct-classification probability of 0.9^3, which is the n-ary analogue of the BSC's $(1-\varepsilon)^m$ joint success probability from chapter 4's worked example.

To exercise an off-diagonal entry, consider the same input blocks (B_1, B_2, B_1) but the output sequence $(Y_1, Y_2, Y_3) = (2, 2, 1)$ (first input misclassified as class 2, second and third correct). The joint probability is

$$\begin{aligned} Q_{1,2} \cdot Q_{2,2} \cdot Q_{1,1} &= 0.05 \cdot 0.9 \cdot 0.9 \\ &= 0.0405. \end{aligned}$$

The first factor is the probability of class-1 input being reported as class 2 (0.05, off-diagonal); the next two factors are the correct-classification probabilities for the second and third queries.

The joint matrix $Q^{(3)}$ for this classifier has shape 27×27. The unconstrained joint parameter count by the per-source-block convention is $K(L^m - 1) = 78$ for $m = 3$, but the Kronecker factorization brings it down to the $K(L-1) = 6$ that specifies Q itself. For $m = 10$ the gap is starker: $3 \cdot (59\,049 - 1) \approx 1.77 \times 10^5$ unconstrained parameters versus the same 6. The exponential cost of joint queries without independence is the chapter 4 story told with bigger matrices.

Looking ahead

Theorem 9.3.1 closes the structural loop opened in Section 9.2. The two chapter 4 axioms transfer to the n-ary setting, and the Kronecker theorem they force transfers with them. Section 9.4 turns from structure to information, generalizing the BSC capacity formula of chapter 8 to the n-ary case.

9.4 Capacity for n-ary Channels

Section 9.3 settled the structural question: an n-ary channel with $K \times L$ matrix Q has the same Kronecker factorization across queries that the Boolean case enjoyed in chapter 4. This section settles the information-theoretic question: how many bits per use does the channel convey? The formula of chapter 8 §8.2, $C = 1 - H(\varepsilon)$, is the $K = L = 2$ special case of a general definition that lifts without surprise to arbitrary K and L. The lift is mostly definitional; what changes is that the optimization underlying the definition no longer has a single closed form, and a class of structurally symmetric

channels has to be identified to recover one. The two sub-families that do admit closed forms are stated, the general case is bridged to the numerical literature, and the worked example computes capacity for a four-state strongly symmetric channel.

Recall: capacity for the BSC

Chapter 8 §8.1 defined channel capacity as the maximum mutual information achievable by choosing the input distribution freely:

$$C \;=\; \max_{P_X} I(X;Y),$$

the expression (8.5) of Section 8.1, with P_X ranging over distributions on the input alphabet and the conditional $\Pr[Y \mid X]$ fixed by the channel matrix. For the binary symmetric channel of chapter 2 §2.1, the closed form (8.6) is

$$C \;=\; 1 - H(\varepsilon) \quad \text{bits per channel use,}$$

attained at the uniform input $P_X(\top) = P_X(\bot) = 1/2$ (Proposition 8.2.1). The argument leading to that form used two facts about the BSC. First, the conditional entropy $H(Y \mid X)$ equals $H(\varepsilon)$ for every input, because the two rows of Q are permutations of each other; averaging over any input distribution gives the same $H(\varepsilon)$. Second, the output marginal entropy $H(Y)$ is maximized when the output is uniform on $\{\top, \bot\}$, which by column-symmetry of Q happens precisely at uniform input. Both facts generalize to a class of n-ary channels with the same symmetry structure; both fail for channels without it, and the general optimization has to fall back on numerical methods. The two strands organize the rest of the section.

The general n-ary case

For an n-ary channel (X, Y, Q) in the sense of Definition 9.2.1, the capacity definition copies over verbatim. An input distribution P_X is now a probability vector on the K blocks $B_1, \ldots, B_K$, with $P_X(B_i) \geq 0$ summing to one. The output distribution P_Y it induces is

$$P_Y(j) \;=\; \sum_{i=1}^{K} P_X(B_i)\, Q_{ij}, \qquad j = 1, \ldots, L,$$

a K-fold average of the rows of Q weighted by the input distribution. The mutual information (8.3) between input and output is

$$I(X;Y) \;=\; H(Y) - H(Y \mid X),$$

where $H(Y \mid X) = \sum_{i=1}^{K} P_X(B_i)\, H(Q_{i\cdot})$ is the input-averaged row entropy of Q and $H(Y)$ is the entropy of the induced output distribution P_Y. The capacity is the maximum over input distributions on the K blocks:

$$C \;=\; \max_{P_X} \big[H(Y) - H(Y \mid X)\big], \tag{9.1}$$

the same object as (8.5), now ranging over the $(K-1)$-dimensional simplex of distributions on input blocks rather than the one-dimensional interval $[0, 1]$ that parametrized the BSC input. The dimension of the optimization grew, but the object did not.

The general optimization has no closed form. The function $I(X;Y)$ viewed as a function of P_X on the input simplex is concave (a standard fact from [CT06]), so the maximum exists and is unique up to ties on the boundary, but the maximizing P_X has to satisfy a system of stationarity conditions whose solution is implicit. The standard numerical recipe is the *Blahut-Arimoto algorithm*, a fixed-point iteration over alternating updates of P_X and a posterior $\Pr[X \mid Y]$. Each iteration tightens a lower bound on $I(X;Y)$, the iterates converge to the capacity-achieving distribution under mild conditions, and the convergence rate is geometric. Cover and Thomas [CT06] chapter 10 develops the algorithm in detail and proves its convergence; the chapter 9 notebook implements it for the worked examples that close this section and for the exercises. The takeaway for the reader is that the general n-ary capacity is computable to any precision, just not via a single analytic expression. The two sub-families below recover analytic expressions by imposing enough structural symmetry on Q to collapse the maximization to a single calculation.

Symmetric and strongly symmetric channels

The BSC's two key properties were that its rows are permutations of each other (which made $H(Y \mid X)$ input-independent) and that its columns are permutations of each other (which made P_Y uniform at uniform input). Lifting each property to the n-ary case identifies a sub-family of channels for which capacity has a closed form, and combining both properties identifies a still smaller sub-family for which the closed form is especially clean.

Symmetric n-ary channel. An n-ary channel is *symmetric* if every row of Q is a permutation of every other row: there is a single multiset of probabilities $\mathbf{r} = (r_1, r_2, \ldots, r_L)$ summing to one, and each row of Q is some rearrangement of $\mathbf{r}$. Under this hypothesis, every row of Q has the same

entropy $H(\mathbf{r}) = -\sum_j r_j \log_2 r_j$, so the conditional entropy $H(Y \mid X) = \sum_i P_X(B_i)\, H(Q_{i\cdot}) = H(\mathbf{r})$ is input-independent. The capacity definition then reduces to

$$C \;=\; \max_{P_X} H(Y) \;-\; H(\mathbf{r}),$$

and the remaining maximization is over the output marginal entropy alone. The output alphabet has L values, so $H(Y) \le \log_2 L$ with equality at the uniform output distribution $P_Y(j) = 1/L$. The symmetric hypothesis does not by itself guarantee that uniform input produces uniform output: that requires the column structure of the next sub-family. The bound $C \le \log_2 L - H(\mathbf{r})$ is nevertheless tight whenever an input distribution making P_Y uniform exists, which is generic for symmetric channels (every row permutation of $\mathbf{r}$ contributes the same total mass to every output, and the uniform input distribution suffices). The closed form for a symmetric n-ary channel is therefore

$$C \;=\; \log_2 L \;-\; H(\mathbf{r}), \tag{9.2}$$

with $\mathbf{r}$ any row of Q (the choice does not matter, since all rows have the same entropy). The result is the n-ary analogue of $C = 1 - H(\varepsilon)$: the term $\log_2 L$ is the maximum possible output entropy of an L-valued channel, the term $H(\mathbf{r})$ is the irreducible per-use uncertainty from the row distribution, and the difference is the per-use information bound.

Strongly symmetric n-ary channel. A symmetric channel is *strongly symmetric* if, in addition, the columns of Q are also permutations of one another. The two hypotheses together force a very rigid structure on Q. A standard parametrization writes a single per-input error probability $p_e \in [0, 1)$ and spreads the off-diagonal mass uniformly across the other $L - 1$ outputs:

$$Q_{ij} \;=\; \begin{cases} 1 - p_e & j = i, \\ \dfrac{p_e}{L-1} & j \neq i. \end{cases}$$

The matrix is square with $K = L$, the diagonal is the common correct-transmission probability $1 - p_e$, and every off-diagonal entry is the same $p_e/(L-1)$. The BSC of chapter 2 is the $L = 2$ instance with $p_e = \varepsilon$. Each row of Q has the form $\mathbf{r} = (1 - p_e,\, p_e/(L-1), \ldots, p_e/(L-1))$, with one "correct" coordinate and $L - 1$ uniform "error" coordinates, and the row entropy is

$$H(\mathbf{r}) \;=\; H\!\left(1 - p_e,\; \frac{p_e}{L-1},\; \ldots,\; \frac{p_e}{L-1}\right).$$

Substituting into (9.2) gives the closed form that the rest of the section uses.

Proposition 9.4.1 (Strongly symmetric n-ary capacity)**.** *The capacity of a strongly symmetric n-ary channel with output alphabet size L and total error probability $p_e \in [0, 1)$ per input is*

$$C \;=\; \log_2 L \;-\; H\Big(1 - p_e,\; \tfrac{p_e}{L-1},\; \ldots,\; \tfrac{p_e}{L-1}\Big) \quad \textit{bits per channel use,} \tag{9.3}$$

attained by the uniform input distribution $P_X(B_i) = 1/L$ for every input block.

At $L = 2$ and $p_e = \varepsilon$, the row distribution $\mathbf{r} = (1 - \varepsilon, \varepsilon)$ has entropy $H(\varepsilon)$, giving $C = \log_2 2 - H(\varepsilon) = 1 - H(\varepsilon)$. The strongly-symmetric formula recovers (8.6) exactly, confirming that the n-ary capacity result subsumes the BSC capacity of chapter 8 §8.2 as the $L = 2$ specialization. The endpoints carry across too: $p_e = 0$ gives $H(\mathbf{r}) = 0$ and $C = \log_2 L$ (noiseless L-ary channel transmits $\log_2 L$ bits per use); $p_e = (L - 1)/L$ gives the uniform row $\mathbf{r} = (1/L, \ldots, 1/L)$, $H(\mathbf{r}) = \log_2 L$, and $C = 0$ (maximally noisy L-ary channel transmits nothing). The strongly-symmetric family is the cleanest generalization of the BSC: a single error parameter parametrizes the whole family, and the closed form mirrors the chapter 8 result with $\log_2 L$ replacing the ceiling of one bit.

Worked example: a 4-state strongly symmetric channel

Take $L = 4$ and $p_e = 0.1$. The channel matrix is

$$Q \;=\; \begin{pmatrix} 0.9 & 0.0333 & 0.0333 & 0.0333 \\ 0.0333 & 0.9 & 0.0333 & 0.0333 \\ 0.0333 & 0.0333 & 0.9 & 0.0333 \\ 0.0333 & 0.0333 & 0.0333 & 0.9 \end{pmatrix},$$

writing the off-diagonal entry $p_e/(L - 1) = 0.1/3 \approx 0.0333$ to four significant figures. Each row sums to $0.9 + 3 \cdot 0.0333 \approx 1$; the diagonal is the common 90% correct transmission probability; every off-diagonal entry is the same 3.33%. The channel is strongly symmetric by construction (rows and columns are both permutations of each other), so Proposition 9.4.1 applies.

The output-alphabet ceiling is

$$\log_2 L \;=\; \log_2 4 \;=\; 2 \text{ bits per use.}$$

The row entropy is

$$H(\mathbf{r}) \;=\; -0.9\log_2 0.9 \;-\; 3\cdot\tfrac{0.1}{3}\log_2\tfrac{0.1}{3}.$$

The two terms compute as follows. The diagonal contribution is

$$-0.9\log_2 0.9 \;\approx\; -0.9\cdot(-0.152) \;\approx\; 0.137 \text{ bits.}$$

The off-diagonal contribution is

$$-3\cdot 0.0333\cdot\log_2 0.0333 \;\approx\; -3\cdot 0.0333\cdot(-4.908) \;\approx\; 0.490 \text{ bits.}$$

Adding gives

$$H(\mathbf{r}) \;\approx\; 0.137 + 0.490 \;=\; 0.627 \text{ bits.}$$

The capacity is the difference:

$$C \;=\; \log_2 4 - H(\mathbf{r}) \;\approx\; 2 - 0.627 \;\approx\; 1.373 \text{ bits per channel use.}$$

A four-symbol channel with a 90% per-symbol correct-transmission probability conveys about 1.37 bits per use, well below the noiseless ceiling of 2 bits and well above the BSC's 0.531 bits per use at the same per-symbol error rate of 0.1. The gap to the ceiling shrinks as L grows for a fixed p_e, because the off-diagonal mass spreads more thinly across more outputs and the row entropy grows sublinearly in L at fixed p_e. The chapter 9 notebook verifies the numerical value $C \approx 1.373$ against a Blahut-Arimoto run on the same Q, confirming both the strongly-symmetric closed form and the general numerical recipe.

Where capacity goes next

The capacity expression (9.1) closes the information-theoretic loop for chapter 9. Every n-ary channel of the form fixed by Definition 9.2.1 has a well-defined per-use information bound; the strongly symmetric sub-family has the closed form (9.3) that Proposition 9.4.1 records; and the general case yields to Blahut-Arimoto when a closed form is not available. Section 9.5 previews how concrete sketches instantiate the chapter 9 framework: a CountMin sketch is an n-ary channel with $L = \infty$ and a one-sided error profile; a HyperLogLog is an n-ary channel with $L = \infty$ and a relative-error profile; a MinHash is an n-ary channel with $L = [0, 1]$ output. Each sketch carries its own Q, its own capacity, and its own specialization of the chapter 9 machinery. Chapter 10 instantiates the framework in detail, building three sketches from scratch with explicit error bounds and composition rules.

9.5 Bridge to Sketches

A *sketch* is a function from a sequence of items to a compact summary, with bounded reconstruction error. The input is a stream or multiset $x_1, x_2, \ldots, x_n$ drawn from some universe, the output is a small data structure (often of size sublinear in n), and a query routine reconstructs an answer to a fixed family of questions about the input from the summary alone. The reconstruction is not exact: a frequency query returns an estimate $\hat{f}$ rather than the true count f, a cardinality query returns an estimate $\hat{n}$ rather than the true distinct-item count n, a similarity query returns an estimate of Jaccard overlap rather than the true overlap. What makes a sketch useful, and what distinguishes it from a lossy compression scheme, is that the error is bounded by an analytic expression in the summary size and the input distribution: the user trades a known accuracy for a known space budget, picks an operating point on the curve, and the data structure delivers it. The three sketches that chapter **??** builds (CountMin, HyperLogLog, MinHash) are the canonical instances of this trade-off for frequency, cardinality, and similarity respectively.

Each of the three sketches is a Bernoulli[T] instance in the sense of this chapter. CountMin is an approximate counter: the type is $T = \mathbb{N}$, the channel matrix Q has an input row for every true count and an output column for every reported estimate, and the sketch's one-sided overestimation guarantee corresponds to a Q with all probability mass at or above the diagonal. HyperLogLog is an approximate cardinality estimator: the type is again $T = \mathbb{N}$, but the channel matrix Q has its mass concentrated within a multiplicative band around the diagonal (relative error rather than the additive-or-one-sided error of CountMin). MinHash is an approximate similarity estimator: the type is $T = [0, 1]$ (Jaccard similarities live in the unit interval), and the channel matrix Q becomes a continuous kernel rather than a discrete matrix, with rows indexed by true similarities and an output distribution concentrated within a band of width $O(1/\sqrt{r})$ around the diagonal at sketch size r. The unifying lens of Definition 9.2.1 is that all three sketches share the same structural skeleton: an input type, an output type, a row-stochastic kernel Q specifying the conditional output distribution given each input, and a per-query independence property inherited from Axioms 1 and 2 of chapter 4 (Axiom 1, Axiom 2). The differences among the three sketches are differences in the structure of Q, not differences in framework.

Chapter **??** builds these three sketches from scratch. Each one comes with a Bernoulli[T] type identifying its input and output spaces, an error bound naming the constants in the analytic relationship between summary

size and reconstruction accuracy, and a composition rule describing how the sketch behaves under the natural operations of its setting (merging two sketches built on disjoint streams, querying a sketch built by aggregating heterogeneous sources, combining a frequency sketch with a cardinality sketch in a single estimation pipeline). The chapter 9 n-ary-channel framework is the unifying lens that ties the three sketches together: each one's error bound is a statement about the entries of Q, each one's composition rule is a statement about how Q-matrices combine, and each one's capacity (the bits per query the summary can convey about the input) falls out of Proposition 9.4.1 or its specialization to the sketch's particular symmetry structure. Chapter 9 prepares the reader for chapter 10 not by anticipating particular formulas but by fixing the vocabulary in which those formulas are expressed: input type, output type, channel matrix, factorization across queries, capacity per use. The three sketches that follow are the first non-Boolean instances of Bernoulli[T] that the book examines in detail.

Bibliographic Notes

The approximate-maps manuscript. The reformulation of the noisy-bit atom Bernoulli[bool] into a general type-lifted noisy value Bernoulli[T], together with the n-ary channel matrix Q that organizes its per-block conditional output distributions, follows Towell [Tow26g]. That manuscript is the primary source for the chapter: the move from the predicate framing of Part II to the T-valued framing of Part III in Section 9.1, the formal $K \times L$ row-stochastic channel of Definition 9.2.1, the n-ary Kronecker factorization of Theorem 9.3.1 and its parameter-count corollary Corollary 9.3.1.1, and the capacity formulation of Section 9.4 are all taken from it. The chapter's contribution beyond the manuscript is the pedagogical scaffolding: the explicit recall of Section 4.4 and Section 8.2 as the model-order-2 special cases that the chapter generalizes, the 3-class classifier and integer-valued counter as concrete instances of Bernoulli[T] for non-Boolean T, and the bridge-to-sketches discussion of Section 9.5 that prepares chapter **??** by exhibiting CountMin, HyperLogLog, and MinHash as Bernoulli[T] instances of the same shape.

N-ary channels and the Blahut-Arimoto algorithm. The information-theoretic background for the n-ary discrete memoryless channel is standard textbook material, and the depth reference for readers who want more than the working summary of Section 9.4 is Cover and Thomas [CT06]. Their

chapter 7 develops the discrete memoryless channel, defines capacity as the supremum of mutual information over input distributions on a finite alphabet, proves the noisy-channel coding theorem in both directions, and treats the symmetric and weakly symmetric special cases whose closed-form capacities the chapter quotes. Their chapter 10 covers the Blahut-Arimoto algorithm, the iterative coordinate ascent on the KL-divergence form of mutual information that the chapter's notebook implements, with proofs of monotone improvement and convergence to the capacity-achieving input distribution. The Blahut-Arimoto recipe is the default numerical tool for any channel that does not admit a closed-form capacity; it is what makes the general n-ary capacity of Proposition 9.4.1 a computable quantity rather than a definition.

Shannon's original framework. The channel-capacity vocabulary that chapter 8 §8.2 introduces for the binary symmetric channel and that this chapter generalizes to arbitrary $K \times L$ matrices originates in Shannon [Sha48]. The 1948 paper introduces the entropy $\mathcal{H}(()X)$ recalled in Section 8.1, the channel capacity $C = \max_{P_X} I(X;Y)$ that Section 9.4 specializes to the n-ary discrete case, and the source-coding theorem that connects the two through the operational characterization of capacity as the largest rate at which information can be transmitted with vanishing error. Shannon's framing of the channel as a probabilistic mapping from input alphabet to output alphabet, conditional on the input distribution and producing an output distribution governed by a fixed transition kernel, is exactly the framing that Definition 9.2.1 adopts: the chapter's Q is Shannon's transition matrix, the chapter's K and L are the cardinalities of his input and output alphabets, and the chapter's capacity is his capacity. The 1948 paper remains worth reading on its own terms, both for the conceptual clarity of its development and for the perspective it gives on how thoroughly Shannon's vocabulary has shaped every subsequent treatment of noisy information transmission, this chapter included.

Forward to chapter 10. The sketch literature that Section 9.5 previews (CountMin, HyperLogLog, MinHash, and the streaming-algorithms tradition that produced them) is taken up in chapter **??** §10.6, where the original papers are cited alongside the survey treatments that have become the standard secondary sources. The strategy of chapter 9 is deliberately to fix the framework first and defer the named constructions: the three sketches are presented in chapter 10 as the first non-Boolean instances of Bernoulli[T]

that exercise the full $K \times L$ channel machinery developed here, and the bibliographic discussion belongs with the constructions rather than with the abstract framework that subsumes them.

Exercises

1. $\star$ **Write a 3x3 channel matrix.** For a 3-class classifier (cats, dogs, birds) with 90% per-class accuracy and the off-diagonal 10% split uniformly across the two competing labels, write the 3×3 row-stochastic matrix Q in the sense of Definition 9.2.1. Label rows by input block, columns by output value, and verify each row sums to one.

2. $\star$ **Parameter count.** For a $K \times L$ channel matrix with $K = 5$ and $L = 3$, count the free parameters using Corollary 9.3.1.1. Contrast with the $K \cdot L = 15$ entry count, and confirm the $K = L = 2$ specialization recovers the (ε, ω) pair of Corollary 4.4.1.2.

3. $\star$ **BSC limit of the n-ary capacity formula.** The strongly-symmetric formula (9.3) reads $C = \log_2 L - H(1 - p_e, p_e/(L-1), \ldots, p_e/(L-1))$. Substitute $L = 2$ and $p_e = \varepsilon$ to recover $C = 1 - H(\varepsilon)$, matching Proposition 8.2.1. Show the row-distribution substitution and confirm the $\log_2 2 = 1$ leading term.

4. $\star\star$ **Notebook-tied n-ary capacity.** Open the chapter 9 companion notebook `python/notebooks/ch09-approximate-functions.ipynb`.

   (a) Construct the strongly symmetric channel matrix Q of Proposition 9.4.1 with $K = L = 5$ and $p_e = 0.1$; verify row-stochasticity.

   (b) Compute the closed-form capacity from Equation (9.3), to four decimal places.

   (c) Run the Blahut-Arimoto fixed-point iteration on Q and confirm convergence to (b) within 10^{-6}. Report the capacity-achieving input distribution (uniform, $P_X(B_i) = 1/5$, by the strongly-symmetric structure).

5. $\star\star$ **Joint probability via the n-ary Kronecker theorem.** Take the $K = 3$ classifier matrix Q of Item 1, with blocks $(B_1, B_2, B_3) = (\text{cats}, \text{dogs}, \text{birds})$ and output alphabet $\{1, 2, 3\}$.

   (a) For input sequence (B_1, B_2, B_3, B_1) and output sequence $(1, 2, 1, 3)$, write the joint probability as the product $\prod_{t=1}^{4} Q_{i_t, j_t}$ produced by Theorem 9.3.1.

(b) Substitute the entries from Item 1 and compute the numerical value.

(c) In one sentence, identify which two chapter 4 axioms make the factorization a plain product over time steps.

6. ⋆⋆ **Composition of two n-ary channels.** Let Q_1 be a $K_1 \times L_1$ row-stochastic matrix (input blocks $\to$ intermediate alphabet) and Q_2 be an $L_1 \times L_2$ row-stochastic matrix (intermediate $\to$ output).

(a) Show that the composed channel is the $K_1 \times L_2$ matrix product $Q = Q_1 Q_2$, with $Q_{i,k} = \sum_j (Q_1)_{i,j}(Q_2)_{j,k}$ giving the probability that input block i produces output k after both stages.

(b) Verify Q is row-stochastic: show $\sum_k Q_{i,k} = 1$ for every i using row-stochasticity of Q_1 and Q_2 separately.

(c) In one sentence, state why this rule makes a multi-stage approximate pipeline (e.g., coarse classifier feeding fine classifier) itself an n-ary channel in the sense of Definition 9.2.1.

7. ⋆⋆ **Sketch as an n-ary channel.** The CountMin sketch (previewed in Section 9.5 and developed in **??**) estimates the frequency of each stream item and reports a non-negative integer.

(a) Identify the input partition: two items share a block when their true stream frequencies agree. State the model order K in terms of the multiset of true frequencies.

(b) Identify the output cardinality: the estimator returns a non-negative integer, so $L = \infty$.

(c) Describe qualitatively what the rows of the channel matrix look like: the row indexed by true frequency n is supported on $\{n, n+1, n+2, \ldots\}$ (CountMin over-counts but never under-counts). In one sentence, relate this one-sided support to the FNR-zero shape from Theorem 8.3.1.

8. ⋆⋆ **Asymmetric 3-class with a rare class.** Take the channel matrix Q of Item 1 (90% accuracy, uniform off-diagonal) on a workload with one rare class: $P_X(B_3) = 0.01$, $P_X(B_1) = P_X(B_2) = 0.495$.

(a) Compute the PPV of Definition 7.4.1 for the rare class: the probability that the true label is B_3 given the classifier reports B_3. Use Bayes' rule with the workload prior and the column of Q indexed by output 3.

(b) Compare to the per-class accuracy 0.90. The PPV falls far below the diagonal: false positives from the two common classes swamp true positives from the rare class.

(c) In one sentence, connect this to Section 7.4: per-row accuracies of Q are workload-independent, PPV depends on Q and the workload prior jointly, and Bayes' rule converts between the two.

9. ⋆⋆⋆ **Transfer of Axiom 2 from binary to n-ary.** Axiom 2 (Axiom 2 of Section 4.3) states that the per-block rates ε and ω of a Boolean Bernoulli predicate are conditionally independent given the objective function.

(a) Restate the axiom in the n-ary setting: the K row distributions $Q_{1,\cdot}, \ldots, Q_{K,\cdot}$ of Q are mutually conditionally independent given the objective function. Show that the Boolean form is the $K = 2$ instance, with the two rows parametrized by (ω) and (ε) respectively.

(b) Argue informally that the chapter 4 proof transfers verbatim: every step appeals to independence at the per-block level and not to the binary nature of the alphabet, so the same argument carries when 2 is replaced by K throughout.

(c) In one sentence, identify which fact of the n-ary Kronecker theorem (Theorem 9.3.1) depends on this lifted Axiom 2 rather than on Axiom 1 alone.

10. ⋆⋆⋆ **Capacity bounded by output entropy.** For any n-ary channel (X, Y, Q) in the sense of Definition 9.2.1, show that $C \leq \log_2 L$ where $L = |Y|$.

(a) Start from $C = \max_{P_X}[H(Y) - H(Y \mid X)]$. Use $H(Y \mid X) \geq 0$ to obtain $C \leq \max_{P_X} H(Y)$.

(b) Apply the maximum-entropy bound on a finite alphabet: $H(Y) \leq \log_2 L$, with equality at uniform $P_Y(j) = 1/L$. Combine with (a).

(c) Characterize when equality $C = \log_2 L$ is achieved: simultaneously, $H(Y \mid X) = 0$ (the channel is deterministic given input) and some input distribution makes P_Y uniform. Identify this regime: the channel is a noise-free function from input blocks to output values, with at least one input block per output value.

11. ⋆⋆⋆ **T-valued Bloom-like construction.** The standard Bloom filter of Section 5.2 stores bits at hashed positions. A T-valued generalization stores small numerals from $T = \{1, \ldots, L\}$ at hashed positions and reports the modal numeral at the k cells hashed by the query key.

    (a) Sketch the construction: m cells holding values in T, k independent hash functions, and a majority-vote query. At insertion, (x, v) writes v to the k cells hashed by x (collisions resolved by overwrite or per-cell rule).

    (b) Cast the result as a $K \times L$ channel matrix in the sense of Definition 9.2.1: input blocks indexed by true numeral v, output values by reported numerals, and $Q_{v,w}$ the probability the query reports w given true v.

    (c) Derive $Q_{v,v}$ and the off-diagonal $Q_{v,w}$ for $w \neq v$ in terms of the load factor n/m and alphabet size L. In one sentence, identify the FNR-zero analog (diagonal is 1 when no collisions occur at the k hashed positions) and connect to the FNR-zero Bloom filter of Section 5.2 as the $L = 2$ specialization.

Chapter 10

Sketches and Estimators

After this chapter you will be able to:

- Construct CountMin, HyperLogLog, and MinHash sketches from scratch, and identify each as a Bernoulli[T] instance under the chapter 9 framework.
- State the error bound and confidence parameter for each sketch, and derive the space/accuracy trade-off.
- Compose sketches via the type-driven algebra (the chapter 13 unification).

10.1 Sketches as Bernoulli of T

What is a sketch?

A *sketch* is a function from a stream or multiset of items, drawn from some universe U, to a compact summary, together with a query routine that reconstructs an answer to a fixed family of questions about the input from the summary alone. The reconstruction is approximate: the query returns an estimate $\hat{\theta}$ of some quantity $\theta(x_1, \ldots, x_n)$ of the stream, and the error $\hat{\theta} - \theta$ is bounded by a known analytic expression in the summary size and the input. What makes a sketch a sketch, and not merely a compressed representation, is that the size of the summary grows sublinearly (often poly-logarithmically) in the stream length n, the per-item update cost is constant or near-constant, and the error bound is a clean function of the design parameters. The reader already saw the preview at Section 9.5, which framed the three sketches of this chapter as the canonical instances of

the trade-off for frequency, cardinality, and similarity queries respectively; this section makes that framing the operating definition for the rest of Part III.

Three canonical sketches, three types

The three sketches built in this chapter differ chiefly in what they estimate, which is to say in their output type.

CountMin estimates per-item frequency. Given a stream $x_1, x_2, \ldots, x_n$ and a query item $a \in U$, CountMin returns an estimate $\hat{f}_a$ of the true count $f_a = |\{i : x_i = a\}|$. The output type is $T = \mathbb{N}$, and the error is *one-sided*: the construction guarantees $\hat{f}_a \geq f_a$ deterministically, with the over-estimate bounded in probability by a quantity that scales with the stream length and the sketch width. This one-sided structure is the direct analogue of the Bloom filter's one-sided membership report (Chapter 5): the construction sets counters monotonically, no operation ever decreases a counter, and the worst case is that some other item's hashes coincide with a's and inflate the reported count. False under-estimation, the analogue of a Bloom false negative, is impossible by construction.

HyperLogLog estimates cardinality. Given a stream that may contain duplicates, HyperLogLog returns an estimate $\hat{n}$ of the number of distinct items $n = |\{x_1, \ldots, x_n\}|$. The output type is again $T = \mathbb{N}$, but the error structure is qualitatively different: the estimate is approximate in both directions, with a relative error of order $1/\sqrt{m}$ in the register count m. The headline result is that $m = 2048$ registers of six bits apiece (a kilobyte and a half) suffice to estimate the cardinality of a stream of billions of distinct items to about two and a half percent relative accuracy. The space cost is logarithmic in the cardinality being estimated rather than linear in it, which is the qualitative win that makes HyperLogLog the standard sketch for distinct counts in production analytics systems.

MinHash estimates Jaccard similarity. Given two sets $A, B \subseteq U$, MinHash returns an estimate $\hat{J}(A, B)$ of the Jaccard similarity $J(A, B) = |A \cap B|/|A \cup B|$. The output type is $T = [0, 1]$ (the unit interval), reflecting the fact that the quantity being estimated is a probability rather than a count. The error is symmetric: $\hat{J} - J$ is centered at zero with variance $J(1 - J)/k$ in the signature length k, and the deviations on either side of the true value have the same distribution. The sketch is built from the

simple observation that the minimum of a random hash function over a set is equally likely to come from any element, so the probability that two sets share their minimum is exactly the fraction of elements they share. The full construction is the obvious generalization: run the trick k times with independent hashes, count the matches, divide by k.

Why the chapter 9 framework unifies them

The three sketches above estimate three different things over three different domains, and the conventional treatment introduces each one as its own structure with its own analysis. The chapter 9 framework gives all three the same shape. Each sketch is a function

$$s \;:\; U^* \longrightarrow T,$$

where U^* is the space of finite streams over the universe U and T is the output type. The query algorithm is the projection from a stream to its sketch value, and the error structure is summarized by a row-stochastic channel matrix Q in the sense of Definition 9.2.1: rows of Q are indexed by the true value θ of the queried quantity, columns by the reported value $\hat{\theta}$, and the entry $Q_{\theta,\hat{\theta}}$ is the conditional probability that the sketch reports $\hat{\theta}$ when the input has true value θ. The matrix Q is typically very large or even continuous (HyperLogLog's θ ranges over the positive integers; MinHash's ranges over the unit interval), but in every case it has structure: CountMin's Q is upper triangular (one-sided), HyperLogLog's concentrates within a multiplicative band around the diagonal (relative error), and MinHash's is approximately binomial along each row (signature counts).

The structural payoff is composition. The independence properties that drive the per-sketch analyses (pairwise-independent hashes for CountMin, independent hash registers for HyperLogLog, independent permutation samples for MinHash) all amount to the same statement in the channel-matrix language: the joint behaviour of multiple sketch queries, or of a single sketch built by aggregating multiple streams, factors as a Kronecker product of the per-query or per-substream matrices. Theorem 9.3.1 is the chapter 9 result that makes this factorization a theorem rather than a per-sketch lemma. Merging two CountMin sketches, merging two HyperLogLog sketches, querying a MinHash signature at k positions are all instances of the same Kronecker structure, with the per-instance content amounting to filling in the correct entries of the elementary matrices.

Signpost

The next three sections build the three sketches from scratch, each one following the same template: problem setup, construction with pseudocode, error-bound theorem, identification of the Bernoulli[T] type, and merge or composition rule. Section 10.2 builds CountMin; Section 10.3 builds HyperLogLog; Section 10.4 builds MinHash and its locality-sensitive-hashing extension. Section 10.5 closes the chapter by handing the three constructions off to chapter 13, where the algebra of Bernoulli[T] composition is axiomatized and the sketches of this chapter become its first non-trivial instances.

10.2 CountMin

Problem setup

A data stream presents items $x_1, x_2, \ldots, x_n$ from a universe U in some order, with possible repetition. For an item $a \in U$, write $f_a = |\{i : x_i = a\}|$ for its true frequency in the stream, and write $F = \sum_b f_b = n$ for the total stream length. The frequency-estimation query asks, for a given a, to report an estimate $\hat{f}_a$ of f_a. Storing the exact frequencies costs space proportional to the number of distinct items, which in the heavy-hitter regime that motivates streaming sketches can run to the billions. CountMin gives up exactness and returns a small data structure of size polylogarithmic in the stream length, with a query that runs in time independent of F and an error bound on $\hat{f}_a - f_a$ controllable by two design parameters.

The construction

Fix a width w and a depth d, both positive integers, and pick d hash functions

$$h_1, h_2, \ldots, h_d \ : \ U \longrightarrow \{1, 2, \ldots, w\}.$$

The h_i are drawn from a pairwise-independent family: for any two distinct items $a, b \in U$ and any two cells $u, v \in \{1, \ldots, w\}$, $\mathrm{P}\left(h_i(a) = u \text{ and } h_i(b) = v\right) = 1/w^2$. Pairwise independence is enough for the analysis below, and it is much cheaper to construct than the full independence assumed for the Bloom filter in Definition 5.2.1.

The data structure is a $d \times w$ integer table C with rows indexed by $i \in \{1, \ldots, d\}$ and columns indexed by $j \in \{1, \ldots, w\}$, initialized to all zeros.

Definition 10.2.1 (CountMin sketch)**.** *The* CountMin sketch *of parameters* (w, d) *is the data structure* $(C, h_1, \ldots, h_d)$ *together with the two operations:*

- UPDATE(a)*: for* $i = 1, \ldots, d$*, set* $C[i][h_i(a)] \leftarrow C[i][h_i(a)] + 1$*.*
- QUERY(a)*: return* $\hat{f}_a = \min_i C[i][h_i(a)]$*.*

After processing a stream by calling UPDATE *once per item, the query returns the estimated frequency of any item* a*.*

Algorithm 3 states the two operations in pseudocode.

Algorithm 3: CountMin update and query

```
1  procedure UPDATE(a)
2  |  for i ← 1 to d do
3  |  |  C[i][h_i(a)] ← C[i][h_i(a)] + 1;
4  |  end
5  function QUERY(a)
6  |  m ← ∞;
7  |  for i ← 1 to d do
8  |  |  if C[i][h_i(a)] < m then m ← C[i][h_i(a)];
9  |  end
10 |  return m;
```

Both operations run in $O(d)$ time per call, independent of the stream length F and the universe size $|U|$. The data structure uses $d \cdot w$ counters of storage, also independent of F. The minimum across rows in QUERY is the central trick: any single row i records a count $C[i][h_i(a)]$ that is at least f_a (one increment per occurrence of a lands in that cell), plus the contributions of any other items whose hash h_i collides with a's. Taking the minimum across d independent rows attenuates the collision noise: a row only inflates the estimate if some heavy collider hashes to the same cell, and the rows are independent enough that this happens in all d rows only with small probability.

Why the estimate never under-shoots

For every row i, the cell $C[i][h_i(a)]$ accumulates a count of f_a from a's own updates, plus a non-negative count from any other item $b \neq a$ whose hash

$h_i(b)$ equals $h_i(a)$. Both contributions are non-negative, so $C[i][h_i(a)] \geq f_a$ for every i, and therefore

$$\hat{f}_a \;=\; \min_i C[i][h_i(a)] \;\geq\; f_a.$$

The estimate is always at least the true count. This is the deterministic counterpart of the Bloom filter's deterministic no-false-negatives property (Definition 5.2.1): the only failure mode is over-estimation, and the error is one-sided. The analysis below bounds the size of the over-estimate in probability.

The error bound

Theorem 10.2.1 (CountMin bound)**.** *Fix $\epsilon, \delta \in (0,1)$, and run CountMin with width $w = \lceil e/\epsilon \rceil$ and depth $d = \lceil \ln(1/\delta) \rceil$. For any item $a \in U$, the estimate $\hat{f}_a$ returned by* QUERY(a) *satisfies*

$$f_a \;\leq\; \hat{f}_a \;\leq\; f_a + \epsilon F$$

with probability at least $1 - \delta$, where $F = \sum_b f_b$ is the total stream length.

Proof. Fix the query item a and a single row $i \in \{1, \ldots, d\}$. Decompose the cell $C[i][h_i(a)]$ at the end of the stream into the contribution from a itself and the contribution from collisions:

$$C[i][h_i(a)] \;=\; f_a \;+\; \sum_{b \neq a} f_b \cdot \mathbf{1}[h_i(b) = h_i(a)],$$

where $\mathbf{1}[\cdot]$ is the indicator. Write $X_i = \sum_{b \neq a} f_b \cdot \mathbf{1}[h_i(b) = h_i(a)]$ for the row-i collision mass, so that $C[i][h_i(a)] = f_a + X_i$ and $X_i \geq 0$.

By pairwise independence of h_i, the marginal probability that any specific $b \neq a$ collides with a under h_i is $\mathrm{P}\left(h_i(b) = h_i(a)\right) = 1/w$. Linearity of expectation gives

$$\mathbb{E}[X_i] \;=\; \sum_{b \neq a} f_b \cdot \mathrm{P}\left(h_i(b) = h_i(a)\right) \;=\; \frac{F - f_a}{w} \;\leq\; \frac{F}{w}.$$

Apply Markov's inequality to the non-negative random variable X_i with threshold ϵF:

$$\mathrm{P}\left(X_i \geq \epsilon F\right) \;\leq\; \frac{\mathbb{E}[X_i]}{\epsilon F} \;\leq\; \frac{1}{\epsilon w}.$$

With $w = \lceil e/\epsilon \rceil \geq e/\epsilon$, this bound becomes

$$\mathrm{P}\left(X_i \geq \epsilon F\right) \;\leq\; \frac{1}{e}.$$

The events $\{X_i \geq \epsilon F\}$ for $i = 1, \ldots, d$ depend on independently drawn hash functions $h_1, \ldots, h_d$, so they are mutually independent. The query reports the minimum across rows, and $\hat{f}_a - f_a > \epsilon F$ exactly when *every* row over-shoots:

$$\mathrm{P}\left(\hat{f}_a > f_a + \epsilon F\right) \;=\; \mathrm{P}\left(X_i \geq \epsilon F \text{ for all } i\right) \;\leq\; (1/e)^d.$$

With $d = \lceil \ln(1/\delta) \rceil \geq \ln(1/\delta)$, $(1/e)^d \leq \delta$, which is the claimed bound. The lower bound $\hat{f}_a \geq f_a$ is the deterministic argument of the previous subsection. □

Two design points are worth flagging. First, the bound is additive in F, not in f_a: heavy hitters with f_a comparable to F are estimated with small *relative* error, while light items with $f_a \ll \epsilon F$ may be reported as ϵF and the relative error is essentially uninformative. CountMin is the right sketch when the question of interest is which items are heavy, not what the counts of the light items are. Second, the space cost is

$$d \cdot w \;=\; \lceil \ln(1/\delta) \rceil \cdot \lceil e/\epsilon \rceil \;=\; O\big((1/\epsilon)\ln(1/\delta)\big)$$

counters, which is logarithmic in the failure probability and inverse in the additive error parameter. The original analysis is due to Cormode and Muthukrishnan [CM05]; the proof above is the standard Markov-plus-union-bound presentation.

The Bernoulli[T] type

CountMin instantiates the framework of Chapter 9 with output type $T = \mathbb{N}$. The query is a function from streams to non-negative integers, and its error behaviour is summarized by a channel matrix Q in the sense of Definition 9.2.1: rows are indexed by the true count f_a and columns by the reported estimate $\hat{f}_a$, with entry

$$Q_{f_a, \hat{f}_a} \;=\; \mathrm{P}\left(\textsc{Query}(a) = \hat{f}_a | \text{true count is } f_a\right).$$

The deterministic lower bound $\hat{f}_a \geq f_a$ has a clean expression in Q: every row places all its mass on columns at or above the diagonal, so Q is upper triangular. This is the analogue, for a count-valued output, of the upper-triangular channel matrix associated with the Bloom filter as a one-sided approximate set: the deterministic lower bound on the estimate corresponds structurally to the deterministic "no false negatives" guarantee of the membership report. The probabilistic upper bound from Theorem 10.2.1 fills

in the row-wise shape: each row's mass is concentrated near the diagonal, with at most a δ fraction of probability sitting more than ϵF columns to the right of f_a. The matrix is large (rows and columns indexed by all of $\mathbb{N}$, in principle), but its structure is summarized by the two parameters (ϵ, δ).

Composition: merging two CountMin sketches

A common operational pattern is that two CountMin sketches C_1 and C_2 are built independently on two disjoint substreams of a single larger stream, and the analysis wants a sketch of the combined stream. If both sketches use the same parameters (w, d) and the same hash functions $h_1, \ldots, h_d$, the merge is entry-wise addition:

$$(C_1 \oplus C_2)[i][j] \;=\; C_1[i][j] + C_2[i][j] \quad \text{for all } i \in \{1, \ldots, d\},\ j \in \{1, \ldots, w\}.$$

The merged table is exactly the CountMin sketch that would have been built by running UPDATE on the concatenation of the two substreams, because UPDATE only increments counters and addition is commutative. QUERY on the merged sketch returns $\min_i (C_1 + C_2)[i][h_i(a)]$, which is the estimate of the total count $f_a^{(1)} + f_a^{(2)}$ in the combined stream. The same (ϵ, δ) bound from Theorem 10.2.1 applies, with the new F being the sum of the two substream lengths.

In the language of Chapter 9, this merge is a Bernoulli[$\mathbb{N}$] composition. The channel matrix of the merged sketch is the channel matrix that Theorem 9.3.1 would assign to the joint query of the two substream sketches under the natural product of integer counts. Crucially the merge does not double the failure probability: the two sketches share their hashes, so the row-collision events on the merged sketch are the same events as on each constituent, not the union of two separate event sets. Sharing the hash family is the discipline that lets independently built sketches add up to one correct sketch of the union.

A worked example

Take $w = 10$ and $d = 5$, a small but illustrative configuration. The corresponding error parameters from Theorem 10.2.1 are $\epsilon \le e/w \approx 0.272$ (the additive bound is at most $0.272 \cdot F$) and $\delta \le e^{-d} \approx 0.00674$ (the failure probability is at most 0.67%).

Consider a stream of $F = 1000$ items drawn from a universe of 50 distinct items, with frequency profile

$$f_{a_1} = 400,\ f_{a_2} = 200,\ f_{a_3} = 100,\ f_{a_4} = \cdots = f_{a_{20}} = 10,\ f_{a_{21}} = \cdots = f_{a_{50}} = 4,$$

which sums to $400 + 200 + 100 + 17 \cdot 10 + 30 \cdot 4 = 990$, and pad with 10 further occurrences of a_1 to reach exactly $F = 1000$ (so $f_{a_1} = 410$). Query for the heavy hitter a_1 with $f_{a_1} = 410$. The expected per-row over-estimate is

$$\mathbb{E}[X_i] \;=\; (F - f_{a_1})/w \;=\; 590/10 \;=\; 59,$$

so each row reports a value near $410 + 59 = 469$ in expectation, and the minimum across $d = 5$ rows is pulled below this mean by the order statistic. A typical realization (verified in the chapter notebook with seed 0) returns $\hat{f}_{a_1} = 432$, an over-estimate of 22, well inside the headline bound $\epsilon F = 0.272 \cdot 1000 = 272$. Now query for a light item a_{30} with $f_{a_{30}} = 4$. The expected per-row over-estimate is $(1000-4)/10 = 99.6$, so even the minimum across five rows will likely report something on the order of 50 to 90. The estimate is correctly bounded above by $f_{a_{30}} + \epsilon F = 4 + 272 = 276$, but it is essentially uninformative about the true count of 4: the additive error term is twenty times the true frequency. The headline bound holds, but it holds in a regime where light items are washed out by the collision mass of heavier ones. This is exactly the heavy-hitter design point of CountMin: heavy items are estimated tightly relative to their own count, and light items are reported only up to the additive ϵF slack.

Looking ahead

CountMin estimates the frequency of a specific item. The next section, Section 10.3, turns to a complementary streaming question: how many *distinct* items have been seen, regardless of which ones. The answer is not a per-item count but a single integer, and the sketch that estimates it works on a completely different principle, the leading-zero count of a random hash, with a space cost that is logarithmic in the cardinality being estimated rather than logarithmic in the failure probability.

10.3 HyperLogLog

Problem setup

The streaming question of this section is complementary to the frequency-estimation question of Section 10.2. Given a stream $x_1, x_2, \ldots, x_N$ of items from a universe U, possibly with repetition, estimate the cardinality

$$n \;=\; |\{x_1, x_2, \ldots, x_N\}|$$

of the set of distinct items seen, ignoring multiplicities. Exact counting requires space that grows with n: any deterministic algorithm that reports the exact cardinality of an arbitrary stream must, in essence, remember which distinct items it has already encountered, and a standard communication-complexity argument establishes that this is $\Theta(n)$ bits in the worst case. The HyperLogLog sketch gives up exactness and returns an estimate $\hat{n}$ with relative error on the order of $1/\sqrt{m}$, using m small-integer registers and a single hash evaluation per stream item. The resulting space cost, for fixed target accuracy, is *doubly logarithmic* in n: a few kilobytes of state suffices to estimate cardinalities into the billions.

The leading-zero trick

Let $h : U \to [0, 1]$ be a uniformly random hash, and think of $h(a)$ as an infinite binary fraction $0.b_1 b_2 b_3 \cdots$. Define $\rho(x)$ to be the position of the first 1 bit in this expansion: $\rho(0.001\ldots) = 3$, $\rho(0.1\ldots) = 1$, and so on. Equivalently, $\rho(x)$ counts the number of leading zeros and adds one.

For a uniformly distributed $h(a)$ the bit b_k is an independent fair coin, so

$$\mathrm{P}\left(\rho(h(a)) \geq k\right) \;=\; \mathrm{P}\left(b_1 = b_2 = \cdots = b_{k-1} = 0\right) \;=\; 2^{-(k-1)},$$

and the indicator that $\rho(h(a)) \geq k$ is exactly the event of seeing $k-1$ leading zeros. The expected value of $\rho(h(a))$ over a single draw is therefore $\sum_{k \geq 1} \mathrm{P}\left(\rho \geq k\right) = \sum_{k \geq 0} 2^{-k} = 2$, but the tail is heavy: the probability of seeing at least k leading zeros is 2^{-k}, so the maximum of $\rho(h(a_1)), \ldots, \rho(h(a_n))$ across n independent items grows as $\log_2 n$. Concretely, in n independent draws of a Geometric$(1/2)$ variable, the maximum is approximately $\log_2 n$ with standard deviation $O(1)$: each unit increase in the threshold halves the probability of exceeding it, so the maximum sits at the value where the expected count of exceedances drops from above one to below one.

The trick of HyperLogLog is to invert this relationship. Track only the running maximum of $\rho(h(a))$ over the items seen, and read off an estimate of $\log_2 n$ from it. The data the algorithm needs to maintain is a single small integer (the current maximum ρ), not the items themselves.

A single-register estimator

The crudest version of the algorithm is the obvious one. Maintain a single register R, initialized to zero. On each stream item a, compute $\rho(h(a))$ and set $R \leftarrow \max(R, \rho(h(a)))$. At query time, report

$$\hat{n} \;=\; 2^R.$$

This single-register estimator captures the right scaling: as the true cardinality n grows, R grows as $\log_2 n$, and 2^R grows as n. But the multiplicative variance is enormous. The maximum of n independent geometric variables has standard deviation on the order of one bit, so the estimate 2^R has multiplicative standard deviation on the order of $2^{O(1)}$, that is, a constant factor of roughly two in either direction. An estimate that is off by a factor of two is useful for an order-of-magnitude check and useless for anything quantitative. The fix is averaging: run many independent copies of the algorithm and combine them in a way that pulls the estimate toward its mean.

Multiple registers (the construction)

Running m truly independent hash functions and tracking m independent registers would solve the variance problem at m-fold cost in hash evaluations. HyperLogLog avoids that overhead by reusing a single hash via *stochastic averaging*: each item is routed to one of m buckets by the first $\log_2 m$ bits of its hash, and only the remaining bits feed the leading-zero count for that bucket. The m registers then play the role of independent observations of the single-register estimator, each on a stream of expected length n/m, without ever evaluating a second hash.

Definition 10.3.1 (HyperLogLog sketch)**.** *Fix a register count $m = 2^p$ for some integer $p \geq 4$, and a hash $h : U \to \{0,1\}^{32}$ (or 64; the bit width sets the maximum representable cardinality). The* HyperLogLog sketch *is a register array $M[0..m-1]$ of small non-negative integers, initialized to zero, together with the operations:*

- Update(a)*: compute $h(a)$; split it into the first p bits j (the bucket index) and the remaining bits w; set $M[j] \leftarrow \max(M[j], \rho(w))$.*
- Estimate()*: return*

$$\hat{n} \;=\; \alpha_m \cdot \frac{m^2}{\sum_{j=0}^{m-1} 2^{-M[j]}},$$

  where α_m is the bias-correction constant given below.

The constants are $\alpha_{16} = 0.673$, $\alpha_{32} = 0.697$, $\alpha_{64} = 0.709$, and $\alpha_m = 0.7213/(1 + 1.079/m)$ for $m \geq 128$.

Algorithm 4 states the two operations in pseudocode.

Algorithm 4: HyperLogLog update and estimate

```
procedure UPDATE(a)
    x ← h(a);
    j ← first p bits of x;
    w ← remaining bits of x;
    M[j] ← max(M[j], ρ(w));
function ESTIMATE()
    Z ← Σ_{j=0}^{m−1} 2^{−M[j]};
    return α_m · m^2 / Z;
```

Three pieces of the estimator deserve comment. The sum $Z = \sum_j 2^{-M[j]}$ is the *harmonic mean* (up to a factor of m) of the per-bucket estimates $2^{M[j]}$. Harmonic averaging suppresses the contribution of an outlier register that, by bad luck, accumulated an unusually long run of leading zeros and would otherwise pull the arithmetic mean of the per-bucket estimates upward by a constant multiplicative factor. The factor m^2 converts an estimate of n/m per bucket into an estimate of n total, and the bias-correction α_m removes a small constant-factor bias introduced by the choice of harmonic mean and the discrete geometric distribution of ρ. The asymptotic form $\alpha_\infty = 0.7213\ldots$ comes from an integral that Flajolet et al. [Fla+07] compute in their Theorem 1; the small-m corrections are tabulated values that fit the same integral evaluated at finite m.

The space cost is m registers, each large enough to hold ρ of the bit-string after the prefix. For a 32-bit hash and $p = 8$ ($m = 256$), each $M[j]$ takes values in $\{0, 1, \ldots, 24\}$ and fits in 5 bits. The whole sketch is $256 \cdot 5 = 1280$ bits, or 160 bytes, regardless of how many items the stream contains.

The error bound

The bias correction is calibrated to make $\mathbb{E}[\hat{n}] = n$ to first order in $1/m$. The fluctuations of $\hat{n}$ around n are governed by the central limit theorem applied to the registers.

Proposition 10.3.1 (HyperLogLog standard error)**.** *For a HyperLogLog sketch with $m \geq 128$ registers built from a uniformly random hash, the relative standard error of the cardinality estimate $\hat{n}$ satisfies*

$$\frac{\sigma(\hat{n})}{n} \approx \frac{1.04}{\sqrt{m}}$$

in the regime where n is large compared to m. With $m = 2048$ (an 11-bit bucket prefix), the relative standard error is approximately 2.3%.

The constant 1.04 is the asymptotic value of the per-register standard-deviation factor computed by Flajolet et al. [Fla+07] in their main analysis; the proof is an exact generating-function calculation that we will not reproduce here. The dependence on $1/\sqrt{m}$ matches the central-limit intuition: m buckets give m approximately independent observations of n/m, and the relative error of their averaged estimator scales as $1/\sqrt{m}$ regardless of n. Doubling the relative accuracy quadruples the register count; this is the space-accuracy duality of the sketch.

Two regime boundaries are worth noting. When n is small relative to m (so that many registers are still zero), the estimator under-counts and the algorithm switches to a linear-counting correction: $\hat{n} = m \ln(m/V)$, where V is the count of zero registers. When n approaches the maximum representable cardinality (set by the hash bit width), a saturation correction applies. Both corrections are routine and are documented in the original paper. The bulk of the operating range is governed by Proposition 10.3.1.

The Bernoulli$[T]$ type

HyperLogLog instantiates the framework of Chapter 9 with output type $T = \mathbb{N}$, the same as CountMin, but with a fundamentally different channel structure. The query is a function from streams to non-negative integers, and its error behaviour is summarized by a channel matrix Q in the sense of Definition 9.2.1: rows are indexed by the true cardinality n and columns by the reported estimate $\hat{n}$, with entry

$$Q_{n,\hat{n}} \;=\; \mathrm{P}\left(\textsc{Estimate}() = \hat{n} \middle| \text{true cardinality is } n\right).$$

Unlike CountMin, the matrix is *not* triangular. HyperLogLog errs in both directions: an unlucky sequence of hash bits can produce a register that overestimates $\log_2 n$ in some buckets while others underestimate, and the harmonic-mean combination can shake out either above or below n. The shape of each row is approximately log-normal in $\hat{n}/n$: by Proposition 10.3.1, the multiplicative error has constant relative standard deviation $1.04/\sqrt{m}$ across the operating range, and the asymptotic analysis of Flajolet et al. [Fla+07] shows the limiting distribution of $\log \hat{n} - \log n$ is approximately normal with that standard deviation. Each row of Q therefore concentrates on a roughly geometric interval around n, symmetric on a multiplicative scale rather than an additive one. The contrast with CountMin's one-sided

additive error is the main qualitative distinction between the two channel matrices.

Merging two HyperLogLog sketches

Two HyperLogLog sketches $M^{(1)}$ and $M^{(2)}$ built on the same hash function and the same register count m on two streams S_1 and S_2 admit a particularly clean merge:

$$(M^{(1)} \sqcup M^{(2)})[j] \;=\; \max\big(M^{(1)}[j], M^{(2)}[j]\big) \quad \text{for all } j \in \{0, 1, \ldots, m-1\}.$$

The merged register array is exactly the array that would have resulted from running UPDATE on the union stream $S_1 \cup S_2$ (treating each stream as a multiset and taking the union of their distinct items). The argument is one line: for each bucket j, the bucket-j register on the union stream is the maximum of $\rho(w)$ over all items in $S_1 \cup S_2$ whose hash prefix is j, which is the maximum of the per-stream bucket-j maxima. The merge is idempotent in duplicate items: an item appearing in both streams contributes the same $\rho(w)$ to its bucket in both sketches, and max of two equal values is that value.

In the language of Chapter 9, this merge is a Bernoulli[$\mathbb{N}$] composition. The channel matrix of the merged sketch is the channel matrix that Theorem 9.3.1 would assign to the joint query of the two stream sketches under the cardinality-of-union operation on multisets. As with CountMin, sharing the hash function across the two sketches is the discipline that lets independently constructed sketches add up to a correct sketch of the union: were the two sketches built on independent hashes, the merge would be a coarse upper bound on the union cardinality with no clean error analysis.

A worked example

Take $m = 256$ (so $p = 8$) and simulate a stream of $n = 10000$ distinct items, each hashed with a 32-bit hash. The expected relative standard error from Proposition 10.3.1 is $1.04/\sqrt{256} = 0.065$, so a single run typically returns an estimate within ± 650 of 10000.

After processing the stream, each register $M[j]$ records the maximum ρ of the $\approx 10000/256 \approx 39$ items that landed in bucket j. The expected maximum of 39 independent draws of ρ is approximately $\log_2 39 \approx 5.3$, so most of the registers contain values in $\{4, 5, 6, 7\}$, with a small minority at 3 or 8 from the tails of the maximum-of-geometrics distribution. A representative

register array (verified in the chapter notebook with seed 0) is concentrated in this range, with distribution

$$M \approx (5,5,6,4,7,5,6,5,4,6,5,7,5,6,5,4,\ldots)$$

for the first sixteen entries, and a full sum $\sum_j 2^{-M[j]}$ on the order of $256/39 \approx 6.5$. The estimate is

$$\hat{n} = \alpha_{256} \cdot \frac{256^2}{Z} \approx 0.7182 \cdot \frac{65536}{6.5} \approx 7240,$$

which is in the wrong tail by about 2.8 standard deviations on a single seed; averaging over a hundred independent seeds returns an empirical mean estimate of 9970 and an empirical relative standard deviation of 0.063, both within the expected range from Proposition 10.3.1. The example reinforces the point that the headline bound is a standard error, not a deterministic bound: any single sketch can land a few standard deviations away from n, and the value of the sketch is that the spread shrinks as $1/\sqrt{m}$ with the register count.

Looking ahead

HyperLogLog estimates the cardinality of a single multiset. Section 10.4 turns to a similarity question on two sets: estimate the Jaccard similarity $J(A,B) = |A \cap B|/|A \cup B|$ from compact signatures of A and B, without recomputing either intersection or union explicitly. The construction is again hash-based, again maintains a small array per set, and again admits a clean Bernoulli[T] analysis, this time with $T = [0,1]$ and a symmetric two-sided error bound from Hoeffding's inequality.

10.4 MinHash and LSH

Problem setup

The similarity question on two sets is the third streaming primitive of this chapter. Given two sets $A, B \subseteq U$, the *Jaccard similarity* is

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|},$$

a value in $[0,1]$ that is 0 when the sets are disjoint and 1 when they coincide. Exact computation of $J(A,B)$ requires a pass through both sets and total

work $O(|A|+|B|)$; storing the sets themselves costs space linear in their sizes, which is a deal-breaker when the inputs are documents over a vocabulary of millions of shingles and the comparison must be made across a corpus of millions of documents. The MinHash sketch (Broder [Bro97]) reduces each set to a signature of k small integers, with k a tunable accuracy parameter, and estimates $J(A, B)$ from the two signatures alone in time $O(k)$. Pairwise similarity queries that would otherwise be quadratic in the corpus size become quadratic in the signature length, and the further extension to Locality-Sensitive Hashing (LSH) reduces the corpus-wide comparison to a sub-linear approximate nearest-neighbour search.

The MinHash trick

Fix a hash $h : U \to [0, 1]$ that is uniform across the universe and injective on $A \cup B$ (no hash collisions among items of interest). For a non-empty set $S \subseteq U$, write $\min h(S) = \min_{a \in S} h(a)$ for the smallest hash value attained on S, and write $\arg\min h(S) = \arg\min_{a \in S} h(a)$ for the item of S at which that minimum is attained. The central observation is the following.

Proposition 10.4.1 (MinHash collision probability)**.** *Let $A, B \subseteq U$ be non-empty finite sets with $A \cup B$ injectively hashed by h. Then*

$$\mathrm{P}\left(\min h(A) = \min h(B)\right) \;=\; J(A, B).$$

Proof. The minimum hash over $A \cup B$ is attained at a unique element $a^* \in A \cup B$ (by injectivity). The event $\{\min h(A) = \min h(B)\}$ holds exactly when $a^* \in A \cap B$: if a^* lies in both A and B, then a^* is the argmin of h restricted to A and also the argmin of h restricted to B, and the two minima coincide; if a^* lies in only one of the two sets, the other set's minimum is strictly larger. By the uniformity of h across $A \cup B$, every element of $A \cup B$ is equally likely to be the argmin, so

$$\mathrm{P}\left(a^* \in A \cap B\right) \;=\; \frac{|A \cap B|}{|A \cup B|} \;=\; J(A, B),$$

which is the claimed identity. □

The proof is short because the identity is essentially combinatorial. What the proof says, restated, is that the minimum hash over a union is itself a uniform random sample from the union, and the probability this sample comes from a particular subset is the subset's proportion. A single hash h converts the set-similarity question into a Bernoulli coin flip with bias $J(A, B)$. Estimating J is then a matter of repeating the experiment.

The construction

Pick k hash functions $h_1, h_2, \ldots, h_k$, each independent and each uniform across U. For each set $A \subseteq U$, compute the *MinHash signature*

$$\sigma(A) \;=\; \big(\sigma_1(A), \sigma_2(A), \ldots, \sigma_k(A)\big) \;\in\; [0,1]^k,$$

where $\sigma_i(A) = \min_{a \in A} h_i(a)$. The signature is a length-k vector that summarizes A: it discards the items of A themselves and retains only the k minima.

Definition 10.4.1 (MinHash sketch)**.** *The* MinHash sketch *of parameter k for a set $A \subseteq U$ is the signature $\sigma(A) = (\sigma_1(A), \ldots, \sigma_k(A))$ where each $\sigma_i(A) = \min_{a \in A} h_i(a)$ for independent uniform hashes h_i. The* MinHash estimator *of the Jaccard similarity $J(A, B)$ is*

$$\hat{J}(A,B) \;=\; \frac{1}{k}\,\big|\{i \in \{1,\ldots,k\} : \sigma_i(A) = \sigma_i(B)\}\big|,$$

the fraction of signature components on which the two sets agree.

Algorithm 5 states the signature construction and the pairwise estimator in pseudocode.

Algorithm 5: MinHash signature and Jaccard estimate

```
function Signature(A)
    for i ← 1 to k do
        σ_i ← min_{a∈A} h_i(a);
    end
    return (σ_1, σ_2, ..., σ_k);
function Estimate(σ(A), σ(B))
    c ← 0;
    for i ← 1 to k do
        if σ_i(A) = σ_i(B) then c ← c + 1;
    end
    return c/k;
```

The signature for A takes $O(k \cdot |A|)$ time to compute (one pass per hash function) and $O(k)$ space to store. The pairwise estimator takes $O(k)$ time, independent of the original set sizes. The signature is also *streaming*: items can be inserted into A one at a time, with each insertion updating each σ_i in $O(1)$ via $\sigma_i \leftarrow \min(\sigma_i, h_i(a))$. Deletion is not supported, which is the price of the min-based summary; an item that once attained the minimum cannot be removed without rescanning the set.

Error bound

Each component of the signature gives a Bernoulli trial for the Jaccard event. Let $Y_i = \mathbf{1}[\sigma_i(A) = \sigma_i(B)]$ for $i = 1, \dots, k$. By Proposition 10.4.1, each Y_i is Bernoulli with parameter $J = J(A, B)$, and the Y_i are independent because the hash functions h_i are independent. The estimator is

$$\hat{J} \;=\; \frac{1}{k} \sum_{i=1}^{k} Y_i,$$

the sample mean of k independent Bernoulli(J) variables.

Proposition 10.4.2 (MinHash error bound)**.** *For a MinHash sketch of parameter k built from independent uniform hashes, the estimator $\hat{J}$ of the Jaccard similarity $J = J(A, B)$ satisfies*

$$\mathbb{E}[\hat{J}] \;=\; J, \qquad \mathrm{Var}[\hat{J}] \;=\; \frac{J(1-J)}{k},$$

and, by Hoeffding's inequality, for every $\epsilon > 0$,

$$\mathrm{P}\left(|\hat{J} - J| \;\geq\; \epsilon\right) \;\leq\; 2\exp\!\left(-2k\epsilon^2\right).$$

Proof. Linearity of expectation gives $\mathbb{E}[\hat{J}] = \mathbb{E}[Y_1] = J$, and independence of the Y_i gives $\mathrm{Var}[\hat{J}] = \mathrm{Var}[Y_1]/k = J(1-J)/k$. Hoeffding's inequality for a sum of k independent random variables each bounded in $[0, 1]$ states that the deviation of the sample mean from its expectation exceeds ϵ in absolute value with probability at most $2\exp(-2k\epsilon^2)$, which is the claimed tail bound applied to $\hat{J}$. □

The bound is symmetric in over-estimation and under-estimation: the MinHash estimator is two-sided in the same sense that the HyperLogLog estimator is two-sided (Proposition 10.3.1), and unlike CountMin, which is one-sided. Inverting the Hoeffding bound for a target failure probability δ gives an additive accuracy $\epsilon = \sqrt{\ln(2/\delta)/(2k)}$, so $\hat{J}$ falls within $\pm\epsilon$ of J with probability at least $1 - \delta$ when $k \geq \ln(2/\delta)/(2\epsilon^2)$. The signature length grows linearly in $\ln(1/\delta)$ and quadratically in $1/\epsilon$.

The Bernoulli[T] type

MinHash instantiates the framework of Chapter 9 with output type $T = [0, 1]$, the unit interval of similarity values. The query is a function from pairs

of signatures to similarity estimates, and its error behaviour is summarized by a channel matrix Q in the sense of Definition 9.2.1: rows are indexed by the true Jaccard $J \in [0,1]$ and columns by the reported estimate $\hat{J} \in \{0, 1/k, 2/k, \ldots, 1\}$, with entry

$$Q_{J,\hat{J}} \;=\; \mathrm{P}\left(\textsc{Estimate}(\sigma(A), \sigma(B)) = \hat{J} | J(A,B) = J\right).$$

Because $k\hat{J}$ is the sum of k independent Bernoulli(J) variables, each row of Q is a Binomial(k, J) distribution on the $k+1$ possible values of $\hat{J}$:

$$Q_{J,j/k} \;=\; \binom{k}{j} J^j (1-J)^{k-j}, \qquad j = 0, 1, \ldots, k.$$

The matrix is row-stochastic by construction and is symmetric in the following sense: the row conditional on true similarity J is the mirror image, under the substitution $\hat{J} \mapsto 1 - \hat{J}$, of the row conditional on true similarity $1 - J$. The contrast with CountMin's upper-triangular Q and with HyperLogLog's approximately log-normal rows is that MinHash's Q has a closed binomial form. The two-sided concentration of Proposition 10.4.2 is the chapter 9 channel-matrix view of the Binomial distribution's concentration around its mean.

LSH for approximate nearest neighbour

Estimating $J(A,B)$ for a single pair (A,B) from signatures of length k takes $O(k)$ time. Doing the same for every pair in a corpus of N documents takes $O(kN^2)$ time, which is intractable once N is in the millions. Locality-Sensitive Hashing (Indyk and Motwani [IM98]) is a structural device that uses the MinHash signatures themselves as bucket keys, so that pairs with high Jaccard similarity collide into the same bucket with high probability and pairs with low similarity do not. The corpus-wide similarity search reduces to a bucket lookup: for a query document Q, hash its signature into the same scheme and inspect only the documents sharing a bucket with Q.

The construction partitions the length-k MinHash signature into b *bands* of r *rows* each, with $k = br$. For each band, the r signature entries are concatenated into a single key and hashed into a bucket. Two sets A and B become *candidate neighbours* if they share a bucket in at least one of the b bands. The probability that a single band agrees on all r rows is J^r (each row agrees independently with probability J by Proposition 10.4.1), so the probability that at least one band agrees is

$$p(J) \;=\; 1 - (1 - J^r)^b.$$

The shape of p as a function of J is the celebrated S-curve of LSH: small J gives $p(J) \approx bJ^r$, which is close to zero when J^r is small; large J gives $p(J)$ close to one. The transition is sharp, with the steep portion concentrated around the threshold $J^* \approx (1/b)^{1/r}$. Choosing (b, r) tunes the threshold: a larger r steepens the curve and pushes the threshold right (more selective), and a larger b raises the entire curve (more permissive).

For approximate nearest neighbour with target similarity J^*, the operational recipe is to pick (b, r) so that $p(J)$ is small below J^* and close to one above J^*, then build the b bucket tables once over the corpus. A query hashes its signature into each of the b band tables in $O(b)$ time and returns the union of the matching buckets as the candidate set; an exact pairwise Jaccard (or a MinHash estimate) on the small candidate set then filters down to the true neighbours. The expected candidate-set size is governed by the corpus's similarity distribution and the chosen (b, r), and is typically a small constant times the number of true neighbours when the parameters are tuned to the target J^*. The resulting query time is sub-linear in N: the bucket lookup is $O(b)$, and the candidate-filtering work is governed by the actual neighbour density, not the corpus size.

A worked example

Take two sets A and B with $|A| = |B| = 100$ and overlap $|A \cap B| = 30$. The union has $|A \cup B| = |A| + |B| - |A \cap B| = 170$, and the true Jaccard similarity is

$$J \;=\; \frac{30}{170} \;\approx\; 0.176.$$

Build MinHash signatures of length $k = 200$ from independent uniform hashes. By Proposition 10.4.2, the estimator $\hat{J}$ has expectation $J \approx 0.176$ and variance

$$\mathrm{Var}[\hat{J}] \;=\; \frac{J(1-J)}{k} \;=\; \frac{0.176 \cdot 0.824}{200} \;\approx\; 7.26 \cdot 10^{-4},$$

so the standard error of $\hat{J}$ is approximately 0.027. A single sketch typically returns $\hat{J}$ within ± 0.027 of 0.176, and the Hoeffding bound of Proposition 10.4.2 gives, for $\epsilon = 0.05$,

$$\mathrm{P}\left(|\hat{J} - J| \geq 0.05\right) \;\leq\; 2\exp(-2 \cdot 200 \cdot 0.0025) \;=\; 2e^{-1} \;\approx\; 0.736,$$

which is a loose bound at this scale because Hoeffding ignores the $J(1-J)$ factor that pulls the variance well below $1/4$ when J is small. The standard-error estimate above, derived from the exact binomial variance, is the practically relevant accuracy figure. A representative realization (verified in the

chapter notebook with seed 0) returns $\hat{J} = 0.185$, a deviation of 0.009 from the true value, well inside one standard error. To halve the standard error to 0.013, quadruple the signature length to $k = 800$; this is the same $1/\sqrt{k}$ space-accuracy scaling as HyperLogLog and the same logarithmic-in-failure-probability dependence as CountMin, adapted to the symmetric two-sided Bernoulli setting.

Looking ahead

The MinHash construction closes the catalogue of three sketches that this chapter builds from scratch. The next section, Section 10.5, recaps the common pattern (construction, Bernoulli[T] type, error bound, composition) across CountMin, HyperLogLog, and MinHash, and hands off to chapter 13's type algebra, where the composition rules informally invoked throughout this chapter are axiomatized and proved in their full generality.

10.5 Bridge

The three sketches built in this chapter share one skeleton and differ only in the structure of the output type and the shape of the channel matrix that ties true input to reported estimate. CountMin (Section 10.2) is the frequency sketch: its output type is $T = \mathbb{N}$, its query is one-sided (the estimate $\hat{f}_a$ never underestimates the true count f_a), and its channel matrix Q places all probability mass at or above the diagonal, with row f_a supported on $\{f_a, f_a + 1, \ldots\}$ and tail probability controlled by $\Pr[\hat{f}_a \geq f_a + \epsilon F] \leq \delta$ from Theorem 10.2.1. HyperLogLog (Section 10.3) is the cardinality sketch: its output type is again $T = \mathbb{N}$, but its error is two-sided and approximately log-normal, with the standard relative error $1.04/\sqrt{m}$ of Proposition 10.3.1 fixing the multiplicative width of the channel-matrix band around the diagonal. MinHash (Section 10.4) is the similarity sketch: its output type is the unit interval $T = [0, 1]$, its per-coordinate agreement is a Bernoulli draw with success probability $J(A, B)$, and the Hoeffding bound of Proposition 10.4.2 controls the width of the channel kernel around the diagonal at signature length k. Three sketches, three Bernoulli[T] types, one common pattern: a construction, an error analysis bounding Q row-by-row, a composition rule for combining sketches built on different streams.

The composition rules of the three sketches share a deeper structure that the present chapter notes in passing but does not axiomatize. Merging two CountMin tables is entry-wise integer addition on the $d \times w$ counter array; merging two HyperLogLog register banks is entry-wise maximum on

the m-register array; merging two MinHash signatures (in the natural sense of forming the signature of the union set from the two component signatures) is entry-wise minimum on the k-coordinate array. Each operation is associative, commutative, and has an identity (the all-zero table, the all-zero register bank, the all-∞ signature respectively). Each operation respects the chapter 9 channel structure: the merge of two Q-matrices is itself a Q-matrix for the merged input, with the rows of the merged matrix computable from the rows of the components by a known formula. Chapter 13 treats this pattern systematically. It introduces the composition algebra of Bernoulli$[T \times U]$ (product types, for sketches over pairs), Bernoulli$[T + U]$ (sum types, for sketches over tagged unions of input domains), and Bernoulli$[L\,T]$ (list types, for sketches whose output is a sequence rather than a single value), and shows that the three sketches of the present chapter are instances of those constructors. The chapter 13 algebra is what makes the family of sketches combinatorial rather than ad hoc: once the type constructors are fixed, new sketches compose by structure.

Chapter 11 turns from streaming and estimation to the algorithmic side of the same picture: randomized algorithms whose answer is approximate in a one-sided sense, with Miller-Rabin primality testing as the canonical worked example. The move is from the data-structure setting of the present chapter (a summary built from a stream, queried many times) to the algorithm-execution setting (a Monte Carlo procedure run with fresh randomness, possibly many times, against a single fixed input). The channel framework of chapter 9 carries over without modification: a randomized one-sided algorithm is a Bernoulli[{accept, reject}] instance with a known false-positive or false-negative rate, and amplification by repetition is composition of the same channel matrix with itself. Chapter 12 lifts the framework from approximate functions to approximate relations, which is the necessary generalization for join queries and multi-set membership tests. Chapter 13 unifies the three threads (the chapter 10 sketches, the chapter 11 randomized algorithms, the chapter 12 relations) under a single algebraic vocabulary, and is the end of Part III.

Bibliographic Notes

The Count-Min sketch. The construction of Section 10.2 and the additive error bound of Theorem 10.2.1 are due to Cormode and Muthukrishnan [CM05]. Their paper introduces the $d \times w$ counter table with d pairwise-independent row hashes, the update rule that increments one counter per

row, the query rule that returns the minimum across rows, and the analysis that gives the $\hat{f}_a \leq f_a + \epsilon F$ guarantee with probability at least $1 - \delta$ when $w = \lceil e/\epsilon \rceil$ and $d = \lceil \log(1/\delta) \rceil$. The proof in their Theorem 1 is the Markov-plus-union-bound argument reproduced in Section 10.2; the chapter adds nothing to the proof itself. What the chapter adds is the reading of Count-Min as a Bernoulli[$\mathbb{N}$] instance: the channel matrix Q has rows indexed by true counts and columns by estimated counts, the upper-triangular shape encodes the one-sided $\hat{f}_a \geq f_a$ guarantee, and the merge operation (entry-wise addition of two tables with matched parameters) is a composition under the chapter 9 framework rather than a CountMin-specific recipe. Cormode and Muthukrishnan also catalog applications (heavy hitters, quantile estimation, inner-product estimation) that the chapter does not cover; their paper is the right entry point for readers who want the full operational repertoire.

The HyperLogLog cardinality estimator. The HyperLogLog construction of Section 10.3 and the $1.04/\sqrt{m}$ standard relative error of Proposition 10.3.1 are due to Flajolet et al. [Fla+07], building on the earlier LogLog and probabilistic-counting work of Flajolet and Martin. Their Theorem 1 is an exact generating-function calculation that gives both the bias correction α_m and the asymptotic standard deviation factor; the chapter quotes the constants but does not reproduce the derivation. The limiting log-normal shape of the estimation error, which underwrites the symmetric-on-a-log-scale rows of the Q matrix in Section 10.3, is also their result, as are the small-m corrections and the tabulated α_m values that production HyperLogLog implementations use. The merge operation (entry-wise maximum on the register array) is the natural composition under the underlying max-of-leading-zeros structure and is treated as a chapter 9 composition rather than as a HyperLogLog-specific operation; the equivalence between the merged sketch and a HyperLogLog of the union stream is left as an exercise at the end of the chapter.

MinHash and Jaccard similarity. The MinHash sketch of Section 10.4 is due to Broder [Bro97], whose original setting was near-duplicate detection across a large web corpus. The key probabilistic identity, $\Pr[\min h(A) = \min h(B)] = J(A, B)$ for a uniform hash h, is from that paper; the k-hash signature, the estimator $\hat{J} = (1/k)|\{i : \sigma_i(A) = \sigma_i(B)\}|$, and the Hoeffding concentration bound of Proposition 10.4.2 are the standard treatment that follows from the identity. The reading of MinHash as a Bernoulli[$[0, 1]$] instance, with rows of Q indexed by true Jaccard similarity and columns

by the empirical fraction of agreeing signature positions, is again the chapter 9 framework rather than the original formulation, but the underlying Binomial-in-k structure of $\hat{J}$ is already implicit in Broder's analysis.

Locality-Sensitive Hashing. The LSH construction of Section 10.4 is due to Indyk and Motwani [IM98], who developed it for approximate nearest neighbor search in high dimensions and proved that the curse of dimensionality can be partially evaded by hashing to buckets in which nearby points collide with high probability and far points collide with low probability. The MinHash-plus-band structure of the chapter is the specialization of their general LSH framework to the Jaccard metric: MinHash signatures provide the locality-sensitive hash family, the band-and-row parameter sweep controls the precision/recall trade-off, and the bucketing turns an $O(N^2)$ pairwise comparison into an $O(N)$-amortized indexed lookup. Indyk and Motwani's paper covers LSH families for Hamming distance and ℓ_p norms in addition to the Jaccard case the chapter develops; their framework is the right entry point for readers who want the general theory rather than its MinHash specialization.

Exercises

1. ⋆ **CountMin parameter selection.** Fix the CountMin guarantee at $\epsilon = 0.01$ and $\delta = 0.001$ (at most 1% over-estimation, with failure probability at most one in a thousand). From Theorem 10.2.1, compute the sketch dimensions $w = \lceil e/\epsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$. Report both integers, the total cell count $w \cdot d$, and a one-sentence interpretation of which parameter buys accuracy and which buys confidence.

2. ⋆ **HyperLogLog accuracy.** The standard relative error of Proposition 10.3.1 is $1.04/\sqrt{m}$, where m is the register count. Tabulate the relative error for $m \in \{64, 256, 1024\}$ as a percentage to two significant figures. In one sentence, identify the quartering-of-error-per-quadrupling-of-m scaling and connect to the $\sqrt{m}$ space-accuracy trade-off discussed in Section 10.3.

3. ⋆ **MinHash variance.** For two sets with true Jaccard similarity $J = 0.5$ and a MinHash signature of length $k = 100$, compute the variance $\mathrm{Var}\left(\hat{J}\right) = J(1-J)/k$ of Proposition 10.4.2, the standard deviation $\sqrt{\mathrm{Var}\left(\hat{J}\right)}$, and a one-standard-deviation confidence interval

$\hat{J} \pm \sigma$ for the estimate. Report all three to three decimal places.

4. ⋆⋆ **Notebook-tied sketch comparison.** Open the chapter 10 companion notebook `python/notebooks/ch10-sketches-estimators.ipynb`.

   (a) Generate a synthetic stream of $N = 10\,000$ items drawn from a Zipfian distribution over 1 000 distinct keys. Run CountMin with $(\epsilon, \delta) = (0.01, 0.001)$ and record the empirical maximum over-estimation $\max_a(\hat{f}_a - f_a)$ across 20 trials.

   (b) Run HyperLogLog with $m = 256$ on the same stream's distinct-element subset; report the empirical relative error against the true distinct count, and compare with the theoretical $1.04/\sqrt{256} \approx 6.5\%$.

   (c) Split the stream into two halves A, B. Compute MinHash signatures of length $k = 200$ for the two halves' item sets and report $\hat{J}(A, B)$ against the true Jaccard. Compare empirical with theoretical error from Proposition 10.4.2.

5. ⋆⋆ **CountMin bound proof.** Reproduce the proof of Theorem 10.2.1 step by step.

   (a) Compute the per-row expected over-estimation $\mathbb{E}[\hat{f}_a^{(i)} - f_a] = (N - f_a)/w \le N/w$ by linearity of expectation over collisions.

   (b) Apply Markov's inequality with $\epsilon = e/w$ to obtain a per-row failure probability at most $1/e$.

   (c) Take the minimum over d rows; the joint failure probability is at most $(1/e)^d = e^{-d}$. Set $e^{-d} \le \delta$ and solve for $d \ge \ln(1/\delta)$.

6. ⋆⋆ **HyperLogLog at low cardinality.** Argue informally why the harmonic-mean estimator of Definition 10.3.1 under-estimates the true cardinality n when n is small relative to m (say $n < 5m/2$). The issue is that most registers remain at their initial value zero, so the ρ-statistic concentrates on small values and the geometric extrapolation breaks down. State the standard fix: switch to the linear-counting estimator $\hat{n} = m \ln(m/V)$, where V is the count of zero registers, and identify why this estimator is exact in the limit $n \to 0$ (under the Poissonization argument sketched in Section 10.3).

7. ⋆⋆ **MinHash for documents.** A document with $n = 1000$ distinct words is represented by a MinHash signature of length $k = 200$, each signature entry being a 64-bit hash.

(a) Compute the signature storage cost in bits and bytes.

(b) Compute the full-set storage cost assuming each word averages 8 bytes (so $n \cdot 8 \cdot 8 = 64\,000$ bits) and state the compression ratio.

(c) In one sentence, identify what is lost in the compression (the full set), what is preserved (Jaccard similarity to other signed sets, within the Proposition 10.4.2 bound), and why this trade-off is the right one for near-duplicate detection at scale.

8. ⋆⋆ **Channel matrix for CountMin.** Cast the CountMin sketch of Definition 10.2.1 as an n-ary channel in the sense of Definition 9.2.1.

(a) Identify the input partition: blocks are indexed by the true count $f_a \in \{0, 1, 2, \ldots\}$, so the model order is $K = \infty$ in principle and $K = N + 1$ for a stream of length N.

(b) Identify the output alphabet: $L = \infty$, the non-negative integers reported by query.

(c) Describe the structure of Q. Show that $Q_{n,\hat{n}} = 0$ for $\hat{n} < n$ (CountMin never under-counts, so the matrix is upper triangular with respect to the natural ordering), and that $\sum_{\hat{n} \geq n} Q_{n,\hat{n}} = 1$ (row-stochasticity). In one sentence, connect the upper-triangular shape to the FNR-zero structure of the Bloom filter of Definition 5.2.1, of which CountMin is the multi-set analog.

9. ⋆⋆⋆ **CountMin expected overshoot, exact form.** Theorem 10.2.1 gives a probabilistic upper bound on the over-estimation $\hat{f}_a - f_a$. Here, derive the exact *expectation* in a single row i, not merely a bound on its tail.

(a) Write $\hat{f}_a^{(i)} - f_a = \sum_{b \neq a} f_b \cdot \mathbb{I}\{h_i(b) = h_i(a)\}$, where h_i is the row's hash function. Apply linearity of expectation.

(b) Use the assumption that h_i is fully random (or pairwise independent), so $\Pr[h_i(b) = h_i(a)] = 1/w$ for any fixed $b \neq a$.

(c) Conclude $\mathbb{E}[\hat{f}_a^{(i)} - f_a] = (N - f_a)/w$, where $N = \sum_b f_b$ is the total stream length. Contrast with the probabilistic bound of Theorem 10.2.1: in expectation the over-estimation is exactly $(N - f_a)/w \leq N/w = \epsilon N/e$; the factor of e in the bound is the Markov slack.

10. ⋆ ⋆ ⋆ **HyperLogLog merge.** Section 10.3 states that two HyperLogLog sketches over the same hash function and register count m

merge by taking the entry-wise maximum of their register arrays. Show that this merge is exactly a HyperLogLog sketch of the union stream.

(a) Recall the register update rule: register j stores the maximum ρ-value (leading-zero count plus one) observed over all stream items hashing to bucket j.

(b) Let S_A, S_B be the register arrays of the two sketches, and $S_{A\cup B}$ the register array of a single sketch built on $A \cup B$. Argue that $S_{A\cup B}(j) = \max(S_A(j), S_B(j))$ entry-wise: each bucket's stored value is a maximum, and the maximum over $A \cup B$ equals the maximum over A combined with the maximum over B.

(c) In one sentence, contrast this with CountMin, whose merge rule is entry-wise *sum* (compatible with the additive count semantics) rather than entry-wise max (compatible with the cardinality semantics). The merge rule is dictated by the algebra of the underlying Bernoulli[T] type, as previewed in Section 10.1.

11. $\star\star\star$ **LSH parameter sweep.** The locality-sensitive hashing (LSH) scheme from Section 10.5 partitions a length-k MinHash signature into b bands of r rows each (so $k = b \cdot r$). Two items are reported as candidates when at least one band has all r signature entries agree.

(a) For two items with Jaccard similarity J, a single row of MinHash agrees with probability J (by Proposition 10.4.1). Derive the candidate probability $p(J) = 1 - (1 - J^r)^b$ by union over bands and intersection over rows within a band.

(b) Plot or tabulate $p(J)$ for $(b, r) = (20, 5)$ over $J \in \{0.1, 0.2, \ldots, 0.9\}$. Identify the S-curve shape: a sharp threshold near $J^* = (1/b)^{1/r}$ that separates high-recall (large J) from low-precision (small J) candidate pairs.

(c) In one sentence, identify the design trade-off: increasing r sharpens the S-curve (fewer false-positive candidates from low-J pairs, at the cost of fewer true candidates from moderate-J pairs); increasing b shifts the S-curve left (more recall, more candidates to verify). The choice of (b, r) is the LSH analog of choosing (w, d) in Theorem 10.2.1.

Chapter 11

Randomized Algorithms

After this chapter you will be able to:

- Restate Miller-Rabin formally as a Bernoulli[bool] with $\omega = 0$ and $\varepsilon \leq 1/4$ per round, closing chapter 3 §3.4's informal walkthrough with a formal proof.
- Distinguish Las Vegas (random runtime, exact output) from Monte Carlo (fixed runtime, random output) and identify when each is appropriate.
- State the RP and BPP complexity classes via the chapter 7 PPV framework, and apply the Chernoff bound for sample-complexity analysis.

11.1 Miller-Rabin Formal

Chapter 3 §3.4 walked through the Miller-Rabin primality test as an extended example, showing step by step how a single round behaves as a Bernoulli[bool] instance with one-sided error and how k rounds compose by the chain rule of Theorem 3.3.1. The exposition there was deliberately informal in one specific respect: the per-round bound $\varepsilon \leq 1/4$ was stated as a fact about strong-liar sets and then applied. The number-theoretic origin of that bound was acknowledged but deferred. This section closes the deferral. We restate the §3.4 walkthrough (see Section 3.4) as a single formal theorem with an explicit appeal to Rabin's original result, then derive the k-round bound as a corollary of Theorem 3.3.1.

The formal statement

Fix an odd integer $n > 3$. Write $n-1 = 2^r d$ with d odd and $r \geq 1$. For each candidate witness $a \in \{2, 3, \ldots, n-2\}$, let MillerRabinRound$(n, a)$ denote the single-round witness test of Section 3.4: it returns COMPOSITE when a certifies the compositeness of n via the square-roots-of-1 criterion, and PROBABLY-PRIME otherwise. Define the witness predicate

$$W_n(a) \;:=\; \text{MillerRabinRound}(n, a),$$

viewed as a Boolean-valued function on the witness pool $\{2, \ldots, n-2\}$ (with PROBABLY-PRIME identified with $\top$ and COMPOSITE with $\bot$, as in chapter 3 §3.4 Step 2).

Theorem 11.1.1 (Miller-Rabin as a Bernoulli[bool])**.** *Let $n > 3$ be an odd integer, and let a be drawn uniformly at random from $\{2, \ldots, n-2\}$. The single-round witness predicate $W_n(a)$ is a* Bernoulli[bool] *instance (in the sense of Definition 4.5.1) whose error rates satisfy*

$$\omega \;=\; 0 \tag{11.1}$$

for every n, and

$$\varepsilon \;\leq\; \frac{1}{4} \tag{11.2}$$

for every odd composite $n > 3$.

If k independent witnesses $a_1, \ldots, a_k$ are drawn uniformly with replacement from $\{2, \ldots, n-2\}$, and the k-round test declares PROBABLY-PRIME *only when every $W_n(a_i)$ does, then the k-round error rates satisfy*

$$\omega^{(k)} \;=\; 0 \quad \textit{and} \quad \varepsilon^{(k)} \;\leq\; \left(\frac{1}{4}\right)^k \tag{11.3}$$

for every odd composite $n > 3$.

Proof sketch

The two claims of Theorem 11.1.1 have different status. The first claim, $\varepsilon \leq 1/4$ per round, is a number-theoretic fact about the structure of $\mathbb{Z}/n\mathbb{Z}$. The second claim, the k-round bound, is a structural consequence that the Bernoulli framework supplies once the first is in hand. We sketch each in turn.

Proof sketch of Theorem 11.1.1. Zero false-negative rate. Suppose n is prime. For any $a \in \{2, \ldots, n-2\}$, Fermat's little theorem gives $a^{n-1} \equiv 1 \pmod{n}$. The sequence $a^d, a^{2d}, a^{4d}, \ldots, a^{2^r d} \pmod{n}$ terminates at 1, and in $\mathbb{Z}/n\mathbb{Z}$ (which is a field when n is prime) the only square roots of 1 are ± 1. Tracing the sequence backwards from $a^{2^r d} = 1$, the last value before 1 must itself be ± 1. The witness test passes. Hence $W_n(a) = \textsc{probably-prime}$ for every witness whenever n is prime, which gives $\omega = 0$.

Per-round false-positive bound. Suppose n is odd composite. Define the *strong-liar set* for n,

$$L_n \;=\; \{\, a \in \{2, \ldots, n-2\} : W_n(a) = \textsc{probably-prime} \,\}.$$

Rabin's theorem [Rab80], building on the earlier deterministic analysis of Miller [Mil76], establishes the strong-liar bound

$$|L_n| \;\le\; \frac{n-3}{4}.$$

This is the number-theoretic content of Miller-Rabin; the proof goes through the structure of $\mathbb{Z}/n\mathbb{Z}$ via the Chinese Remainder Theorem and a careful counting of solutions to $x^{2^j d} \equiv \pm 1 \pmod{n}$ for composite n. We do not reproduce it here; §11.6 records the standard secondary references. With $|L_n| \le (n-3)/4$ and a drawn uniformly from a pool of size $n-3$,

$$\varepsilon \;=\; \Pr[W_n(a) = \textsc{probably-prime} \mid n \text{ composite}] \;=\; \frac{|L_n|}{n-3} \;\le\; \frac{1}{4},$$

which is the bound of equation (11.2).

The k-round bound. The argument here is exactly the chapter 3 §3.4 Step 3 calculation, now phrased as an application of the framework's Axiom 1 (Axiom 1, Section 4.2) followed by the composition theorem. Recast the per-round test as a predicate $\tilde{P}\colon \{2, \ldots, n-2\} \to \{\top, \bot\}$ on the witness pool, so that distinct witnesses $a_i \neq a_j$ are distinct input elements. Drawing the a_i independently and uniformly makes the events $\{a_i \in L_n\}$ mutually independent (this is sampling-with-replacement independence, which holds by the protocol, not by any deeper property of L_n). Axiom 1 therefore applies to $\tilde{P}$. The k-round test is the k-fold AND of $\tilde{P}(a_1), \ldots, \tilde{P}(a_k)$, an identity-with-error chain in the sense of Theorem 3.3.1 with $\omega_i = 0$ and $\varepsilon_i \le 1/4$ for each i.

Substituting into the composition theorem gives

$$\omega^{(k)} \;=\; 1 - \prod_{i=1}^{k} (1 - \omega_i) \;=\; 1 - 1 \;=\; 0,$$

and, in the FNR-zero special case where errors at later rounds cannot cancel an earlier false positive, the false-positive rate of the chain reduces to the product

$$\varepsilon^{(k)} \;\le\; \prod_{i=1}^{k} \varepsilon_i \;\le\; \left(\frac{1}{4}\right)^{k},$$

which is equation (11.3). The bound is uniform over all odd composite $n > 3$, since the per-round bound was. □

The proof has the shape that chapter 3 §3.4 promised. One ingredient is borrowed from number theory and credited to its origin [Mil76; Rab80]; the rest is the Bernoulli framework applied mechanically. The choice of k to drive $\varepsilon^{(k)}$ below a target failure probability δ is a sample-complexity question that we take up in Section 11.4.

What changed from §3.4

The §3.4 walkthrough and the present section are about the same algorithm, the same per-round bound, and the same k-round bound. What changed is the status of the per-round bound in the surrounding argument.

In §3.4, the bound $\varepsilon \le 1/4$ was introduced as a fact to be applied. We named it the Miller-Rabin theorem and attached it to the strong-liar set, but we did not prove it and we did not cite the original source. The pedagogical reason was clear: §3.4 was Part I climax 1, whose purpose was to demonstrate that the chapter 3 composition machinery could carry a real algorithm. Pausing to prove a 1980 number-theory theorem would have distracted from the framework story. The bound was a black box, and the framework was what filled the rest of the page.

In §11.1, the same bound is stated as part of Theorem 11.1.1 and credited to its origins [Rab80; Mil76]. The proof sketch makes the division of labor explicit: number theory supplies the strong-liar bound, the framework supplies everything else. We have not strengthened the §3.4 result, nor have we recomputed it. What we have done is reframe the §3.4 walkthrough as the proof sketch of a single formal theorem, with the per-round bound cited rather than asserted, and the k-round bound derived as a corollary of Theorem 3.3.1 rather than as a self-contained product calculation.

This reframing is the contribution of §11.1, and it is small by design. The chapter 11 promise is not to re-derive Miller-Rabin from scratch; it is to retire the §3.4 informal layer by replacing it with a formal layer that names its borrowed pieces. The remaining sections of chapter 11 extend the same kind

of formalization to the broader landscape of randomized algorithms: the Las Vegas versus Monte Carlo distinction, the complexity classes RP and BPP, and the sample-complexity bounds that govern how many repetitions are needed to reach a target error rate.

11.2 Las Vegas vs Monte Carlo

The chapter 3 §3.4 walkthrough and the formalization of Section 11.1 both treat a single example: Miller-Rabin. That example happens to live at one corner of the randomized-algorithm landscape. To navigate the rest of the landscape one needs a coordinate system, and the standard one separates randomized algorithms by where the randomness is allowed to surface. It either surfaces in the runtime, while the output stays exactly correct, or it surfaces in the output, while the runtime stays predictably bounded. These two regimes are named after the casinos that gave them their public reputations: Las Vegas and Monte Carlo. This section lays out the two definitions, shows the trade-off that forces the choice, walks through three canonical examples, sketches the conversions between the two regimes, and identifies which regime maps onto the Bernoulli[bool] vocabulary that the rest of Part III speaks.

Two definitions

Definition 11.2.1 (Las Vegas algorithm)**.** *A randomized algorithm A on inputs x is a* Las Vegas algorithm *when its output $A(x)$ is deterministically correct on every input and every coin sequence (whenever A halts), and its runtime $T_A(x)$ is a random variable whose distribution may depend on x. The algorithm is* expected-polynomial-time Las Vegas *when $\mathrm{E}[T_A(x)]$ is bounded by a polynomial in $|x|$ for every x.*

The contract a Las Vegas algorithm signs is asymmetric. Output correctness is absolute: no coin flip will ever cause a wrong answer. Runtime is a random variable, and a bad coin sequence can make the algorithm take longer than its expected value. There is no fixed worst-case time bound; an adversarial coin flip is allowed to extend the runtime, as long as the expected runtime stays inside the promised polynomial envelope.

Definition 11.2.2 (Monte Carlo algorithm)**.** *A randomized algorithm A on inputs x is a* Monte Carlo algorithm *when its runtime $T_A(x)$ is deterministically bounded on every input and every coin sequence (typically by a polynomial in $|x|$), and its output $A(x)$ is a random variable that agrees*

with the true answer $f(x)$ only with bounded probability. The probability of disagreement is the algorithm's error rate*, and Monte Carlo algorithms are classified further by whether that rate is one-sided or two-sided. A one-sided Monte Carlo decision algorithm has $\varepsilon \leq \alpha$ and $\omega = 0$ (or the mirror image); a two-sided one has $\varepsilon, \omega \leq \alpha$ for some common bound $\alpha < 1/2$.*

The Monte Carlo contract is the mirror image. Runtime is locked down: every coin sequence completes in at most $T(|x|)$ steps. Output is random, and a bad coin sequence is allowed to return the wrong answer, as long as the probability of that bad sequence is below the advertised error rate.

The two definitions are exhaustive of the practically useful regimes but not of the formal possibilities. A randomized algorithm whose runtime is unbounded *and* whose output may be wrong is rarely interesting; it loses both contracts simultaneously and offers nothing in exchange. The opposite, an algorithm with bounded worst-case runtime *and* deterministically correct output, is simply a deterministic algorithm and needs no randomness at all. Las Vegas and Monte Carlo carve up the non-trivial middle.

The trade-off

The two regimes look like a matter of taste but they are not. They are two faces of a single trade-off, which the following observation makes precise.

Remark. *If $f\colon X \to Y$ is a total function whose graph an algorithm wishes to compute, then for any randomized algorithm A at most one of the following holds with probability* 1 *on every input: (i) $A(x) = f(x)$, or (ii) $T_A(x) \leq T(|x|)$ for a fixed function T. An algorithm that satisfies both for every input on every coin sequence is deterministic.* △

The remark is almost a tautology, but its content is the design tension of the field. A worst-case bounded runtime over all coin sequences leaves the algorithm no room to keep flipping coins until the answer is right. If every coin sequence completes in at most $T(|x|)$ steps, and on some coin sequence the answer the algorithm has computed at step $T(|x|)$ happens to be wrong, the algorithm must return that wrong answer. Conversely, an algorithm that insists on returning the right answer must keep working until it is sure, and on bad coin sequences that takes longer than any fixed bound would allow.

Each regime gives up exactly one of the two guarantees. Las Vegas keeps the output correct and pays in the runtime tail: a bad sample of pivots, a bad sample of hash seeds, and the algorithm runs longer than expected. Monte

Carlo keeps the runtime bounded and pays in the output: a bad sample of witnesses, a bad sample of contractions, and the algorithm reports a wrong answer. Which side to pay on is a question about the application. When an answer that is sometimes wrong is unsafe (a medical decision, a financial settlement) but a longer wait is tolerable, Las Vegas is the sensible choice. When timing is part of the contract and a small error rate is tolerable (a network filter, a real-time approximate query), Monte Carlo is.

Three examples

The two definitions become concrete once one has a few canonical examples in mind. The following three are representative and recur throughout the rest of Part III.

Randomized quicksort (Las Vegas). Sort an array of n elements by recursively partitioning around a pivot chosen uniformly at random from the current sub-array. The output is always the correctly sorted array: every comparison was a comparison, and the partition step preserves the multiset of elements. Runtime is a random variable. The expected number of comparisons is $\mathrm{E}[T(n)] = O(n \log n)$, but a bad sequence of pivot choices can make a particular run take $\Theta(n^2)$ comparisons. The worst-case sequence is exponentially unlikely: $\Pr[T(n) \geq cn \log n] \leq n^{-\Omega(c)}$ for a suitable constant. The algorithm is Las Vegas because the contract is on output correctness, and the runtime tail is the price paid.

Karger min-cut (Monte Carlo). Find a global minimum cut of an n-vertex multigraph by repeatedly contracting a uniformly random edge until only two super-vertices remain, then reporting the set of edges between them as the cut. The runtime is deterministically $O(n^2)$ on every coin sequence. The output is random; on a fixed input graph G with a fixed minimum cut C, Karger's analysis gives

$$\Pr[\text{algorithm returns } C] \;\geq\; \frac{2}{n(n-1)}.$$

A single run has only a $\Theta(1/n^2)$ chance of returning the right cut. The algorithm is salvaged by amplification: running it $k = cn^2 \log n$ independent times and taking the smallest cut found drives the failure probability below $1/n^c$. Each run is fixed-cost; the cost of confidence is the count of runs. This is Monte Carlo through and through.

Miller-Rabin (Monte Carlo). The primality test of Section 11.1. The k-round version has runtime deterministically bounded by $O(k \log^3 n)$ on every coin sequence (each round does a modular exponentiation, which is $O(\log^3 n)$ with schoolbook modular arithmetic). The output is random: on composite inputs, the algorithm may report PROBABLY-PRIME with probability at most $(1/4)^k$, by Theorem 11.1.1. On prime inputs, the algorithm always reports PROBABLY-PRIME, so the error is one-sided. This is the Monte Carlo regime, and it is exactly the regime in which the Bernoulli[bool] vocabulary of Part II applies directly.

The three examples cover the corners of the design space. Randomized quicksort sacrifices runtime tail to keep output exact. Karger min-cut sacrifices output to keep runtime fixed and uses amplification to recover confidence. Miller-Rabin sacrifices output in a one-sided way and uses the composition theorem of chapter 3 to amplify confidence in the same way.

Conversion in both directions

The Las Vegas and Monte Carlo regimes are not strictly disjoint as a matter of design. Each can be converted to the other, at a cost.

Las Vegas to Monte Carlo. Given a Las Vegas algorithm A with expected runtime $\mathrm{E}[T_A(x)] \leq \mu(|x|)$, define the Monte Carlo algorithm A' that runs A for at most $c \cdot \mu(|x|)$ steps and returns "failure" if A has not halted by then. By Markov's inequality, A' returns failure with probability at most $1/c$, and otherwise returns the same correct output as A. Picking $c = 10$ gives a Monte Carlo algorithm with error rate ≤ 0.1 and worst-case runtime $\leq 10\,\mu(|x|)$. The conversion is mechanical, and it explains why expected polynomial time is essentially the same resource as worst-case polynomial time with a bounded error rate.

Monte Carlo to Las Vegas (if a verifier exists). Given a Monte Carlo algorithm A with error rate ε *and* a deterministic polynomial-time verifier V that, given an input x and a candidate output y, decides whether $y = f(x)$, define the Las Vegas algorithm A'' that repeatedly runs $A(x)$, verifies the output with V, and returns the first verified result. The expected number of trials is $1/(1-\varepsilon)$, so the expected runtime is bounded. The output is always correct because V accepts only the true answer. The conversion requires the verifier, which is a non-trivial assumption: some problems admit Monte Carlo algorithms but no known polynomial-time verifier. This asymmetry is why the complexity class ZPP (the Las Vegas analogue of P) is contained

in both RP and coRP, but the reverse containment is open in general; see Section 11.3.

The Bernoulli[bool] correspondence

The Monte Carlo regime is the one that Part II's vocabulary fits exactly. A Monte Carlo *decision* algorithm, on input x, returns a Boolean report $A(x) \in \{\top, \bot\}$ that is the algorithm's guess at the true predicate value $P(x) \in \{\top, \bot\}$. The error structure is precisely the 2×2 channel of Definition 4.5.1:

$$\varepsilon \;=\; \Pr[A(x) = \top \mid P(x) = \bot], \qquad \omega \;=\; \Pr[A(x) = \bot \mid P(x) = \top].$$

A Monte Carlo decision algorithm *is* a Bernoulli[bool] instance, in the sense of the type-membership criterion of chapter 4. The algorithm's stated error rate is the framework's ε (or its mirror ω in the symmetric case). The k-fold amplification calculation of Theorem 11.1.1 is just the composition theorem applied to k independent instances of the same Bernoulli predicate. Everything chapter 7 says about PPV, NPV, and the derived measures applies to a Monte Carlo decision algorithm without modification, with prevalence playing the role of the base rate at which the input distribution presents true positives.

Las Vegas algorithms occupy a different level of abstraction. The output of a Las Vegas algorithm is not a Bernoulli random variable; it is deterministically equal to the true answer. The randomness lives in the runtime, which is a non-negative real-valued random variable, not a Boolean. Framework-level objects such as ε and ω are not defined for a Las Vegas algorithm because there is no error to rate. The right framework for analyzing a Las Vegas algorithm is the theory of runtime tail bounds: Markov's inequality, Chebyshev's inequality, and the Chernoff-style concentration inequalities we will see in Section 11.4. These are about the runtime distribution, not about output error.

The asymmetry is principled. Part III is about the algebra of approximate answers, and approximate answers are what Monte Carlo algorithms produce. Las Vegas algorithms produce exact answers and then ask the user to wait; their analysis belongs to a different chapter of algorithmics, the one about expected runtime, and the Bernoulli framework has nothing to add to it. From this point on in Part III, when we say "randomized algorithm" without further qualification, we mean a Monte Carlo decision algorithm, and we mean it as a Bernoulli[bool] instance.

11.3 RP, BPP, and Bernoulli Type

The Monte Carlo regime of Definition 11.2.2 is the regime in which a single coin sequence may produce a wrong answer with bounded probability. Once one accepts that contract, the obvious next question is which decision problems admit a polynomial-time algorithm of that shape, and how the admissible shapes carve up by where the error is allowed to land. The answer is the family of randomized complexity classes RP, coRP, ZPP, and BPP. This section states each one in its standard form, then reads it back through the Part II vocabulary: a language is in RP exactly when its decision algorithm sits on the FNR-zero axis of the perfect filter line of Definition 6.4.1; coRP is the FPR-zero axis; ZPP is the origin; BPP is the interior of the rate plane below a fixed bound. The correspondence is not an analogy. The complexity-class definitions and the chapter 6 perfect-filter-line classification are the same axes seen from two angles.

RP: one-sided error in the false-positive direction

Definition 11.3.1 (Randomized Polynomial Time, RP)**.** *A language $L \subseteq \{0,1\}^*$ is in the class* RP *when there exists a polynomial-time randomized algorithm A and a polynomial p such that on input $x \in \{0,1\}^*$, A reads at most $p(|x|)$ random bits, runs in time at most $p(|x|)$, and satisfies*

$$\begin{aligned} x \in L &\;\Rightarrow\; \Pr[A(x) = 1] \;\geq\; 1/2, \\ x \notin L &\;\Rightarrow\; \Pr[A(x) = 1] \;=\; 0. \end{aligned}$$

The probability is over A's coin flips alone; x is fixed in both lines.

Read the two lines through the 2×2 channel of Definition 4.5.1, with $A(x) = 1$ identified with the report IN-L and the truth value $\mathbf{1}_{x \in L}$ identified with the predicate $P(x)$. The second line is the statement that the algorithm never reports IN-L on an input that is not in L:

$$\varepsilon \;=\; \Pr[A(x) = 1 \mid x \notin L] \;=\; 0.$$

The first line is the statement that on inputs that are in L, the algorithm reports IN-L with probability at least $1/2$, which is the false-negative bound

$$\omega \;=\; \Pr[A(x) = 0 \mid x \in L] \;\leq\; 1/2.$$

An RP algorithm is, channel-for-channel, a Bernoulli[bool] instance with $\varepsilon = 0$ and $\omega \leq 1/2$. The class RP is the complexity-class incarnation of the

FNR-zero axis of the chapter 6 perfect filter line, polynomially constrained and polynomially advised. Where chapter 5 §5.4 named the Bloom filter as the canonical FNR-zero approximate set, RP plays the same role for decision algorithms: a witness seen is a witness believed, and the only thing that can go wrong is failing to see the witness when one exists. The constant $1/2$ in the first line is conventional; any constant in $(0, 1)$ would give the same class by amplification, a point we return to below.

The canonical RP problem is Non-Zero Multivariate Polynomial, which asks, given an arithmetic circuit computing a polynomial f over a finite field, whether f is not the zero polynomial. The Schwartz-Zippel-DeMillo-Lipton lemma says that if f is not zero, a uniformly random assignment of values to its variables produces a non-zero output with probability at least $1 - d/|F|$, where d is the total degree and $|F|$ the field size. If f is the zero polynomial, every assignment produces zero. Reporting non-zero when the random assignment evaluates to non-zero is exactly the RP shape: silent on zero polynomials (no false positives), occasionally wrong on non-zero polynomials (false negatives at rate $\leq d/|F|$).

coRP and ZPP: the mirror axis and the origin

Definition 11.3.2 (Co-Randomized Polynomial Time, coRP)**.** *A language L is in* coRP *when its complement $\bar{L}$ is in RP. Equivalently, there exists a polynomial-time randomized algorithm A with*

$$\begin{aligned} x \in L &\Rightarrow \Pr[A(x) = 1] = 1, \\ x \notin L &\Rightarrow \Pr[A(x) = 1] \leq 1/2. \end{aligned}$$

The two lines are the mirror image of Definition 11.3.1. Reading them as a channel, $\omega = 0$ and $\varepsilon \leq 1/2$. A coRP algorithm is a Bernoulli[bool] instance on the FPR-zero axis of the perfect filter line: an answer of not-in-L is always correct; the only error mode is reporting in-L when the input is not in L. Miller-Rabin (Theorem 11.1.1) is the canonical coRP algorithm for the language Composites: the algorithm reports composite only with a witness in hand, and that report is always correct; the failure mode is reporting prime on a composite input. Equivalently, the language Primes is in RP via the same algorithm with output negated, and historically this was the textbook example of a non-trivial RP language until the discovery of the deterministic polynomial-time AKS primality test in 2002.

Definition 11.3.3 (Zero-error Probabilistic Polynomial Time, ZPP)**.** *A language L is in* ZPP *when $L \in \text{RP} \cap \text{coRP}$. Equivalently, there exists an*

expected-polynomial-time Las Vegas algorithm (Definition 11.2.1) for L: an algorithm that always returns the correct answer and whose expected runtime is polynomial in $|x|$.

The equivalence is the Las Vegas / Monte Carlo conversion of §11.2. Given an RP algorithm A_+ and a coRP algorithm A_- for the same language L, run them in parallel on input x until one of them produces a definite answer. A_+ produces a definite IN-L on every coin sequence that triggers its acceptance branch; A_- produces a definite NOT-IN-L on every coin sequence that triggers its rejection branch. On every input, at least one of the two algorithms is allowed to be definite on a random fraction $\geq 1/2$ of its coin sequences (the fraction depends on whether $x \in L$ or not). The expected number of parallel rounds to obtain a definite answer is therefore bounded by a constant, and the overall runtime is expected polynomial. Conversely, an expected-polynomial-time Las Vegas algorithm gives an RP algorithm by the truncation trick of §11.2 (run for a constant multiple of the expected time, report NOT-IN-L on timeout) and a coRP algorithm by the opposite truncation. ZPP is the complexity-class incarnation of the origin of the rate plane: $\varepsilon = \omega = 0$, exact decisions, paid for in runtime tail.

BPP: the interior of the rate plane

Definition 11.3.4 (Bounded-error Probabilistic Polynomial Time, BPP)**.** *A language L is in* BPP *when there exists a polynomial-time randomized algorithm A such that*

$$\begin{aligned} x \in L &\Rightarrow \Pr[A(x) = 1] \geq 2/3, \\ x \notin L &\Rightarrow \Pr[A(x) = 1] \leq 1/3. \end{aligned}$$

The two lines are now both bounded away from 1/2 on the right side, but neither is forced to 0 or 1. Read as a channel, $\omega \leq 1/3$ and $\varepsilon \leq 1/3$, so the BPP algorithm sits in the interior of the rate plane, off both axes of the perfect filter line. The constants 2/3 and 1/3 are conventional; any pair (p, q) with $p > 1/2 > q$ gives the same class by amplification (Theorem 11.3.2 below; the proof is deferred to Section 11.4).

The Part II vocabulary applies directly. At workload prevalence $\pi = \Pr[x \in L]$ (which depends on the input distribution the algorithm is deployed against, not on the algorithm itself), the positive predictive value of a BPP algorithm is given by Definition 7.4.1:

$$\text{PPV} = \frac{\pi(1-\omega)}{\pi(1-\omega) + (1-\pi)\varepsilon} \geq \frac{\pi \cdot 2/3}{\pi \cdot 2/3 + (1-\pi) \cdot 1/3} = \frac{2\pi}{2\pi + (1-\pi)} = \frac{2\pi}{\pi + 1}.$$

The expression looks innocent until one substitutes a small π. At $\pi = 0.01$ the bound gives PPV $\geq 0.0198/1.01 \approx 0.0196$: the algorithm's report of IN-L is correct only about 2% of the time, despite the algorithm's 2/3-versus-1/3 accuracy on each distribution. The other 98% of positive reports are false alarms. The negative predictive value of Definition 7.4.2 similarly depends on the workload. The two definitions are not artifacts of medical statistics; they are the unavoidable book-keeping for any decision algorithm whose per-class accuracy is bounded away from 1, including a generic BPP algorithm. A BPP algorithm with a 1/3 error rate looks impressive in isolation and unimpressive at low prevalence, and Definition 7.4.1 is the calculation that explains why.

The conventional gap 2/3 versus 1/3 exists for exactly this reason: it forces a non-trivial signal in both directions, so that amplification can drive both errors to zero rather than only one. A class defined with the gap closed (both probabilities equal to 1/2 on the respective inputs) is the class PP, which is much larger than BPP and is not believed to admit efficient amplification.

Amplification: error reduction by repetition

The constants in the definitions of RP, coRP, and BPP look arbitrary, because they are. Any constant strictly inside $(0, 1)$ for RP, and any constant gap around 1/2 for BPP, gives the same class. The reason is amplification: k independent repetitions plus the appropriate aggregator drive the error rate to zero exponentially in k.

Theorem 11.3.1 (RP amplification)**.** *Let A be an RP algorithm for L with $\omega \leq 1/2$ per run. Define $A^{(k)}$ to be k independent runs of A on x, reporting* IN-L *when any run reports* IN-L *and* NOT-IN-L *otherwise. Then $A^{(k)}$ is a polynomial-time RP algorithm for L with $\omega \leq 2^{-k}$ and $\varepsilon = 0$.*

The proof is the chapter 3 composition theorem (Theorem 3.3.1) applied to k independent draws from the same Bernoulli predicate, with the OR aggregator. The $\varepsilon = 0$ property is preserved by OR (any IN-L report must be true), and the ω at the OR is the product of the per-run ωs by independence (Axiom 1). Bringing ω from 1/2 to 2^{-30} costs a factor of 30 in runtime; bringing it to 2^{-60} costs a factor of 60. This is the perfect-filter-line analog of the k-round Miller-Rabin bound of Theorem 11.1.1, with the only difference being that Miller-Rabin's per-round bound is 1/4 rather than 1/2. coRP amplifies the same way with the AND aggregator, since coRP is RP on the mirror axis.

Theorem 11.3.2 (BPP amplification)**.** *Let A be a BPP algorithm for L with $\omega, \varepsilon \leq 1/3$ per run. Define $A^{(k)}$ to be k independent runs of A on x, reporting* IN-L *when at least $k/2$ runs report* IN-L *and* NOT-IN-L *otherwise. Then $A^{(k)}$ is a polynomial-time BPP algorithm with $\omega, \varepsilon \leq \exp(-\Omega(k))$.*

The BPP amplification rate is exponential in k, but the exponent is no longer the per-run error rate itself; it is the Chernoff exponent for a majority of independent Bernoulli draws. The proof rests on the Chernoff bound (Theorem 11.4.1) applied to the count of majority votes for the wrong answer; we defer it to Section 11.4, which develops the Chernoff machinery in the form needed for sample-complexity calculations across the whole chapter. The qualitative content is what matters here: BPP error decays exponentially in the number of repetitions even though the BPP algorithm cannot drive either error to zero on a single coin sequence. Bringing the per-vote error from $1/3$ to 2^{-30} costs $O(k)$ runs with $k = \Theta(30)$, and the implicit constant is small.

The Bernoulli[bool] correspondence at scale

The four definitions of this section read in the chapter 4 vocabulary as four shapes in the rate plane:

Class	ε bound	ω bound
RP	$= 0$	$\leq 1/2$
coRP	$\leq 1/2$	$= 0$
ZPP	$= 0$	$= 0$
BPP	$\leq 1/3$	$\leq 1/3$

The first three rows trace out the perfect filter line of Definition 6.4.1, with RP and coRP as the two axes and ZPP as their intersection at the origin. The fourth row, BPP, is the interior of the rate plane below the constant $1/3$. The chapter 6 classification of approximate sets by where they sit on the perfect filter line and the classical taxonomy of randomized complexity classes are the same classification applied to two different families of objects: arbitrary Bernoulli[bool] instances in chapter 6, polynomially constrained decision algorithms here. The constraints differ; the geometry of the rate plane is the same.

This is more than a slogan. The constraint that propagates the most is amplification. A point on the FNR-zero axis can be slid arbitrarily close to the origin by independent repetition with OR; the chapter 3 composition theorem says so and Theorem 11.3.1 is the complexity-class-specific

restatement. A point in the interior can be slid arbitrarily close to the origin by independent repetition with majority vote; the Chernoff bound says so and Theorem 11.3.2 is the complexity-class-specific restatement. The two amplification theorems are special cases of the same phenomenon: any Bernoulli[bool] instance whose per-trial error is bounded away from 1/2 on both inputs can be driven to negligible error by enough independent repetitions and the right aggregator. The aggregator depends on which axis or interior region the instance sits in; the existence of amplification does not.

The connection runs the other way too. When chapter 12 lifts the Bernoulli[bool] framework to relations and when chapter 13 unifies all of Part III via the algebra of approximate types, every theorem about RP, coRP, ZPP, and BPP will lift along with it. A randomized algorithm that decides membership in a binary relation $R \subseteq A \times B$ with ε, ω in the BPP regime sits in a class one might call BPP_R, the relational analog of BPP, and amplification will work for it for the same reason it works for the language version. The plumbing the rest of Part III installs is what makes that lift mechanical rather than a theorem-by-theorem retread.

11.4 Sample Complexity

Every randomized algorithm of the preceding three sections has a knob labelled k: the number of Miller-Rabin rounds, the number of BPP repetitions for majority vote, the number of Karger trials. Each k-knob does the same job: drive a per-trial error rate below a target failure probability δ. The sample-complexity question is how to set the knob. Given a target relative error ϵ and a target failure probability δ, how many independent samples k are needed? The answer for every algorithm in this chapter is the same family of bounds, derived from a single theorem about sums of independent Bernoulli random variables. This section states that theorem, sketches its proof via the moment-generating function, and applies it to Miller-Rabin and to BPP amplification. The closing remarks compare the bound to the variance-based confidence interval of Section 7.2: same estimator, two different tails.

The multiplicative Chernoff bound

Fix iid Bernoulli random variables $X_1, \ldots, X_n$ with success probability p, and let $X = \sum_{i=1}^{n} X_i$ be their sum. Write $\mu = \mathbb{E}[X] = np$ for the mean. The textbook variance bound of Proposition 7.2.2 controls $\Pr[|X - \mu| \geq t]$ polynomially in t. The Chernoff bound improves the polynomial decay to

exponential decay in $\mu\epsilon^2$, at the price of an upper tail expressed in multiplicative form $(1+\epsilon)\mu$ rather than additive form $\mu + t$.

Theorem 11.4.1 (Multiplicative Chernoff bound)**.** *Let $X_1, \dots, X_n$ be independent Bernoulli random variables with $\Pr[X_i = 1] = p$, and set $X = \sum_{i=1}^{n} X_i$ and $\mu = np$. For every $\epsilon \in (0, 1]$,*

$$\Pr[X \geq (1+\epsilon)\mu] \;\leq\; \exp\!\left(-\frac{\mu\epsilon^2}{3}\right), \tag{11.4}$$

and symmetrically

$$\Pr[X \leq (1-\epsilon)\mu] \;\leq\; \exp\!\left(-\frac{\mu\epsilon^2}{2}\right). \tag{11.5}$$

Proof sketch. The argument is the moment-generating-function (MGF) plus Markov recipe, optimized over the free parameter. Fix $t > 0$. By Markov's inequality applied to the random variable e^{tX},

$$\Pr[X \geq a] \;=\; \Pr[e^{tX} \geq e^{ta}] \;\leq\; e^{-ta}\,\mathbb{E}[e^{tX}].$$

Independence of the X_i factors the MGF of the sum into the product of the per-variable MGFs:

$$\mathbb{E}[e^{tX}] \;=\; \prod_{i=1}^{n} \mathbb{E}[e^{tX_i}] \;=\; \prod_{i=1}^{n} \bigl(1 - p + pe^{t}\bigr) \;=\; \bigl(1 + p(e^{t} - 1)\bigr)^{n}.$$

The pointwise inequality $1 + x \leq e^{x}$ valid for all real x (applied with $x = p(e^t - 1) \geq 0$) gives

$$\mathbb{E}[e^{tX}] \;\leq\; \exp\bigl(np(e^{t} - 1)\bigr) \;=\; \exp\bigl(\mu(e^{t} - 1)\bigr).$$

Substituting $a = (1+\epsilon)\mu$ into the Markov step yields

$$\Pr[X \geq (1+\epsilon)\mu] \;\leq\; \exp\bigl(-t(1+\epsilon)\mu + \mu(e^{t} - 1)\bigr).$$

The right-hand exponent is minimized over $t > 0$ at $t = \ln(1+\epsilon)$, where it equals $-\mu\bigl((1+\epsilon)\ln(1+\epsilon) - \epsilon\bigr)$. The function $\phi(\epsilon) := (1+\epsilon)\ln(1+\epsilon) - \epsilon$ satisfies $\phi(\epsilon) \geq \epsilon^2/3$ for $\epsilon \in (0, 1]$ (a Taylor-remainder estimate; the constant $1/3$ is the value that makes the inequality tight at $\epsilon = 1$). Substituting gives (11.4). The lower-tail bound (11.5) follows from the same recipe with $t < 0$ and the matching estimate $\phi(-\epsilon) \geq \epsilon^2/2$. □

The two bounds are not symmetric in the constant: the upper tail has exponent $\mu\epsilon^2/3$ and the lower tail has the tighter $\mu\epsilon^2/2$. This asymmetry is a feature of the Bernoulli case (large deviations above the mean are harder than below because of the $\{0,1\}$ ceiling) and matters in sample-complexity calculations only when the dominant failure mode is on a specific side. When both tails are bad and a two-sided bound is wanted, taking the union of the two events and using the worse constant gives

$$\Pr[|X-\mu| \geq \epsilon\mu] \;\leq\; 2\exp\!\left(-\frac{\mu\epsilon^2}{3}\right),$$

which is the form used in most of what follows.

Application: Miller-Rabin round count

The simplest application is also the cleanest, because Miller-Rabin sits on the FNR-zero axis and the Chernoff bound is not needed at all for the count. The per-round false-positive rate of Theorem 11.1.1 is $\varepsilon \leq 1/4$, and the k-round composition theorem reduces $\varepsilon^{(k)}$ to the product of per-round rates: $\varepsilon^{(k)} \leq (1/4)^k$. Choosing k to drive $\varepsilon^{(k)}$ below a target δ requires solving $(1/4)^k \leq \delta$, which gives

$$k \;\geq\; \log_4(1/\delta) \;=\; \frac{\log_2(1/\delta)}{2}.$$

At $\delta = 10^{-6}$, $\log_2(10^6) \approx 19.93$, so $k \geq 10$ rounds suffice. At $\delta = 10^{-12}$, $k \geq 20$ rounds. At $\delta = 2^{-128}$ (cryptographic-strength confidence), $k \geq 64$. The bound is sharp up to the per-round constant $1/4$: doubling the desired confidence in bits costs a constant number of additional rounds, not a constant factor.

This calculation does not use Theorem 11.4.1 at all. The reason is that Miller-Rabin's error is one-sided (ε only, with $\omega = 0$), so a single failure event collapses the chain. The relevant bound is the chain rule of Theorem 3.3.1, not a concentration bound on a sum. Chernoff enters when the failure event is a tail count rather than a single witness, which is the BPP case below.

Application: BPP amplification

The BPP majority-vote amplification of Theorem 11.3.2 deferred its Chernoff calculation to this section. We can now supply it. Let A be a BPP algorithm with per-run accuracy bounded by $1/2+\epsilon_0$ on every input, where $\epsilon_0 > 0$ is the margin above the $1/2$ threshold. (The conventional $2/3$ versus

1/3 of Definition 11.3.4 corresponds to $\epsilon_0 = 1/6$.) Run A on input x for k independent rounds, and report the majority of the k outputs. We bound the probability that the majority is wrong.

Let $X_i = 1$ if the ith run gives the correct answer, 0 otherwise. By the BPP guarantee, $\Pr[X_i = 1] \geq 1/2 + \epsilon_0$ on every x, and by Axiom 1 the X_i are mutually independent. Let $X = \sum_{i=1}^{k} X_i$ be the number of correct runs. The majority vote is wrong exactly when $X < k/2$. The mean of X is $\mu = k(1/2 + \epsilon_0)$, and the threshold $k/2$ sits below μ by the deviation

$$\mu - k/2 \;=\; k\epsilon_0,$$

which in multiplicative form is

$$k/2 \;=\; (1 - \epsilon')\mu \quad \text{with} \quad \epsilon' \;=\; \frac{\epsilon_0}{1/2 + \epsilon_0}.$$

Applying the lower-tail bound (11.5) gives

$$\Pr[X < k/2] \;\leq\; \Pr[X \leq (1 - \epsilon')\mu] \;\leq\; \exp\!\left(-\frac{\mu(\epsilon')^2}{2}\right).$$

Substituting $\mu = k(1/2 + \epsilon_0)$ and $(\epsilon')^2 = \epsilon_0^2/(1/2 + \epsilon_0)^2$, the right side simplifies to

$$\exp\!\left(-\frac{k\epsilon_0^2}{2(1/2 + \epsilon_0)}\right) \;\leq\; \exp\!\left(-\frac{2k\epsilon_0^2}{3}\right)$$

when $\epsilon_0 \leq 1/2$. Loosening the constant to a clean $\exp(-k\epsilon_0^2/3)$ and asking that this be at most δ,

$$k \;\geq\; \frac{3\ln(1/\delta)}{\epsilon_0^2}.$$

For the conventional BPP gap $\epsilon_0 = 1/6$, this is $k \geq 108\ln(1/\delta)$, so $\delta = 10^{-6}$ requires $k \approx 1490$ rounds. The constant 108 looks large; in practice the proof can be tightened by using (11.5) with its better $\mu\epsilon^2/2$ exponent throughout and the per-step inequalities at their tightest, which trims the constant to single digits. The qualitative point is what matters: BPP amplification is $O((1/\epsilon_0^2)\log(1/\delta))$ rounds, exponential decay in k rather than polynomial, and the exponent is the square of the margin above 1/2.

The cost asymmetry between BPP amplification and RP amplification is worth naming. RP amplification (Theorem 11.3.1) brings ω from 1/2 to 2^{-k} in k rounds: a bit of confidence per round. BPP amplification brings the same confidence in $(3/\epsilon_0^2)k$ rounds, with a quadratic dependence on the margin. A BPP algorithm with a 2/3-versus-1/3 gap pays a $\Theta(1/\epsilon_0^2) = \Theta(36)$

overhead per bit of confidence, relative to a one-sided RP algorithm. This is the price of the two-sided contract: when both errors can occur on the same coin sequence, a single repetition is not enough; a vote across many repetitions is needed, and votes concentrate exponentially in the margin squared rather than exponentially in the count.

Connection to the variance-based interval

The Chernoff bound and the variance-based confidence interval of Section 7.2 are two tail bounds for the same estimator. The binomial estimator $\hat{p} = X/n$ studied there had variance $p(1-p)/n$ (Proposition 7.2.2); Chebyshev's inequality applied to that variance gives

$$\Pr[|\hat{p} - p| \geq \epsilon] \;\leq\; \frac{p(1-p)}{n\epsilon^2} \;\leq\; \frac{1}{4n\epsilon^2},$$

where the second step uses $p(1-p) \leq 1/4$. The bound is polynomial in $n\epsilon^2$: doubling n halves the failure probability. The Chernoff bound on the same estimator (rewriting $\hat{p} = X/n$ with $X = \sum X_i$ and $\mu = np$, taking absolute deviation ϵ in the additive sense rather than multiplicative) gives

$$\Pr[|\hat{p} - p| \geq \epsilon] \;\leq\; 2\exp\bigl(-2n\epsilon^2\bigr)$$

by Hoeffding's inequality, the additive cousin of Theorem 11.4.1 specialized to bounded $\{0,1\}$ random variables. The bound is exponential in $n\epsilon^2$: doubling n squares the failure probability.

The two bounds describe the same estimator, but they answer different questions at different cost. Chebyshev gives a bound that requires only the variance, so it works for any distribution with the right second moment; Chernoff requires the full MGF, so it is restricted to distributions whose MGF is finite. The variance-based bound is what chapter 7 §7.2 used to construct an unbiased point estimate with a crisp Cramer-Rao certificate; the Chernoff bound is what chapter 11 uses to specify sample complexity. The two are not in competition; they are the polynomial and the exponential tails of the same binomial. When the question is "what is the variance of $\hat{p}$," the chapter 7 answer suffices. When the question is "how many samples to drive the failure probability below δ," the chapter 11 answer is needed, because the polynomial bound forces $n = \Theta(1/\delta)$ samples whereas the exponential bound asks only $n = \Theta(\log(1/\delta))$. The gap between $1/\delta$ and $\log(1/\delta)$ is the gap between a system that needs 10^6 samples for $\delta = 10^{-6}$ confidence and one that needs 20. This is why the Chernoff bound (and not Chebyshev) is the workhorse of randomized-algorithm analysis.

11.5 Bridge

The four prior sections of this chapter trace one figure from four angles. Section 11.1 closes the chapter 3 §3.4 deferral by restating the Miller-Rabin witness predicate as a formal Bernoulli[bool] instance with $\omega = 0$ and $\varepsilon \leq 1/4$ per round (Theorem 11.1.1), which makes the per-round bound a theorem with citations to Rabin (1980) rather than an unmotivated assertion. Section 11.2 separates that contract from its sibling: Las Vegas algorithms (Definition 11.2.1) trade unbounded worst-case runtime for a deterministic output, while Monte Carlo algorithms (Definition 11.2.2) trade a bounded error rate for a fixed runtime, and the Monte Carlo regime is exactly the regime in which Bernoulli[bool] analysis applies. Section 11.3 carves up the Monte Carlo regime into the classical complexity classes RP, coRP, ZPP, and BPP, and reads each one as a location on the chapter 6 rate plane: RP (Definition 11.3.1) sits on the FNR-zero axis of the perfect filter line of Definition 6.4.1, coRP on the FPR-zero axis, ZPP at the origin, BPP in the bounded interior. The one-sided versus two-sided distinction of Section 5.4 is the same distinction that separates RP from BPP. Section 11.4 closes the arc by quantifying how many independent rounds drive the chapter 4 Axiom 1 composition bound below a target δ: the multiplicative Chernoff bound (Theorem 11.4.1) gives the tight $k = \Theta(\log(1/\delta))$ count that makes randomized algorithms practical at all, and the same machinery sharpens the variance-based confidence interval of the chapter 7 binomial estimator (Section 7.2) from polynomial to exponential decay. Five sections, one Bernoulli[bool]: a per-round bound, a runtime contract, a location on the rate plane, a sample budget.

The chapter 11 picture extends in two directions before Part III closes. Chapter 12 lifts the analysis from Bernoulli[bool] and the approximate functions of chapter 9 to Bernoulli[R] for binary relations $R \subseteq A \times B$. A random approximate relation is what is needed once the question stops being "does x belong to S" and starts being "do x and y stand in some relation," which is the natural setting for join queries, equality predicates over pairs, and multi-table membership tests. The randomized-algorithm vocabulary of the present chapter carries over without modification: a Monte Carlo relation-test algorithm is a Bernoulli[R] instance with a known per-pair error rate, and amplification by repetition is the chapter 3 composition theorem specialized to the corresponding two-state channel.

Chapter 13 closes Part III by giving the type-driven algebra under which the chapter 10 sketches, the present chapter's randomized algorithms, and the chapter 12 relations are all instances of a small set of constructors over

Bernoulli[T]. Sketches over product types Bernoulli[$T \times U$], sketches over sum types Bernoulli[$T + U$], sketches over list types Bernoulli[$L\,T$], and Monte Carlo decision procedures as Bernoulli[bool] all compose by structure once the type constructors are fixed. The present chapter is the algorithmic instance of that algebra; the next two chapters supply the relational instance and the algebraic vocabulary that unifies them.

Chapter 12

Approximate Relations

Placeholder. Content drafted in Plan 3.

Chapter 13

The Algebra of Approximate Types

Placeholder. Content drafted in Plan 3.

Part IV

Optimality and Construction

Chapter 14

Optimal Bernoulli Sets

Placeholder. Content drafted in Plan 4.

Chapter 15

Perfect Hash Filters

Placeholder. Content drafted in Plan 4.

Part V

The Boundary and Beyond

Chapter 16

Approximate but Not Bernoulli

Placeholder. Content drafted in Plan 4.

Chapter 17

Cipher Maps and Trapdoor Computing

Placeholder. Content drafted in Plan 4.

Chapter 18

Encrypted Search as Relaxed Trapdoor

Placeholder. Content drafted in Plan 4.

Appendix A

Probability Refresher

Placeholder. Content drafted in Plan 5.

Appendix B

The Five Core Principles

Placeholder. Cheat-sheet of the Two Axioms, Parametric Parsimony, Space-Accuracy Duality, Composition Closure, and the Boolean Thread. Content drafted in Plan 5.

Appendix C

Library Walkthrough

Placeholder. Tour of the `bernoulli` *Python package as a unified entry point. Content drafted in Plan 5.*

Appendix D

Selected Exercise Solutions

Placeholder. Roughly half of the end-of-chapter exercises are worked here; the rest are reader practice. Content drafted in Plan 5.